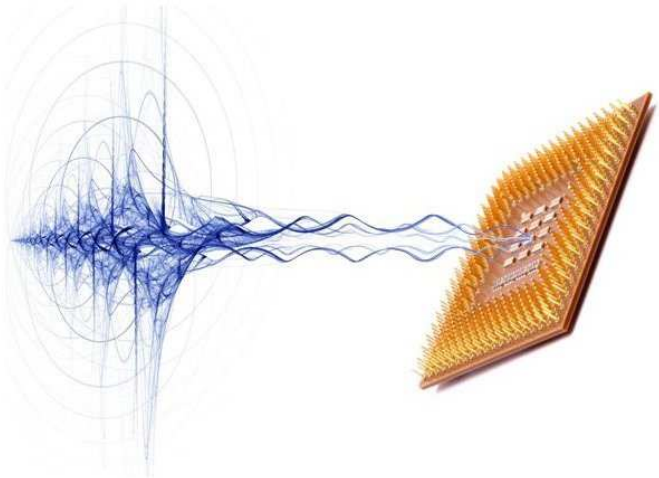
Électronique et Informatique Industrielle 2nde année - EII2
29 septembre 2014

Traitement Numérique du Signal

Polycopié de cours



Olivier Sentieys

ENSSAT - Université de Rennes 1
olivier.sentieys@enssat.fr
http://www.irisa.fr/cairn
6 Rue de Kerampont - CS 80518
22305 LANNION - France

IRISA — ENSSAT
Institut de Recherche en Informatique et Systèmes Aléatoires
École Nationale Supérieure de Sciences Appliquées et de Technologie
Technopôle Anticipa Lannion

Table des matières

Introduction **3**
Remarques sur la notation . . . 4
Présentation du document . . . 4
Utilisation de Matlab et Scilab . . . 4

1 Filtrage numérique **5**
1.1 Introduction au filtrage numérique . . . 5
1.2 Représentation d'un filtre numérique . . . 6
1.3 Spécification d'un filtre numérique . . . 8
1.3.1 Spécifications des filtres passe-bas et passe-haut . . . 9
1.3.2 Spécifications des filtres passe-bande et réjecteur-de-bande . . . 9
1.4 Classification des filtres numériques . . . 10
1.4.1 Filtres récursifs RII . . . 10
1.4.2 Filtres non récursifs RIF . . . 11
1.5 Analyse fréquentielle des filtres numériques . . . 12
1.6 Structures des filtres RII et RIF . . . 12
1.6.1 Structure des filtres RIF . . . 12
1.6.2 Structure des filtres RII . . . 14

2 Synthèse des filtres RII **17**
2.1 Introduction et rappels en filtrage analogique . . . 17
2.1.1 Introduction . . . 17
2.1.2 Rappels en filtrage analogique . . . 17
2.2 Méthode de l'invariance impulsionnelle . . . 21
2.3 Transformation d'Euler . . . 22
2.4 Transformation bilinéaire . . . 24

3 Synthèse des filtres RIF **29**
3.1 Introduction . . . 29
3.2 Filtres à phase linéaire . . . 29
3.2.1 Filtre RIF à phase linéaire de type I . . . 31
3.2.2 Filtre RIF à phase linéaire de type II . . . 31
3.2.3 Filtre RIF à phase linéaire de type III . . . 32
3.2.4 Filtre RIF à phase linéaire de type IV . . . 32
3.3 Méthode de synthèse par fenêtrage . . . 34
3.3.1 Caractéristiques des principales fenêtres . . . 36

3.3.2 Choix de la fenêtre dans la méthode de synthèse . . . 37
3.4 Méthode de synthèse par échantillonnage en fréquence . . . 39

4 Transformée de Fourier Rapide 41
4.1 Naissance de la TFR . . . 41
4.2 TFR partagée dans le temps (DIT) . . . 42
4.3 TFR partagée dans les fréquences (DIF) . . . 45
4.4 Autres graphes de la TFR . . . 46
4.5 Annexes au chapitre sur la TFR . . . 48
4.5.1 Graphe d'une TFR DIT radix 2 sur 16 points . . . 48
4.5.2 Graphe d'une TFR DIF radix 2 sur 16 points . . . 49
4.5.3 Graphe d'une TFR à géométrie constante sur 16 points . . . 50
4.5.4 Graphe d'une TFR DIF radix 4 sur 16 points . . . 51
4.5.5 Algorithme DIF de la TFR radix 2 sur I points complexes . . . 52

5 Analyse spectrale de signaux numériques 55
5.1 Introduction . . . 55
5.2 Troncature d'un signal discrétisé . . . 55
5.2.1 Opération dans le domaine temporel . . . 55
5.2.2 Conséquences dans le domaine fréquentiel . . . 56
5.3 Analyse spectrale par TFD . . . 56
5.4 Zéro-Padding . . . 58
5.5 Paramètres d'une analyse spectrale . . . 59
5.6 Conclusion (méthodologie) . . . 62

6 Systèmes multi-cadences 65
6.1 Réduction de la fréquence d'échantillonnage . . . 65
6.2 Augmentation de la fréquence d'échantillonnage . . . 66
6.2.1 Élévateur de fréquence d'échantillonnage . . . 68
6.2.2 Interpolation . . . 69
6.2.3 Multiplication de la fréquence d'échantillonnage par un facteur rationnel . . . 69

7 Arithmétique virgule fixe 71
7.1 Présentation de l'arithmétique virgule fixe . . . 71
7.1.1 Les différents types de codage . . . 71
7.1.2 Définition des règles de l'arithmétique virgule fixe . . . 78
7.1.3 Multiplication en binaire à base de décalage . . . 80
7.2 Codage en virgule fixe . . . 83
7.2.1 Détermination de la dynamique des données . . . 85
7.2.2 Détermination de la position de la virgule . . . 92
7.2.3 Détermination de la largeur des données . . . 99
7.3 Evaluation de la précision . . . 103
7.3.1 Processus de quantification et de dépassement . . . 103
7.3.2 Modélisation du bruit . . . 105
7.3.3 Filtrage du bruit . . . 111
7.4 Exemple . . . 116
7.4.1 Description de l'application . . . 117

7.4.2 Détermination de la specification virgule fixe . . . 117
7.4.3 Code C de l'application . . . 120
7.4.4 Evaluation de la précision . . . 121

8 Processeurs de traitement du signal 123
8.1 Introduction . . . 123
8.1.1 Les solutions architecturales . . . 123
8.1.2 Domaine d'application des processeurs de traitement du signal . . . 126
8.2 Description de l'unité de traitement . . . 127
8.2.1 Représentation des données . . . 127
8.2.2 Présentation des différents éléments du chemin de données . . . 129
8.2.3 Architecture des chemins de données . . . 134
8.3 Description de l'unité mémoire . . . 136
8.3.1 Modèles de localisation des opérandes . . . 136
8.3.2 Spécificité des DSP . . . 138
8.3.3 Mode d'adressage . . . 138
8.4 Description de l'unité de contrôle . . . 141
8.4.1 Type de code et Pipeline . . . 141
8.4.2 Format du jeu d'instructions . . . 142
8.4.3 Structure de contrôle . . . 144

A Tables des transformées 147
A.1 Transformées en Z . . . 147
A.2 Transformées de Fourier . . . 148

B Abaques de filtrage analogique 151
B.1 Filtres de Butterworth . . . 151
B.2 Filtres de Bessel . . . 151
B.3 Filtres de Chebyshev . . . 154
B.4 Filtres elliptiques de Cauer . . . 155

0pt0pt 0pt0pt 0pt0pt

Introduction

Le traitement numérique du signal est une notion qu'il n'est pas facile de définir simplement étant donné le nombre important d'applications relevant de cette discipline. En première approximation, on peut tout d'abord tenter d'expliciter chacun des mots de cette expression. *Traitement* signifie que l'on est en présence d'un processus de séquencement d'opérations programmées. Une séquence d'opérations s'applique ici à une suite de données *Numériques* qui vont représenter sous une forme discrète un paramètre variable, ou *Signal*, qui le plus souvent est extérieur au processus de traitement.
Il s'agit donc d'appliquer un traitement ou une analyse de l'information à une séquence de nombres discrets qui représente un signal provenant pour la majorité des applications du monde physique qui nous entoure.
Le traitement numérique d'un signal nécessite un support matériel permettant d'effectuer le traitement de l'information, ce peut-être du matériel électronique spécifique à une tâche particulière ou du matériel moins spécialisé comme peut-l'être un ordinateur.
Il faut pouvoir communiquer entre le monde physique extérieur et le processus par lequel s'effectue le traitement ; le signal extérieur, s'il est défini sur un support continu, doit être représenté sous une forme discrète. Cela veut dire que l'on accepte de perdre de l'information : entre deux valeurs consécutives et discrètes du signal nous faisons l'hypothèse de ne disposer d'aucune autre information. Nous verrons qu'une analyse du problème à traiter permet de définir correctement ce qui est conservé ou non dans la forme numérique d'un signal.
Il faut bien sûr définir la séquence des opérations qui transforme le signal numérique. Cette opération correspond à un objectif de traitement bien précis, par exemple supprimer l'écho des lignes téléphoniques, reconnaître une signature radar, etc...

Le développement de l'électronique permit le traitement du signal analogique ; un signal électronique *analogique* suit continument le signal physique qui lui est relié via un capteur. C'est le développement des calculateurs numériques qui a conduit à l'essor du traitement numérique du signal. C'est en effet une approche souple - les traitements correspondent à des logiciels, les supports matériels sont polyvalents -, les états discrets d'un calculateur sont stables - ce n'est en effet pas le cas des systèmes analogiques fortement sensibles aux dérives dues par exemple aux conditions de température ou aux problèmes de vieillissement -. Cependant, l'approche numérique peut se révéler parfois complexe pour des applications très simples. La quantité d'information pouvant être traitée est corrélée à la vitesse de calcul ; la réalisation d'un traitement sous forme analogique ira toujours plus vite que par une forme numérique avec un calculateur et du logiciel.

Remarques sur la notation Dans la suite du document, on notera par f la fréquence en Hz, f_e la fréquence d'échantillonnage, T la période d'échantillonnage et $\omega = 2\pi f$ la pulsation en rad/s. Pour des raisons de simplicité d'expression, on peut faire abstraction de T si on sait que toutes les grandeurs utilisées seront relatives à T ou f_e.

Pour ces raisons, on peut utiliser dans le domaine fréquentiel, la variable $\Omega = 2\pi f.T = 2\pi f/f_e$ appelée pulsation relative. Un filtre numérique exprimé selon la variable Ω (ou un signal discret) sera donc périodique de période 2π (c'est à dire f_e) (voir figure 1.7) et son gabarit sera défini entre 0 et π (c'est à dire entre 0 et $f_e/2$).

Présentation du document Ce document s'intéresse à l'analyse, la conception et l'implantation des applications de traitement numérique du signal. Ce document correspond à la suite du polycopié de cours de Signaux et Systèmes discrets de première année. Les chapitres 1 à 6, rédigés par O. Sentieys, concernent l'analyse et la conception d'algorithmes de traitement numérique du signal. Les filtres numériques sont analysés dans le chapitre 1 et la synthèse des filtres RII et RIF est détaillée dans les chapitres 2 et 3. La transformée de Fourier rapide est présentée dans le chapitre 4. Le chapitre 5 concerne l'analyse spectrale de signaux numériques. Les systèmes multi-cadences sont analysés dans le chapitre 6. Les chapitres 7 et 8, rédigés par D. Ménard (maintenant Professeur à l'INSA de Rennes) et O. Sentieys, concernent l'implantation d'algorithmes de traitement numérique du signal. Le chapitre 7 détaille le processus de codage en virgule fixe d'une application de TNS. Le chapitre 8 présente l'architecture des processeurs de traitement du signal et les outils de développement associés.

Utilisation de Matlab et Scilab Matlab, acronyme de *Matrix Laboratory*, est un environnement logiciel interactif puissant dédié au calcul numérique et à la visualisation ; il est très utilisé dans les divers domaines des sciences pour l'ingénieur, tant pour l'analyse que la conception. Il existe également un nombre important de *toolboxes* qui étendent les possibilités de Matlab à divers domaines spécialisés au moyen de fonctions supplémentaires : traitement du signal, automatique, traitement d'images, optimisation, réseaux de neurones, logique floue *etc...* Il faut considérer Matlab comme étant avant tout un outil de calcul matriciel. Scilab (http ://www-rocq.inria.fr/scilab) est un équivalent libre de droit de Matlab qui se révèle donc intéressant si on ne possède pas de licence Matlab. Même si elle est proche dans sa philosophie, la syntaxe des commandes pour l'utilisation de Scilab n'est pas totalement compatible avec Matlab.
La plupart des exemples ou figures utilisés ont été réalisés sous Matlab ou Scilab et seront illustrés sous Matlab à travers des exemple au cours du document afin d'effectuer l'apprentissage de ce type de logiciel, aujourd'hui indispensable pour tout traiteur de signaux.

Remerciements Merci à Michel Corazza, Hervé Chuberre, Olivier Berder et Olivier Boëffard pour leur aide dans l'élaboration de ce document.

Chapitre 1

Filtrage numérique

1.1 Introduction au filtrage numérique

Il est difficile de donner une définition formelle de la notion de filtrage. L'ingénieur électronicien pense souvent à une modification des caractéristiques fréquentielles d'un signal donné d'entrée. D'un point de vue théorique, le domaine fréquentiel est couplé au domaine temporel, le filtrage modifie donc également la réponse dans ce dernier.

A une séquence d'échantillons d'un signal d'entrée à temps discret $x(n)$, un filtre numérique, défini par sa réponse impulsionnelle $h(n)$ ou par sa fonction de transfert en z $H(z)$, répond par une séquence d'échantillons d'un signal de sortie $y(n)$ (figure 1.1).

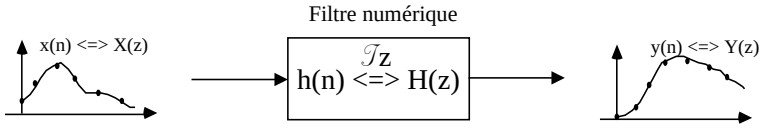


FIGURE 1.1 – Représentation sous forme de fonction de transfert en z

Des exemples de filtrage sont donnés ci après.

- Réduction de bruit pour des signaux radio, des images issues de capteurs, ou encore des signaux audio.
- Modification de certaines zones de fréquence dans un signal audio ou sur une image.
- Limitation à une bande fréquentielle pré-définie.
- Fonctions spéciales (dérivation, intégration, transformée de Hilbert, ...).
- Dans l'exemple du code DTMF (Digital Tone Multiple Frequency) utilisé en téléphonie, le signal transmis est la somme de deux sinusoïdes dont les fréquences sont normalisées (voir figure 1.2 gauche). Il résulte du choix de la touche appuyée sur votre téléphone. Ce principe est souvent qualifié de *fréquences vocales*. A la réception, pour reconnaître le numéro composé, une série de bancs de filtres est utilisée (voir figure 1.2 droite). Une première discrimination de deux zones fréquentielles est réalisée par un filtre passe haut et un filtre passe-bas. Puis, dans chaque zone, une série de filtres passe-bande suivis d'un détecteur permet de déterminer la présence d'une fréquence particulière.

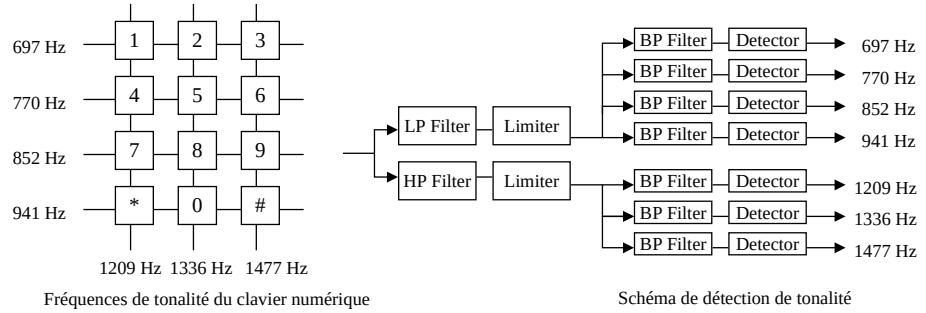


FIGURE 1.2 – Exemple du code DTMF en téléphonie

1.2 Représentation d'un filtre numérique

Un filtrage numérique peut être représenté en utilisant plusieurs types de spécifications.

1. **Fonction de transfert en z.** Ce mode de représentation est le plus usuel. Il permet de lier l'entrée et la sortie dans le plan z par $Y(z) = H(z).X(z)$. On posera dans la suite :

$$H(z) = \frac{N(z)}{D(z)} = \frac{\sum_{i=0}^{N} b_i.z^{-i}}{1 + \sum_{i=1}^{N} a_i.z^{-i}} \tag{1.1}$$

où $N(z)$ est le polynôme du numérateur de la fonction de transfert, tandis que $D(z)$ est son dénominateur. N est ici *l'ordre* du filtre. Dans le cas où $H(z)$ possède des pôles, on parlera de filtres RII (pour Réponse Impulsionnelle Infinie). Si $N(z) = 1$, on parlera de filtre tous-pôles. Dans le cas où $D(z) = 1$, le filtre ne possède que des zéros. Cette famille de filtre correspond au cas des filtres RIF (pour Réponse Impulsionnelle Finie). Celle ci n'a pas d'équivalent en filtrage analogique, et nous verrons que ses propriétés en font une fonction très utilisée en traitement numérique du signal.

L'équation 1.1 peut également être représentée en mettant en avant les pôles et les zéros.

$$H(z) = b_0 \frac{\prod_{i=1}^{N}(z - z_i)}{\prod_{i=1}^{N}(z - p_i)} \tag{1.2}$$

où p_i sont les pôles et z_i sont les zéros de $H(z)$. On rappelle ici que la stabilité du filtre sera déterminé par l'appartenance des pôles au cercle unité (i.e. $|p_i| < 1$), et que des zéros appartenant au cercle unité caractériseront un filtre à minimum de phase.
La figure 1.3 montre plusieurs versions de représentations de $H(z)$. La forme directe (figure1.3.a) peut être décomposée en produit ou en somme de fonctions de transfert d'ordre inférieur, généralement d'ordre 2. L'équation 1.3 et la figure 1.3.b représentent la forme parallèle, tandis que l'équation 1.4 et la figure 1.3.c représentent la forme cascade.

$$H(z) = \sum_{i=1}^{M} H_i(z) = \sum_{i=1}^{M} \frac{b_0 + b_1.z^{-1} + b_2.z^{-2}}{1 + a_1.z^{-1} + a_2.z^{-2}} \tag{1.3}$$

$$H(z) = \prod_{i=1}^{M} H_i(z) = \prod_{i=1}^{M} \frac{b_0 + b_1.z^{-1} + b_2.z^{-2}}{1 + a_1.z^{-1} + a_2.z^{-2}} \tag{1.4}$$

2. **Réponse impulsionnelle.** La réponse impulsionnelle est la fonction en z inverse de $H(z)$.

$$H(z) = \sum_{n=0}^{\infty} h(n).z^{-n} \tag{1.5}$$

Comme en filtrage analogique, la sortie d'un filtre $y(nT)$ est le résultat de la convolution du signal d'entrée représenté de manière temporelle $x(nT)$ avec la réponse impulsionnelle du filtre $h(nT)$. On a alors $y(nT) = x(nT) * h(nT)$, ou, si on fait abstraction de la période d'échantillonnage T :

$$y(n) = x(n) * h(n) = \sum_{k=0}^{\infty} x(k).h(n-k) = \sum_{k=0}^{\infty} x(n-k).h(k) \tag{1.6}$$

Dans le cas où $x(n)$ est une impulsion $\delta(n)$, on retrouve bien $y(n) = h(n)$.

Selon les cas où h(n) est à support infini ou fini, on retrouvera respectivement les deux types de filtres RII et RIF.

3. **Équation aux différences.** Une transformation en z inverse de l'équation 1.1 permet d'aboutir à la forme suivante :

$$y(n) = \sum_{i=0}^{N} b_i.x(n-i) - \sum_{i=0}^{N} a_i.y(n-i) \tag{1.7}$$

On identifie ici deux parties distinctes : une partie fonction de la valeur courante et des valeurs précédentes de l'entrée $x(n)$, et une partie fonction des valeurs précédentes de la sortie $y(n)$. Selon si les a_i sont non nuls ou nuls, on parlera donc de filtres *récursifs* ou de filtres *non récursifs*.

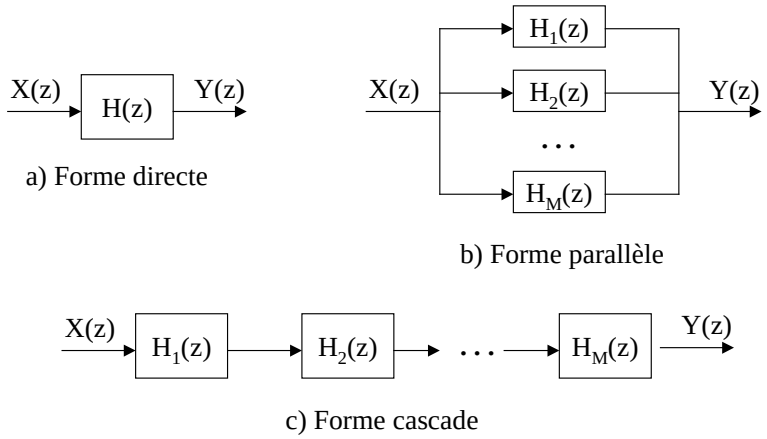


FIGURE 1.3 – Représentations sous forme de fonctions de transfert en z

1.3 Spécification d'un filtre numérique

Avant qu'un filtre numérique soit conçu et implanté, nous avons besoin de définir ses spécifications. Un filtre doit laisser passer certaines fréquences, alors qu'il doit en atténuer (voire éliminer) d'autres. Nous devons donc pouvoir représenter ces contraintes. Il y a quatre filtres de bases :

1. les *filtres passe-bas* laissent passer les fréquences inférieures à une fréquence de coupure f_c et bloquent celles qui lui sont supérieures (figure 1.4.a),
2. les *filtres passe-haut* bloquent les fréquences inférieures à une fréquence de coupure f_c et laissent passer celles qui lui sont supérieures (figure 1.4.b),
3. les *filtres passe-bande* laissent passer les fréquences autour d'une fréquence centrale f_0 (ou comprises entre f_1 et f_2) et bloquent les autres (figure 1.4.c),
4. les *filtres réjecteur-de-bande* bloquent les fréquences autour d'une fréquence centrale f_0 (ou comprises entre f_1 et f_2) et laissent passer les autres (figure 1.4.d).

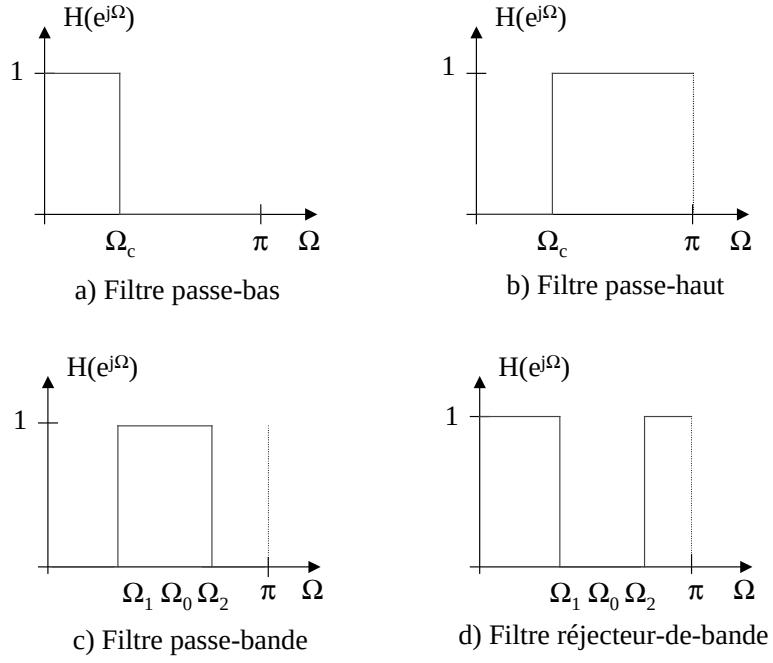


FIGURE 1.4 – Réponses fréquentielles idéales des 4 filtres de base

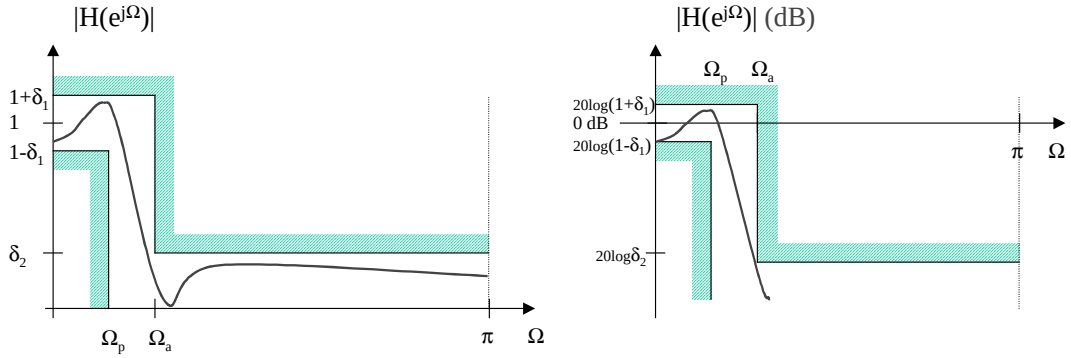
Les filtres représentés en figure 1.4 sont idéaux. Dans un cas réel il ne peut y avoir de discontinuités. Le passage entre zones passantes et zones atténuées se fait par des zones dites "de transition" dont la largeur va exprimer la *sélectivité* du filtre. Les bandes passantes et atténuées ne sont également pas idéales, elles contiennent des ondulations dont l'amplitude est exprimée par les paramètres d'*ondulation en bande passante* et d'*atténuation*.

Pour toutes ces raisons, la spécification d'un filtre est habituellement réalisée à partir d'un *gabarit* fréquentiel, défini entre 0 et π.

1.3.1 Spécifications des filtres passe-bas et passe-haut

Un filtre passe-bas possède trois zones : la bande passante ($0 \leq \Omega \leq \Omega_p$), la bande de transition ($\Omega_p \leq \Omega \leq \Omega_a$) et la bande atténuée ($\Omega_a \leq \Omega \leq \pi$). La figure 1.5.a montre une représentation graphique du gabarit fréquentiel linéaire d'un filtre passe-bas, tandis que la figure 1.5.b représente un gabarit fréquentiel en dB. Un filtre passe-haut verrait ses bandes atténuées et passantes inversées, on aurait dans ce cas $\Omega_p > \Omega_a$. δ_1 est l'ondulation en bande passante, δ_2 est l'atténuation.

La sélectivité du filtre est définie dans le tableau 1.1.



a) Gabarit fréquentiel linéaire b) Gabarit fréquentiel en dB

FIGURE 1.5 – Gabarit fréquentiel d'un filtre passe-bas

1.3.2 Spécifications des filtres passe-bande et réjecteur-de-bande

Un filtre passe-bande possède plusieurs zones : la bande passante ($\Omega_{p-} \leq \Omega \leq \Omega_{p+}$), deux bandes de transition et deux bandes atténuées ($0 \leq \Omega \leq \Omega_{a-}$ et $\Omega_{a+} \leq \Omega \leq \pi$). La figure 1.6 montre une représentation graphique du gabarit fréquentiel linéaire d'un filtre passe-bande. Un filtre réjecteur-de-bande verrait ses bandes atténuées et passantes inversées.

Le tableau 1.1 résume les paramètres des différents gabarits étudiés.

	Passe-bas	Passe-haut	Passe-bande	Réjecteur-de-bande
Sélectivité s	$\frac{\Omega_p}{\Omega_a}$	$\frac{\Omega_a}{\Omega_p}$	$\frac{\Omega_{p+}-\Omega_{p-}}{\Omega_{a+}-\Omega_{a-}}$	$\frac{\Omega_{a+}-\Omega_{a-}}{\Omega_{p+}-\Omega_{p-}}$
Ondulation	δ_1	δ_1	δ_1	δ_1
Atténuation	δ_2	δ_2	δ_2	δ_2
Fréquence centrale Ω_0	-	-	$\sqrt{\Omega_{p+}.\Omega_{p-}}$	$\sqrt{\Omega_{p+}.\Omega_{p-}}$
Largeur de bande B	-	-	$\frac{\Omega_{p+}-\Omega_{p-}}{\Omega_0}$	$\frac{\Omega_{p+}-\Omega_{p-}}{\Omega_0}$

TABLE 1.1 – Paramètres de spécification d'un filtre numérique

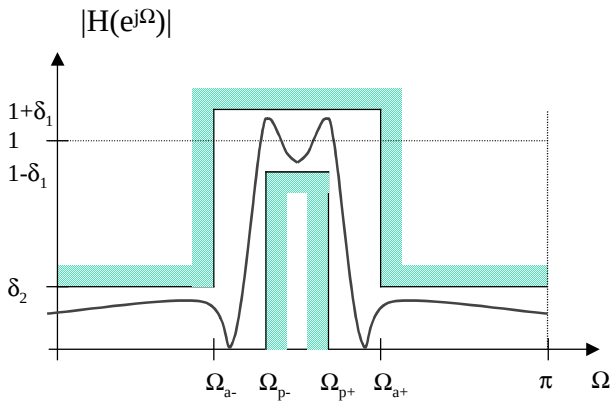


FIGURE 1.6 – Gabarit fréquentiel linéaire d'un filtre passe-bande

1.4 Classification des filtres numériques

Les filtres numériques peuvent être classés selon plusieurs critères :

1. la longueur de la réponse impulsionnelle implique deux types de filtres RII et RIF,
2. le type de représentation, ou de structure, implique deux types de filtres récursifs et non récursifs.

Nous verrons qu'à l'exception d'un cas particulier, les filtres récursifs et non récursifs sont respectivement équivalents aux filtres RII et RIF.

1.4.1 Filtres récursifs RII

Les filtres analogiques ont nécessairement une *réponse impulsionnelle infinie*. Les filtres numériques RII se comportent de manière similaire, mis à part les effets dus à la discrétisation. Cette catégorie de filtre est également caractérisée par une fonction de transfert en z contenant des pôles, et une équation aux différences récursives, c'est à dire lorsque la sortie $y(n)$ dépend à la fois des entrées et des sorties précédentes.
Les équations 1.8 et 1.9 montrent la fonction de transfert en z et l'équation aux différences correspondante de la forme générale d'un filtre RII. N est appelé ici l'ordre du filtre.

$$H(z) = \frac{N(z)}{D(z)} = \frac{\sum_{i=0}^{N} b_i.z^{-i}}{1 + \sum_{i=1}^{N} a_i.z^{-i}} \tag{1.8}$$

$$y(n) = \sum_{i=0}^{N} b_i.x(n-i) - \sum_{i=0}^{N} a_i.y(n-i) \tag{1.9}$$

A partir de l'équation 1.8, deux cas se présentent :

1. si $N(z)$ n'est pas divisible par $D(z)$, on a un nombre infini de termes dans la division

polynomiale de $N(z)$ par $D(z)$:

$$\begin{aligned} H(z) &= \sum_{n=0}^{N} c_n.z^{-n} \\ h(n) &= c_n \; pour \; n = 0 \ldots \infty \end{aligned}$$

$H(z)$ est un filtre RII,

2. si $N(z)$ est divisible par $D(z)$, on a un nombre fini de termes dans la division polynomiale de $N(z)$ par $D(z)$:

$$\begin{aligned} H(z) &= \sum_{n=0}^{N-1} c_n.z^{-n} \\ h(n) &= c_n \; pour \; n = 0 \ldots N-1 \end{aligned}$$

$H(z)$ est un filtre RIF.

Exemple 1.4.1 : Démontrer que le filtre moyenneur
$H(z) = \frac{1}{M}\frac{1-z^{-M}}{1-z^{-1}} = \frac{1}{M}\sum_{i=0}^{M-1} z^{-i}$
peut être exprimé de manière récursive : $y(n) = s(n-1) + \frac{1}{M}[e(n) - e(n-M)]$
ou de manière non récursive : $y(n) = \frac{1}{M}\sum_{i=0}^{M-1} e(n-i)$

Les principales caractéristiques des filtres RII sont :

1. une bande de transition qui peut être étroite ;
2. des méthodes de synthèse par transposition des méthodes pour les filtres analogiques (voir chapitre 2) ;
3. une instabilité potentielle due à des pôles situés en dehors du cercle unité (i.e. $|p_i| \geq 1$ quel que soit i) ;
4. une instabilité numérique (i.e. après quantification des coefficients et du signal) potentielle due au rebouclage.

1.4.2 Filtres non récursifs RIF

Les filtres RIF ne peuvent pas être dérivés des filtres analogiques. Il sont cependant très largement utilisés car ils possèdent des propriétés uniques (phase linéaire, stabilité, flexibilité). Les équations 1.10 et 1.11 montrent la fonction de transfert en z et l'équation aux différences correspondante de la forme générale d'un filtre RII. N est appelé ici la longueur de la réponse impulsionnelle du filtre.

$$H(z) = \sum_{i=0}^{N-1} b_i.z^{-i} \tag{1.10}$$

$$y(n) = \sum_{i=0}^{N-1} b_i.x(n-i) = \sum_{i=0}^{N-1} h(i).x(n-i) \tag{1.11}$$

On remarque en exploitant l'équation 1.11 que les coefficients b_i du filtre sont également les valeurs de la réponse impulsionnelle $h(n)$, qui se trouve donc être limitée dans le temps.

$$H(z) = \sum_{i=0}^{N-1} b_i.z^{-i} \Longleftrightarrow h(n) = \sum_{i=0}^{N-1} b_i.\delta(n-i) \tag{1.12}$$

$$h(n) = \begin{cases} b_n & pour\ 0 \leq n \leq N-1 \\ 0 & ailleurs \end{cases} \tag{1.13}$$

Les principales caractéristiques des filtres RIF sont :

1. une bande de transition qui sera toujours plus large qu'un filtre RII ayant le même nombre de coefficients ;
2. des méthodes de synthèse permettant de dériver n'importe quelle réponse fréquentielle (voir chapitre 3) ;
3. une stabilité inhérente ($\sum_{n=0}^{N-1} |h(n)| < \infty$) ;
4. une plus grande stabilité numérique que les RII ;
5. une phase qui peut être exactement linéaire, par conséquent un temps de propagation de groupe constant et une absence de distorsion harmonique dans le signal ;
6. une plus grande facilité d'implantation dans un système numérique de traitement.

1.5 Analyse fréquentielle des filtres numériques

L'analyse fréquentielle est la représentation de la fonction de transfert du filtre dans le domaine des fréquences, c'est à dire celui de $\mathcal{F}$ourier. La fonction de transfert en Ω est la transformée de $\mathcal{F}$ourier du signal $h(n)$.

$$H(e^{j\Omega}) = \sum_{n=0}^{\infty} h(n).e^{-jn\Omega} = H(z)/_{z=e^{j\Omega}} \tag{1.14}$$

Cette fonction correspond à un signal discret, elle est donc périodique de période 2π. C'est pour cette raison que l'on a l'habitude d'utiliser la notation $H(e^{j\Omega})$ plutôt que $H(\Omega)$. La figure 1.7 représente un exemple de filtre passe-bas. En traitement du signal, on étudie généralement le module et la phase (ou argument) de la fonction complexe $H(e^{j\Omega})$.

$$H(e^{j\Omega}) = H_r(e^{j\Omega}) + jH_i(e^{j\Omega}) = |H(e^{j\Omega})|e^{j\Phi(\Omega)} \tag{1.15}$$

1.6 Structures des filtres RII et RIF

1.6.1 Structure des filtres RIF

La structure d'un filtre est un graphe flots de données (GFD) dans lequel les nœuds sont des opérations (les additions sont habituellement représentées par des cercles contenant un + et les multiplications par des triangles associés aux coefficients multiplicande) et les arcs les dépendances, c'est à dire le flot des données issues du signal. Certains arcs sont valués

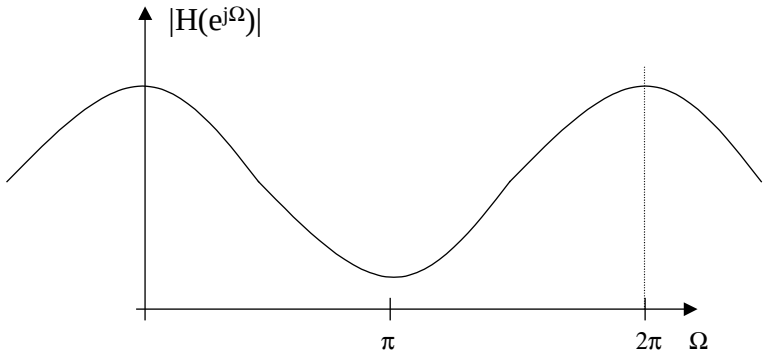


FIGURE 1.7 – Exemple de filtre passe-bas

d'un coefficient z^{-1} représentant un délai d'une période d'échantillonnage. Ce coefficient se représente également sous la forme d'un registre.

L'équation :

$$y(n) = \sum_{i=0}^{N} b_i.x(n-i) = b_0.x(n) + b_1.x(n-1) + \ldots + b_{N-1}.x(n-N+1) + b_N.x(n-N)$$

représente le comportement temporel d'un filtre RIF. On peut en déduire immédiatement la structure directe d'un filtre RIF qui est représentée à la figure 1.8.a. La structure transposée de la figure 1.8.b est obtenue après manipulation de cette équation.

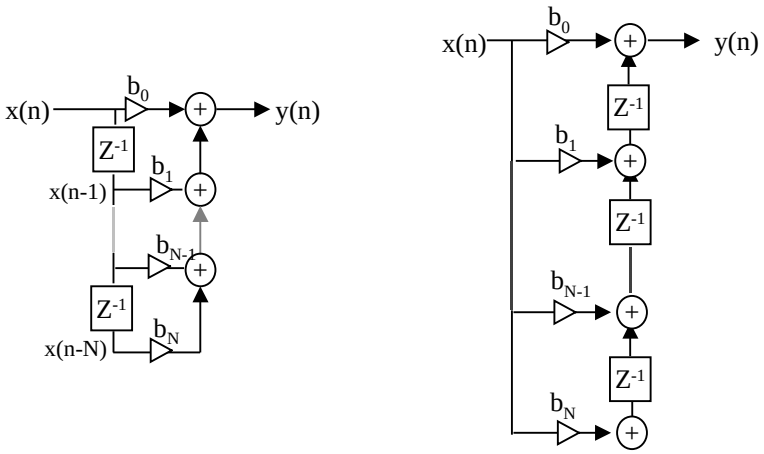


FIGURE 1.8 – Structures des filtres RIF

1.6.1.1 Complexité d'implantation d'un filtre RIF

Un filtre RIF nécessite $N + 1$ opérations de multiplication, N opérations d'addition pour chaque nouvel échantillon à filtrer. On peut également exprimer la complexité en nombre

de multiplication-accumulation (MAC), qui, dans le cas du filtre RIF, vaut $N+1$. Le coût mémoire d'un filtrage RIF est de $2(N+1)$ ($N+1$ coefficients b_i et $N+1$ points mémoire pour le vecteur des entrées $x(i)$).
Si la fréquence d'échantillonnage du signal vaut F_e, cela signifie que le calcul d'un filtre devra être réalisé en un temps T_{calcul} inférieur à $T_e = \frac{1}{F_e}$.

Sur un processeur de type *DSP* capable d'exécuter une multiplication-accumulation (MAC) à chaque cycle, de puissance de calcul P_{calcul} exprimée en *MIPS (Million d'Instruction Par Seconde)*, le temps de calcul sera : $T_{calcul} = (N+1).T_{cycle} = (N+1)/P_{calcul}$. Aussi, la puissance de calcul d'un DSP pour l'implantation d'un filtrage RIF vaut :

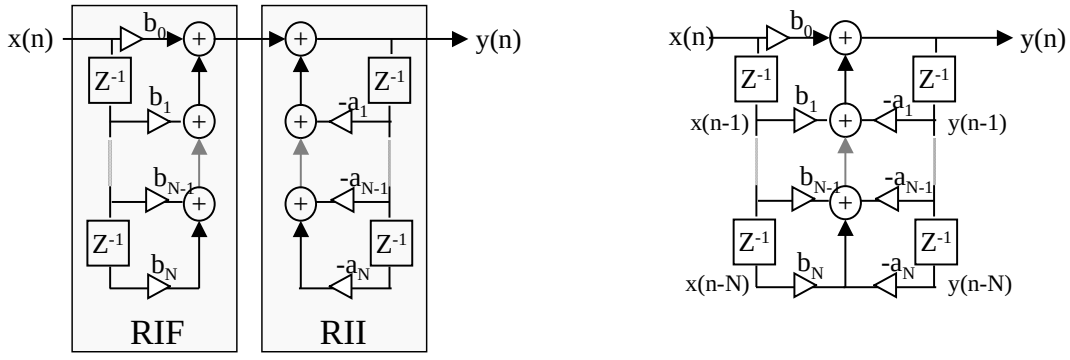
$$P_{calcul}\ (MIPS) > (N+1).F_e/10^6 \tag{1.16}$$

1.6.2 Structure des filtres RII

L'équation suivante (1.17) montre que l'on peut représenter un filtre RII $H(z)$ sous la forme du produit de 2 structures, dont une est une filtre RIF $N(z)$, et l'autre un filtre RII *tout-pôle* $1/D(z)$.

$$H(z) = \frac{N(z)}{D(z)} = [N(z)] \times \left[\frac{1}{D(z)}\right] = \left[\sum_{i=0}^{N} b_i.z^{-i}\right] \times \left[\frac{1}{1+\sum_{i=1}^{N} a_i.z^{-i}}\right] \tag{1.17}$$

La structure directe d'un filtre RII est donc obtenue en mettant en cascade un filtre RIF sous forme directe, et la représentation immédiate du filtre tout-pôle $1/D(z)$. Celle ci est donnée à la figure 1.9.a. En réunissant les additions du centre de la figure, on obtient la forme directe classique utilisée dans la littérature représentée à la figure 1.9.b.



a) Cascade **b) Structure directe**

FIGURE 1.9 – Structures directes des filtres RII

Il est bien entendu possible de représenter différemment un filtre RII en utilisant la propriété de commutativité de la multiplication. On a alors :

$$H(z) = \left[\frac{1}{D(z)}\right] \times [N(z)] = \left[\frac{1}{1+\sum_{i=1}^{N} a_i.z^{-i}}\right] \times \left[\sum_{i=0}^{N} b_i.z^{-i}\right] \tag{1.18}$$

On peut donc échanger sur la figure 1.9.a les 2 blocs RIF et RII. Les deux lignes à retard permettant de mémoriser les signaux x(n) se retrouvant communes, il est possible de les réunir en obtenant un vecteur unique de registres $w(n)$, comme représenté à la figure 1.10.a.

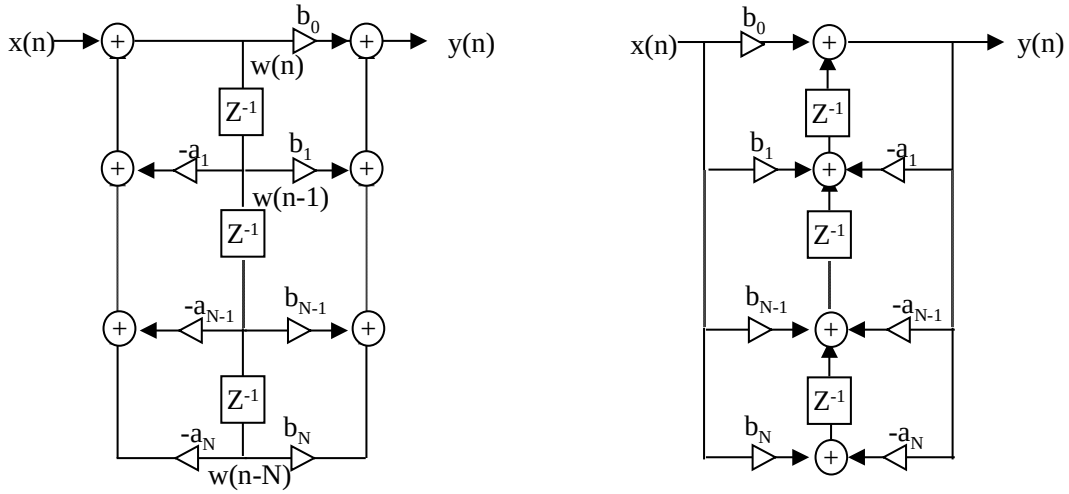


FIGURE 1.10 – Structures canoniques des filtres RII

On peut également représenter la structure canonique sous la forme d'un système d'équation (1.19) faisant apparaître le signal $w(n)$.

$$\begin{cases} W(z) = \frac{1}{D(z)}.X(z) \\ Y(z) = N(z).W(z) \end{cases} \quad \begin{cases} w(n) = x(n) - \sum_{i=1}^{N} a_i.w(n-i) \\ y(n) = \sum_{i=0}^{N} b_i.w(n-i) \end{cases} \tag{1.19}$$

1.6.2.1 Complexité d'implantation d'un filtre RII

Un filtre RII nécessite $2N+1$ opérations de multiplication, $2N$ opérations d'addition pour chaque nouvel échantillons à filtrer ou $2N+1$ *MAC*. Le coût mémoire d'un filtrage RII en structure directe est de $4N+3$ ($2N+1$ coefficients b_i et $2(N+1)$ points mémoire pour les vecteurs des entrées $x(n)$ et des sorties $y(n)$). La structure canonique permet de diminuer le coût mémoire qui ne necessite plus que $N+1$ points mémoire pour le vecteur $w(n)$.
Si la fréquence d'échantillonnage du signal vaut F_e, cela signifie que le calcul du filtre devra être réalisé en un temps T_{calcul} inférieur à $T_e = \frac{1}{F_e}$.

Sur un processeur de type *DSP* capable d'exécuter une multiplication-accumulation (MAC) à chaque cycle, de puissance de calcul P_{calcul} exprimée en *MIPS (Million d'Instruction Par Seconde)*, Le temps de calcul sera : $T_{calcul} = (2N+1).T_{cycle} = (2N+1)/P_{calcul}$. Aussi, la puissance de calcul d'un DSP pour l'implantation d'un filtrage RIF vaut :

$$P_{calcul}\ (MIPS) > (2N+1).F_e/10^6 \tag{1.20}$$

Exemple 1.6.1 : Cellule élémentaire du premier ordre

Soit le système qui, à la suite de données $x(n)$, fait correspondre la suite $y(n)$ telle que :

$$y(n) = x(n) + b.y(n-1)$$

où b est une constante.

1. Donner les réponses impulsionnelles et indicielles de ce système. par deux méthodes (suite numérique, transformée en Z). Que peut on dire de la stabilité du filtre.
2. Étudier l'analogie avec le système continu de constante de temps t, échantillonné avec la période T.
3. Étudier la réponse fréquentielle du filtre.
4. Donner la structure de réalisation du filtre.

Exemple 1.6.2 : Cellule du second ordre purement récursive

Soit le système qui, à la suite de données $x(n)$, fait correspondre la suite $y(n)$ telle que :

$$y(n) = x(n) - b_1.y(n-1) - b_2.y(n-2)$$

1. Donner la fonction de transfert en Z du système.
2. En déduire la réponse impulsionnelle du filtre numérique.
3. Étudier la réponse fréquentielle du filtre. On regardera plus particulièrement l'influence des coefficients b_1 et b_2 sur les pôles de la fonction de transfert $H(z)$.
4. Tracer le diagramme des pôles et zéros.
5. Donner les structures de réalisation.

Chapitre 2

Synthèse des filtres RII

2.1 Introduction et rappels en filtrage analogique

2.1.1 Introduction

La synthèse d'un filtre numérique est la recherche d'une fonction $H(z)$ (ou $h(n)$) correspondant à la spécification sous forme de gabarit. La recherche de cette fonction peut être réalisée selon diverses méthodes.

- La méthode la plus courante est l'utilisation des méthodes de synthèse des filtres analogiques aboutissant à une fonction $H(p)$ correspondant aux spécifications. Une fonction permettant le passage du plan p dans le plan z (i.e. $p = f(z)$) est ensuite utilisée pour obtenir $H(z)$.
  Cette fonction doit maintenir la stabilité du filtre analogique et maintenir, au mieux, les caractéristiques de la réponse fréquentielle $H(e^{j\Omega})$ du filtre numérique. Nous étudions dans la suite du chapitre trois types de transformation f.
- Une synthèse directe en z est également possible selon les mêmes principes que la synthèse analogique, mais elle ne sera pas présentée ici vu le nombre important d'outils développés dans le contexte analogique.
- Enfin, des méthodes d'optimisation issues de l'analyse numérique peuvent être utilisées afin de rechercher une fonction $H(z)$ s'approchant le plus possible d'une fonction prototype. La minimisation d'un critère d'erreur entre la courbe idéale et la courbe réelle est alors appliquée.

La figure 2.1 résume la méthodologie de synthèse de filtres analogiques et numériques RII. Les premières phases de synthèse analogique (normalisation, approximation et dénormalisation) sont explicités dans la section suivante, tandis que les différentes transformations $p \rightarrow z$ sont présentées aux sections 2.2 à 2.4.

2.1.2 Rappels en filtrage analogique

Les filtres analogiques sont spécifiés de manière équivalente aux filtres numériques (voir section 1.3). Les fréquences sont exprimées en fonction de f (en Hz) ou de ω, la pulsation (en rad/s). La fonction de transfert $H(p)$ est exprimée en $p = j\omega$. La synthèse de cette fonction $H(p)$ est composée de trois étapes principales :

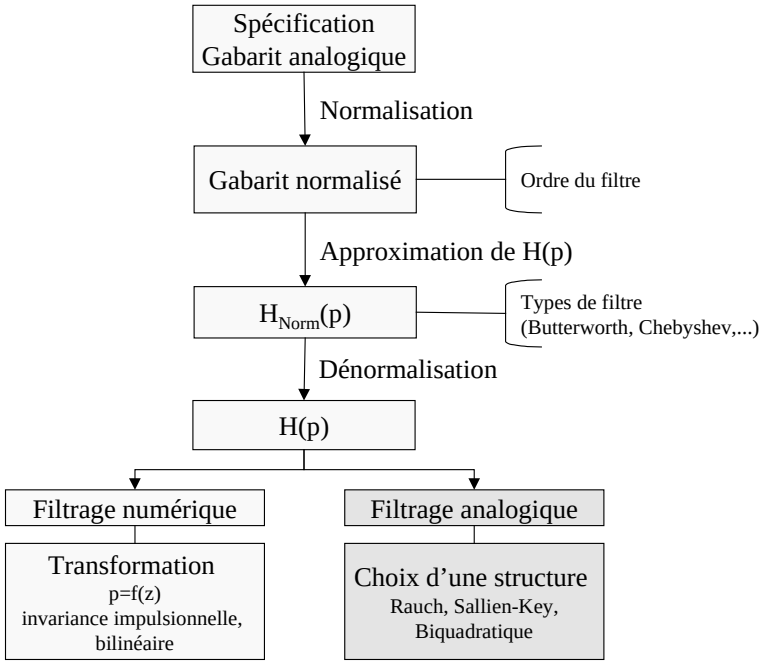


FIGURE 2.1 – Méthodologie de synthèse de filtres analogique et numérique RII

1. normalisation du gabarit,
2. approximation de la fonction de transfert normalisée $H_{Norm}(p)$,
3. dénormalisation de $H_{Norm}(p)$ aboutissant à la fonction de transfert $H(p)$.

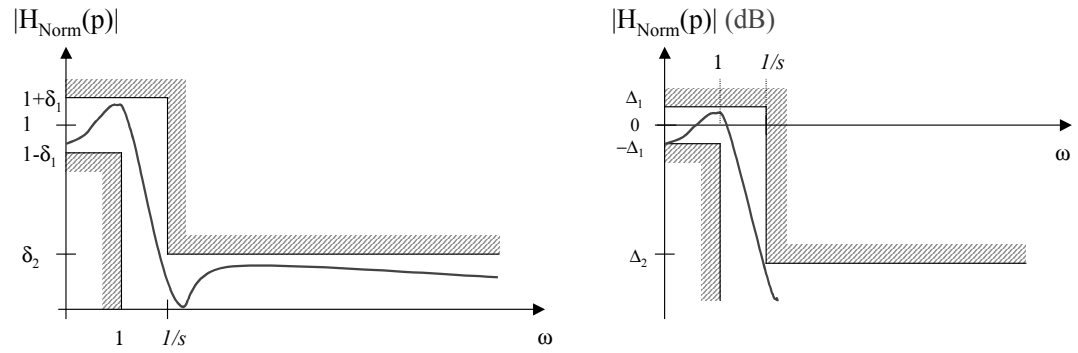
2.1.2.1 Normalisation

Cette première phase permet d'aboutir au gabarit passe-bas prototype (ou gabarit normalisé) à partir de n'importe quel des 4 types principaux de filtres (passe-bas, passe-haut, passe-bande, réjecteur-de-bande). Celui-ci (voir figure 2.2) est un gabarit passe-bas possédant une pulsation de coupure normalisée à 1 et une pulsation de bande atténuée valant $\frac{1}{s}$, s étant la sélectivité définie au paragraphe 1.3 (voir tableau 1.1). Les valeurs d'ondulation et d'atténuation sont inchangées.

2.1.2.2 Approximation de la fonction de transfert

La phase suivante consiste en la recherche d'une fonction $H_{Norm}(p)$ entrant dans le gabarit prototype passe-bas défini avant. Il existe plusieurs fonctions d'approximation, nous exposons ci-après les 4 fonctions les plus utilisées.
Pour la détermination de l'ordre du filtre et des coefficients de la fonction de transfert $H(p)$, on pourra soit utiliser les formules et tableaux données ci-après, soit utiliser les abaques fournies en annexe B.

1. **Filtres de Butterworth**
   Un filtre passe-bas de Butterworth d'ordre N et de pulsation de coupure ω_c est défini



a) Gabarit prototype linéaire b) Gabarit prototype en dB

FIGURE 2.2 – Gabarit prototype passe-bas

par le carré de sa réponse fréquentielle :

$$\left|H^B(\omega)\right|^2 = \frac{1}{1+\left(\frac{\omega}{\omega_c}\right)^{2N}} \tag{2.1}$$

Les principales propriétés des filtres de Butterworth sont :
— l'amplitude est une fonction monolithique décroissante sur ω,
— la réponse fréquentielle est plate dans les bandes passante et atténuée,
— le gain maximum est pour $\omega = 0$ et vaut 1,
— $|H^B(\omega_c)| = \sqrt{0.5}$, ω_c est donc la pulsation de coupure à -3dB,
— l'atténuation asymptotique dans les hautes fréquences vaut $20.N$ dB/décade.
L'ordre du filtre est déterminé par la relation :

$$N \geq \frac{log\left(\frac{1}{\delta_2\sqrt{\delta 1}}\right)}{log\left(\frac{1}{s}\right)} \tag{2.2}$$

N	a_1	a_2	a_3	a_4	a_5
2	1.4142				
3	2	2			
4	2.6131	3.4142	2.6131		
5	3.2361	5.2361	5.2361	3.2361	
6	3.8637	7.4641	9.1416	7.4641	3.8637

TABLE 2.1 – Coefficients du polynôme du dénominateur d'un filtre passe-bas normalisé de Butterworth de degrés 2 à 6

2. **Filtres de Chebyshev**
Un filtre passe-bas de Chebyshev de type I d'ordre N et de pulsation de coupure ω_c est défini par le carré de sa réponse fréquentielle :

$$\left|H^C(\omega)\right|^2 = \frac{1}{1+\epsilon^2 T_N^2\left(\frac{\omega}{\omega_c}\right)}, \tag{2.3}$$

où $T_N(x)$ est un polynôme de Chebyshev de degré N, ϵ un paramètre permettant de régler l'amplitude de l'ondulation en bande passante ou en bande atténuée.

Les principales propriétés des filtres de Chebyshev sont :

— pour un filtre de type I, l'amplitude dans la bande passante possède une ondulation respectant :

$$0 \leq \omega \leq \omega_c \Longrightarrow \frac{1}{1+\epsilon^2} \leq \left|H^C(\omega)\right|^2 \leq 1$$

— pour un filtre de type II, ces ondulations sont situées dans la bande atténuée ($\omega \geq \omega_c$),

$$|H^C(0)| = \begin{cases} 1/(1+\epsilon^2), & N \textit{ impair} \\ 1, & N \textit{ pair} \end{cases}$$

— l'atténuation asymptotique dans les hautes fréquences vaut $20.N$ dB/décade, mais l'ordre du filtre sera plus faible qu'un Butterworth à spécifications équivalentes.

3. **Filtres elliptiques**
Les filtres elliptiques possèdent des ondulations à la fois en bande passante et en bande atténuée. Ils sont optimaux en terme de sélectivité, c'est à dire qu'ils donnent l'ordre minimal à spécification donnée. Un filtre passe-bas elliptique d'ordre N et de pulsation de coupure ω_c est défini par le carré de sa réponse fréquentielle :

$$\left|H^E(\omega)\right|^2 = \frac{1}{1+\epsilon^2 R_N^2\left(\frac{\omega}{\omega_c}\right)}, \tag{2.4}$$

où $R_N(x)$ est une fonction rationnelle de Chebyshev de degré N, ϵ un paramètre permettant de régler l'amplitude de l'ondulation en bande passante ou en bande atténuée. L'ondulation en bande passante, équivalente à un chebyshev-I, est comprise entre 1 et $1/(1+\epsilon^2)$.

4. **Filtres de Bessel**
Les filtres de Bessel sont intéressants pour leur phase proche de la linéarité. En contrepartie, ils sont ceux ayant la sélectivité la plus faible.

2.1.2.3 Dénormalisation

Cette dernière étape permet de passer de la fonction de transfert normalisée $H_{Norm}(p_N)$ à la fonction de transfert $H(p)$. Nous appelons ici p_N la variable de la fonction normalisée. La dénormalisation consiste donc en l'application d'une fonction f permettant de transformer cette variable p_N en la variable $p = j\omega$: $p_N = f(p)$. La fonction f dépend du type de filtre spécifié. Le tableau 2.2 résume les fonctions de dénormalisation pour les 4 types de filtres.

passe-bas	passe-haut	passe-bande	réjecteur-de-bande
$p_N = \frac{p}{\omega_c}$	$p_N = \frac{\omega_c}{p}$	$p_N = \frac{1}{B}\left(\frac{p}{\omega_0} + \frac{\omega_0}{p}\right)$	$p_N = \left[\frac{1}{B}\left(\frac{p}{\omega_0} + \frac{\omega_0}{p}\right)\right]^{-1}$

TABLE 2.2 – Fonctions de dénormalisation

Exemple 2.1.1 : Synthèse d'un filtre passe-haut

Soit le filtre défini par le gabarit suivant :

f_p	f_a	δ_1	δ_2
3 kHz	1 kHz	-3 dB	-20 dB

1. Dessiner le gabarit du filtre. En déduire le gabarit du filtre prototype.
2. Pour les 4 types de filtre précédents, déterminer l'ordre du filtre et la fonction de transfert normalisée $H_{Norm}(p)$.
3. En déduire les fonctions de transfert $H(p)$ des 4 types de filtre.

2.2 Méthode de l'invariance impulsionnelle

La méthode de l'invariance impulsionnelle consiste à effectuer la synthèse d'un filtre numérique dont la réponse impulsionnelle $h(nT)$ est l'échantillonnage de la réponse impulsionnelle $h_a(t)$ du filtre analogique équivalent (figure 2.3). On a alors :

$$h(nT) \triangleq h_a(t)|_{t=nT} \tag{2.5}$$

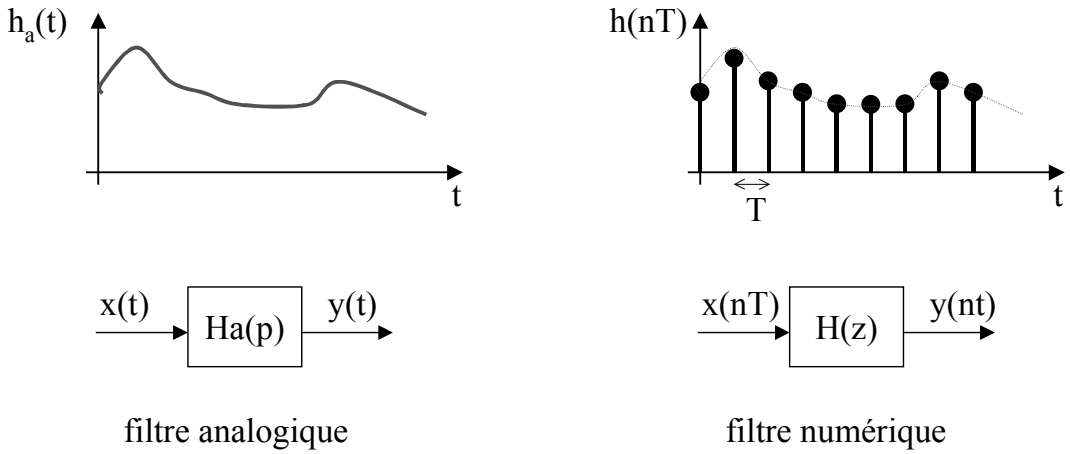


FIGURE 2.3 – Conservation de la réponse impulsionnelle entre les filtres analogique et numérique

Dans le cas où la fonction de transfert du filtre analogique $H_a(p)$ possède N pôles simples p_i, on peut écrire :

$$\begin{aligned} H_a(p) &= \frac{N(p)}{D(p)} = \sum_{i=1}^{N} \frac{k_i}{p+p_i} \\ h_a(t) &= \sum_{i=1}^{N} k_i e^{-p_i t} \end{aligned}$$

Si on échantillonne la réponse impulsionnelle du filtre analogique par $h(nT) = h_a(t)|_{t=nT}$, on obtient la fonction de transfert du filtre numérique $H(z)$:

$$\begin{aligned} h(nT) &= \sum_{i=1}^{N} k_i e^{-p_i nT} \\ H(z) &= \sum_{i=1}^{N} k_i \frac{z}{z - e^{-p_i T}} = \sum_{i=1}^{N} k_i \frac{1}{1 - e^{-p_i T} z^{-1}} \end{aligned}$$

Si on étend cette formulation de $H(z)$ à tout type de fonction $H_a(p)$, on peut, en utilisant la formule des résidus, obtenir une relation directe entre $H_a(p) \longrightarrow H(z)$:

$$H(z) = \sum_{p_i \ de \ H_a(p)} Res\left(\frac{H_a(p)}{1 - e^{pT} z^{-1}}, p_i\right) \tag{2.6}$$

Cette méthode de synthèse est simple. Elle conserve la **réponse temporelle** du filtre analogique équivalent et également la stabilité du filtre. Par contre, la réponse fréquentielle du filtre n'est pas conservée. En effet, cette transformation suit logiquement le théorème d'échantillonnage et le phénomène de recouvrement de spectre. Son utilisation doit respecter les contraintes dues à l'échantillonnage, en particulier une bande passante inférieure à la moitié de la fréquence d'échantillonnage. La réponse fréquentielle est donnée par la formulation :

$$H(e^{j\omega T}) = H(z)|_{z=e^{j\omega T}} = \frac{1}{T} \sum_{k} H_a(j\omega + j\frac{2\pi k}{T}) \tag{2.7}$$

Par conséquent, la méthode de l'invariance impulsionnelle n'est applicable qu'aux filtres à bande limitée satisfaisant l'équation :

$$|H_a(j\omega)| \simeq 0 \ pour \ |\omega| > \omega_B \ avec \ \omega_B < \pi f_e \tag{2.8}$$

La figure 2.4 montre le résultat de la réponse fréquentielle d'un filtre numérique dans le cas où le filtre analogique équivalent n'est pas à bande limitée. Le phénomène de recouvrement de spectre est ici important. On remarque également que la réponse fréquentielle de $H(z)$ possède un facteur multiplicatif $1/T$ (équation 2.7). Ce facteur, dont la valeur est très élevée (e.g. 10000 pour une fréquence d'échantillonnage de 10kHz), doit être atténué si on veut un filtre réalisable. En pratique, on normalisera $H(z)$ par $(\times T)$ ou par $(/H(z = e^{j0})$. Dans ce dernier cas, on obtiendra une valeur en $\omega = 0$ équivalente au filtre analogique.

2.3 Transformation d'Euler

L'effet de recouvrement de la méthode de l'invariance impulsionnelle est due à l'utilisation d'une fonction non algébrique et non bijective entre le plan p et le plan z. Nous allons donc chercher à utiliser une transformation algébrique entre ces deux plans.
La transformation d'Euler consiste à utiliser l'approximation simple d'une dérivée :

$$\frac{de(t)}{dt}|_{t=nT} = \lim_{\Delta t \to 0} \frac{\Delta e(nT)}{\Delta t} \simeq \frac{e[nT] - e[(n-1)T]}{T} \tag{2.9}$$

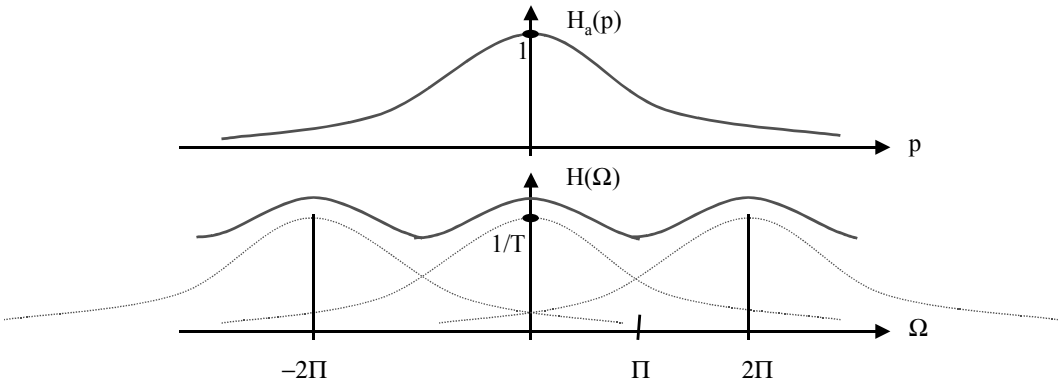


FIGURE 2.4 – Réponses fréquentielles des filtres analogique et numérique

Si on considère un système linéaire discret $H(z)$ réalisant cette approximation, il aura la forme :

$$s(nT) = \frac{e[nT] - e[(n-1)T]}{T} \tag{2.10}$$

Ce qui donne dans le plan z :

$$S(z) = \frac{1 - z^{-1}}{T} E(z) \tag{2.11}$$

$$H(z) = \frac{1 - z^{-1}}{T} \tag{2.12}$$

Dans le plan p, la dérivation est obtenue par la fonction de transfert $H_a(p) = p$. Par conséquent, la transformation d'Euler réalisant l'approximation d'une dérivée est définie par :

$$H(z) \triangleq H(p)\big|_{p=\frac{1-z^{-1}}{T}} \tag{2.13}$$

La transformation inverse donnée par :

$$z = \frac{1}{1 - pT} \tag{2.14}$$

permet de chercher une relation entre les fréquences analogique (ω_a) et numérique (ω). On obtient :

$$z = \frac{1}{1 - j\omega_a T} = e^{j\omega T} \tag{2.15}$$

$$z = \frac{1}{2}\left[1 + \frac{1 + j\omega_a T}{1 - j\omega_a T}\right] = \frac{1}{2}\left[1 + e^{j2\arctan(\omega_a T)}\right] \tag{2.16}$$

La variable z parcourt donc un cercle de centre $z = 1/2$ et de rayon $1/2$. Le cercle unité du plan z est donc transformé, impliquant une modification importante de la réponse fréquentielle du filtre donnée par l'équation 2.16. Cependant, il n'y a pas de phénomène de recouvrement de spectre. La stabilité est conservée.

La section suivante montre une transformation utilisant le même principe, mais dont les performances en terme de réponse fréquentielle sont meilleures.

2.4 Transformation bilinéaire

La transformation bilinéaire est issue d'un système linéaire discret $H(z)$ réalisant l'approximation d'une intégrale par la méthode des rectangles (voir figure 2.5). On peut l'exprimer comme un filtre récursif donné par :

$$s(nT) = s((n-1)T) + T\frac{e(nT) + e((n-1)T)}{2} \tag{2.17}$$

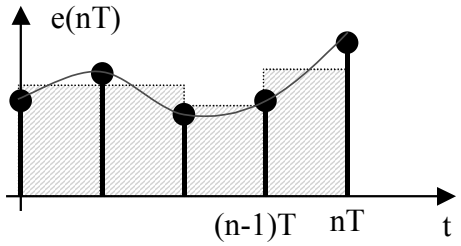


FIGURE 2.5 – Approximation d'une intégrale par la méthode des rectangles

Ce qui donne dans le plan z :

$$S(z) = z^{-1}S(z) + T\frac{E(z) + z^{-1}E(z)}{2} \tag{2.18}$$

$$H(z) = \frac{S(z)}{E(z)} = \frac{T}{2}\frac{1+z^{-1}}{1-z^{-1}} \tag{2.19}$$

Dans le plan p, l'intégration est obtenue par la fonction de transfert $H_a(p) = 1/p$. Par conséquent, la transformation bilinéaire réalisant l'approximation d'une intégrale est définie par :

$$H(z) \triangleq H(p)\big|_{p=\frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}} \tag{2.20}$$

La transformation inverse donnée par :

$$z = \frac{2/T + p}{2/T - p} = \frac{2/T + j\omega_a}{2/T - j\omega_a} \tag{2.21}$$

permet de chercher une relation entre les fréquences analogique (ω_a) et numérique (ω). On obtient :

$$z = \frac{2 + j\omega_a T}{2 - j\omega_a T} = e^{j\omega T} \tag{2.22}$$

$$|z| = 1 \tag{2.23}$$

$$\arg(z) = \arctan(\frac{\omega_a T}{2}) + \arctan(\frac{\omega_a T}{2}) = 2\arctan(\frac{\omega_a T}{2}) \tag{2.24}$$

$$z = e^{j\omega T} \Rightarrow \arg(z) = \omega T \tag{2.25}$$

Les équations 2.24 et 2.25 impliquent la relation entre les pulsations analogique et numérique :

$$\frac{\omega_a T}{2} = \tan\left(\frac{\omega T}{2}\right) \tag{2.26}$$

Cette relation est très importante puisqu'elle permet de connaître de manière analytique la transformation entre les réponses fréquentielles des filtres analogique et numérique. Cette relation s'appelle distorsion en fréquence ou *frequency warping*.

La figure 2.6 montre que l'ensemble de l'axe imaginaire du plan p est transformé vers le cercle unité du plan z (cf. équation 2.23 et 2.24) de manière bijective. De plus, le domaine de stabilité (demi-plan gauche du plan p) est transformé vers le disque unité (figure 2.6). Cette transformation évite donc le phénomène de recouvrement de spectre et conserve la stabilité. Cependant, une compression non linéaire de l'axe des fréquences est réalisée comme le montrent l'équation 2.26 et la figure 2.7.

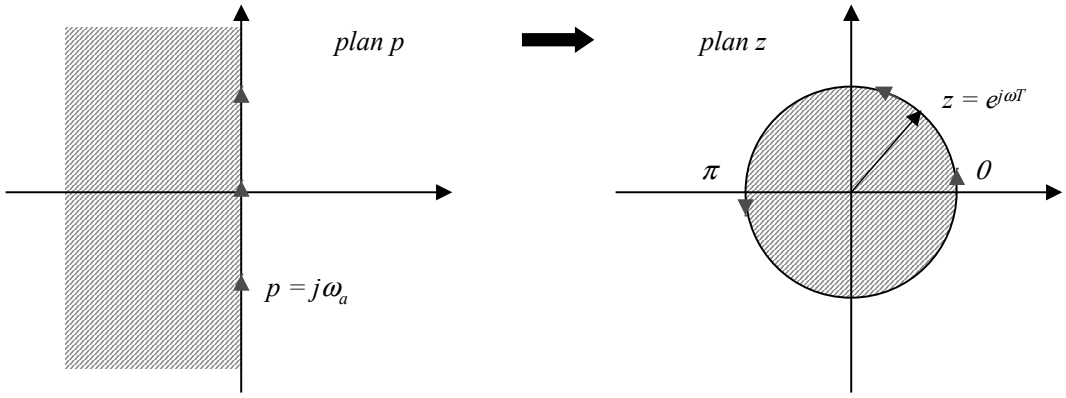


FIGURE 2.6 – Transformation du plan p vers le plan z

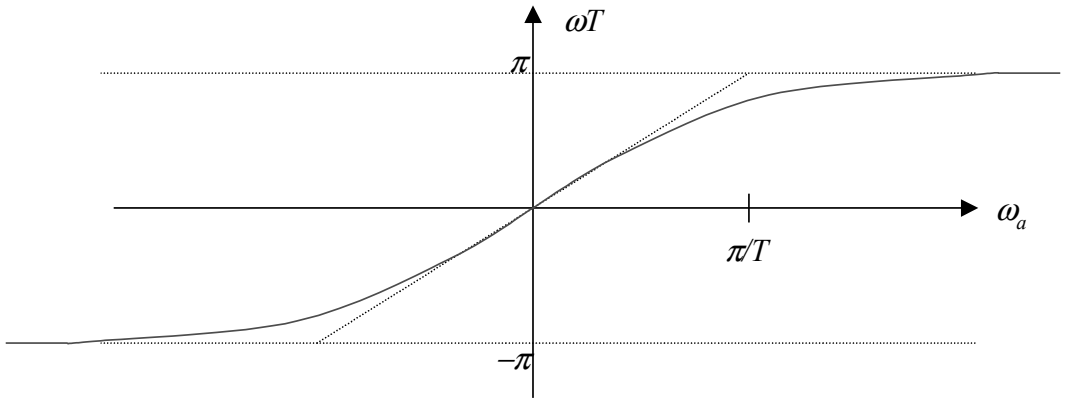


FIGURE 2.7 – Transformation non linéaire entre les fréquences analogique et numérique : distorsion en fréquence

En conséquence, la synthèse de filtre numérique utilisant la transformation bilinéaire est utilisable quand cette compression peut être tolérée ou compensée. La figure 2.8 illustre ce phénomène. Elle montre comment une réponse fréquentielle à temps continu et son gabarit est transformée en une réponse fréquentielle à temps discret. Si les fréquences critiques (i.e. le gabarit) sont modifiées en utilisant une **pré-distorsion** (équation 2.26), alors le filtre

numérique respectera les spécifications initiales.

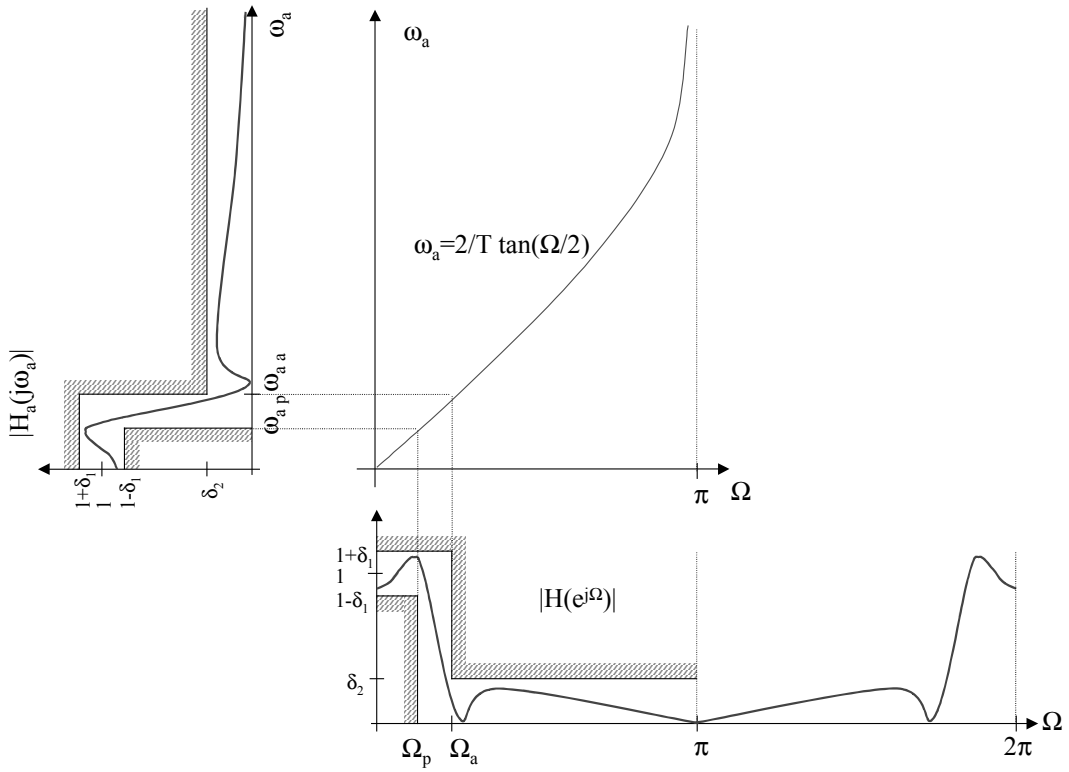


FIGURE 2.8 – Transformation fréquentielle du filtre analogique $H_a(\omega_a)$ vers le filtre numérique $H(\Omega)$

La méthodologie de synthèse par la transformation bilinéaire doit donc suivre l'algorithme suivant :

1. à partir des spécifications fréquentielles, effectuer une pré-distorsion des fréquences critiques du gabarit :
$$\omega_{a\ p,a} = \frac{2}{T} \tan\left(\frac{\Omega_{p,a}}{2}\right), \tag{2.27}$$
2. effectuer la synthèse du filtre analogique $H_a(p)$ (voir section 2.1),
3. utiliser la transformation bilinéaire $H(z) \triangleq H(p)\big|_{p=\frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}}$ pour obtenir le filtre numérique $H(z)$.

La pré-distorsion assure que le filtre numérique ainsi synthétisé entre dans le gabarit initial.

Exemple 2.4.1 : Synthèse d'un filtre IIR du premier ordre

On s'intéresse ici à la synthèse d'un filtre passe-bas RC du premier ordre par les méthodes de l'invariance impulsionnelle et bilinéaire. Soit le filtre :

$$H_a(p) = \frac{1}{1 + \frac{p}{\omega_c}},$$

avec $f_c = 1kHz$ et $f_e = 10kHz$.

1. Donner la réponse fréquentielle du filtre analogique $H_a(\omega_a)$. Donner les valeurs de son module en $f = 0$, 1kHz, 5kHz et 10kHz.
2. Exprimez les filtres numériques obtenus par invariance impulsionnelle ($H_i(z)$) et transformation bilinéaire ($H_b(z)$). Pour $H_b(z)$, il faudra s'assurer de la même fréquence de coupure ω_c que le filtre analogique. Pour $H_i(z)$, il faudra s'assurer de la même valeur en $\Omega = 0$ que le filtre analogique.
3. Donner de manière analytique les réponses fréquentielles des deux filtres numériques. Donner les valeurs de leurs modules en $f = 0$, 1kHz, 5kHz et 10kHz.
4. Tracer ces deux réponses à l'aide de `Matlab` ou `Scilab`.

Chapitre 3

Synthèse des filtres RIF

3.1 Introduction

Un filtre RIF est défini par sa fonction de transfert en z :

$$H(z) = \sum_{n=0}^{N-1} h(n).z^{-n} \tag{3.1}$$

où la réponse impulsionnelle $h(n)$ est de longueur N.

A la différence des filtres RII, les filtres RIF ne sont réalisables que dans le domaine discret. Par conséquent, leurs méthodes de synthèse ne sont pas dérivées des filtres analogiques. On distingue trois principales méthodes, dont deux seront détaillées dans les sections suivantes.

1. **La méthode du fenêtrage (*windowing*)** consiste à appliquer une fenêtre de taille N au filtre idéal équivalent (voir section 3.3).
2. **La méthode de l'échantillonnage fréquentiel** utilise la Transformée de Fourier Discrète inverse depuis une fonction discrète représentative du filtre et définie en fréquence (voir section 3.4).
3. Les méthodes d'optimisation se concentrent sur la minimisation d'un critère d'erreur entre la courbe réelle et le filtre idéal. La plus utilisée est la méthode de Parks and McClellan [PM73], qui reformule le problème de synthèse de filtre sous la forme d'une approximation polynômiale.

3.2 Filtres à phase linéaire

Les filtres RIF ont la particularité de pouvoir présenter une phase linéaire. Puisque pour les systèmes causaux une phase nulle dans la bande passante n'est pas réalisable, une phase exactement linéaire évitera la modification de la forme du signal, c'est à dire le phénomène de distorsion qui peut être très gênant, en particulier dans les systèmes audio. En effet, une phase linéaire implique un temps de propagation de groupe constant, et, par conséquent, l'effet de la phase sur le signal sera un simple décalage temporel. On posera pour ce type de filtre :

$$H(e^{j\Omega}) = A(e^{j\Omega})e^{-j\alpha\Omega+j\beta}, \tag{3.2}$$

où α et β sont des constantes et $A(e^{j\Omega})$ une fonction réelle, éventuellement bipolaire, de Ω que l'on nommera plus tard pseudo-module [1] (ou *amplitude*). Dans ce cas, le temps de propagation de groupe $\tau g(\Omega) = -\frac{d\varphi(\Omega)}{d\Omega}$ sera constant et aura pour valeur α. $\varphi(\Omega)$ est ici la phase linéaire du filtre valant $\varphi(\Omega) = -\alpha\Omega + \beta$. On distinguera par la suite quatre types de filtres selon les valeurs de α et β et de la parité de N.

Le filtre $H(z)$ et sa réponse fréquentielle $H(e^{j\Omega})$ sont donnés par :

$$H(z) = \sum_{n=0}^{N-1} h(n).z^{-n} \tag{3.3}$$

$$H(e^{j\Omega}) = H(z)|_{z=e^{j\Omega}} = \sum_{n=0}^{N-1} h(n).e^{-jn\Omega} \tag{3.4}$$

$$H(e^{j\Omega}) = \Re[H(e^{j\Omega})] + j\Im[H(e^{j\Omega})] \tag{3.5}$$

$$H(e^{j\Omega}) = |H(e^{j\Omega})|e^{j\arg(\Omega)} = A(e^{j\Omega})e^{j\varphi(\Omega)} \tag{3.6}$$

•1er **cas :** $\varphi(\Omega) = -\alpha\Omega$**,** $-\pi \leq \Omega \leq \pi$ Dans ce cas :

$$H(e^{j\Omega}) = A(e^{j\Omega})e^{-j\alpha\Omega} = A(e^{j\Omega})[\cos\alpha\Omega - j\sin\alpha\Omega] \tag{3.7}$$

$$H(e^{j\Omega}) = \sum_{n=0}^{N-1} h(n).e^{-jn\Omega} = \sum_{n=0}^{N-1} h(n)[\cos n\Omega - j\sin n\Omega] \tag{3.8}$$

En identifiant les parties réelles et imaginaires des deux équations précédentes, il est possible d'obtenir le système d'équation suivant :

$$\begin{cases} A(e^{j\Omega})\cos\alpha\Omega = \sum_{n=0}^{N-1} h(n)\cos n\Omega \\ A(e^{j\Omega})\sin\alpha\Omega = \sum_{n=0}^{N-1} h(n)\sin n\Omega \end{cases} \tag{3.9}$$

$$\Leftrightarrow \sum_{n=0}^{N-1} h(n)\cos n\Omega \sin\alpha\Omega - \sum_{n=0}^{N-1} h(n)\sin n\Omega\cos\alpha\Omega = 0 \tag{3.10}$$

$$\Leftrightarrow \sum_{n=0}^{N-1} h(n)\sin[(\alpha - n)\Omega] = 0 \tag{3.11}$$

L'équation 3.11 résume donc la condition sur la réponse impulsionnelle pour que le filtre soit à phase linéaire.

Si $\alpha \neq 0$, on montre que l'équation 3.11 est équivalente à :

$$\begin{cases} \alpha = \frac{N-1}{2} \\ h(n) = h(N-1-n) \ pour \ 0 \leq n \leq \alpha \end{cases} \tag{3.12}$$

La réponse impulsionnelle est donc **symétrique**. α est l'axe de symétrie de $h(n)$. α peut être entier ou non selon si N est impair ou pair.

1. Le pseudo-module est souvent utilisé en remplacement du module car il évite les discontinuités dans le module dues à la valeur absolue et dans la phase dues à la détermination principale $[-\pi, \pi]$ de l'argument. On peut le voir comme une représentation du module intégrant ses variations de signe

•2nd **cas :** $\varphi(\Omega) = \beta - \alpha\Omega$**,** $-\pi \leq \Omega \leq \pi$ Dans ce cas on obtient selon la même démonstration que pour le premier cas la relation suivante :

$$\sum_{n=0}^{N-1} h(n) \sin[(\alpha - n)\Omega + \beta] = 0 \tag{3.13}$$

On montre ensuite que l'équation 3.13 est équivalente à :

$$\begin{cases} \beta = \pm\pi/2 \\ \alpha = \frac{N-1}{2} \\ h(n) = -h(N-1-n) \text{ pour } 0 \leq n \leq \alpha \end{cases} \tag{3.14}$$

La réponse impulsionnelle est donc **antisymétrique** par rapport à α.

On déduit de ces deux cas que selon la parité de N et le type de symétrie de $h(n)$, quatre types distincts de filtres sont à étudier.

3.2.1 Filtre RIF à phase linéaire de type I

Un filtre RIF à phase linéaire de type I est défini par une réponse impulsionnelle symétrique :

$$h(n) = h(N-1-n), \qquad 0 \leq n \leq \alpha \tag{3.15}$$

avec N impair. Le retard $\alpha = \frac{N-1}{2}$ est un entier. La réponse fréquentielle du filtre est :

$$H(e^{j\Omega}) = A(e^{j\Omega})e^{-j\alpha\Omega} = \sum_{n=0}^{N-1} h(n).e^{-jn\Omega} \tag{3.16}$$

En utilisant la relation de symétrie de l'équation 3.15 on obtient :

$$H(e^{j\Omega}) = e^{-j\frac{N-1}{2}\Omega} \sum_{n=0}^{\frac{N-1}{2}} a_n . \cos(n\Omega) \tag{3.17}$$

$$a_0 = h(\frac{N-1}{2}) \tag{3.18}$$

$$a_n = 2h(\frac{N-1}{2} - n), \qquad n = 1, \ldots \frac{N-1}{2} \tag{3.19}$$

3.2.2 Filtre RIF à phase linéaire de type II

Un filtre RIF à phase linéaire de type II est défini par une réponse impulsionnelle symétrique comme dans l'équation 3.15, avec N pair. Le retard $\alpha = \frac{N-1}{2}$ n'est pas un entier. La réponse fréquentielle du filtre est :

$$H(e^{j\Omega}) = e^{-j\frac{N-1}{2}\Omega} \sum_{n=1}^{\frac{N}{2}} b_n . \cos[(n - \frac{1}{2})\Omega] \tag{3.20}$$

$$b_n = 2h(\frac{N}{2} - n), \qquad n = 1, \ldots \frac{N}{2} \tag{3.21}$$

Remarque : si $\Omega = \pi$ alors $\cos[(n - \frac{1}{2})\pi] = 0$ ce qui implique que $H(e^{j\pi}) = 0$. Par consequent, un filtrage ne respectant pas cette contrainte (e.g. un filtre passe-haut) est impossible à réaliser avec un filtre de type II.

3.2.3 Filtre RIF à phase linéaire de type III

Un filtre RIF à phase linéaire de type III est défini par une réponse impulsionnelle antisymétrique :

$$h(n) = -h(N - 1 - n), \qquad 0 \leq n \leq \alpha \tag{3.22}$$

avec N impair. Le retard $\alpha = \frac{N-1}{2}$ est un entier, $\beta = \pm\pi/2$. La réponse fréquentielle du filtre est :

$$H(e^{j\Omega}) = je^{-j\frac{N-1}{2}\Omega} \sum_{n=1}^{\frac{N-1}{2}} c_n . \sin(n\Omega) \tag{3.23}$$

$$c_n = 2h(\frac{N-1}{2} - n), \qquad n = 1, \ldots \frac{N-1}{2} \tag{3.24}$$

Remarque : $H(e^{j0}) = H(e^{j\pi}) = 0$. Par consequent, un filtrage ne respectant pas cette contrainte est impossible à réaliser avec un filtre de type III. Les filtres passe-bande ou certains dérivateurs sont réalisables.

3.2.4 Filtre RIF à phase linéaire de type IV

Un filtre RIF à phase linéaire de type III est défini par une réponse impulsionnelle antisymétrique comme dans l'équation 3.22, avec N pair. Le retard $\alpha = \frac{N-1}{2}$ n'est pas un entier, $\beta = \pm\pi/2$. La réponse fréquentielle du filtre est :

$$H(e^{j\Omega}) = je^{-j\frac{N-1}{2}\Omega} \sum_{n=1}^{\frac{N}{2}} d_n . \sin[(n - \frac{1}{2})\Omega] \tag{3.25}$$

$$d_n = 2h(\frac{N}{2} - n), \qquad n = 1, \ldots \frac{N}{2} \tag{3.26}$$

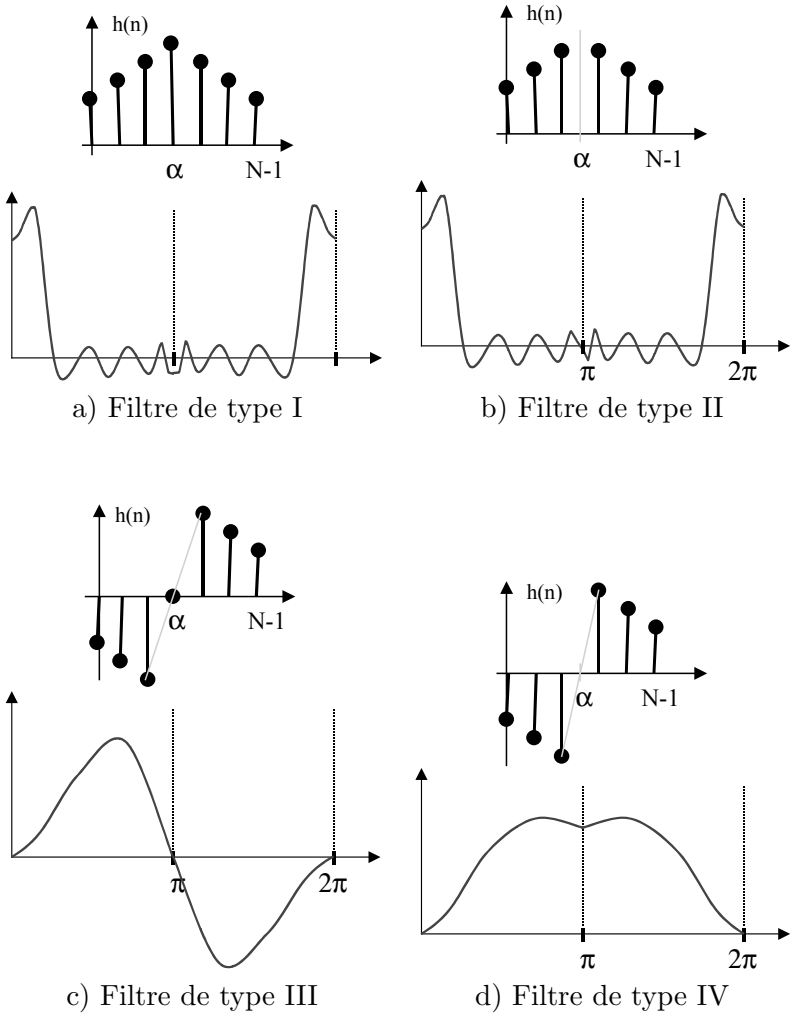
Remarque : $H(e^{j0}) = 0$, $H(e^{j\pi}) \neq 0$. Par consequent, un filtrage ne respectant pas cette contrainte est impossible à réaliser avec un filtre de type IV. Les filtres passe-haut ou certains dérivateurs sont réalisables.

La figure 3.1 résume les caractéristiques des 4 types de filtre RIF à phase linéaire. Elle trace des exemples de réponses impulsionnelle symétriques (types I et II) ou antisymétriques (types III et IV) et des exemples de réponses fréquentielles.

Exemple 3.2.1 : Analyse d'un filtre numérique RIF

Soit un filtre RIF défini par sa réponse impulsionnelle $h(n)$:

$$h(n) = 0.1\delta(n) - 0.3\delta(n-2) + 0.5\delta(n-3) - 0.3\delta(n-4) + 0.1\delta(n-6)$$



a) Filtre de type I b) Filtre de type II

c) Filtre de type III d) Filtre de type IV

FIGURE 3.1 – Les quatre types de filtres RIF à phase linéaire

1. Donner les expressions de l'équation aux différences finies ainsi que la fonction de transfert en Z.
2. Tracer la réponse impulsionnelle $h(n)$ du filtre.
3. Calculer la réponse fréquentielle $H(e^{j\Omega})$ du filtre. Déterminer son module et sa phase. On note que :
$$e^{-j\Omega_1} + e^{-j\Omega_2} = 2 \times e^{-j\frac{(\Omega_1+\Omega_2)}{2}} \times \cos(\frac{\Omega_2 - \Omega_1}{2})$$
4. Donner les valeurs du module en $\Omega = 0,\ \pi/2,\ ,\pi\ ,2\pi$.
5. Tracer approximativement son module. De quel type de filtre s'agit-il ?

3.3 Méthode de synthèse par fenêtrage

Soit un filtre numérique idéal $H(e^{j\Omega})$, périodique de période 2π. Il est décomposable en série de Fourier par sa réponse impulsionnelle :

$$H(e^{j\Omega}) = \sum_{n=-\infty}^{\infty} h(n)e^{-jn\Omega} \tag{3.27}$$

Sa réponse impulsionnelle peut être exprimée en fonction de $H(e^{j\Omega})$ par :

$$h(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} H(e^{j\Omega})e^{jn\Omega}d\Omega \tag{3.28}$$

Ce filtre est non causal et à réponse impulsionnelle infinie. La manière la plus simple d'obtenir un filtre RIF approchant $H(e^{j\Omega})$ est de limiter $h(n)$ par :

$$\hat{h}(n) = \begin{cases} h(n), & 0 \leq n < N \\ 0, & n < 0 \ et \ n \geq N \end{cases}$$

On obtient dans ce cas un filtre numérique RIF approché $\hat{h}(n)$ dont la réponse fréquentielle est modifiée par le phénomène de Gibbs [OS99], c'est à dire l'apparition d'ondulation dans les bandes passantes et atténuées d'amplitude convergeant vers une valeur non nulle lorsque $N \to \infty$. De manière plus générale, on peut représenter $\hat{h}(n)$ comme le produit du filtre idéal $h(n)$ avec une fenêtre à durée finie $w(n)$.

$$\hat{h}(n) = h(n).w(n) \tag{3.29}$$

Dans l'exemple précédent, $w(n)$ est la fenêtre rectangulaire $r(n)$ définie par :

$$r(n) = \begin{cases} 1, & 0 \leq n < N \\ 0, & n < 0 \ et \ n \geq N \end{cases}$$

On peut alors étudier la réponse fréquentielle du filtre RIF ainsi synthétisé $\hat{H}(e^{j\Omega})$:

$$\hat{H}(e^{j\Omega}) = \hat{H}(z)|_{z=e^{j\Omega}} = \sum_{n=0}^{N-1} h(n).w(n).e^{-jn\Omega} \tag{3.30}$$

$$\hat{H}(e^{j\Omega}) = \frac{1}{2\pi}\int_{-\pi}^{\pi} H(e^{j\theta})W(e^{j(\Omega-\theta)})d\theta \tag{3.31}$$

$$= H(e^{j\Omega}) * W(e^{j\Omega}) \tag{3.32}$$

La réponse fréquentielle du filtre $\hat{H}(e^{j\Omega})$ sera donc le résultat de la convolution entre le filtre idéal et la transformée de Fourier de la fenêtre. Les caractéristiques de cette dernière sont donc très importantes. On devra trouver un fenêtrage dont le $W(e^{j\Omega})$ se rapproche le plus des caractéristiques de $\delta(\Omega)$.

La figure 3.2 montre ce phénomène de convolution. Les effets sur le filtre idéal sont :
- l'apparition d'ondulations en bandes passante et atténuée,
- une zone de transition moins rapide déterminée par la largeur du lobe principal.

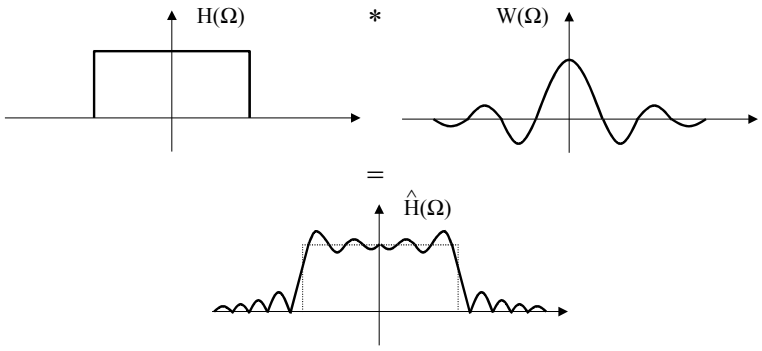


FIGURE 3.2 – Processus de convolution de la réponse fréquentielle du filtre idéal par la fenêtre

---

Exemple 3.3.1 : Calcul de la TF de la fenêtre rectangulaire

Soit la fenêtre rectangulaire $r(n)$ définie par : $r(n) = \begin{cases} 1, & 0 \leq n < N \\ 0, & n < 0 \ et \ n \geq N \end{cases}$

1. Calculer la transformée de Fourier $R(e^{j\Omega})$ de $r(n)$.
2. Tracer sa réponse fréquentielle entre $-\pi$ et π. Donner en particulier la valeur en 0 (amplitude du lobe principal), la largeur du lobe principal et l'amplitude du lobe secondaire.

Exemple 3.3.2 : Filtrage passe bas RIF à phase linéaire

Soit le filtre passe-bas idéal défini par :

$$H(e^{j\Omega}) = \begin{cases} e^{-j\alpha\Omega} & pour \quad -\Omega_c \leq \Omega \leq \Omega_c \\ 0 & pour \quad \Omega_c < \Omega \leq \pi \ et \ -\pi \leq \Omega < -\Omega_c \end{cases}$$

1. Tracer la réponse fréquentielle de $H(e^{j\Omega})$ en module et phase entre -2π et 2π.
2. Calculer et tracer $h(n)$.
3. Expliquer comment obtenir un filtre RIF de longueur N à partir de ce filtre. On utilisera la fenêtre étudiée dans l'exemple précédent. Donner et tracer les valeurs de $\hat{h}(n)$.
4. Donner une formule analytique de la réponse fréquentielle du filtre $\hat{H}(e^{j\Omega})$.
5. Tracer son module et sa phase.

3.3.1 Caractéristiques des principales fenêtres

Les principales fenêtres utilisées sont détaillées à la figure 3.3. Elles sont définies par les équations ci dessous :

Rectangulaire

$$w(n) = \begin{cases} 1, & 0 \leq n < N \\ 0, & n < 0 \; et \; n \geq N \end{cases} \tag{3.33}$$

Triangulaire (Bartlett)

$$w(n) = \begin{cases} \frac{2n}{N-1}, & 0 \leq n \leq \frac{N-1}{2} \\ 2 - \frac{2n}{N-1}, & \frac{N-1}{2} \leq n \leq N-1 \\ 0, & n < 0 \; et \; n \geq N \end{cases} \tag{3.34}$$

Hanning

$$w(n) = \begin{cases} 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), & 0 \leq n \leq N-1 \\ 0, & n < 0 \; et \; n \geq N \end{cases} \tag{3.35}$$

Hamming

$$w(n) = \begin{cases} 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), & 0 \leq n \leq N-1 \\ 0, & n < 0 \; et \; n \geq N \end{cases} \tag{3.36}$$

Blackman

$$w(n) = \begin{cases} 0.42 - 0.5\cos\left(\frac{2\pi n}{N-1}\right) + 0.08\cos\left(\frac{4\pi n}{N-1}\right), & 0 \leq n \leq N-1 \\ 0, & n < 0 \; et \; n \geq N \end{cases} \tag{3.37}$$

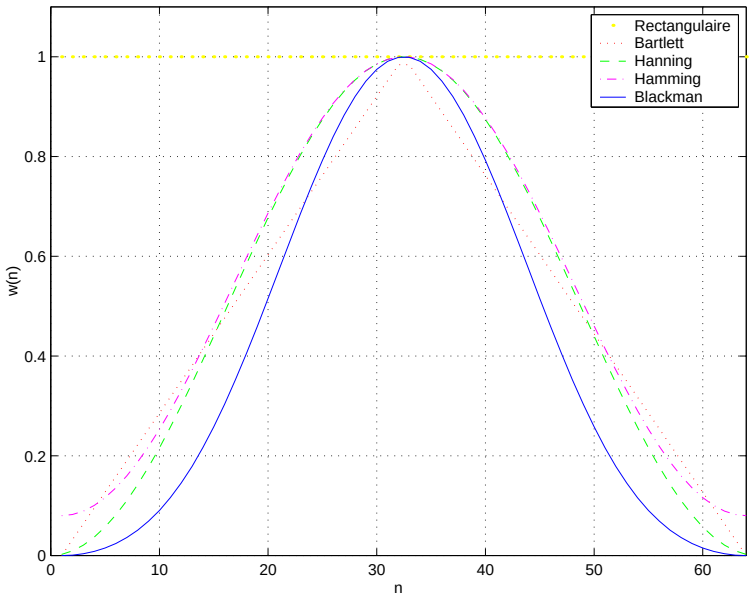


FIGURE 3.3 – Réponses temporelles des principales fenêtres

Ces fenêtres sont les principales fonctions utilisées en analyse spectrale et en synthèse de filtre RIF. Leurs réponses fréquentielles $20\log_{10}|W(e^{j\Omega})|$sont données à la la figure 3.4 avec

$N = 64$. On voit que selon la fenêtre, la largeur du lobe principal et l'amplitude du plus grand lobe secondaire diffèrent. Par exemple, la fenêtre rectangulaire possède le lobe principal le plus étroit ($4\pi/N$), mais le lobe secondaire le moins atténué ($-13dB$). Le tableau 3.1 résume ces caractéristiques. On y voit en particulier qu'en utilisant une fenêtre sans discontinuité et plus lisse (e.g. Hamming ou Blackman), on peut réduire de manière importante l'amplitude des lobes secondaires parasites. La colonne de droite est en relation avec la synthèse RIF et sera détaillée dans la section suivante.

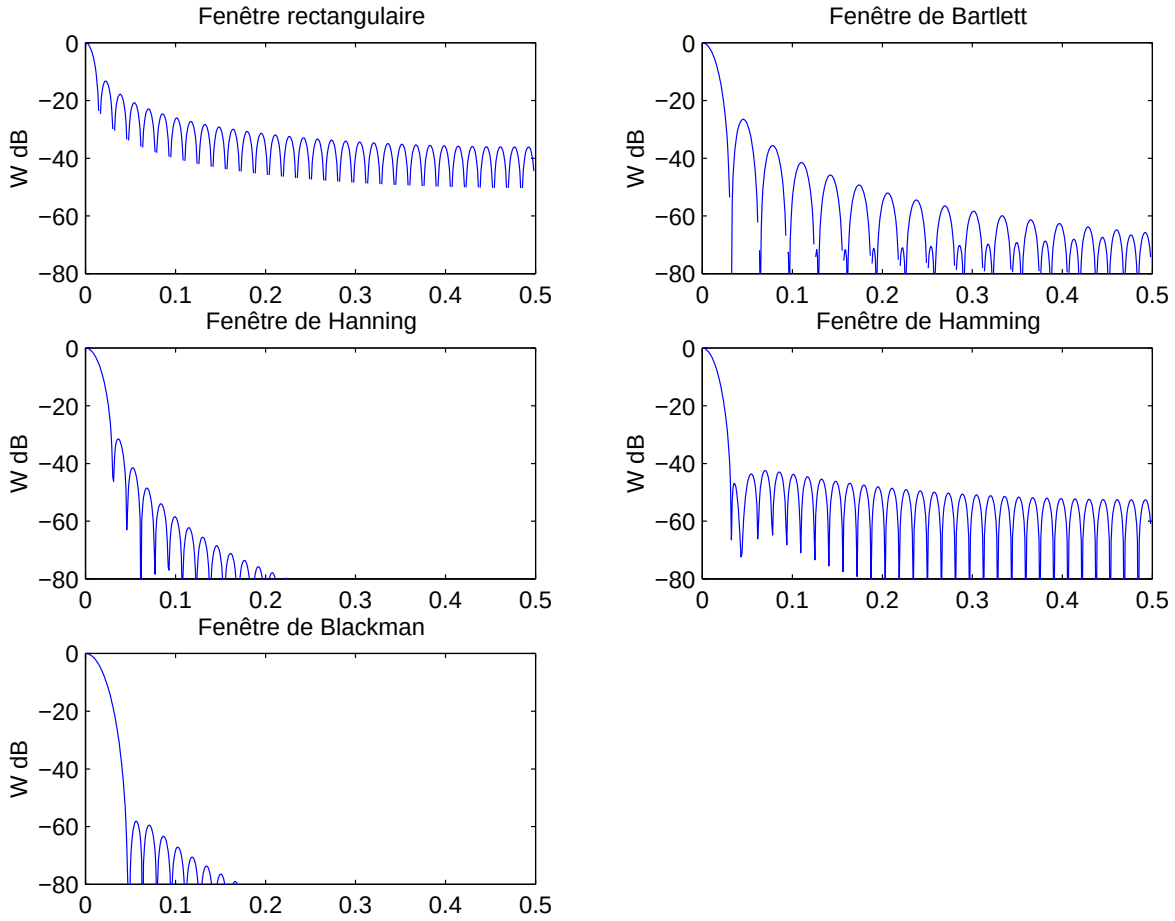


FIGURE 3.4 – Réponses fréquentielles des principales fenêtres

3.3.2 Choix de la fenêtre dans la méthode de synthèse

On voit donc que le choix de la fenêtre aura une influence sur les performances du filtre. En particulier, les caractéristiques importantes et leur influence sur le filtre synthétisé sont détaillées ci dessous.

— La largeur de la zone de transition $\Delta\Omega = |\Omega_p - \Omega_a|$ définie à la section 1.3.2 sera directement fonction de la largeur du lobe principal $\Delta\Omega_m$ donnée dans le tableau 3.1. Lorsque la fréquence de coupure ou de transition du filtre n'est pas trop proche de 0 ou de π, on peut considérer que $\Delta\Omega \cong \frac{\Delta\Omega_m}{2}$. On voit donc que **la transition $\Delta\Omega$ sera**

Type de fenêtre	Rapport d'amplitude entre lobe principal et lobe secondaire λ	Largeur du lobe principal $\Delta\Omega_m$	Atténuation minimale en bande atténuée ΔA
Rectangulaire	$-13dB$	$4\pi/N$	-21dB
Bartlett	$-25dB$	$8\pi/N$	-25dB
Hanning	$-31dB$	$8\pi/N$	-44dB
Hamming	$-41dB$	$8\pi/N$	-53dB
Blackman	$-57dB$	$12\pi/N$	-74dB

TABLE 3.1 – Caractéristiques des principales fenêtres

d'autant plus faible que N sera grand.

— Les ondulations en bande passante et en bande atténuée seront égales, on notera $\Delta A = \delta_1 = \delta_2$ et on parlera de filtre *equiripple*. Cette atténuation est donnée par la quatrième colonne du tableau 3.1. On voit que ΔA **ne dépend que du type de fenêtre et non de** N.

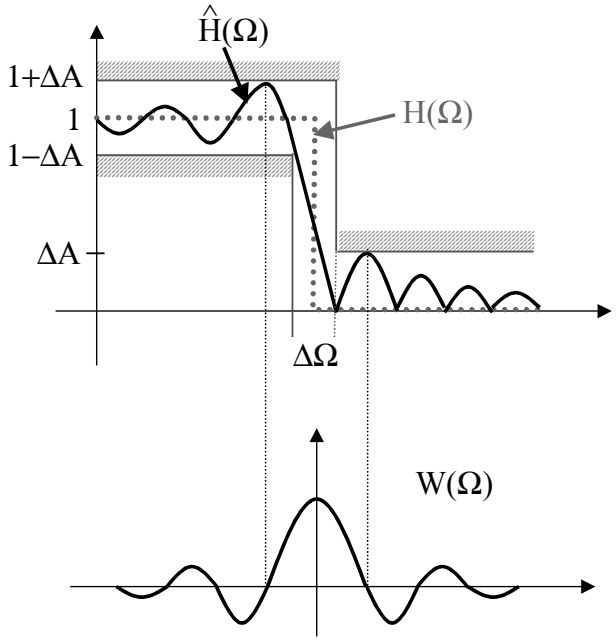


FIGURE 3.5 – Illustration de l'effet du fenêtrage sur le filtre idéal

La procédure de synthèse d'un filtre numérique RIF par fenêtrage à partir de la réponse impulsionnelle d'un filtre idéal $h(n)$ consiste à :

1. choisir en fonction de l'atténuation ΔA le type de fenêtre à utiliser,
2. choisir en fonction de la largeur de la zone de transition $\Delta\Omega$ et du type de la fenêtre $w(n)$ la longueur de la réponse impulsionnelle N.

Le filtre RIF résultant $\hat{h}(n)$ est ensuite défini par $\hat{h}(n) = h(n).w(n)$. Il sera de longueur N.

Remarque : un décalage de α de la réponse impulsionnelle du filtre idéal $h(n)$ peut être nécessaire afin de rendre le filtre $\hat{h}(n)$ à phase linéaire. On aura alors : $\hat{h}(n) = h(n-\alpha).w(n)$.

Le calcul de la largeur de la zone de transition $\Delta\Omega$ est ici présenté de façon empirique. Dans la plupart des cas, cette approximation reste suffisante, mais il est quelque fois nécessaire d'itérer plusieurs fois l'algorithme de synthèse afin d'assurer que le filtre entre dans le gabarit spécifié. Aussi un dernier type de fenêtre dit de Kaiser peut être utilisée. Cette méthode, permettant d'obtenir une formulation exacte des paramètres $\Delta\Omega$ et ΔA, ne sera pas développée ici, mais on peut se rapporter à [OS99] pour l'utiliser.

Exemple 3.3.3 : Caractéristique d'un filtre
Soit un filtre défini par une zone de transition $\Delta\Omega = 0.1\pi$ et une atténuation $> 30dB$, déterminer la taille et le type de fenêtre à utiliser.

3.4 Méthode de synthèse par échantillonnage en fréquence

La méthode de synthèse par échantillonnage en fréquence est appliquée depuis la réponse fréquentielle d'un filtre continu idéal $H(e^{j\Omega})$. A partir de celle-ci et d'une valeur de N fixée, on réalise un échantillonnage en fréquence de pas $\Omega_e = \frac{2\pi}{N}$ définie par :

$$\hat{H}(e^{jk\Omega_e}) \triangleq H(e^{j\Omega})|_{\Omega=\frac{2k\pi}{N}} \tag{3.38}$$

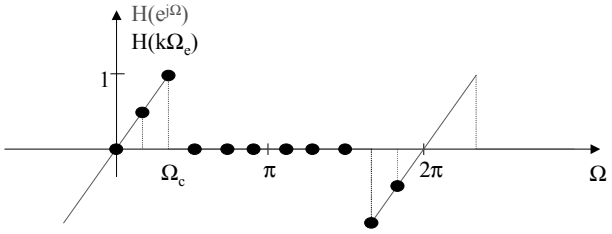


FIGURE 3.6 – Échantillonnage fréquentiel d'un filtre dérivateur

La figure 3.6 montre un exemple de dérivateur défini à partir d'un filtre continu. On remarquera en particulier que prendre N points entre $[0, 2\pi[$ à un pas de $\frac{2\pi}{N}$ élimine l'échantillon en $\Omega = 2\pi$. Le filtre RIF $\hat{h}(n)$ est alors trouvé par TFD inverse de $\hat{H}(e^{jk\Omega_e})$:

$$\hat{h}(n) = \frac{1}{N}\sum_{k=0}^{N-1} \hat{H}(e^{jk\Omega_e}) e^{2j\pi\frac{n.k}{N}},\ n = 0, 1 \ldots N-1 \tag{3.39}$$

$$\hat{h}(n) = 0,\ ailleurs \tag{3.40}$$

Cette méthode de synthèse est très simple et permet de réaliser toute forme de filtre. De plus, elle peut être combinée avec la méthode du fenêtrage. La fonction de transfert en Z du filtre

sera :

$$\hat{H}(z) = \sum_{n=0}^{N-1} \hat{h}(n) z^{-n} \tag{3.41}$$

En retravaillant cette fonction de transfert, on obtient :

$$\hat{H}(z) = \sum_{n=0}^{N-1} \hat{h}(n) z^{-n} = \sum_{n=0}^{N-1} \left[\frac{1}{N} \sum_{k=0}^{N-1} \hat{H}(e^{jk\Omega_e}) e^{2j\pi \frac{n.k}{N}} \right] z^{-n} \tag{3.42}$$

$$\hat{H}(z) = \frac{1}{N} \sum_{k=0}^{N-1} \hat{H}(e^{jk\Omega_e}) \left[\sum_{n=0}^{N-1} e^{2j\pi \frac{n.k}{N}} z^{-n} \right] \tag{3.43}$$

$$\hat{H}(z) = \frac{1}{N} \sum_{k=0}^{N-1} \hat{H}(e^{jk\Omega_e}) \frac{1 - e^{2j\pi \frac{N.k}{N}} z^{-N}}{1 - e^{2j\pi \frac{k}{N}} z^{-1}} \tag{3.44}$$

$$\hat{H}(z) = \frac{1 - z^{-N}}{N} \sum_{k=0}^{N-1} \hat{H}(e^{jk\Omega_e}) \frac{1}{1 - e^{2j\pi \frac{k}{N}} z^{-1}} \tag{3.45}$$

La fonction de transfert du filtre $\hat{H}(z)$ peut donc être également exprimée comme une mise en parallèle de N filtres RII du premier ordre complexe. Cette décomposition permet d'obtenir directement les coefficients de cette structure du filtre à partir de la réponse fréquentielle recherchée.

Exemple 3.4.1 : Synthèse d'un filtre passe-bas

Soit un filtre passe-bas idéal défini par :

$$H(e^{j\Omega}) = \begin{cases} 1 & pour \quad |\Omega| < \Omega_c = \frac{4\pi}{N} \\ 0 & pour \quad \frac{4\pi}{N} \leq |\Omega| < \pi \end{cases}$$

1. Tracer le filtre idéal entre $[0, 2\pi]$. Tracer ensuite le filtre échantillonné $\hat{H}(e^{jk\Omega_e})$.
2. Calculer $\hat{h}(n)$ par TFD inverse.
3. Calculer deux versions de $\hat{H}(z)$.

Chapitre 4

Transformée de Fourier Rapide

4.1 Naissance de la TFR

La transformation de Fourier rapide (TFR), ou encore Fast Fourier Transform (FFT), est directement issue d'une réorganisation du calcul des matrices de la transformée de Fourier discrète (TFD) dont la définition est donnée dans les équations 4.1 et 4.2. $X(k)$ est la TFD du signal $x(n)$. N points du signal $x(n)$ donnent N points de la TFD $X(k)$.

$$X(k) = \sum_{n=0}^{N-1} x(n).e^{(-2j\pi\frac{n.k}{N})},\ k = 0, 1 \dots N-1 \tag{4.1}$$

$$x(n) = \frac{1}{N}\sum_{k=0}^{N-1} X(k).e^{(2j\pi\frac{n.k}{N})},\ n = 0, 1 \dots N-1 \tag{4.2}$$

x(n) et X(k) sont, dans le cas général, des nombres complexes.
Nous pouvons réécrire l'équation 4.1 sous forme matricielle en faisant ainsi apparaître la complexité de l'algorithme de la TFD.

$$\begin{pmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & W_N^1 & W_N^2 & \cdots & W_N^{N-1} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & W_N^2 & W_N^4 & \cdots & W_N^{2(N-1)} \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & \cdots & W_N^{(N-1)^2} \end{pmatrix} \times \begin{pmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{pmatrix} \tag{4.3}$$

avec W_N le twiddle factor égal à $e^{(-2j\frac{\pi}{N})}$

La TFD revient à calculer un produit matrice-vecteur où chaque élément est de type complexe. La complexité de calcul de la TFD est donc de N^2 multiplications, et de $N(N-1)$ additions sur des nombres complexes. Ceci revient à une complexité de $4N^2$ multiplications réelles et $N(4N-2)$ additions réelles. Cet algorithme se comporte donc en $O(N^2)$, mais ne possède pas de problèmes d'adressage car les $x(n)$ et les W_N^k sont rangés dans l'ordre en mémoire.
En exploitant les propriétés des W_N^k (symétrie, périodicité, ...) :

$$W_N^{k(N-n)} = (W_N^{kn})^* \tag{4.4}$$
$$W_N^{k(n+N)} = W_N^{n(k+N)} = W_N^{kn} \tag{4.5}$$
$$W_N^{n+N/2} = -W_N^n \tag{4.6}$$
$$W_N^{2kn} = W_{N/2}^{kn} \tag{4.7}$$

on peut réduire le nombre d'opérations arithmétiques par un rapport de 2, mais la complexité globale reste en $O(N^2)$ [OS75].
De nombreux travaux ont essayé de réduire la complexité de la transformée de Fourier discrète, mais il a fallu attendre 1965 et la paire Cooley et Tuckey qui ont publié un algorithme applicable quand N est le produit de 2 ou plusieurs entiers. Cette publication allait engendrer de nombreuses recherches sur les algorithmes de calculs des transformées ; la FFT était née. Depuis, et encore de nos jours, beaucoup d'algorithmes de traitement numérique du signal sont basés sur l'utilisation de cet algorithme efficace (ou de ses nombreux dérivés) pour réduire leur complexité.

Le principe fondamental de la TFR (en français dans le texte) repose sur la décomposition du calcul d'une séquence de N échantillons en TFD successives sur un nombre inférieur de points. Selon la manière dont ce principe est implanté, on obtient différents algorithmes, tous comparables au niveau de leur complexité.

— TFR partagée dans le temps ou DIT (decimation in time) :
$x(n)$ est décomposé en sous séquence.
— TFR partagée en fréquences ou DIF (decimation in frequency) :
$X(k)$ est décomposé en sous séquence.

4.2 TFR partagée dans le temps (DIT)

Depuis la formule 4.1, on partage $X(k)$ en une somme sur les termes d'indices pairs et une autre sur les termes d'indices impairs, dans notre exemple on se limitera au cas où N est une puissance de deux.

$$X(k) = \sum_{n\ pair} x(n).W_N^{nk} + \sum_{n\ impair} x(n).W_N^{nk} \tag{4.8}$$
$$X(k) = \sum_{n=0}^{N/2-1} x(2n).W_N^{2nk} + \sum_{n=0}^{N/2-1} x(2n+1).W_N^{(2n+1)k} \tag{4.9}$$
$$X(k) = \sum_{n=0}^{N/2-1} x(2n).W_{N/2}^{nk} + W_N^k \sum_{n=0}^{N/2-1} x(2n+1).W_{N/2}^{nk} \tag{4.10}$$
$$X(k) = G(k) + W_N^k.H(k) \tag{4.11}$$

où $G(k)$: TFD sur les $N/2$ points d'indices pairs,
$H(k)$: TFD sur les $N/2$ points d'indices impairs.

De plus, on peut réduire la complexité de calcul des échantillons $N/2$ à $N-1$ en calculant $X(k+N/2)$, puis en appliquant les propriétés de symétrie (voir équation 4.13). On obtient

ainsi le calcul d'un *papillon de la TFR.*

$$X(k+\frac{N}{2}) = \sum_{n=0}^{N/2-1} x(2n).W_{N/2}^{n(k+N/2)} + W_N^{(k+N/2)} \sum_{n=0}^{N/2-1} x(2n+1).W_{N/2}^{n(k+N/2)} \quad (4.12)$$

$$X(k+\frac{N}{2}) = G(k) - W_N^k.H(k) \quad (4.13)$$

L'application de ces équations est représentée graphiquement à la figure 4.1. On remarque qu'une TFD d'ordre N est décomposée en 2 TFD d'ordre $N/2$ suivies d'une recombinaison dont la complexité est en $O(N)$.

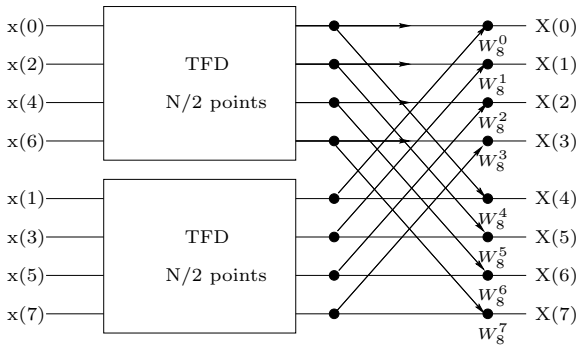


FIGURE 4.1 – Décomposition DIT de la TFD

Comme $N/2$ est également un nombre pair, on peut de nouveau partager chacune des TFD d'ordre $N/2$ en deux TFD d'ordre $N/4$ jusqu'à arriver à une TFD sur deux points pour laquelle le graphe flot est de complexité $O(N)$. Finalement il en résulte un graphe flot possédant log_2N étages de recombinaison (complexité $O'N$)) comme représenté à la figure 4.3 ; la complexité de la transformée de Fourier est enfin réduite à $O(Nlog_2N)$.

L'opération élémentaire est ici appelée papillon DIT. On peut le représenter sous sa forme générale entre deux étages m et $m+1$ avec $m = 1, 2, \ldots, log_2N$ (figure 4.2 et système d'équations 4.14).

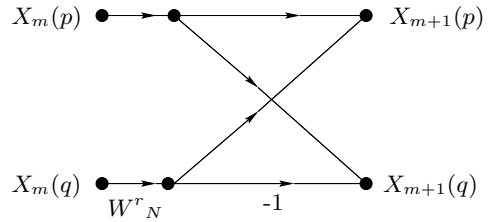


FIGURE 4.2 – Papillon DIT de la TFR

$$\begin{cases} X_{m+1}(p) = X_m(p) + W_N^r.X_m(q) \\ X_{m+1}(q) = X_m(p) - W_N^r.X_m(q) \end{cases} \quad (4.14)$$

Un papillon DIT, dont les équations sont données ci dessus (4.14) nécessite 1 multiplication, 1 addition et 1 soustraction sur des nombres complexes. Le graphe complet nécessite le calcul de $\frac{N}{2} \times log_2 N$ papillons ce qui donne donc en unité de calcul :

- $\frac{N}{2} log_2 N$ multiplications de nombres complexes,
- $N log_2 N$ additions/soustractions de nombres complexes, ou,

- $2N log_2 N$ multiplications de nombre réels [1],
- $3N log_2 N$ additions/soustractions de nombre réels [2].

Cette complexité précise peut servir à évaluer le temps de calcul d'une TFR complète sur n'importe quel processeur.

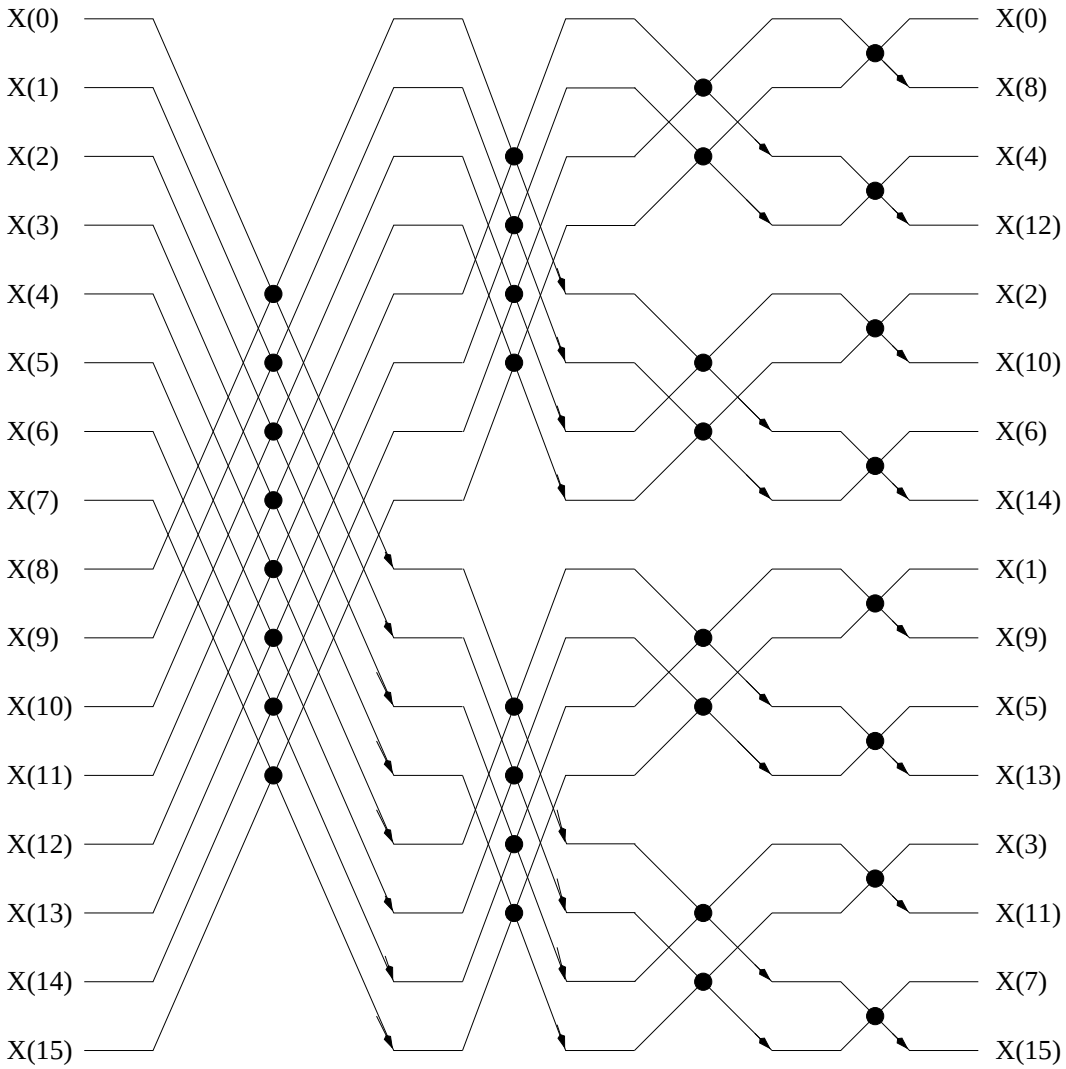


FIGURE 4.3 – Graphe d'une TFR DIF sur 16 échantillons

1. 1 multiplication complexe requiert 4 multiplications et 2 additions sur des nombres réels.
2. 1 addition complexe requiert 2 additions sur des nombres réels.

Si $X_m(.)$ et $X_{m+1}(.)$ peuvent être placés en mémoire sur le même vecteur on parle de calcul **en place**. De plus, le fait de faire apparaître la séparation pair/impair au départ (DIT) puis de séparer à nouveau en $N/4$, $N/8$,... groupes d'échantillons, implique en entrée des échantillons dont les adresses suivent un ordre appelé **bit-reverse** tel que :

— si la représentation en binaire de l'indice (n) est $(n_b \dots n_2 n_1 n_0)$ alors celle de son bit-reverse (n') est $(n_0 n_1 n_2 \dots n_b)$.

A l'opposé de la TFD, la TFR génère un calcul d'adresse non négligeable. Entre deux étages m et $m+1$, les W^k ne varient pas linéairement avec les $x(k)$. De plus, il est nécessaire de présenter à l'algorithme les échantillons dans un ordre non régulier (bit-reverse) ce qui engendre un calcul d'adresse supplémentaire. Ces problèmes ne sont pas négligeables - surtout sur un processeur général - car ils rajoutent à la complexité élémentaire une complexité d'adressage.

4.3 TFR partagée dans les fréquences (DIF)

Depuis la formule 4.1, on partage X(k) en une somme sur les $N/2$ premiers termes et une autre sur les $N/2$ autres termes :

$$X(k) = \sum_{n=0}^{N/2-1} x(n).W_N^{nk} + \sum_{n=N/2}^{N-1} x().W_N^{nk} \tag{4.15}$$

$$X(k) = \sum_{n=0}^{N/2-1} x(n).W_N^{nk} + W_N^{kN/2} \sum_{n=0}^{N/2-1} x(n+N/2).W_N^{nk} \tag{4.16}$$

$$X(k) = \sum_{n=0}^{N/2-1} \left[x(n) + (-1)^k.x(n+N/2)\right] W_N^{nk} \tag{4.17}$$

On effectue une séparation des indices pairs et impairs :

$$X(2p) = \sum_{n=0}^{N/2-1} \left[x(n) + x(n+N/2)\right] W_N^{2pk} \tag{4.18}$$

$$X(2p+1) = \sum_{n=0}^{N/2-1} \left[x(n) - x(n+N/2)\right] W_N^{2pk}.W_N^n \tag{4.19}$$

$$X(2p) = \sum_{n=0}^{N/2-1} \left[x(n) + x(n+N/2)\right] W_{N/2}^{pk} \tag{4.20}$$

$$X(2p+1) = \sum_{n=0}^{N/2-1} \left(\left[x(n) - x(n+N/2)\right] W_{N/2}^{pk}\right).W_N^n \tag{4.21}$$

De même que pour le partage temporel, le remaniement des équations amène le calcul de deux TFD d'ordre inférieur selon le schéma de la figure 4.4.

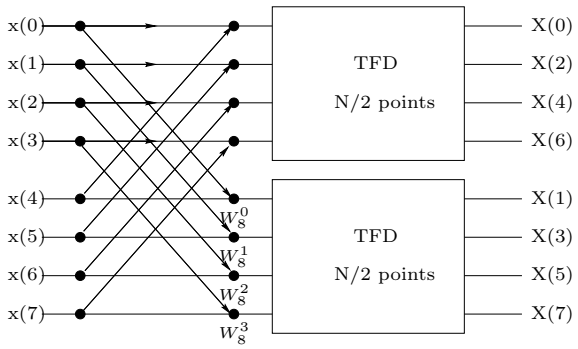


FIGURE 4.4 – Décomposition DIF de la TFD

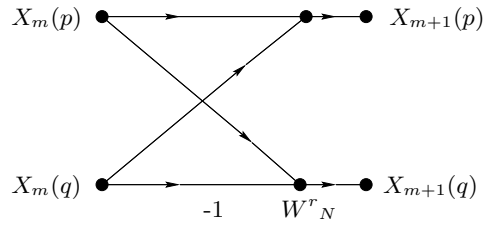


FIGURE 4.5 – Papillon DIF de la TFR

On peut également continuer à diviser ces transformées d'ordre inférieur comme dans le cas précédent (DIT) jusqu'à arriver au papillon unitaire dont le graphe est décrit à la figure 4.5 et dans le système d'équations 4.22.

$$\begin{cases} X_{m+1}(p) = X_m(p) + X_m(q) \\ X_{m+1}(q) = [X_m(p) - X_m(q)].W_N^r \end{cases} \tag{4.22}$$

La complexité élémentaire est identique à la TFR partagée dans le temps, la seule différence réside dans le calcul d'adresse de la table des W^k lors des étapes, qui est plus simple à programmer dans le cas DIF. De plus ce sont les échantillons en sortie qui sont placés en ordre bit-reverse.

4.4 Autres graphes de la TFR

Divers paramètres permettent de modifier la structure du graphe flot de données de la Transformée de Fourier Rapide (mode d'adressage, structure graphique, base du calcul), nous allons essayer d'en énumérer les principaux.

Le graphe à géométrie constante se caractérise par le fait que chaque passe du calcul possède la même structure graphique que les autres passes. Celui-ci est directement issu de la décomposition de la matrice $[W]$ de la transformée de Fourier discrète en matrices de Good [Ka91] selon la relation ci-dessous.

$$W = \prod_{i=1}^{log_2 N} D_i.C_i$$

avec : D_i des matrices diagonales, et C_i les matrices de Good.

Un exemple pour N=4 est représenté sous forme matricielle dans l'équation 4.23 puis sous son équivalent de graphe dans la figure 4.6 ci-dessous.

$$\begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{pmatrix} \times \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & W \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{pmatrix} \times \begin{pmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{pmatrix} \quad (4.23)$$

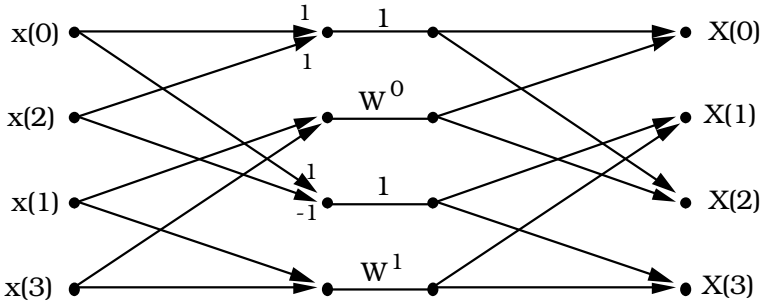


FIGURE 4.6 – Graphe d'une TFR à géométrie constante sur 4 échantillons

Un autre paramètre donnant des graphes de calcul différents est celui du bit-reverse. En effet on peut choisir l'entrée comme la sortie selon l'ordre normal ou l'ordre bit-reverse. Ceci a pour effet de modifier la forme du graphe ainsi que du calcul des indices des W_i. Cependant, si pour éviter un calcul d'adressage supplémentaire on désire obtenir un graphe dont à la fois les entrées comme les sorties sont dans l'ordre naturel, on passe dans une forme de calcul dit "non en place". Cette forme possède l'inconvénient majeur de perdre la notion de papillon. C'est à dire qu'entre deux passes successives on est obligé de mémoriser les vecteurs intermédiaires, ce qui a pour effet de doubler la mémoire nécessaire à l'algorithme.

On peut également utiliser d'autres bases de calcul de la transformée (nos exemples étant en base 2) qui consistent à séparer la TFD en plus de deux (quatre par exemple) TFD d'ordre inférieur ce qui a pour effet de réduire un peu encore la complexité ($O(Nlog_4 N)$ pour une TFR base (ou radix) 4). Par contre le papillon élémentaire voit son nombre de calcul augmenter. On peut citer l'exemple de la TFR DIF en base 4 dont la complexité en opération est $\frac{3N}{4} log_4 N$ multiplications complexes et $4Nlog_4 N$ additions complexes. Un exemple de graphe radix 4 est donné en annexe du document.

4.5 Annexes au chapitre sur la TFR

4.5.1 Graphe d'une TFR DIT radix 2 sur 16 points

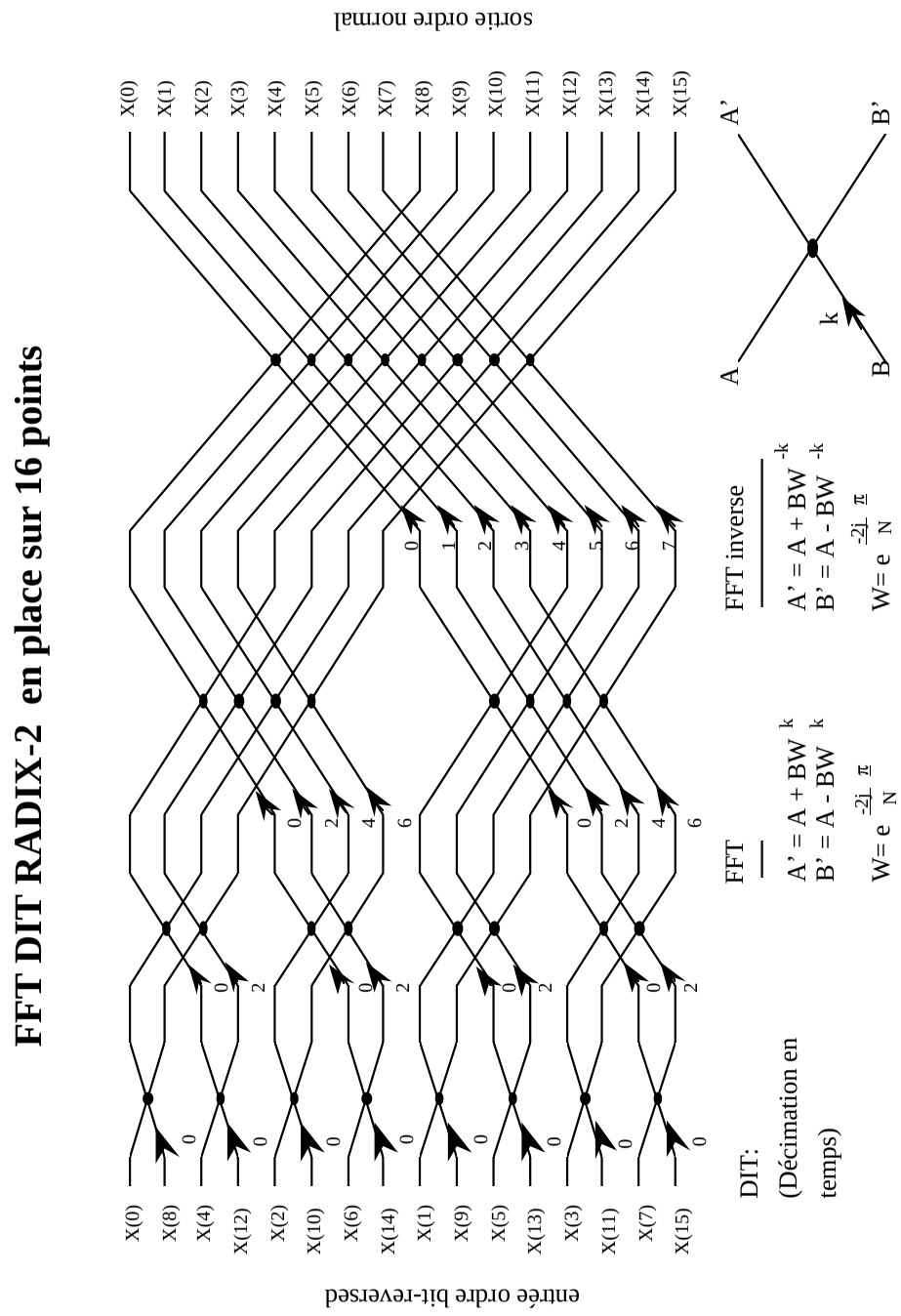


FIGURE 4.7 – Graphe d'une TFR DIT radix 2 sur 16 points

4.5.2 Graphe d'une TFR DIF radix 2 sur 16 points

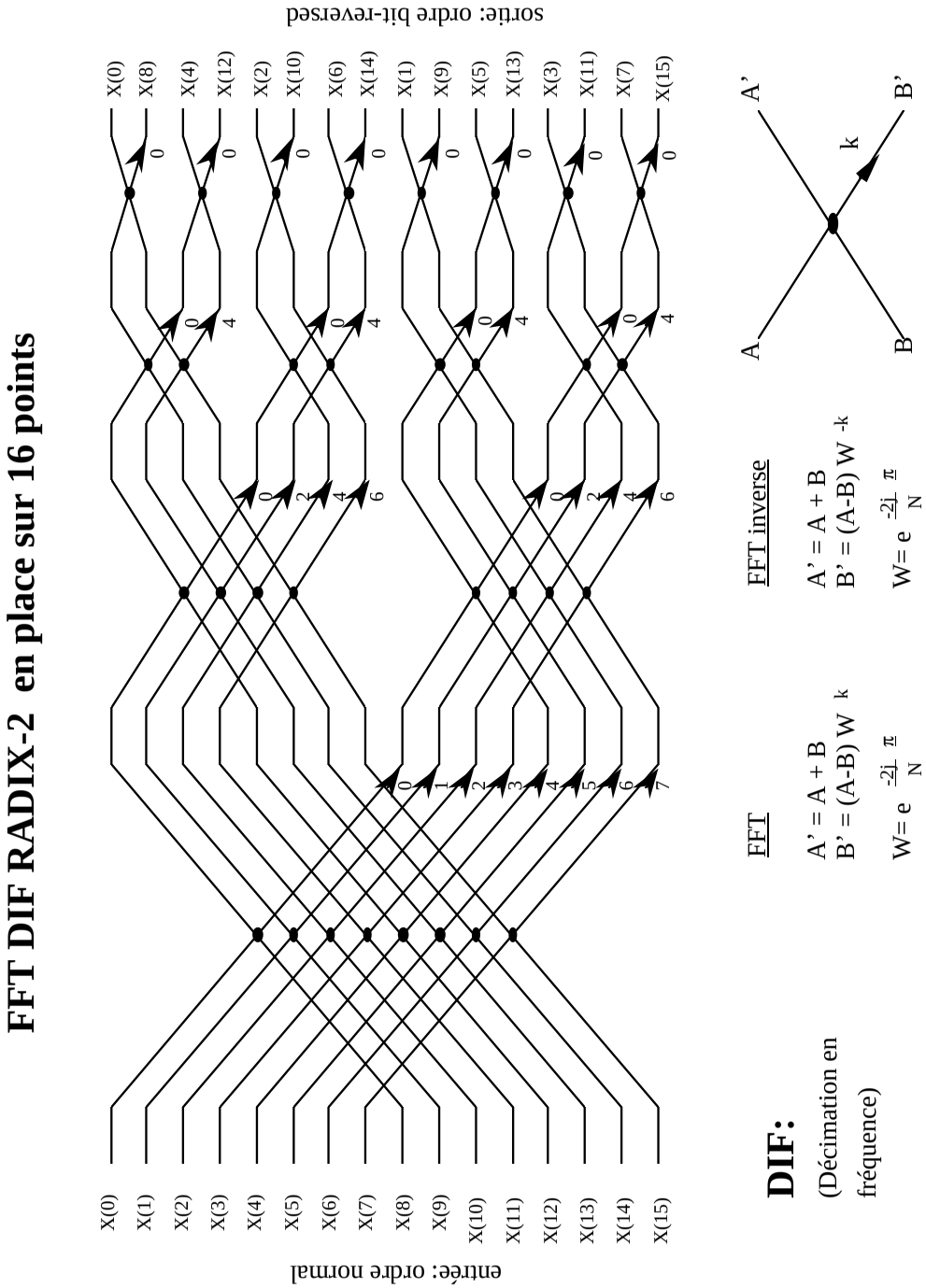


FIGURE 4.8 – Graphe d'une TFR DIF radix 2 sur 16 points

4.5.3 Graphe d'une TFR à géométrie constante sur 16 points

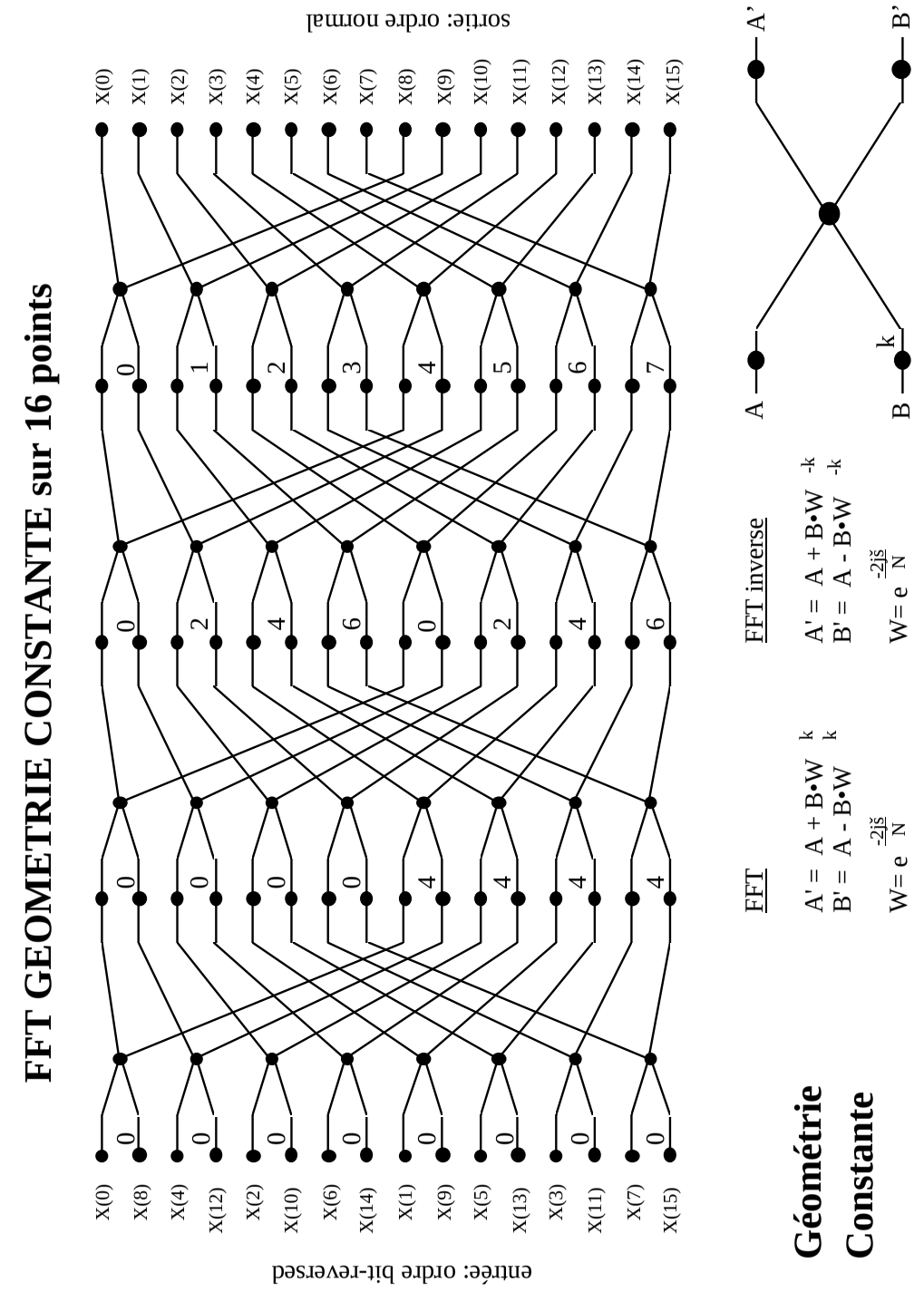


FIGURE 4.9 – Graphe d'une TFR à géométrie constante sur 16 points

4.5.4 Graphe d'une TFR DIF radix 4 sur 16 points

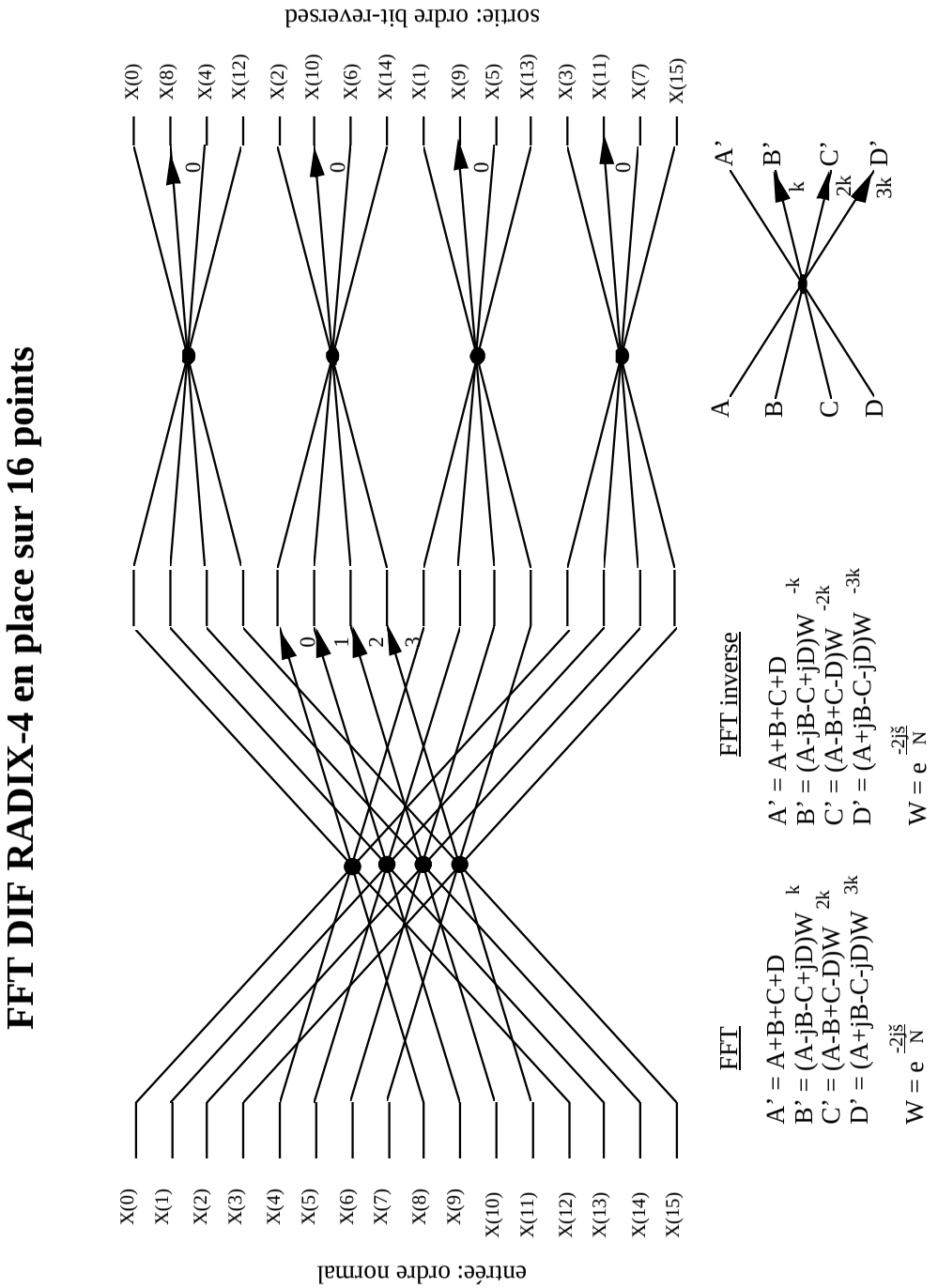


FIGURE 4.10 – Graphe d'une TFR DIF radix 4 sur 16 points

4.5.5 Algorithme DIF de la TFR radix 2 sur I points complexes

Complexité de calcul : $O(I.log_2 I)$
Nombre total de passes : $log_2 I$
Nombre de groupes dans une passe : 2^{passe}
Nombre de papillons calculés : $I.log_2 I/2$
Complexité d'un papillon : 2 additions et 1 multiplication complexes, ou 6 additions et 4 multiplications réelles.

```
PROCEDURE FFT (X, isign  )
    [I]complexe X: -- echantillons temporels et spatiaux
    int isign:     -- FFT / FFT inverse

        variables locales:

        [I]real: Wr, Wi:
        -- racines Nième de l'unité
        -- Wr[k]:=  COS(2p k / I)
        -- Wi[k]:= isign * SIN(2p k / I)
        int k,K:
        real C,S:

        int I1: -- nombre d'echantillons par groupe
        int I2: -- nombre de papillons par groupe
        int I3: -- nombre de groupe

        int i,l: -- indice des échantillons du papillons
        complexe t: -- variable temporaire

        DEBUT

             I2:=I;
             I3:=1;
             POUR passe DE 1 à LOG2I FAIRE

                I1:=I2; I2:= I2 / 2;    K:= I / I1;
                k:=0;

                POUR j DE 0 à I2-1 FAIRE

                     C:=Wr[k];  -- COS (2p k / I)
                     S:=Wi[k];  -- SIN(2p k / I)
                     k:=k + K;

                     POUR i DE j à I-1 PAR I1 FAIRE
                        l:=i + I2;

                        --- papillon X[i] X[l]
                        --- X[i] = X[i] + X[l]
                        --- X[l] = (X[i] - X[l])
                        t.r:=X[i].r - X[l].r;
                        t.i:=X[i].i - X[l].i;
                        X[i].r := X[i].r + X[l].r;
```

```
                X[i].i := X[i].i + X[l].i;
                X[l].r := C*t.r + S*t.i;
                X[l].i := C*t.i - S*t.r;
            FAIT
            I3:=I3 * 2;        -- nombre de groupes
        FAIT
    FAIT

    -- bit reverse : remettre la sortie dans l'ordre normal
    j:=1
    POUR i DE 1 à I-1 FAIRE
        SI i<j ALORS
            t:=X[j]
            X[j]:=X[i]
            X[i]:=t
        FINSI
        k:=I/2
        TANT QUE k<j FAIRE
            j:=j-k
            k:=k/2
        FAIT
        j:=j+k
    FAIT

    SI isign=-1 ALORS
        --- normaliser X dans le cas de la FFT inverse
        POUR tous les X[i] FAIRE
            X[i]:=X[i] / I
        FAIT
    FINSI

FIN
```

Chapitre 5

Analyse spectrale de signaux numériques

5.1 Introduction

Une des applications majeures utilisant la transformée de Fourier Discrète (TFD) en TNS est l'analyse du contenu fréquentiel (spectre) de signaux continus. Le principe de l'analyse de Fourier numérique d'un signal analogique déterministe est représenté à la figure 5.1. La numérisation du signal continu $x_c(t)$ est effectuée de manière classique par un filtre anti-repliement suivi d'un convertisseur analogique-numérique. Le signal discret $x(n)$ est ensuite multiplié par une fonction $w(n)$ nommée *fenêtre temporelle* sur N points. Ce fenêtrage est une conséquence de la contrainte de durée finie imposée par la TFD qui ne peut s'effectuer sur un nombre infini d'échantillons. Le signal résultant $x_N(n)$, dont on pourra ensuite analyser le spectre par TFD (ou par TFR) puis par un calcul du module et éventuellement de la phase, est donc représentatif d'une observation limitée du signal d'entrée sur un horizon d'observation $T_0 = N.T$. L'effet de cette troncature temporelle, indispensable au calcul numérique de l'analyse spectrale, va être étudié dans les sections suivantes.

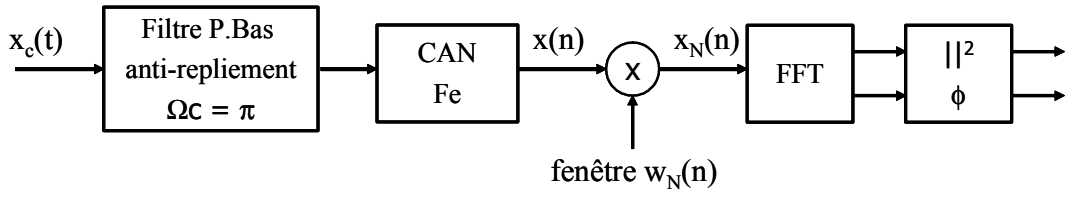


FIGURE 5.1 – Principe de l'analyse de Fourier numérique d'un signal analogique déterministe

5.2 Troncature d'un signal discrétisé

5.2.1 Opération dans le domaine temporel

Soit N le nombre d'échantillons manipulables par le calculateur numérique utilisé, le signal tronqué $x_N(n)$ est le résultat de la multiplication du signal discret $x(n)$ de durée infinie par une fenêtre $w(n)$ de durée $T_0 = NT$, appelée fenêtre d'observation.

La version tronquée du signal discrétisé s'écrit donc finalement :

$$x_N(n) = x(n) \cdot w(n) \tag{5.1}$$

La fenêtre la plus intuitive est une fenêtre sans pondération, c'est à dire une fenêtre rectangulaire notée $w_r(n)$ définie par :

$$w_r(n) = \begin{cases} 1 & \text{si } 0 \leq n \leq N-1 \\ 0 & \text{ailleurs} \end{cases} \tag{5.2}$$

La version tronquée du signal discrétisé s'écrit donc finalement dans le cas de la fenêtre rectangulaire :

$$x_N(n) = x(n) \cdot w_r(n) = \sum_{k=0}^{N-1} x(k) \cdot \delta(n-k) \tag{5.3}$$

5.2.2 Conséquences dans le domaine fréquentiel

La TF de (5.1) donne

$$\begin{aligned} X_N(e^{j\Omega}) &= X(e^{j\Omega}) * W(e^{j\Omega}) \\ &= \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{j\Theta}).W(e^{j(\Omega-\Theta)})\, d\Theta \end{aligned} \tag{5.4}$$

où $W(e^{j\Omega})$ est la représentation de la fenêtre dans le domaine fréquentiel.

Dans le cas de la fenêtre rectangulaire, on a :

$$W(e^{j\Omega}) = W_r(e^{j\Omega}) = e^{-j\Omega\frac{N-1}{2}} \frac{\sin(N\Omega/2)}{\sin(\Omega/2)} \tag{5.5}$$

Une représentation de la fenêtre $w_r(n)$ ainsi que du module de sa TF est donnée à la figure 5.2.

Exemple 5.2.1 : Calculer la TF $W_r(e^{j\Omega})$ de la fenêtre rectangulaire $w_r(n)$ et démontrez qu'elle s'écrit comme dans l'équation 5.5.

Les conséquences obtenues dans le domaine fréquentiel sont issues de la convolution spectrale illustrée par l'équation 5.4. On obtient donc :

- un lissage de la représentation spectrale qui implique une perte de finesse de l'analyse en fréquence et un masquage des raies trop proches en fréquence,
- des ondulations dans la réponse fréquentielle dues aux effets des lobes secondaires de $W(e^{j\Omega})$, entraînant du bruit et du masquage en amplitude.

5.3 Analyse spectrale par TFD

La troncature du signal sur N points n'est pas le seul effet d'approximation obtenu lors de l'analyse spectrale. En effet, l'équation 5.4 de convolution spectrale est une fonction de la variable continue Ω. Ce calcul n'est donc pas réalisable sur un processeur numérique et doit donc être approximé par la transformée de Fourier discrète (TFD, voir polycopié EII1).

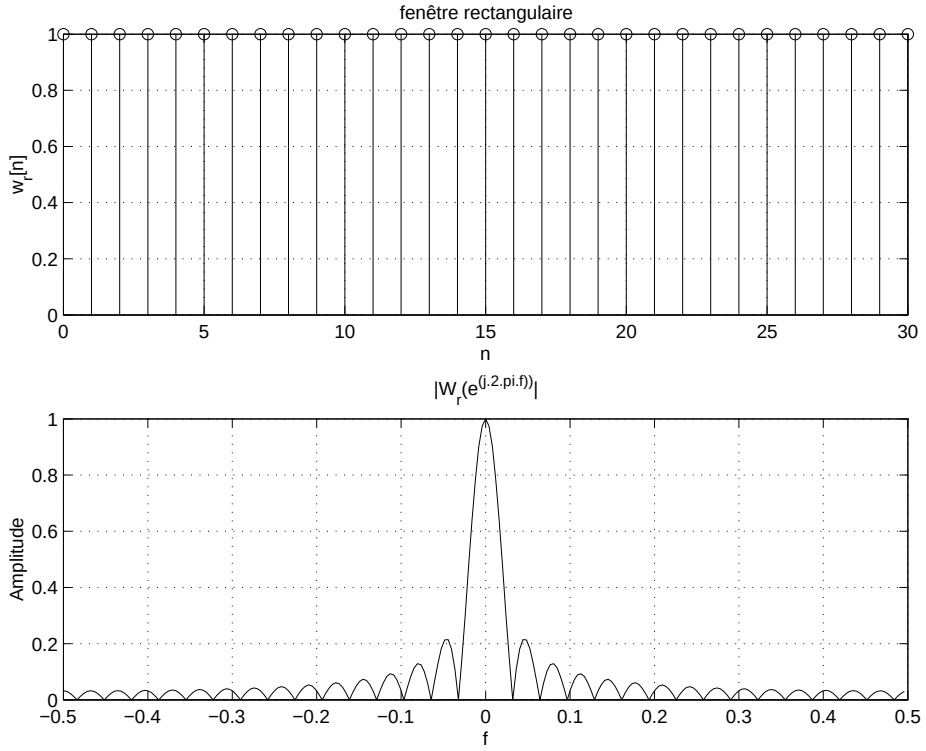


FIGURE 5.2 – Fenêtre rectangulaire de $N = 31$ points et le module de sa transformée de Fourier.

La dernière étape de l'analyse spectrale est donc le calcul de le TFD du signal fenêtré $x_N(n)$. On obtient alors :

$$X_N(k) = \sum_{n=0}^{L-1} x_N(n) e^{-j\frac{2\pi kn}{N}}, \qquad k = 0, 1, \ldots, L-1, \tag{5.6}$$

où la longueur de la fenêtre N est inférieure ou égale à la longueur de la TFD L. On prendra le plus souvent $L = N$, sauf dans le cas du *zero-padding* (voir section 5.4).

Comme illustrée dans le chapitre sur la TFD (voir polycopié EII1), la relation entre la TF $X_N(e^{j\Omega})$ de $x_N(n)$ (équation 5.4) et $X_N(k)$ s'écrit :

$$X_N(k) = X_N(e^{j\Omega})\big|_{\Omega=2\pi k/L} \tag{5.7}$$

L'espace entre deux points fréquentiels de l'analyse est donc de $2\pi/L$ et la relation entre les points d'indice k de la TFD et les fréquences continues f_k est donnée par :

$$\Omega_k = \frac{2\pi k}{L} \Leftrightarrow f_k = \frac{k}{LT} \tag{5.8}$$

Exemple 5.3.1 : Déterminez l'expression de la TFD, calculée avec une fenêtre rectangulaire ($N = 8$) du signal $x(n) = 2cos2\pi f_o n$ avec $f_o = 0,2$. Calculez la valeur numérique des $X(k)$ et dessinez à la main ce résultat.

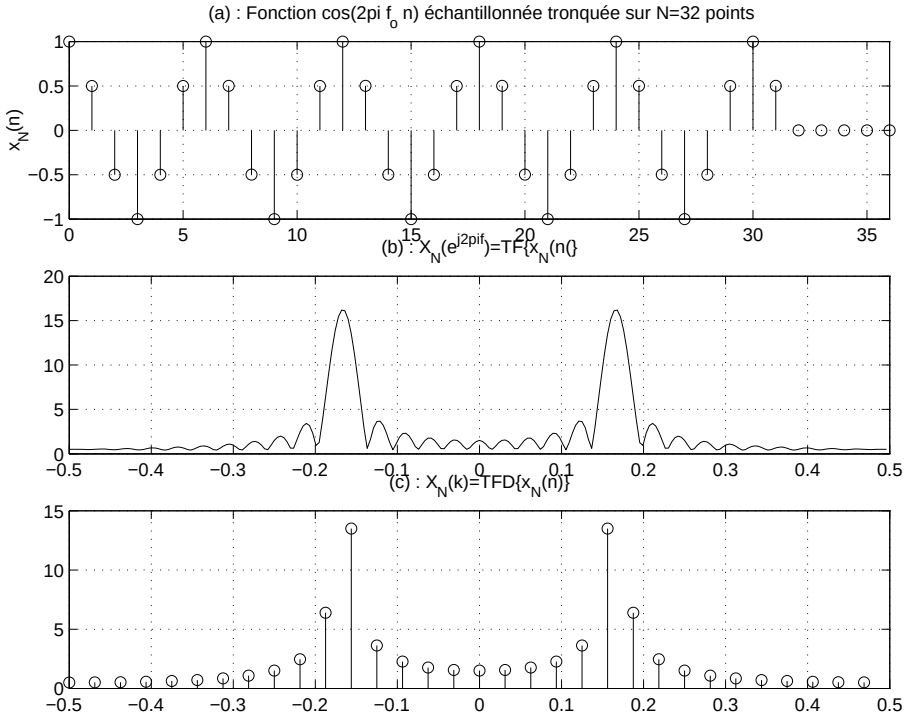


FIGURE 5.3 – Analyse de Fourier et TFD d'un signal sinusoïdal discret

La figure 5.3 illustre l'analyse spectrale par TFD sur un signal sinusoïdal discret. Le signal $x(t) = cos(2\pi f_0 t)$ est échantillonné puis tronqué sur $N = 32$ points par une fenêtre rectangulaire (voir figure 5.3(a)). La figure 5.3(b) représente la TF $X_N(e^{j\Omega})$ de $x_N(n)$ et illustre le problème de la troncature temporelle présenté dans la section précédente. En effet, la TF d'un cosinus étant composée de deux impulsions de Dirac située en f_0 et $-f_0$, alors la TF du signal fenêtré est composée de la somme des TF de la fenêtre décalées en f_0 et $-f_0$ *(ce que vous devez avoir trouvé si vous avez résolu l'exemple page 57).*
La figure 5.3(c) représente le résultat du calcul de la TFD sur $L = 32$ points du signal $x_N(n)$ et souligne donc l'erreur due à l'échantillonnage en fréquence effectué lors de la TFD. La TFD $X_N(k)$ est une version échantillonnée de $X_N(e^{j\Omega})$, dans laquelle l'espace entre deux points successifs représente Fe/L (ici $Fe/32$). Une erreur visible ici est donc que plusieurs raies spectrales peuvent être distinguées après TFD, pour uniquement deux raies effectives dans le spectre théorique de $x(t) = cos(2\pi f_0 t)$.

5.4 Zéro-Padding

Faire du *zéro-padding* consiste à augmenter artificiellement le nombre d'échantillons en ajoutant des zéros :

$$\{\, x(0)\,,\, x(1)\,,\, \ldots\,,\, x(N-1)\,,\, 0\,,\, 0\,,\, \ldots\,,\, 0\,\} \tag{5.9}$$

jusqu'à obtenir un nouveau nombre d'échantillons égal à L. Ainsi, si l'on calcule maintenant la TFD sur ces données, cela entraîne que les $X(k)$ seront calculés en L fréquences : $f_k = k/L$; elles seront toujours situées dans $[0\ ;\ 1[$, mais comme elles sont plus nombreuses, elles seront plus rapprochées. Cela aura pour vertu de **révéler plus de détails** invisibles sinon.

La figure 5.4 illustre par exemple quatre analyses de Fourier faites sur une fenêtre carrée de taille initiale : $N = 8$ points. Ces quatre analyses seront effectuées successivement à l'aide de N points, $2N$ points (donc les N échantillons de la fenêtre auxquels on ajoute N zéros), $4N$ points (donc il y a $3N$ zéros) et enfin $8N$ points.

5.5 Paramètres d'une analyse spectrale

Deux paramètres peuvent être affinés pour améliorer l'analyse :
- N, la longueur de l'analyse (ou T_0),
- le type de fenêtre $w_N(n)$ à appliquer sur le signal.

Cette section illustre ce résultat sur des signaux de type sinusoïdal.

Soit le signal discret constitué de trois raies

$$x(n) = \cos 2\pi f_1 n + \cos 2\pi f_2 n + 0,25 \cos 2\pi f_3 n \tag{5.10}$$

avec $f_1 = 0,175$, $f_2 = 0,2$ et $f_3 = 0,4$. Sa TF s'écrit immédiatement

$$\begin{aligned} X\left(e^{j2\pi f}\right) &= \frac{1}{2}\left[\delta(f+0,175) + \delta(f-0,175) + \delta(f+0,2) + \delta(f-0,2)\right. \\ &\quad \left. +0,25\,\delta(f+0,4) + 0,25\,\delta(f-0,4)\right] \end{aligned} \tag{5.11}$$

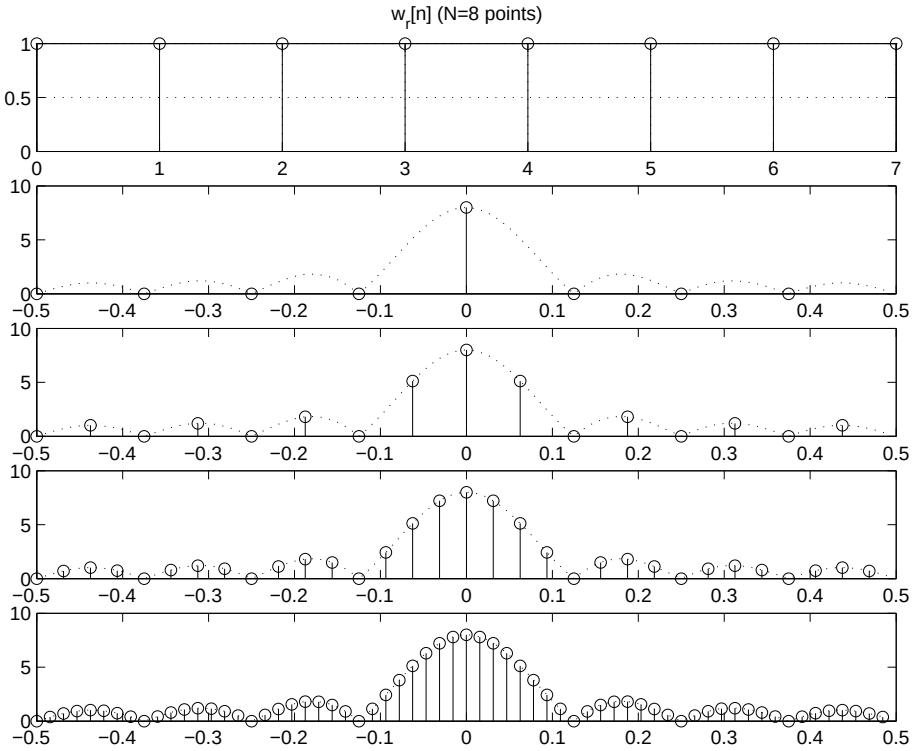


FIGURE 5.4 – Effet du *zéro-padding* sur l'analyse de Fourier d'une fenêtre rectangulaire de $N = 8$ points.

Le spectre $X_N\left(e^{j2\pi f}\right)$ de $x_N(n)$ utilisant une fenêtre rectangulaire est donc donné par

$$\begin{aligned} X_N\left(e^{j2\pi f}\right) &= \frac{1}{2}\Big[W_r(e^{j2\pi(f+0{,}175)} + W_r(e^{j2\pi(f-0{,}175)} + W_r(e^{j2\pi(f+0{,}2)} + W_r(e^{j2\pi(f-0{,}2)} \\ &\quad +0,25\,W_r(e^{j2\pi(f+0{,}4)} + 0,25\,W_r(e^{j2\pi(f-0{,}4)}\Big] \end{aligned} \tag{5.12}$$

Ce résultat est illustré à la Figure 5.5. Il est donc clair que d'une part la taille de la fenêtre influe sur l'aptitude de l'analyse de Fourier à séparer deux raies et que d'autre part les lobes secondaires influent nécessairement sur l'aptitude de l'analyse de Fourier à détecter des raies de faible amplitude plus ou moins éloignées d'une de forte amplitude.

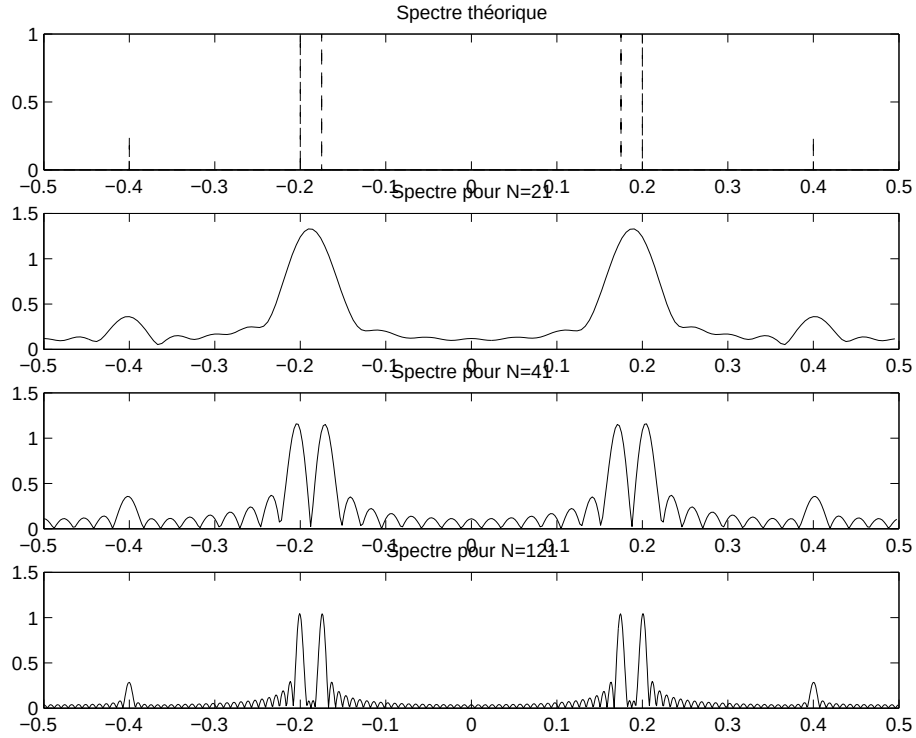


FIGURE 5.5 – Spectre (théorique puis utilisant des fenêtres rectangulaires de différentes tailles N) de trois sinusoïdes

La relation (5.4) montre clairement que la qualité du résultat de la TF, c'est-à-dire l'adéquation avec le résultat théorique que l'on aurait sans troncature, dépend du type de fenêtre utilisé. La section 3.3.1 du chapitre 3 page 36 a résumé les principales des fenêtres utilisées en TNS ainsi que leurs caractéristiques. Le tableau 5.1 résume ces caractéristiques. La figure 5.6 représente un exemple de TF d'une fonction $w(n)$ de fenêtrage et en définit ses paramètres :

- largeur du lobe principal : $\Delta\Omega_m$,
- rapport d'amplitude entre lobe principal et lobe secondaire : $\lambda = 20\log\frac{|W(\Omega_s)|}{|W(0)|}$, où Ω_s est la fréquence correspondant au maximum d'amplitude des lobes secondaires.

Dans une analyse spectrale, on définit deux type de résolution : la résolution en fréquence et la résolution en amplitude.

Type de fenêtre	Rapport d'amplitude entre lobe principal et lobe secondaire λ	Largeur du lobe principal $\Delta\Omega_m$
Rectangulaire	$-13dB$	$4\pi/N$
Bartlett	$-25dB$	$8\pi/N$
Hanning	$-31dB$	$8\pi/N$
Hamming	$-41dB$	$8\pi/N$
Blackman	$-57dB$	$12\pi/N$

TABLE 5.1 – Caractéristiques des principales fenêtres

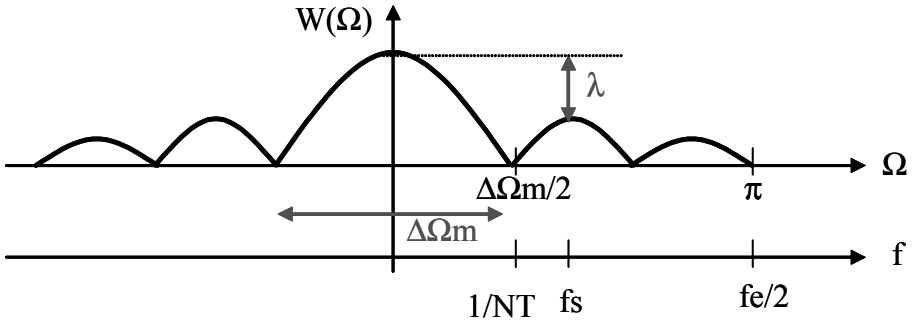


FIGURE 5.6 – Paramètres d'une fenêtre $w(n)$

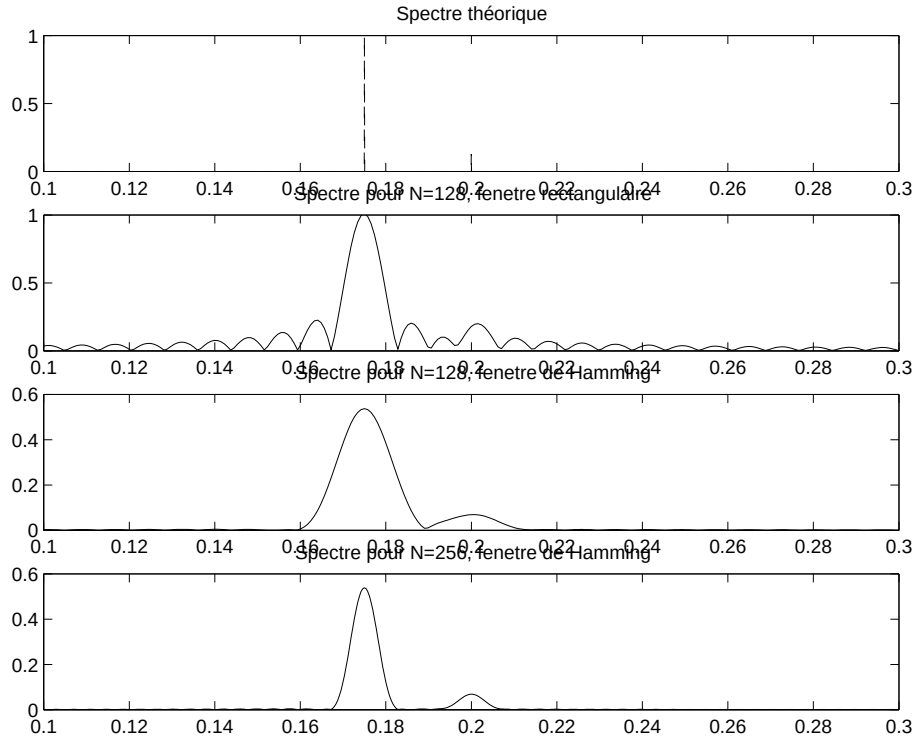


FIGURE 5.7 – Spectre (théorique puis utilisant des fenêtres rectangulaires et de Hamming de différentes taille N) de trois sinusoïdes

— **Résolution en fréquence** : la finesse en fréquence est la capacité de l'analyseur à distinguer deux raies spectrales proches l'une de l'autre. Clairement, ce phénomène, connu aussi sous le nom de masquage fréquentiel, dépend de la largeur du lobe principal de la réponse fréquentielle $W_N(e^{j\Omega})$ de la fenêtre ($\Delta\Omega_m$ du tableau 5.1). En effet, deux raies proches d'un espace $< \Delta\Omega_m$ se verront confondues après l'analyse par TFD. La figure 5.5 illustre ce phénomène puisque l'analyse pour $N = 21$ ne permet pas de distinguer les raies en $f = 0.2$ et $f = 0.175$, tandis que des valeurs de $N = 41$ ou $N = 121$ le permettent.

On définit donc la résolution en fréquence comme : $\Delta\Omega_m$.
Cette résolution peut être améliorée en augmentant le nombre de points N de l'analyse et dépend du type de fenêtre utilisée.

— **Résolution en amplitude** : la finesse en amplitude est la capacité de l'analyseur à distinguer des raies spectrales de faible amplitude ou à distinguer une raie spectrale de faible amplitude proche d'une autre plus importante. Dans cas, ce sont les lobes secondaires de de la réponse fréquentielle $W_N(e^{j\Omega})$ de la fenêtre qui viennent masquer une raie de faible amplitude en ajoutant un bruit à l'analyse. La figure 5.7 illustre ce phénomène puisque l'analyse pour $N = 128$ par fenêtre rectangulaire ne permet pas de distinguer la raie en $f = 0.2$ d'amplitude 0.25 parmi le bruit dû aux lobes secondaires, tandis qu'une fenêtre de Hamming et des valeurs de $N = 128$ ou $N = 256$ le permettent.

On définit donc la résolution en amplitude comme le rapport d'amplitude entre lobe principal et lobe secondaire : λ.
Cette résolution n'est pas fonction du nombre de points N de l'analyse mais dépend fortement du type de fenêtre utilisée.

5.6 Conclusion (méthodologie)

L'analyse spectrale de signaux déterministes requiert donc trois études préalables importantes :

1. le signal $x(t) \leftrightarrow X(f)$ doit être correctement **échantillonné** pour obtenir les échantillons $x(n)$ qui seront pris en compte par **bloc (segment) de N échantillons** ;
2. ces bloc seront **conditionnés par multiplication par une fenêtre, de taille N,$w(n)$**, à choisir en fonction des performances attendues :
   (a) la résolution en amplitude souhaitée $\lambda \Rightarrow$ choix du type de fenêtre,
   (b) la résolution en fréquence souhaitée $\Delta\Omega_m \Rightarrow$ choix de N ;
3. finalement les blocs fenêtrés, $x_N(n)$ sont transformés dans le domaine spectral par TFD (calculée par l'algorithme FFT). Le résultat (les $X_N(k)$) est une réplique correcte du vrai spectre $X(f)$ si les erreurs suivantes sont suffisamment faibles :
   (a) erreur de recouvrement de spectres (*Aliasing error* en anglais) (bien choisir la fréquence d'échantillonnage et utiliser un filtre passe-bas antirepliement) ;

(b) erreur due à la longueur de la fenêtre (plus elle est longue et moins cette erreur est importante, de plus le choix de la fenêtre permet de diminuer l'erreur due aux lobes secondaires) ;

(c) erreur de reconstruction du spectre (il faut utiliser le *zéro-padding* pour diminuer cette erreur).

On peut donc finalement représenter facilement l'analyse de Fourier numérique d'un signal analogique déterministe comme indiquée à la Figure 5.1.

Exemple 5.6.1 : Simulation avec Matlab Dans un script Matlab, rédigez un programme qui permet de :

1. générer et afficher à l'écran $N = 10$ points d'une sinusoïde $x_1(t) = a \cos 2\pi f_1 t$ de $f_1 = 100Hz$ échantillonée à $F_e = 1000Hz$ et d'amplitude $a = 2$;
   (a) effectuer une analyse de Fourier élémentaire (sans *zéro-padding*) de ce signal et afficher le résultat sur l'intervalle principal de fréquences normalisées $[-1/2\ ;\ +1/2[$;
   (b) refaire une analyse de Fourier de ce signal avec divers choix de *zéro-padding* ;
   (c) explorer l'influence de divers types de fenêtrages effectués sur le signal ;
2. regénérer et afficher à l'écran le même signal que précédemment mais cette fois avec $N = 15$ points ;
   (a) refaire la même étude que précédemment ;
   (b) ajouter une seconde sinusoïde et rendre compte de la pertinence de l'analyse effectuée en fonction de l'écart en fréquence des deux composantes élémentaire du signal.
   (c) considérer deux sinusoïdes proches à la limite de résolution constatée au point précédent et ajouter une troisième sinusoïde faible et éloignée des précédentes ; analysez alors l'influence de différentes fenêtres dans cette situation (résolution des deux sinusoïdes proches et détection de la sinusoïde faible éloignée).

Chapitre 6

Systèmes multi-cadences

Les systèmes de TNS utilisés jusqu'alors ne considèrent qu'une seule fréquence d'échantillonnage Fe. Les systèmes multi-cadences changent la fréquence d'échantillonnage au cours de la chaîne de traitement afin qu'elle soit la plus adaptée aux traitements à réaliser. Par exemple, il est aisé de comprendre qu'après un filtrage sélectif passe-bas il est possible de réduire la fréquence d'échantillonnage à une valeur équivalente à deux fois la bande passante du filtre passe-bas réalisé, sans pour autant transgresser le théorème d'échantillonnage de Shannon. Cette réduction permet de diminuer la complexité du filtrage à réaliser.

Les principaux opérateurs utilisés en traitement du signal multi-cadences sont la décimation (réduction d'un facteur M de la fréquence d'échantillonnage) et l'interpolation (augmentation d'un facteur M de la fréquence d'échantillonnage) ou une combinaison des deux.

6.1 Réduction de la fréquence d'échantillonnage

La réduction de la fréquence d'échantillonnage, ou décimation, ou encore sous-échantillonnage, par un facteur M entier est une opération simple puisqu'il suffit de ne garder que 1 échantillon sur M de la séquence d'origine. Soit un signal continu $x_c(t)$ échantillonné à la période T représenté par la séquence $x(n) = x_c(nT)$, on posera donc le résultat de la décimation de $x(n)$ d'un facteur M par la relation suivante :

$$x_d(n) = x(nM) \tag{6.1}$$

La notation graphique utilisée est $\downarrow M$ (voir figure 6.1). Si la fréquence d'échantillonnage en entrée du décimateur est Fe, alors celle de sortie vaudra $F'e = Fe/M$. La période d'échantillonnage en sortie vaudra quant à elle $T' = MT$.

Afin que l'opération de décimation s'effectue sans recouvrement de spectre, il est nécessaire que le spectre du signal $x_c(t)$ respecte le théorème de Shannon au regard de la fréquence $F'e$. Cela implique que si $X_c(j\omega)$, la TF de $x_c(n)$, est à bande limitée, i.e. $X_c(j\omega) = 0$ pour $|\omega| \leq \omega_0$, alors $x_d(n)$ est une représentation exacte de $x_c(t)$ si $\pi/T' = \pi/(MT) \geq \omega_0$ ou encore $2\pi F'e = 2\pi Fe/M \geq \omega_0$.

Dans le cas où le sous échantillonnage ne pourra s'effectuer sans recouvrement de spectre, il faudra alors en limiter la bande par un filtre passe-bas de fréquence de coupure $F'e/2$.

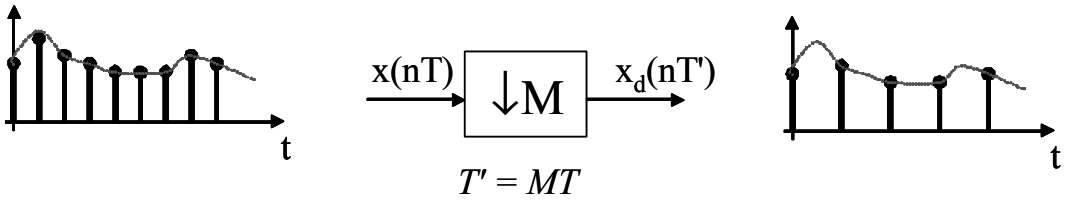


FIGURE 6.1 – Décimation d'un signal

Il est utile de chercher une relation entre le spectre de $x(n)$ et celui de $x_d(n)$. On peut poser :

$$X(e^{j\Omega}) = \frac{1}{T}\sum_{k=-\infty}^{\infty} X_c\left(j\left(\frac{\Omega}{T} - \frac{2\pi k}{T}\right)\right) \tag{6.2}$$

De manière similaire :

$$X_d(e^{j\Omega}) = \frac{1}{T'}\sum_{l=-\infty}^{\infty} X_c\left(j\left(\frac{\Omega}{T'} - \frac{2\pi l}{T'}\right)\right) = \frac{1}{MT}\sum_{l=-\infty}^{\infty} X_c\left(j\left(\frac{\Omega}{MT} - \frac{2\pi l}{MT}\right)\right) \tag{6.3}$$

Afin de trouver la relation entre les équations 6.2 et 6.3 les indices des sommes peuvent être exprimés par $l = i + kM$ où $0 \leq i \leq M-1$ et l'équation 6.3 peut être réécrite de la manière suivante :

$$X_d(e^{j\Omega}) = \frac{1}{M}\sum_{i=0}^{M-1}\left[\frac{1}{T}\sum_{k=-\infty}^{\infty} X_c\left(j\left(\frac{\Omega - 2\pi i}{MT} - \frac{2\pi k}{T}\right)\right)\right] \tag{6.4}$$

On obtient alors la relation

$$X_d(e^{j\Omega}) = \frac{1}{M}\sum_{i=0}^{M-1} X(e^{j(\Omega/M - 2\pi i/M)}) \tag{6.5}$$

Le décimation est illustrée à la figure 6.2. L'équation 6.5 montre que le spectre $X_d(e^{j\Omega})$ est composé de M copies de $X(e^{j\Omega})$ mises à l'échelle par un facteur $1/M$ et décalées par des entiers multiples de 2π.

Par conséquent, un décimateur sera en pratique composé d'un filtre passe-bas idéal de fréquence de coupure $F_c = 1/2T' = 1/2TM$ suivi d'un opérateur de décimation $\downarrow M$ comme représenté à la figure 6.3.

6.2 Augmentation de la fréquence d'échantillonnage

L'augmentation de la fréquence d'échantillonnage, ou interpolation, ou encore sur-échantillonnage, par un facteur L entier est une opération consistant à augmenter le nombre d'échantillons de la séquence d'origine d'un facteur L. Soit un signal continu $x_c(t)$ échantillonné à la période T représenté par la séquence

$$x(n) = x_c(nT), \tag{6.6}$$

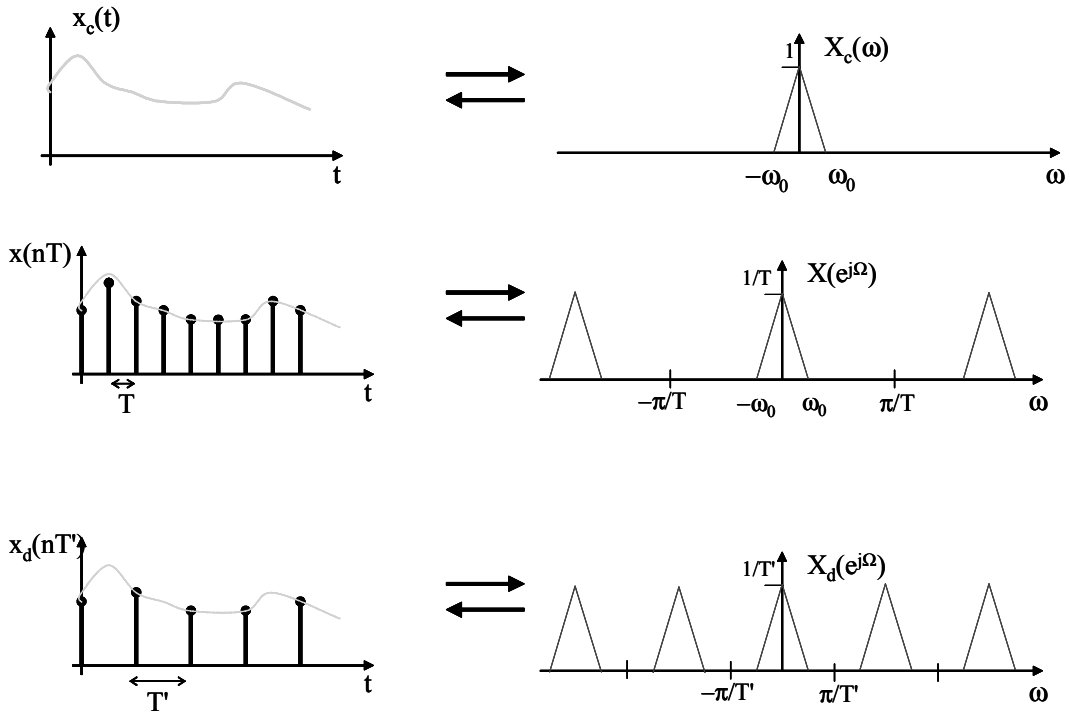


FIGURE 6.2 – Représentation d'un signal continu $x_c(t)$ et de son échantillonnage à 2 fréquences différentes et de leurs spectres respectifs

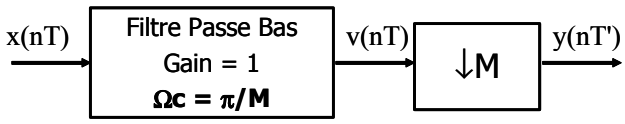


FIGURE 6.3 – Décimateur : filtrage passe-bas suivi d'une décimation

on posera donc le résultat de l'interpolation de $x(n)$ d'un facteur L par la relation suivante :

$$x_i(n) = x_c(nT') = x_c(nT/L), \tag{6.7}$$

avec $T' = T/L$ la période d'échantillonnage en sortie de l'interpolation ou $F'e = Fe.L$ la fréquence d'échantillonnage en sortie de l'interpolation.
À partir des équations 6.6 et 6.7, on peut déduire que :

$$x_i(n) = x_c(nT/L) = x(n/L), \qquad n = 0, \pm L, \pm 2L, \ldots \tag{6.8}$$

6.2.1 Élévateur de fréquence d'échantillonnage

On définit par élévateur de fréquence, le système décrit à la figure 6.4 consistant en l'ajout de $L-1$ zéros entre deux échantillons successifs de la séquence d'entré $x(n)$ défini par :

$$x_e(n) = \begin{cases} x(n/L), & n = 0, \pm L, \pm 2L, \ldots \\ 0, & ailleurs \end{cases} \tag{6.9}$$

ou de manière équivalente :

$$x_e(n) = \sum_{k=-\infty}^{\infty} x(k) \cdot \delta(n - kL) \tag{6.10}$$

La notation graphique utilisée est $\uparrow L$ (voir figure 6.4). Si la fréquence d'échantillonnage en entrée de l' élévateur de fréquence est Fe, alors celle de sortie vaudra $F'e = Fe.L$. La période d'échantillonnage en sortie vaudra quant à elle $T' = T/L$.

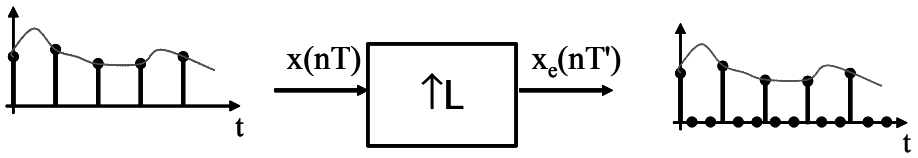


FIGURE 6.4 – Élévateur de fréquence d'un facteur L : ajout de $L-1$ zéros entre deux échantillons

L'opération $\uparrow L$ peut être étudiée dans le domaine fréquentiel afin de comprendre sont utilité dans le cadre de l'interpolation. La TF de $x_e(n)$ s'exprime :

$$X_e(e^{j\omega T'}) = \sum_{n=-\infty}^{\infty} x_e(nT')e^{-j\omega nT'} \tag{6.11}$$

$$= \sum_{n=0,\pm L,\pm 2L,\ldots} x\left(\frac{nT}{L}\right) e^{-j\omega nT'} \tag{6.12}$$

En posant $n = kL$, on obtient :

$$X_e(e^{j\omega T'}) = \sum_{k=-\infty}^{\infty} x(kT)e^{-j\omega kLT'} \quad (6.13)$$

$$= \sum_{k=-\infty}^{\infty} x(kT)e^{-j\omega kT} \quad (6.14)$$

$$= X(e^{j\omega T}) \quad (6.15)$$

L'ajout de zéros n'a donc aucun effet sur le spectre, si ce n'est l'écartement de l'intervalle de périodisation d'un facteur L. Ceci est illustré à la figure 6.5. On s'aperçoit alors qu'un filtrage passe-bas idéal de gain L et de fréquence de coupure $Fc = 1/2T$ permet de retrouver la forme du spectre correspondant à $x_c(t)$ échantillonné à T'.

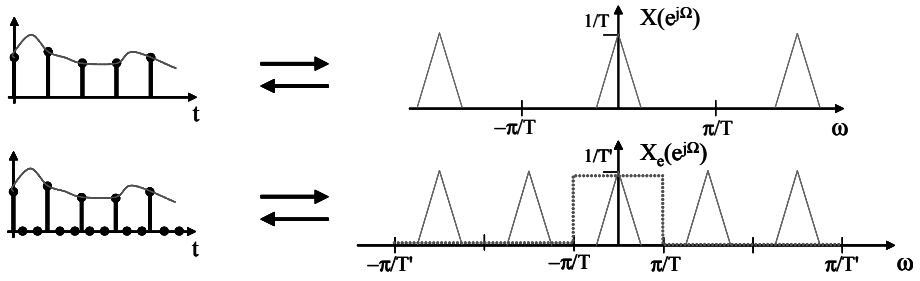


FIGURE 6.5 – Influence sur le spectre d'un signal interpolé par des zéros

6.2.2 Interpolation

Un interpolateur (voir figure 6.6) sera donc défini comme la succession d'un élévateur de fréquence $\uparrow L$, suivi d'un filtrage passe-bas idéal de gain L et de fréquence de coupure $Fc = 1/2T$ (ou $\Omega_c = \pi/L$).

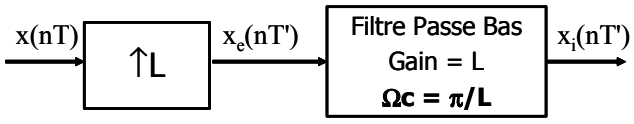


FIGURE 6.6 – Principe de l'interpolation d'un signal

D'autres systèmes d'interpolation sont connus (interpolation linéaire ou par TFD) mais ne seront pas détaillés ici.

6.2.3 Multiplication de la fréquence d'échantillonnage par un facteur rationnel

La figure 6.7 représente un schéma permettant de multiplier la fréquence d'échantillonnage d'un système par un facteur rationnel $R = L/M$. On aura alors $F'e = R.Fe = L.Fe/M$ ou

encore $T' = T/R = T.M/L$. Pour cela, il suffit d'effectuer tout d'abord une interpolation par un facteur $\uparrow L$, puis une décimation par un facteur $\downarrow M$. Les deux filtres présents dans la décimation (voir figure 6.3) et dans l'interpolation (voir figure 6.6) peuvent alors réunis dans un seul et même filtre dont la fréquence de coupure F_c dépendra des valeurs relatives de L et M selon :

— soit $R > 1 \Leftrightarrow M < L \Leftrightarrow F'e > Fe \quad \Rightarrow \quad F_c = \frac{1}{2T}$,
— soit $R < 1 \Leftrightarrow M > L \Leftrightarrow F'e < Fe \quad \Rightarrow \quad F_c = \frac{1}{2T'}$.

On peut également écrire :

$$F_c = min\left(\frac{1}{2T}, \frac{1}{2T'}\right) \tag{6.16}$$

ou encore

$$\Omega_c = min\left(\frac{\pi}{L}, \frac{\pi}{M}\right) \tag{6.17}$$

car le filtre passe-bas travaille à une fréquence d'échantillonnage $L.Fe$ (ou encore $M.F'e$).

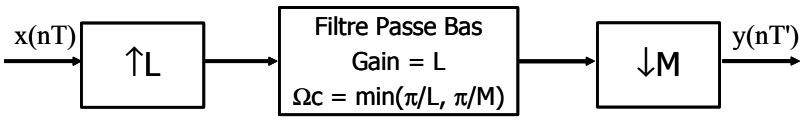


FIGURE 6.7 – Multiplication de la fréquence d'échantillonnage par un facteur rationnel $R = L/M$

Chapitre 7

Arithmétique virgule fixe

Alfred S. se rend auprès de la société chaotique de banque afin d'effectuer un placement à long terme. Le banquier lui propose le placement suivant : « Vous faites un placement initial de $e - 1$ euros ($e = 2.718281459045\ldots$). La première année on multiplie votre capital par 1 et on prélève 1€ de frais. La deuxième année on multiplie votre capital par 2 et on prélève 1€ de frais. La n-ième année on multiplie votre capital par n et on prélève 1€ de frais. Vous pouvez retirer l'argent après 25 ans. Intéressant, n'est-ce pas ? » La banque réalise une simulation sur son calculateur et obtient au bout de 25 ans +4645987753€. Alfred signe logiquement le contrat proposé. Rentré chez lui, il vérifie le calcul sur sa calculatrice de poche et trouve $\approx -140.10^{12}$€ ! En réalité un calcul symbolique donne un résultat d'environ 4 centimes. Cette anecdote prouve qu'un calcul peut être instable, même pour un petit nombre d'opérations (ici 25$\otimes$ et 25$\ominus$).

L'objectif de ce chapitre est de traiter de l'arithmétique utilisée pour l'implantation d'applications de traitement numérique du signal au sein de systèmes embarqués. Dans la première partie, les caractéristiques de l'arithmétique virgule fixe sont présentées. Dans la seconde partie, le processus de codage des données en virgule fixe est détaillé. L'évaluation de la précision d'une spécification virgule fixe est présentée dans la troisième partie. Deux exemples sont entièrement traités afin d'illustrer le processus de conversion en virgule fixe.

7.1 Présentation de l'arithmétique virgule fixe

7.1.1 Les différents types de codage

7.1.1.1 Rappels sur le codage d'un entier

Représentation non signée

Le codage d'un nombre x dans une base B correspond à la décomposition de ce nombre suivant les puissances successives de la base :

$$x = \sum_{i=0}^{n-1} a_i \cdot B^i,$$

valeur décimale A_{10}	code binaire pur $a_3a_2a_1a_0$
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
...	...
15	1 1 1 1

TABLE 7.1 – Représentation binaire non signée

a_i appartenant à l'ensemble des symboles avec $0 \leq a_i \leq B-1$.

x est écrit dans la base B (la notation x_B) sous la forme

$$a_{n-1}a_{n-2}\dots a_0.$$

Les puissances de la base B sont appelées les *poids* et les a_i les *coefficients*. a_{n-1} est le coefficient du poids fort et a_0 est le coefficient du poids faible. En base 2, les poids sont couramment appelés **bit** (Binary Digit).

Un entier positif non signé x est codé en binaire sur b bits par la relation suivante :

$$x = \sum_{i=0}^{b-1} b_i 2^i \tag{7.1}$$

Ce codage sur b bits, permet de coder tout les nombres compris entre 0 et $2^b - 1$.

Représentation signée : signe valeur absolue (SVA)

Pour cette représentation, la donnée x est composée d'un bit de signe S et de $b-1$ bits représentant le module de x. La valeur de cette donnée est la suivante :

$$x = (-1)^S \sum_{i=0}^{b-2} b_i 2^i \tag{7.2}$$

Ce codage a deux inconvénients. La valeur 0 a deux représentations $+0$ et -0 et les opérations arithmétiques ne sont pas faciles à réaliser à partir du cascadage des opérations sur 1 bit. Il est nécessaire d'avoir deux opérateurs différents pour faire une addition suivant le signe relatif des deux opérandes :

- si les opérandes sont de **même signe** : il faut ajouter les valeurs absolues et garder le signe
- si les opérandes sont de **signe différent** : il faut faire une soustraction. Ainsi, les deux opérations à réaliser en fonction du signe sont différentes

Représentation signée en complément à 2 (CA2)

Le complément à deux est le cas spécial du complément à la base B. Pour les nombres positifs le codage binaire pur est conservé. si $A \geq 0$ et alors A est représenté par $a_{n-1}a_{n-2}\dots a_0$ avec :

$$A = \sum_{i=0}^{n-1} a_i 2^i$$

Pour l'opposé de A un codage facilitant les opérations arithmétiques est recherché. Soit A', le nombre codé par $\overline{a_{n-1}}\ \overline{a_{n-2}}\dots\overline{a_0}$ et correspondant au *complément à 1* de A codé $a_{n-1}\,a_{n-2}\dots a_0$. L'addition de ces deux nombres permet d'obtenir la valeur $2^n - 1$:

$$\begin{array}{cccccc} & a_{n-1} & a_{n-2} & \dots & a_0 & \\ + & \overline{a_{n-1}} & \overline{a_{n-2}} & \dots & \overline{a_0} & \\ \hline & 1 & 1 & \dots & 1 & \longrightarrow 2^n - 1 \end{array}$$

Soit A'', le nombre correspondant au *complément à 2 de* A et défini tel que $A'' = A' + 1$. L'addition d'un nombre A et de son complément à 2, A'', permet d'obtenir la valeur 2^n :

$$A + A'' = (A + A') + 1 = (2^n - 1) + 1 = 2^n$$

En limitant le codage à n bits, la valeur est obtenue modulo 2^n. Le bit $n+1$ n'est pas pris en compte et donc $2^n = 0$.

$$\left(A + A''\right)_n = 0$$

Ainsi le complément à 2, A'', est choisi comme opposé de A puisque $A + A'' = 0$ sur n bits (modulo 2^n).

La représentation en code complément à 2 de la donnée x est égale à :

$$x = -2^{b-1}S + \sum_{i=0}^{b-2} b_i 2^i \tag{7.3}$$

On écrira également que si $x > 0$ alors $-x = /x + 1$, où $/x$ est le complément de x.

Le domaine de définition $\mathcal{D}$ correspond à l'intervalle regroupant l'ensemble des valeurs représentables par le code. Les bornes minimales et maximales de cet intervalle sont respectivement X_{min} et X_{max}. Le domaine de définition de ce code n'est pas symétrique par rapport à l'origine, il est composé de $2^{b-1} - 1$ valeurs positives et de 2^{b-1} valeurs négatives :

$$\mathcal{D} = [X_{min}; X_{max}] = [-2^{b-1}; 2^{b-1} - 1] \tag{7.4}$$

La représentation en code complément à 2 est très utilisée car elle possède des propriétés arithmétiques très intéressantes pour l'addition et la soustraction. Le résultat d'une série d'additions sera correct même si les résultats intermédiaires sont en dehors du domaine de définition du codage, il suffit que le résultat final appartienne au domaine de définition. De plus l'implantation dans les processeurs numériques des opérateurs traditionnels utilisant ce code est plus simple car elle nécessite un opérateur unique d'addition et de soustraction que

les données d'entrée soient positives ou négatives.

Un nombre réel pourra être représenté par la multiplication d'un nombre entier par un coefficient $q < 1$. On parlera alors de virgule fixe.

7.1.1.2 Codage virgule fixe

Les données en virgule fixe sont composées d'une partie fractionnaire et d'une partie entière pour lesquelles le nombre de bits alloués reste figé au cours du traitement. L'exposant associé à chaque donnée est implicite et fixe. La figure 7.1 représente une donnée en virgule fixe composée d'un bit de signe et de $b-1$ bits répartis en m bits pour la partie entière et n bits pour la partie fractionnaire. Nous utilisons dans la suite du document la notation (b, m, n) pour définir le format d'une donnée. Nous trouvons aussi dans la littérature la notation $Q_{m.n}$.

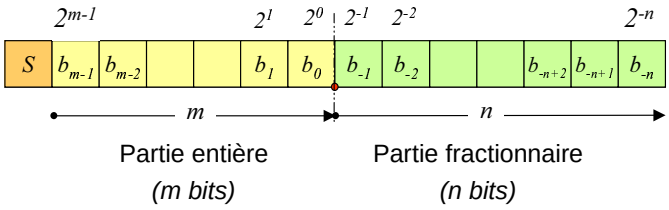


FIGURE 7.1 – Représentation des données en virgule fixe

Le format d'une donnée en virgule fixe est entièrement défini par la longueur de sa partie entière et de sa partie fractionnaire et de la représentation choisie. Nous présentons dans les parties suivantes les propriétés des représentations signe valeur absolue et complément à deux [Ka91].

Représentation signe valeur absolue (SVA)

Pour cette représentation, la donnée x est composée d'un bit de signe S et de $b-1$ bits représentant le module de x. La valeur de cette donnée est la suivante :

$$x = (-1)^S \sum_{i=-n}^{m-1} b_i 2^i \tag{7.5}$$

Ce type de représentation possède deux représentations de la valeur zéro ($10\ldots0$ et $00\ldots0$), ainsi le nombre de valeurs représentables N_c est égal à $2^b - 1$.

Le domaine de définition $\mathcal{D}$ correspond à l'intervalle regroupant l'ensemble des valeurs représentables par le code. Dans le cas d'une représentation signe valeur absolue nous obtenons un domaine de définition symétrique par rapport à l'origine :

$$\mathcal{D}_R = [-2^m + 2^{-n}; 2^m - 2^{-n}] \tag{7.6}$$

Représentation	**cadrage à gauche**	**cadrage à droite**
conditions	$m = 0$ $n = b - 1$	$n = 0$ $m = b - 1$
q	$2^{-(b-1)}$	1
$\mathcal{D}$	$[-1 + q; 1 - q]$	$[-2^{b-1} + q; 2^{b-1} - q]$

TABLE 7.2 – Cas particuliers de la représentation signe valeur absolue

La dynamique d'un code représente la différence entre la valeur minimale et maximale. Pour la représentation SVA, la dynamique est égale à :

$$D = X_{max} - X_{min} = 2.(2^m - 2^{-n}) \tag{7.7}$$

Le pas de quantification, correspondant à la distance q entre deux valeurs successives, est fonction de la dynamique D et du nombre de valeurs représentables N_c :

$$q = \frac{D}{N_c - 1} = 2^{-n} \tag{7.8}$$

Le niveau de dynamique correspond au rapport entre les valeurs absolues maximales et minimales représentables par le code. L'expression du niveau de dynamique exprimé en dB, est la suivante :

$$N_{D\ dB} = 20 \log \left(\frac{\max(|x|)}{\min(|x|)} \right) \simeq 20.b. \log(2) = 6,02.b \tag{7.9}$$

Deux représentations particulières liées à la position de la virgule sont couramment utilisées. Lorsque la virgule est cadrée à droite la valeur codée est entière et lorsque celle-ci est cadrée à gauche la donnée est fractionnaire. Les caractéristiques de ces deux représentations sont présentées dans le tableau 7.2. Plusieurs exemples de codage SVA sont présents dans le tableau 7.4.

Représentation en complément à 2 (CA2)

La représentation en code complément à 2 de la donnée x en virgule fixe est égale à :

$$x = -2^m S + \sum_{i=-n}^{m-1} b_i 2^i \tag{7.10}$$

Ce code a l'avantage de ne posséder qu'une seule représentation de la valeur zéro. Le domaine de définition de ce code n'est pas symétrique par rapport à l'origine, il est composé de $2^{b-1} - 1$ valeurs positives et de 2^{b-1} valeurs négatives :

$$\mathcal{D} = [-2^m; 2^m - 2^{-n}] \tag{7.11}$$

Représentation	**cadrage à gauche**	**cadrage à droite**	**cadrage à n**
condition	$m = 0$	$n = 0$	$n + m = b - 1$
q	$2^{-(b-1)}$	1	$2^{-(n)}$
$\mathcal{D}$	$[-1; 1-q]$	$[-2^{b-1}; 2^{b-1}-q]$	$[-2^m; 2^m-q]$

TABLE 7.3 – Cas particuliers de la représentation complément à 2

Valeur			**Représentation**	
cadrage à gauche	cadrage à droite	$m = 3 \quad n = 2$	C.A.2	S.V.A.
0.96875	31	7.75	011111	011111
0.9375	30	7.5	011110	011110
⋮	⋮	⋮	⋮	⋮
0.3125	1	0.25	000001	000001
0	0	0	000000	000000
0	0	0		100000
-0.3125	-1	-0.25	111111	100001
⋮	⋮	⋮	⋮	⋮
-0.9375	-30	-7.5	100010	111110
-0.96875	-31	-7.75	100001	111111
-1	-32	-8	100000	...

TABLE 7.4 – Exemples de codage

Le pas de quantification est identique à celui de la représentation précédente : $q = 2^{-n}$.

Les caractéristiques des représentations cadrées à gauche et cadrées à droite sont présentées dans le tableau 7.3. Plusieurs exemples de codage en CA2 sont présents dans le tableau 7.4.

7.1.1.3 Codage virgule flottante

Le codage des données en virgule flottante a été normalisée par l'IEEE. Dans cette partie, la norme IEEE 754 est plus particulièrement décrite. Pour cette norme, représentée à la figure 7.2, les données sont composées d'un bit de signe S, d'un exposant u et d'une mantisse.
L'exposant u permet d'obtenir un facteur d'échelle explicite et variable au cours du traitement, celui-ci est une puissance de 2. Ce facteur permet de s'adapter à la valeur à représenter. Afin de ne coder que des nombres positifs, cet exposant sur E bits est codé relativement à la valeur $2^{E-1} - 1$. La mantisse représente la valeur de la donnée divisée par le facteur d'échelle. Afin d'éviter toute ambiguïté, le premier bit de la mantisse représente le coefficient 2^0 et est fixé à 1. La valeur de ce bit restant fixe au cours du traitement, celui-ci n'est pas représenté dans le code. La mantisse est comprise dans l'intervalle $[1; 2[$
La valeur de la donnée x est la suivante :

$$x = (-1)^S.2^u.\Big(1 + \sum_{i=0}^{M-1} C_i 2^{i-M}\Big) \quad \text{avec} \quad u = \sum_{i=0}^{E-1} d_i 2^i - \big(2^{E-1} - 1\big) \tag{7.12}$$

D'après l'équation 7.12 la valeur 0 n'est pas représentable. Les valeurs $00\ldots00_b$ et $11\ldots11_b$

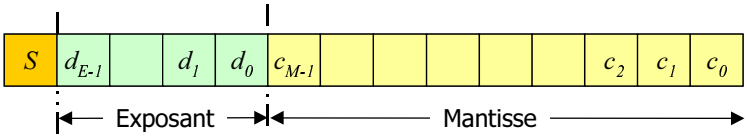


FIGURE 7.2 – Representation des données en virgule flottante

Type de représentation	WL (bits)	M (bits)	E (bits)
simple précision (`float`)	32	23	8
double précision (`double`)	64	52	11
double précision étendue (`long double`)	80	64	15

TABLE 7.5 – Les différents types associés à la norme IEEE 754. Les termes WL, M, et E représentent respectivement la largeur de la donnée, de la mantisse et de l'exposant.

sont réservées pour coder des valeurs particulières (0, ∞). Le domaine de définition est composé des deux sous-intervalles suivants :

$$\mathcal{D}_R = \left[-2^{K_{max}}.(2-2^{-M}); -2^{K_{min}} \right] \bigcup \left[2^{K_{min}}; 2^{K_{max}}.(2-2^{-M}) \right] \tag{7.13}$$

avec

$$\begin{cases} K_{min} = -2^{E-1}+2 \\ K_{max} = 2^{E-1}-1 \end{cases} \tag{7.14}$$

Le pas de quantification est fonction de la valeur représentée. Pour les valeurs de x comprises dans l'intervalle $[-2^{u+1}, -2^u] \bigcup [2^u, 2^{u+1}]$, le pas de quantification est égal à :

$$q = 2^u.2^{-M} \tag{7.15}$$

L'expression 7.16 détermine les bornes minimales et maximales du pas de quantification relatif. Nous pouvons considérer qu'il est pratiquement constant pour l'ensemble des valeurs de x.

$$2^{-(M+1)} < \frac{q}{|\,x\,|} < 2^{-M} \tag{7.16}$$

Le niveau de dynamique de cette représentation est égal à :

$$N_D = 20\log(2^K) \quad \text{avec} \quad K = 2^E - 1 \tag{7.17}$$

Les différents types associés à cette norme IEEE 754 sont présentés dans le tableau 7.5.

7.1.1.4 Comparaison virgule fixe et flottante

Niveau de dynamique

Le niveau de dynamique exprimé en dB est représenté ,à la figure 7.3, en fonction du nombre de bits utilisés pour un codage en virgule fixe et en virgule flottante. Nous avons fixé pour le codage en virgule flottante la taille de l'exposant à 1/4 de la longueur totale.

Lorsque le nombre de bits est inférieur à 16, le niveau de dynamique obtenu avec une représentation en virgule fixe est supérieur à celui d'une représentation en virgule flottante. Cette tendance s'inverse pour un nombre de bits supérieur à 16. Pour N=32, la représentation en virgule flottante montre tout son intérêt, la dynamique disponible permet d'utiliser ce codage dans la majorité des applications sans risque de débordements.

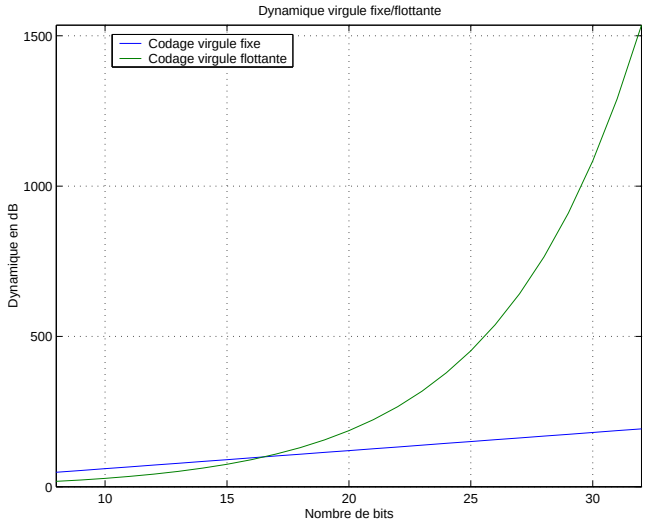


FIGURE 7.3 – Dynamique du codage en virgule fixe et en virgule flottante

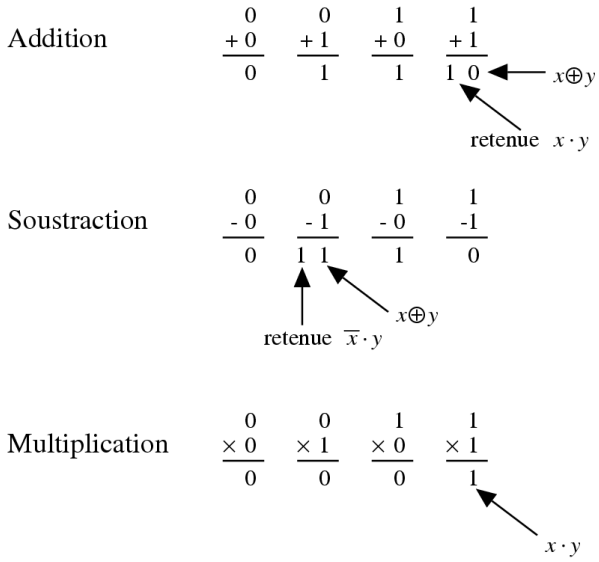
7.1.2 Définition des règles de l'arithmétique virgule fixe

Nous considérons dans la suite de ce document que les données sont codées en virgule fixe avec une représentation en CA2.

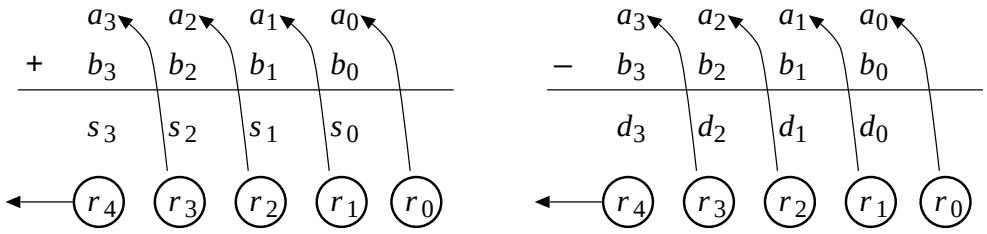
7.1.2.1 Opérations arithmétiques

Opérations arithmétiques sur un bit Les opérations arithmétiques entre 2 opérandes d'un bit sont facilement réalisables à base de portes logiques :

- l'addition est un "OU EXCLUSIF", la retenue pour l'addition un "ET"
- la soustraction est un "OU EXCLUSIF", la retenue un "ET"
- la multiplication est un "ET".



Addition soustraction de nombres non signés Les opérations sur 2 bits avec cascadage des retenues sont utilisées.



Si la retenue du dernier étage est à 1, un dépassement de capacité est présent entrainant un résultat erroné et signifiant que

— la somme des deux nombres est supérieure à 2^{n-1},

— le résultat de la soustraction est négatif

Addition et soustraction de nombres en complément à 2 L'intérêt de la notation des nombres négatifs en complément à 2 est que l'on n'a pas à se préoccuper du signe de A et B pour réaliser l'opération $A + B$ ou $A - B$. On réalise une addition des deux codes par cascadage de retenue et on garde le résultat sur n bits.

14	0000 1110	- 14	1111 0010	-14	1111 0010
+13	0000 1101	+13	0000 1101	-13	1111 0011
+27	0001 1011	-1	1111 1111	-27	1110 0101

Les problèmes de débordement sont plus difficiles à détecter. On le voit quand le signe du résultat est non conforme à l'algèbre.

L'addition des nombres réels en virgule fixe est identique aux nombres entiers (absence de notion de virgule dans les systèmes numériques)

7.1.3 Multiplication en binaire à base de décalage

La multiplication de deux nombres non signés est représentée à la figure 7.4.

				a_3	a_2	a_1	a_0
			$\times$	b_3	b_2	b_1	b_0
				b_0a_3	b_0a_2	b_0a_1	b_0a_0
			b_1a_3	b_1a_2	b_1a_1	b_1a_0	0
		b_2a_3	b_2a_2	b_2a_1	b_2a_0	0	0
	b_3a_3	b_3a_2	b_3a_1	b_3a_0	0	0	0
c_7	c_6	c_5	c_4	c_3	c_2	c_1	c_0

FIGURE 7.4 – Multiplication de nombres non signés

La multiplication de deux nombres signés est représentée à la figure 7.5.

				a_3	a_2	a_1	a_0
			$\times$	b_3	b_2	b_1	b_0
				$-b_0a_3$	b_0a_2	b_0a_1	b_0a_0
			$-b_1a_3$	b_1a_2	b_1a_1	b_1a_0	0
		$-b_2a_3$	b_2a_2	b_2a_1	b_2a_0	0	0
	b_3a_3	$-b_3a_2$	$-b_3a_1$	$-b_3a_0$	0	0	0
c_7	c_6	c_5	c_4	c_3	c_2	c_1	c_0

FIGURE 7.5 – Multiplication de nombres signés

Pour éviter d'avoir à faire des soustractions, le bit de signe des deux opérandes sur N bits est étendu sur $2N$ bits comme représenté à la figure 7.6.

	$\mathbf{a_3}$	$\mathbf{a_3}$	$\mathbf{a_3}$	$\mathbf{a_3}$	a_3	a_2	a_1	a_0
$\times$	$\mathbf{b_3}$	$\mathbf{b_3}$	$\mathbf{b_3}$	$\mathbf{b_3}$	b_3	b_2	b_1	b_0
	$b_0\mathbf{a_3}$	$b_0\mathbf{a_3}$	$b_0\mathbf{a_3}$	$b_0\mathbf{a_3}$	b_0a_3	b_0a_2	b_0a_1	b_0a_0
	$b_1\mathbf{a_3}$	$b_1\mathbf{a_3}$	$b_1\mathbf{a_3}$	b_1a_3	b_1a_2	b_1a_1	b_1a_0	0
	$b_2\mathbf{a_3}$	$b_2\mathbf{a_3}$	b_2a_3	b_2a_2	b_2a_1	b_2a_0	0	0
	$b_3\mathbf{a_3}$	b_3a_3	b_3a_2	b_3a_1	b_3a_0	0	0	0
	$\mathbf{b_3}a_3$	$\mathbf{b_3}a_2$	$\mathbf{b_3}a_1$	$\mathbf{b_3}a_0$	0	0	0	0
	$\mathbf{b_3}a_2$	$\mathbf{b_3}a_1$	$\mathbf{b_3}a_0$	0	0	0	0	0
	$\mathbf{b_3}a_1$	$\mathbf{b_3}a_0$	0	0	0	0	0	0
	$\mathbf{b_3}a_0$	0	0	0	0	0	0	0
	c_7	c_6	c_5	c_4	c_3	c_2	c_1	c_0

FIGURE 7.6 – Multiplication de nombres signés en utilisant l'extension de signe

7.1.3.1 Addition virgule fixe

L'addition de deux opérandes a et b nécessite qu'elles possèdent un format commun. Le type de représentation, la longueur de la partie entière et la longueur de la partie fractionnaire

doivent être identiques pour les deux opérandes. Si cette condition n'est pas respectée il est nécessaire de modifier le format des opérandes afin d'obtenir un format identique (b_c, m_c, n_c). Le format commun garantissant l'absence de perte d'information est le suivant :

$$\begin{aligned} m_c &= \max(m_a, m_b) \\ n_c &= \max(n_a, n_b) \\ b_c &= m_c + n_c + 1 \end{aligned} \tag{7.18}$$

Pour les données ayant un format différent du format commun, il est nécessaire d'étendre le nombre de bits des parties entières et fractionnaires en suivant les règles suivantes :

- *partie fractionnaire :* les $(n_c - n_a)$ bits supplémentaires sont mis à 0.
- *partie entière :* **extension du bit de signe**[1]. Dans le cas du complément à 2 les $(m_c - m_a)$ nouveaux bits prennent la valeur du bit de signe.

Le format du résultat de l'addition de deux opérandes au format (b_c, m_c, n_c) est présenté à l'expression 7.19. Nous obtenons un **débordement** si le résultat de l'addition des deux opérandes n'appartient pas au domaine de définition $\mathcal{D}_c = [-2^{m_c}; 2^{m_c}[$. Dans ce cas un bit supplémentaire est nécessaire pour coder la partie entière du résultat de l'addition. Deux exemples d'addition sont présentés à la figure 7.7.

$$n_{Add} = n_c \tag{7.19}$$

$$m_{Add} = \begin{cases} m_c + 1 & \text{si } a + b \notin \mathcal{D}_c \\ m_c & \text{si } a + b \in \mathcal{D}_c \end{cases} \tag{7.20}$$

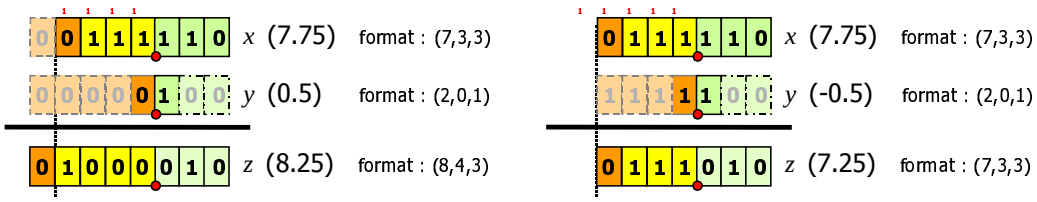


FIGURE 7.7 – Exemples d'addition en virgule fixe

7.1.3.2 Multiplication

Pour une multiplication, les deux opérandes doivent posséder la même représentation mais le nombre de bits réservés pour chaque partie peut être différent. Néanmoins, il est nécessaire avant d'effectuer l'opération, d'étendre le bit de signe. La multiplication de deux nombres en virgule fixe entraîne le doublement du bit de signe, celui-ci peut être éliminé automatiquement à l'aide d'un décalage à gauche. Pour un code en complément à 2 nous pouvons considérer que ce bit de signe redondant appartient à la partie entière. Le format du résultat de la multiplication de deux opérandes a et b est alors le suivant :

1. L'extension de signe dans le cas d'une représentation signe valeur absolue est plus complexe, il faut décaler le bit de signe à la position b_{m_c-1} et mettre à zéro les bits b_{m_c-2} à b_{m_a-1}.

$$\begin{aligned} m_{Mult} &= m_a + m_b + 1 \\ n_{Mult} &= n_a + n_b \\ b_{Mult} &= b_a + b_b \end{aligned} \tag{7.21}$$

Un exemple de multiplication est présenté à la figure 7.8.

Dans le cas de la virgule fixe cadrée à gauche, le résultat de la multiplication de deux opérandes a et b appartenant à l'intervalle $[-1;1[$ reste dans le même intervalle. Il n'y a donc pas de débordement mais juste une augmentation de la précision du résultat. Cette propriété est très intéressante car elle élimine le problème du débordement.

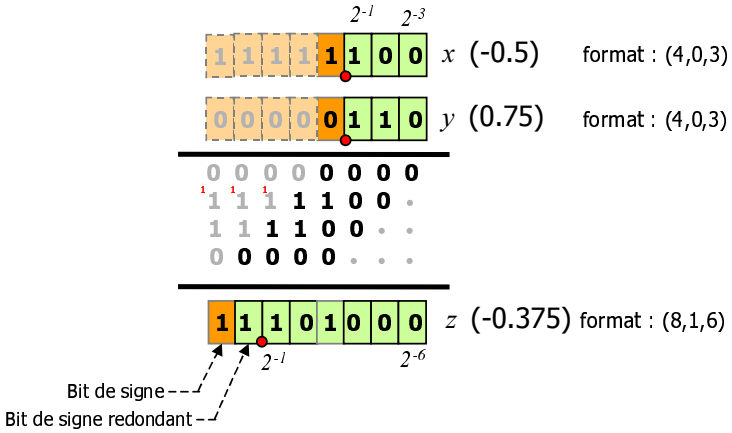


FIGURE 7.8 – Exemple de multiplication en virgule fixe

Codage virgule fixe en complément à 2

— Définition :

$$x = -2^m S + \sum_{i=-n}^{m-1} b_i 2^i$$

— Domaine de définition : $\mathcal{D} = [-2^m; 2^m - 2^{-n}]$
— Pas de quantification : $q = 2^{-n}$

7.2 Codage en virgule fixe

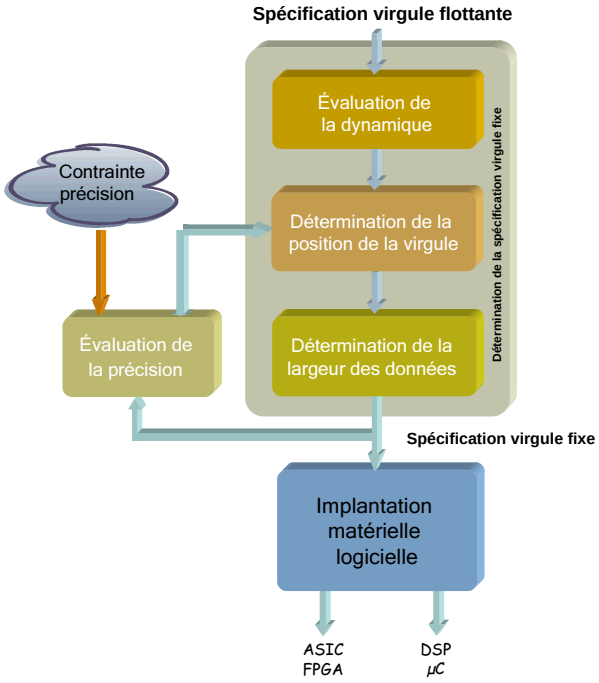


FIGURE 7.9 – Synoptique du processus d'implantation d'une application en virgule fixe

Le synoptique du processus d'implantation d'une application en virgule fixe est présenté à la figure 7.9. La spécification de l'application en virgule fixe est déterminée à partir d'une représentation de l'application sous la forme d'un graphe flot de signal. Ce graphe représente les différents traitements effectués sur le signal. Les noeuds de ce graphe sont les opérations arithmétiques (addition, soustraction, multiplication, division) et les opérations de retard. Avant de débuter le processus de conversion en virgule fixe, le graphe flot de signal représentant l'application doit être défini. Ainsi, dans le cadre des filtres numériques, la structure d'implantation (cascade, parallèle, directe, canonique ou transposée) doit être sélectionnée.

Trois étapes sont nécessaires pour réaliser le codage en virgule fixe. Elles correspondent à l'évaluation de la dynamique des données, à la détermination de la position de la virgule (définition de la largeur de la partie entière) et à la détermination de la largeur des données (définition de la largeur de la partie fractionnaire). Ensuite, la précision des calculs est évaluée afin de vérifier que la spécification virgule fixe respecte la contrainte de précision. Si cette contrainte n'est pas vérifiée alors, cette spécification virgule fixe est modifiée. L'objectif de ce processus de conversion est d'optimiser l'implantation de l'application en s'assurant que l'utilisation de l'arithmétique virgule fixe ne dégrade pas trop les performances de l'application. Les différentes étapes du processus de conversion en virgule fixe sont présentées dans les parties suivantes.

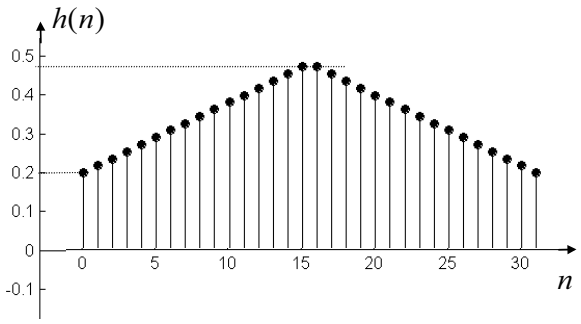


FIGURE 7.10 – Réponse impulsionnelle du filtre RIF considéré

Exemple filtre RIF

Pour illustrer ce processus de conversion, l'exemple d'un filtre à réponse impusionnelle finie (RIF) de taille $N = 32$, est traité. La réponse impulsionnelle de ce filtre est définie ci-dessous et représentée à la figure 7.10 :

$$h(n) = \sum_{i=0}^{N-1} c_i.\delta(n-i) \tag{7.22}$$

avec

$$c_i = \frac{1}{5} + \frac{3.i}{160} \quad \forall\ i \in [0; \frac{N}{2} - 1] \tag{7.23}$$

nous utilisons un filtre RIF symétrique, ainsi nous obtenons

$$c_i = c_{N-1-i} \quad \forall\ i \in [\frac{N}{2}; N-1] \tag{7.24}$$

Le graphe flot de signal de cette application est représenté à la figure 7.11.

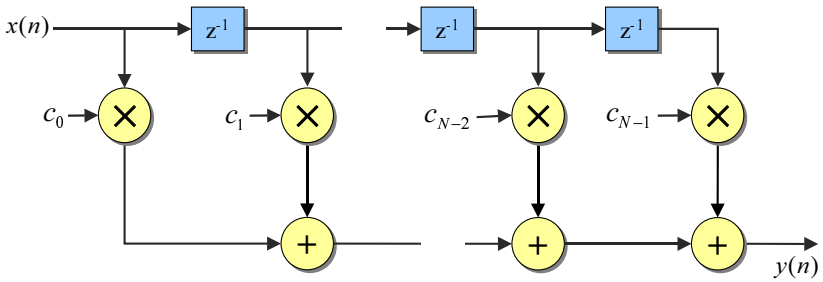


FIGURE 7.11 – Graphe flot de signal du filtre RIF

7.2.1 Détermination de la dynamique des données

Le but de cette première phase est de déterminer le domaine de définition de certaines données de l'application afin d'en déduire la position de la virgule par rapport au bit le plus significatif. Cette position est définie afin de garantir l'absence de débordement.

Deux types d'approche peuvent être utilisés pour déterminer la valeur absolue maximale d'une donnée. Pour la première approche, l'estimation est effectuée à partir des valeurs prises par la donnée et obtenues par simulation de l'algorithme en virgule flottante. La seconde approche utilise une méthode analytique permettant de déterminer l'expression de la dynamique d'une donnée à partir de celle des entrées de l'application.

7.2.1.1 Méthode par simulation

Ce type d'approche permet d'estimer la dynamique d'une donnée à partir de ses caractéristiques déterminées lors de la simulation de l'algorithme en virgule flottante. Cependant, ces approches sont très sensibles au choix des stimuli d'entrée. Elles garantissent uniquement une absence de débordement pour des signaux dont les caractéristiques sont proches de celles des stimuli d'entrée. L'estimation de la dynamique la plus simple consiste à prendre la valeur absolue maximale des échantillons obtenus lors de la simulation. Des méthodes plus évoluées proposent de déterminer la dynamique d'une donnée à partir de ses caractéristiques statistiques.

Exemple filtre RIF

Les simulations ont été réalisées avec trois types de signaux :

1 : signal aléatoire correspondant à un bruit blanc uniformément distribué entre -1 et 1.

2 : signal aléatoire correspondant à un bruit blanc possédant une distribution gaussienne.

3 : signal de type chirp, signal dont la fréquence évolue linéairement au cours du temps entre 0 et $F_e/2$.

Les signaux en entrée et en sortie du filtre, obtenus à l'aide de l'outil Matlab, sont présentés à la figure 7.12. Pour les trois cas testés, le signal d'entrée est compris dans l'intervalle $]-1;1[$. *La dynamique de la sortie y a été mesurée pour les trois cas. La dynamique obtenue est respectivement de 3.4, 2.4 et 9.3. Ces résultats montrent que les valeurs obtenues pour l'évaluation de la dynamique dépendent fortement des signaux utilisés en entrée.*

7.2.1.2 Méthodes analytiques

L'objectif des méthodes analytiques est de déterminer l'expression du domaine de définition d'une donnée à partir de la connaissance de la dynamique des entrées. Les méthodes analytiques basées sur une estimation dans le pire cas garantissent l'absence de débordement mais l'estimation est plus pessimiste. Différentes approches, présentées ci-dessous, peuvent être utilisées.

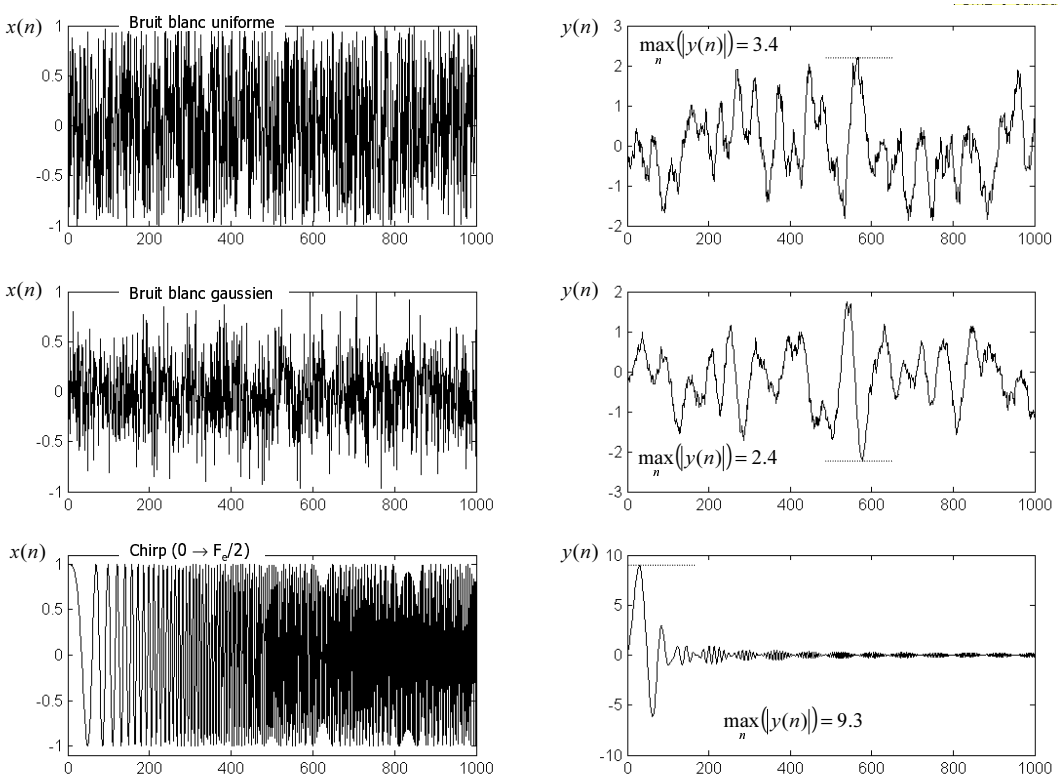


FIGURE 7.12 – Représentation des signaux en entrée et en sortie du filtre pour trois types de signaux d'entrée

L'arithmétique d'intervalles L'arithmétique d'intervalles correspond à une arithmétique définie sur un ensemble d'intervalles au lieu d'un ensemble de nombre réels. Un intervalle est assimilé au domaine de définition d'une variable x et est caractérisé par sa valeur limite inférieure $\underline{x}$ correspondant à la valeur minimale de x et sa valeur limite supérieure $\overline{x}$ correspondant à la valeur maximale de x.

L'arithmétique d'intervalles définit le domaine de définition des données issues des opérations arithmétiques classiques. Considérons la donnée z issue d'une opération arithmétique dont les entrées sont les deux variables x et y. Le domaine de définition de la donnée z pour les différents opérateurs arithmétiques est le suivant :

$$\begin{array}{ll} z = x + y & [\underline{z}, \overline{z}] = [\underline{x} + \underline{y},\ \overline{x} + \overline{y}] \\ z = x - y & [\underline{z}, \overline{z}] = [\underline{x} - \overline{y},\ \overline{x} - \underline{y}] \\ z = x * y & [\underline{z}, \overline{z}] = [\min(\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}),\ \max(\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y})] \end{array} \tag{7.25}$$

Considérons un système composé de N_e entrées $x_i(n)$. Soit $y(n)$ une donnée quelconque de ce système et définie en fonction des entrées de la manière suivante :

$$y(n) = f\left(x_0(n), x_1(n), \ldots x_i(n), \ldots x_{N_e-1}(n)\right) \tag{7.26}$$

Le domaine de définition de la donnée y peut être obtenu à partir de celui des entrées en appliquant à la fonction f, les différentes règles présentées à l'équation 7.25 :

$$[\underline{y}, \overline{y}] = f\left([\underline{x_0}, \overline{x_0}], \dots [\underline{x_i}, \overline{x_i}], \dots [\underline{x_{N_e-1}}, \overline{x_{N_e-1}}]\right) \tag{7.27}$$

Cette technique permet de définir le domaine de définition d'une donnée qui est fonction uniquement des entrées du système. En effet, dans le cas des systèmes récursifs, certaines données $y(n)$ sont fonction des valeurs précédentes $y(n-i)$ de ces données. Le domaine de définition de la donnée y étant fonction de son propre domaine de définition, l'arithmétique d'intervalles ne permet pas de déterminer directement le domaine de définition de ce type de donnée. Ainsi, cette technique ne peut être mise en œuvre de manière simple que pour les systèmes non-récursifs.

Pour déterminer la dynamique des données d'un système non-récursif, le graphe flot de signal représentant le système est parcouru des sources (entrées) vers les racines (sorties) et les règles définies à la relation 7.25 sont appliquées à chaque opérateur. Dans les systèmes récursifs, le graphe flot de signal possède des circuits, ainsi, le processus de parcours du graphe des entrées vers la sortie ne peut pas se terminer.

Exemple filtre RIF

Nous considérons une entrée x comprise dans l'intervalle $]-1;1[$. L'objectif de cette étape est de déterminer le domaine de définition de la sortie de la série d'accumulations correspondant au signal y. Nous propageons le domaine de définition de l'entrée x au sein du graphe flot de signal représentant l'application. Pour la sortie du filtre, la valeur absolue maximale est bornée à l'aide de la relation suivante :

$$y_{max} = \max_n \left(|y(n)|\right) = \sum_{m=0}^{N-1} |c_m| = 10.75 \tag{7.28}$$

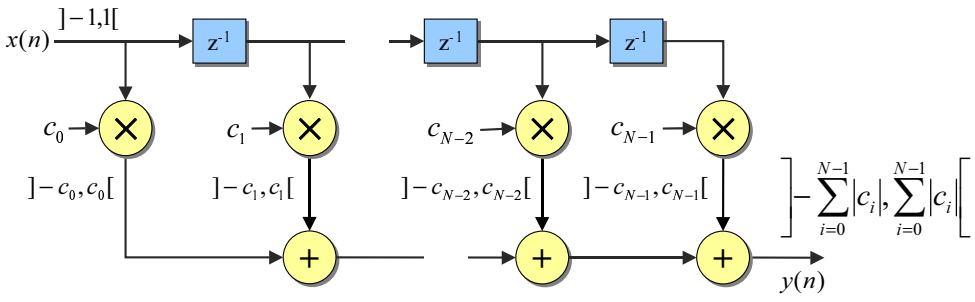


FIGURE 7.13 – Détermination de la dynamique en sortie du filtre à l'aide de l'arithmétique d'intervalle

---

Normes pour les systèmes linéaires Dans cette partie, nous présentons les normes définies pour déterminer la dynamique des données dans le cadre des systèmes linéaires invariant dans le temps. Les expressions des normes sont présentées dans le cas des systèmes linéaires à mono-entrée ou multi-entrées. Pour le système mono-entrée, considérons un système linéaire invariant dans le temps de fonction de transfert $H(z)$, d'entrée x et de sortie y :

$$y(n) = h(n) * x(n) \rightleftharpoons Y(z) = H(z).X(z) \tag{7.29}$$

Pour le système multi-entrées, considérons un système linéaire invariant dans le temps composé de N_e entrées x_i. Soit y, la sortie de ce système définie de la manière suivante :

$$y(n) = \sum_{i=0}^{N_e-1} h_i(n) * x_i(n) \tag{7.30}$$

Ces normes permettent de déterminer la valeur absolue maximale de la sortie y à partir de la valeur absolue maximale des entrées.

Norme L1 D'après la relation 7.29, la valeur absolue de la donnée y est égale à :

$$|y(n)| = \left| \sum_{m=-\infty}^{+\infty} h(m)x(n-m) \right| \tag{7.31}$$

Sachant que la valeur absolue d'une somme de termes est inférieure à la somme des valeurs absolues de ces termes et que l'entrée est bornée par sa valeur absolue maximale l'expression 7.31 peut être bornée. Ainsi, la relation entre la valeur absolue maximale d'une donnée y et celle de l'entrée x du système est la suivante :

$$y_{max} = \max_n (|y(n)|) \leq \max_n (|x(n)|) \sum_{m=-\infty}^{+\infty} |h(m)| \tag{7.32}$$

Aucune hypothèse concernant la nature du signal en entrée ayant été faite pour obtenir l'expression de y_{max}, la relation 7.32 permet de borner la valeur absolue maximale de la donnée y pour tous types de signaux au niveau de l'entrée.

L'extension de la norme L1 pour un système linéaire à plusieurs entrées conduit avec la même démarche à la relation suivante :

$$y_{max} = \max_n (|y(n)|) \leq \sum_{i=0}^{N_e-1} \max_n (|x_i(n)|) \sum_{m=-\infty}^{+\infty} |h_i(m)| \tag{7.33}$$

Exemple filtre FIR

L'expression de la dynamique de la sortie y, obtenue avec la norme L1, est la suivante :

$$y_{max} = \max_n (|x(n)|) \sum_{m=-\infty}^{+\infty} |h(m)| \tag{7.34}$$

Sachant que l'entrée x est comprise dans l'intervalle $]-1;1[$ et en utilisant l'expression de la réponse impulsionnelle du filtre, l'expression de la dynamique de la sortie y est la suivante :

$$y_{max} = 1. \sum_{m=0}^{N-1} |c_m| = 10.75 \tag{7.35}$$

Norme de Chebyshev L'estimation de la dynamique dans un système linéaire basé sur la norme L1 permet de garantir l'absence de débordement pour tous les types de signaux au niveau des entrées. Cette estimation étant relativement pessimiste, d'autres normes ont été proposées afin de tenir compte de la nature des signaux en entrée du système. La norme de Chebyshev permet de déterminer la valeur absolue maximale d'une donnée pour des signaux d'entrée à bande étroite. Dans ce cas, le signal d'entrée est modélisé par une sinusoïde définie de la manière suivante :

$$x(n) = A\cos(\Omega.n) \qquad \text{avec } 0 \leq \Omega < \pi \tag{7.36}$$

D'après les relations 7.29 et 7.36, l'expression de la valeur absolue de y est égale à :

$$|y(n)| = \left|A|.|H(e^{j\Omega})|.|\cos\left(\Omega.n + \arg\left(H(e^{j\Omega})\right)\right)\right| \quad \text{avec } 0 \leq \Omega < \pi \tag{7.37}$$

L'expression 7.37 peut être bornée en utilisant la valeur maximale du module de la réponse fréquentielle de H. Ainsi, pour la norme de Chebyshev, la relation entre la valeur absolue maximale d'une donnée y et celle de l'entrée du système est la suivante :

$$y_{max} = \max_n \left(|y(n)|\right) \leq \max_n \left(|x(n)|\right) \max_\Omega \left(|H(e^{j\Omega})|\right) \tag{7.38}$$

L'extension de la norme de Chebyshev à un système linéaire à plusieurs entrées conduit avec la même démarche à la relation suivante :

$$y_{max} = \max_n \left(|y(n)|\right) \leq \sum_{i=0}^{N_e-1} \max_n \left(|x_i(n)|\right) \max_\Omega \left(|H_i(e^{j\Omega})|\right) \tag{7.39}$$

Cette norme permet d'obtenir des résultats moins pessimistes que la norme L1, mais elle impose des conditions sur la nature du signal d'entrée du filtre d'un point de vue spectral.

Exemple filtre RIF

L'expression de la dynamique de la sortie y, obtenue avec la norme de Chebyshev, est la suivante :

$$y_{max} = \max_n \left(|x(n)|\right) \max_\Omega \left(|H(e^{j\Omega})|\right) \tag{7.40}$$

Le module de la réponse fréquentielle du filtre $H(e^{j\Omega})$ est présenté à la figure 7.14. Le maximum de cette fonction est obtenu pour une fréquence Ω nulle. Ainsi, l'expression de la dynamique de la sortie y est la suivante :

$$y_{max} = \left|H(e^{j0})\right| = \left|\sum_{m=0}^{N-1} c_m z^{-m}\right|_{z=1} = 10.75 \tag{7.41}$$

Nous obtenons les mêmes résultats que pour la norme L1, car nous sommes dans le cas particulier ou les différents éléments de la réponse impulsionnelle du filtre RIF sont tous positifs.

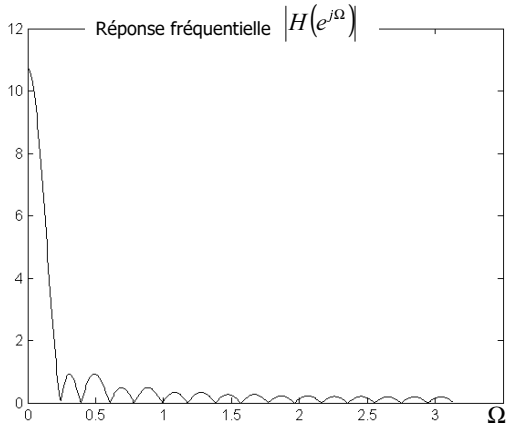


FIGURE 7.14 – Réponse fréquentielle du filtre RIF

Structure cascadée Pour illustrer les concepts détaillés dans cette partie, considérons le système présenté à la figure 7.15. Le système global H est réalisé en cascadant trois sous-systèmes H_1, H_2 et H_3 :

$$H(z) = H_1(z) \times H_2(z) \times H_3(z) \tag{7.42}$$

Les normes L1 et de Chebyshev permettent de majorer la valeur maximale en sortie d'un système. Dans le cas d'une structure cascadée, il est nécessaire de ne réaliser qu'une seule majoration en déterminant la dynamique d'une sortie y_i directement en fonction de l'entrée x du système global. La détermination de la dynamique de la sortie y_i en plusieurs étapes (utilisation des entrées des sous-systèmes) réalise plusieurs majorations conduisant à une surestimation de la dynamique.

Pour illustrer cet effet de surestimation, considérons les deux fonctions de transfert $H_1(z)$ et $H_2(z)$ dont les réponses fréquentielles sont présentées à la figure 7.16. La fonction de transfert globale est $H_{12}(z)$.

En appliquant la norme de Chebyshev, la dynamique de la sortie y_2 du système global est obtenue à partir de l'expression 7.43. Le maximum est obtenu pour une pulsation $\Omega_{12} = 1.6$.

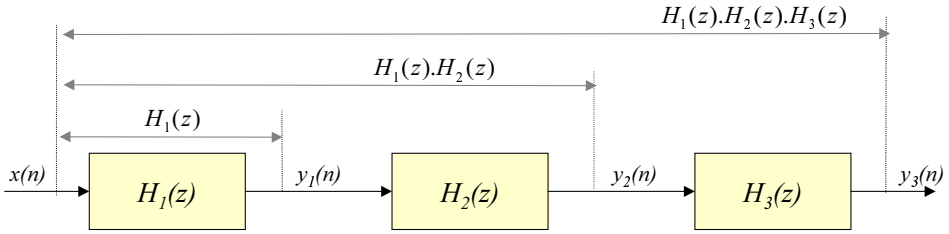


FIGURE 7.15 – Synoptique du système correspondant à une structure cascadée

$$y_{2-max} = \max_n (|x(n)|) \max_\Omega \left(|H_{12}(e^{j\Omega})|\right) = \max_\Omega \left(|H_1.H_2(e^{j\Omega})|\right) = 0,356 \tag{7.43}$$

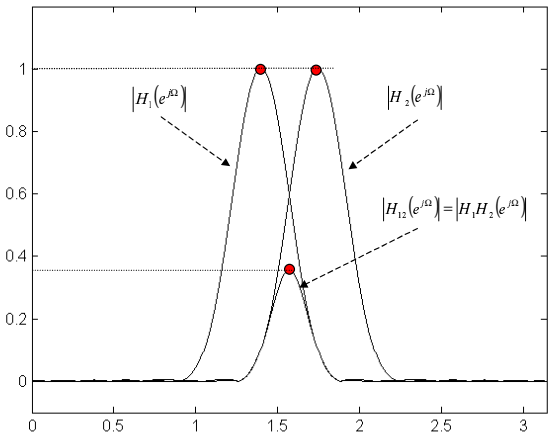


FIGURE 7.16 – Réponse fréquentielle des sous-systèmes H_1 et H_2 et du système global H_{12}

Si nous déterminons la dynamique en plusieurs étapes alors la dynamique de la sortie y_2 en fonction de l'entrée y_1 du sous-système H_2 est obtenue à partir de l'expression 7.44. Le maximum est obtenu pour une pulsation $\Omega_2 = 1.8$.

$$y_{2-max} = \max_n (|y_1(n)|) \max_\Omega \left(|H_2(e^{j\Omega})|\right) \tag{7.44}$$

En introduisant l'expression de la dynamique de la sortie y_1 en fonction de l'entrée globale du système, la dynamique de y_2 en fonction de l'entrée globale x est obtenue à partir de l'expression 7.45. Le maximum est obtenu pour une pulsation $\Omega_1 = 1.4$.

$$y_{2-max} = \max_n (|x(n)|) \max_\Omega \left(|H_1(e^{j\Omega})|\right) \max_\Omega \left(|H_2(e^{j\Omega})|\right) = 1 \times 1 = 1 \tag{7.45}$$

Cet exemple montre que la seconde évaluation de la dynamique de la sortie y surestime celle-ci, car les maximums des réponses fréquentielles des deux cellules cascadées sont obtenus pour

des pulsations différentes. Ainsi, il est nécessaire de toujours considérer la fonction de transfert globale entre la donnée considérée et l'entrée du système.

Evaluation de la dynamique

- Méthodes basées sur la simulation
  - Détermination des valeurs maximales à partir de simulations en virgule flottante.
  - Estimation précise mais elle ne garantit pas l'absence de débordement.

- Méthodes analytiques
  - Méthodes pessimistes mais garantissant l'absence de débordement.

  - **Arithmétique d'intervalle**
    - Propagation de l'intervalle de définition des entrées du système au sein du graphe (acyclique) de l'application
    - Règles de propagation pour les opérateurs arithmétiques :

$$\begin{array}{ll} z = x + y & [\underline{z}, \overline{z}] = [\underline{x} + \underline{y},\ \overline{x} + \overline{y}] \\ z = x - y & [\underline{z}, \overline{z}] = [\underline{x} - \overline{y},\ \overline{x} - \underline{y}] \\ z = x * y & [\underline{z}, \overline{z}] = [\min(\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}),\ \max(\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y})] \end{array}$$

  - **Normes pour les systèmes linéaires invariant dans le temps :**
    - Norme L1 :

$$y_{max} = \max_n (|y(n)|) \leq \max_n (|x(n)|) \sum_{m=-\infty}^{+\infty} |h(m)|$$

    - Norme de Chebyshev :

$$y_{max} = \max_n (|y(n)|) \leq \max_n (|x(n)|) \max_\Omega \left(|H(e^{j\Omega})|\right)$$

    - Structure cascadée : il est nécessaire de ne réaliser qu'une seule majoration en déterminant la dynamique d'une sortie y_i directement en fonction de l'entrée x du système global.

7.2.2 Détermination de la position de la virgule

L'objectif de cette phase est de déterminer la position de la virgule de chaque donnée du graphe flot de signal afin d'aboutir à une spécification correcte d'un point de vue de l'arithmétique virgule fixe. Cette transformation doit garantir l'absence de débordement et respecter les règles de l'arithmétique virgule fixe. La position de la virgule d'une donnée spécifiée à travers son paramètre m doit permettre de représenter l'ensemble des valeurs prises par cette donnée et d'éviter la présence de bits non utilisés au sein de celle-ci afin de maintenir une précision

maximale. Ces deux contraintes permettent d'obtenir pour une donnée x l'inégalité suivante :

$$2^{m_x-1} \leq x_{max} < 2^{m_x} \tag{7.46}$$

Pour une donnée x, la position de sa virgule m_x est directement obtenue à partir de sa dynamique selon la relation suivante :

$$m_x = \left\lfloor \log_2 \left(\max_n \left(|x(n)| \right) \right) \right\rfloor + 1 \tag{7.47}$$

7.2.2.1 Règles associées aux opérateurs arithmétiques

Multiplication La position de la virgule de la sortie d'une multiplication est directement obtenue à partir de celle des entrées. En intégrant le bit de signe redondant à la partie entière du résultat, la position de la virgule en sortie de la multiplication est la suivante :

$$m_z = m_x + m_y + 1 \tag{7.48}$$

Additionneur sans bit de garde Pour les opérations d'addition ou de soustraction, il est nécessaire de définir une position de la virgule commune aux entrées afin d'aligner la position de leur virgule. La position de la virgule de la sortie correspond à la position commune au niveau des entrées. Cette position commune doit garantir l'absence de débordement, ainsi elle est définie à partir de la contrainte la plus forte au niveau des entrées. Si l'opérateur ne possède pas de bit de garde permettant d'accueillir le bit supplémentaire issu d'un éventuel débordement il est nécessaire d'intégrer cette contrainte dans la détermination du format commun. Ainsi, pour un additionneur ne possédant pas de bit de garde, la position de la virgule commune est définie de la manière suivante :

$$m_c = \max\left(m_x, m_y, m_z\right) \tag{7.49}$$

Si la position de la virgule des entrées ou de la sortie n'est pas identique à la position de la virgule commune à l'additionneur, alors il est nécessaire d'introduire des opérations de décalage pour aligner la virgule. Les valeurs du décalage à réaliser pour les entrées et la sortie sont les suivantes :

$$\begin{array}{ll} d_x = m_c - m_x & \text{Décalage à } \textbf{droite} \text{ de } d_x \text{ bits} \\ d_y = m_c - m_y & \text{Décalage à } \textbf{droite} \text{ de } d_y \text{ bits} \\ d_z = m_c - m_z & \text{Décalage à } \textbf{gauche} \text{ de } d_z \text{ bits} \end{array} \tag{7.50}$$

Pour les entrées, un décalage à droite est éventuellement réalisé afin d'ajouter des bits supplémentaires à la partie entière de la donnée. Ces bits sont nécessaires pour éviter les débordements liés à l'accroissement de la dynamique lors de l'addition.
Pour la sortie, un décalage à gauche est éventuellement réalisé afin d'adapter la position de la virgule à la dynamique réelle de la donnée. Ce décalage élimine quelques bits de poids fort qui n'étaient pas utilisés.

Additionneur avec bits de garde Pour les additionneurs possédant des bits de garde, les largeurs des opérandes d'entrée et de sortie sont différentes et les bits les plus significatifs de ces données ne sont plus alignés. Ainsi, les positions des virgules entre les données ne sont plus cohérentes et un référentiel commun doit être utilisé pour analyser les contraintes au niveau des entrées et de la sortie.

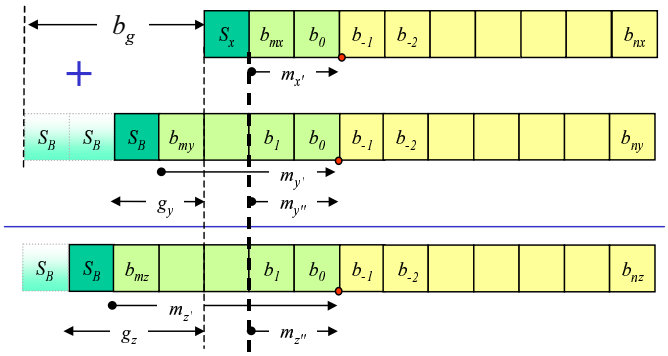


FIGURE 7.17 – Spécification de la position de la virgule en présence de bits de garde

Un paramètre g_x est associé à chaque donnée x pour représenter le nombre de bits de garde utilisés au sein de la donnée. Soient $m_{x'}, m_{y'}, m_{z'}$, les positions des virgules des données référencées par rapport au bit le plus significatif de l'entrée dont la largeur est la plus faible. Ces différents paramètres sont représentés à la figure 7.17. Le passage du référentiel associé à chaque donnée au référentiel commun nécessite de retrancher aux positions de la virgule le nombre de bits de garde utilisés. Ainsi, les expressions de la position des virgules dans le référentiel commun sont les suivantes :

$$\begin{cases} m_{x'} = m_x - g_x \\ m_{y'} = m_y - g_y \\ m_{z'} = m_z - g_z \end{cases} \tag{7.51}$$

Une position de la virgule commune aux entrées et à la sortie de l'additionneur dans le référentiel commun est définie. Cette position est déterminée à partir de la contrainte maximale sur les entrées et la sortie. Cependant, le calcul de la contrainte sur la sortie nécessite de connaître le nombre de bits de garde utilisés au niveau de la sortie. Cette valeur n'étant pas connue avant de déterminer le format commun, celle-ci est fixée à sa valeur maximale B_g, correspondant au nombre de bits de garde disponibles au niveau de l'additionneur. La position de la virgule commune est définie à partir de la relation suivante :

$$m_c = \max\left(m_{x'}, m_{y'}, m_{z'}\right) = \max\left(m_x - g_x, m_y - g_y, m_z - B_g\right) \tag{7.52}$$

Le nombre de bits de garde réellement utilisés par la sortie de l'additionneur est égal à :

$$\begin{cases} g_z = m_z - m_c & \text{si} \quad m_z > m_c \\ g_z = 0 & \text{si} \quad m_z \leq m_c \end{cases} \tag{7.53}$$

En conséquence, les positions des virgules des entrées sont égales à :

$$\begin{cases} m_x = m_c + g_x \\ m_y = m_c + g_y \end{cases} \tag{7.54}$$

Ainsi, la position de la virgule de la sortie de l'additionneur est égale à :

$$m_z = m_c + g_z \tag{7.55}$$

Si g_z est égal à 0, alors aucun bit de garde n'est utilisé et dans ce cas la position de la virgule de la sortie correspond à celle du format commun. Sinon, la position de la virgule de la sortie est obtenue à partir de la dynamique du résultat de l'opération. Si la contrainte la plus forte au niveau de la relation 7.78 correspond à celle sur la sortie alors, l'ensemble des bits de garde sont utilisés pour coder la sortie et la valeur de g_z est égale au nombre de bits de garde disponibles au niveau de l'additionneur.

Le renvoi en mémoire du résultat de l'addition nécessite de repositionner la donnée dans le registre d'accumulation car les bits de garde ne peuvent pas être envoyés directement en mémoire. Ainsi, il est nécessaire de faire un décalage à droite de g_z bits.

La relation 7.78 peut être utilisée pour déterminer la position de la virgule commune dans le cas des additionneurs avec et sans bits de garde. Si l'additionneur ne possède pas de bit de garde, les paramètres g et B_g sont nuls et nous retrouvons les résultats obtenus à l'équation 7.75.

Exemple du filtre RIF

Les différentes cellules du filtre RIF sont traitées de la même manière ainsi, le format des données est identique d'une cellule à l'autre. En particulier, un format commun est utilisé pour la suite d'opérations d'accumulation. En conséquence le graphe flot de signal à traiter se réduit à une cellule telle que présentée à la figure 7.18.

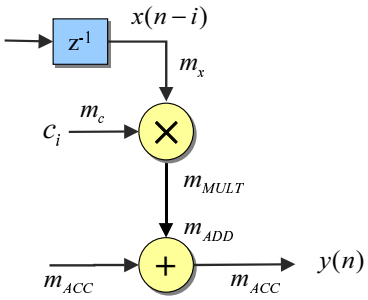


FIGURE 7.18 – Graphe flot de signal représentant une cellule du filtre RIF

L'objectif de cette phase est de déterminer la position de la virgule des différentes données à partir des informations obtenues dans la phase précédente de détermination de la dynamique.

Sachant que les dynamiques des données x et y sont respectivement égales à 0.999 et 10.75, alors la position de la virgule de ces données est la suivante :

$$\begin{aligned} m_x &= \lceil \log_2 (\max_n (|x(n)|)) \rfloor + 1 = \lfloor \log_2 (0.999) \rfloor + 1 = 0 \\ m_y &= \lfloor \log_2 (\max_n (|y(n)|)) \rfloor + 1 = \lfloor \log_2 (10.75) \rfloor + 1 = \lfloor 3.42 \rfloor + 1 = 4 \end{aligned} \tag{7.56}$$

Sachant que la valeur absolue maximale des coefficients c_i est égale à 0.48 alors la position de la virgule des coefficients est la suivante :

$$m_c = \left\lfloor \log_2 \left(\max_i (|c_i|) \right) \right\rfloor + 1 = \lfloor \log_2 (0.48) \rfloor + 1 = -1 \tag{7.57}$$

La position de la virgule de la sortie de la multiplication est calculée directement à partir de celle de ses entrées à l'aide de l'expression 7.74 :

$$m_{MULT} = m_x + m_c + 1 = 0 - 1 + 1 = 0 \tag{7.58}$$

Pour l'additionneur, il est nécessaire de trouver un format commun afin d'aligner la virgule des entrées de l'addition. La sortie y du filtre est obtenue à partir de la variable `ACC` à la fin de la suite d'accumulations. Le format de la variable `ACC` est identique pour toutes les accumulations. Ainsi, la dynamique de la variable `ACC` est égale à celle de y. En conséquence, la position de la virgule de la variable `ACC` correspond à celle de y.

Additionneur sans bit de garde *Dans le cas où l'architecture ne possède pas d'additionneur avec des bits de garde, le format commun associé à l'addition est obtenu à partir de l'expression 7.75 :*

$$m_{ADD} = \max (m_{MULT}, m_{ACC}, m_{ACC}) = \max (0, 4, 4) \tag{7.59}$$

Pour obtenir ce format commun il est nécessaire de modifier la position de la virgule de certaines données et en particulier celle de la donnée issue de la multiplication. La valeur de ce décalage est la suivante :

$$d_{MULT} = m_{ADD} - m_{MULT} = 4 - 0 = 4 \tag{7.60}$$

L'objectif de ce décalage est d'ajouter des bits à la partie entière de la donnée issue de la multiplication afin d'aligner la virgule. Cet ajout de bits est obtenu en réalisant un décalage arithmétique à droite (les nouveaux bits de poids fort sont une copie du bit de signe)

Additionneur avec bits de garde *Dans le cas où l'architecture possède un additionneur avec la possibilité d'utiliser 8 bits de garde, le format commun associé à l'addition est obtenu à partir de l'expression :*

$$\begin{aligned} m_{ADD} &= \max (m_{MULT} - g_{MULT}; m_{ACC} - B_g; m_{ACC} - B_g) \\ &= \max (0 - 0; 4 - 8; 4 - 8) = 0 \end{aligned} \tag{7.61}$$

En conséquence, les positions des virgules des entrées sont égales à :

$$m_{x'} = m_{ADD} + g_x = 0 \tag{7.62}$$

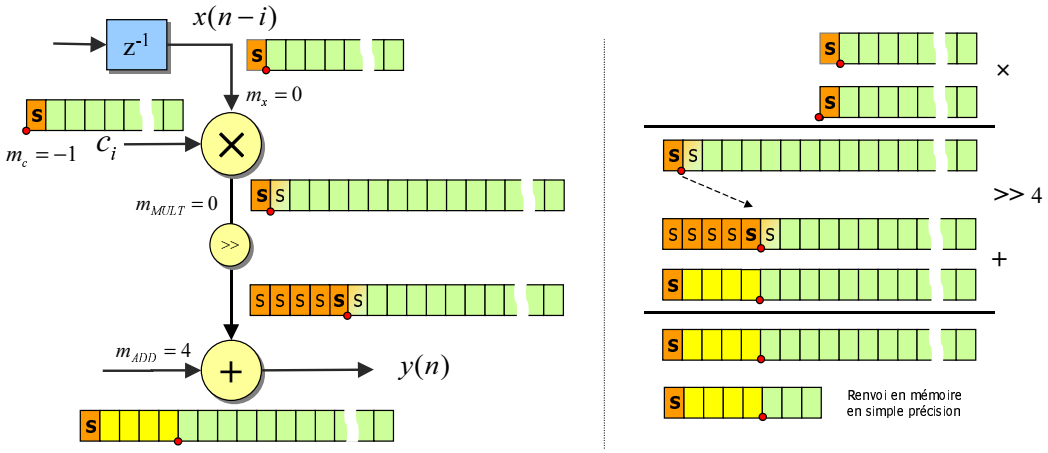


FIGURE 7.19 – Calcul en virgule fixe avec un additionneur sans bit de garde

Le nombre de bits de garde réellement utilisés par la sortie de l'additionneur est égal à :

$$g_{ACC} = m_{ACC} - m_{ADD} = 4 - 0 = 4 \tag{7.63}$$

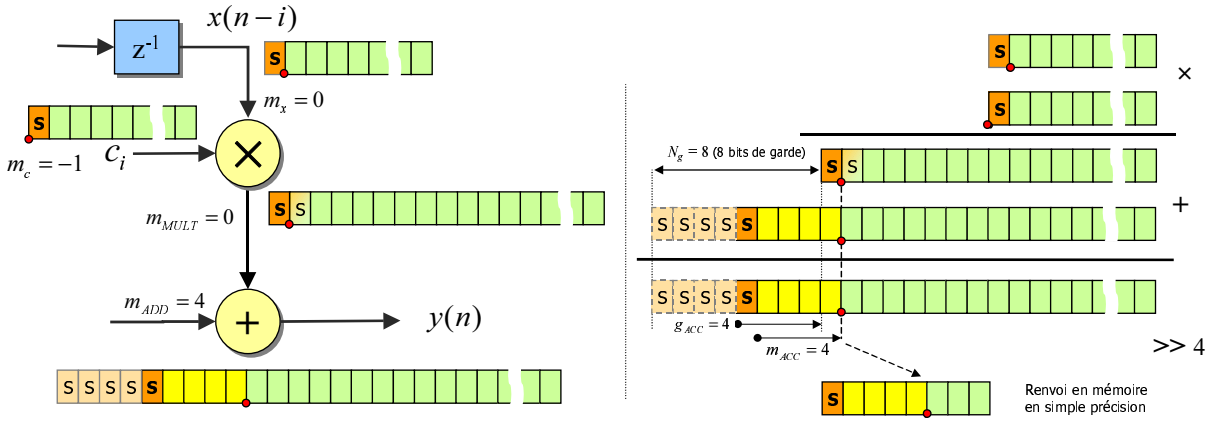


FIGURE 7.20 – Calcul en virgule fixe avec un additionneur possédant 8 bits de garde

Le renvoi en mémoire de la sortie du filtre après la série d'accumulations nécessite de faire un décalage à droite de g_{ACC} bits.

Détermination de la position de la virgule

- Position de la virgule d'une donnée x à partir de sa dynamique

$$m_x = \left\lfloor \log_2 \left(\max_n \left(|x(n)| \right) \right) \right\rfloor + 1 \tag{7.73}$$

- Règles associées aux opérateurs arithmétiques
  - **Multiplieur :**

$$m_z = m_x + m_y + 1 \tag{7.74}$$

  - **Additionneur sans bit de garde :**
    - Format commun pour l'additionneur :

$$m_c = \max\left(m_x, m_y, m_z\right) \tag{7.75}$$

    - Opération de décalage en entrée et en sortie :

$$\begin{array}{ll} d_x = m_c - m_x & \text{Décalage à } \textbf{droite} \text{ de } d_x \text{ bits} \\ d_y = m_c - m_y & \text{Décalage à } \textbf{droite} \text{ de } d_y \text{ bits} \\ d_z = m_c - m_z & \text{Décalage à } \textbf{gauche} \text{ de } d_z \text{ bits} \end{array} \tag{7.76}$$

  - **Additionneur avec bits de garde :**
    - Nouveau référentiel commun :

$$\left\{ \begin{array}{l} m_{x'} = m_x - g_x \\ m_{y'} = m_y - g_y \\ m_{z'} = m_z - g_z \end{array} \right. \tag{7.77}$$

    - Format commun pour l'additionneur :

$$m_c = \max\left(m_{x'}, m_{y'}, m_{z'}\right) = \max\left(m_x - g_x, m_y - g_y, m_z - B_g\right) \tag{7.78}$$

    - Nombre de bits de garde réellement utilisés par la sortie de l'additionneur :

$$\left\{ \begin{array}{lll} g_z = m_z - m_c & \text{si} & m_z > m_c \\ g_z = 0 & \text{si} & m_z \leq m_c \end{array} \right. \tag{7.79}$$

    - Positions des virgules des entrées sont égales à :

$$\left\{ \begin{array}{l} m_x = m_c + g_x \\ m_y = m_c + g_y \end{array} \right. \tag{7.80}$$

    - Position de la virgule de la sortie de l'additionneur :

$$m_z = m_c + g_z \tag{7.81}$$

7.2.3 Détermination de la largeur des données

Pour obtenir le format complet de chaque donnée (largeur de la partie entière et de la partie fractionnaire), la largeur des données doit être déterminée.

7.2.3.1 Implantation logicielle

Dans le cas d'une implantation logicielle de l'application au sein d'un processeur (DSP, μC), la largeur des données dépend des types manipulés par le processeur. Ces types sont liés aux différentes instructions disponibles au sein du jeu d'instructions :

— **Instructions arithmétiques classiques** : elles réalisent le traitement d'une seule opération sur un opérateur en un ou plusieurs cycles d'horloge. Ces instructions manipulent des opérandes de largeur b_{op}.

— **Instructions multi-précision** : elles permettent de manipuler des données de largeur plus importante, obtenues par concaténation de plusieurs données de largeur b_{op}. Une opération utilisant des données en multi-précision est décomposée en une suite d'opérations manipulant des opérandes de largeur b_{op}.

— **Instructions SWP (Sub-Word Parallelism)** : elles permettent de traiter en parallèle des opérandes dont la largeur est inférieure à la largeur b_{op}. Cette technique divise un opérateur manipulant des données de largeur b_{op} afin de pouvoir exécuter en parallèle k opérations sur des fractions de mot de largeur b_{op}/k (voir cours EII3)

Pour les processeurs DSP basés sur une architecture traditionnelle de type MAC, les opérateurs ne manipulent qu'un seul type de donnée. Ainsi le processus de détermination de la largeur des données est simplifié. Il consiste à reporter les largeurs des opérateurs en entrée et en sortie des opérations du graphe flot de l'application.

Certains processeurs DSP plus récents permettent de manipuler différents types de données (instructions classiques, multi-précision, SWP) il est nécessaire de sélectionner le type d'instructions permettant de minimiser le temps d'exécution du code et de respecter les contraintes de précision des calculs.

Exemple filtre RIF

Pour le filtre RIF traité en exemple, nous considérons dans un premier temps un DSP basé sur une architecture traditionnelle double précision de type MAC possédant 8 bits de garde. Le synoptique de la partie opérative est présenté à la figure 7.21.a. En reportant la largeur des opérateurs aux opérations du graphe flot de signal, nous obtenons le graphe présenté à la figure 7.22.

Nous considérons dans un second temps un DSP basé sur une architecture traditionnelle double précision de type MAC sans bit de garde. Le synoptique de la partie opérative est présenté à la figure 7.21.b. En reportant la largeur des opérateurs aux opérations du graphe flot de signal, nous obtenons le graphe présenté à la figure 7.23.

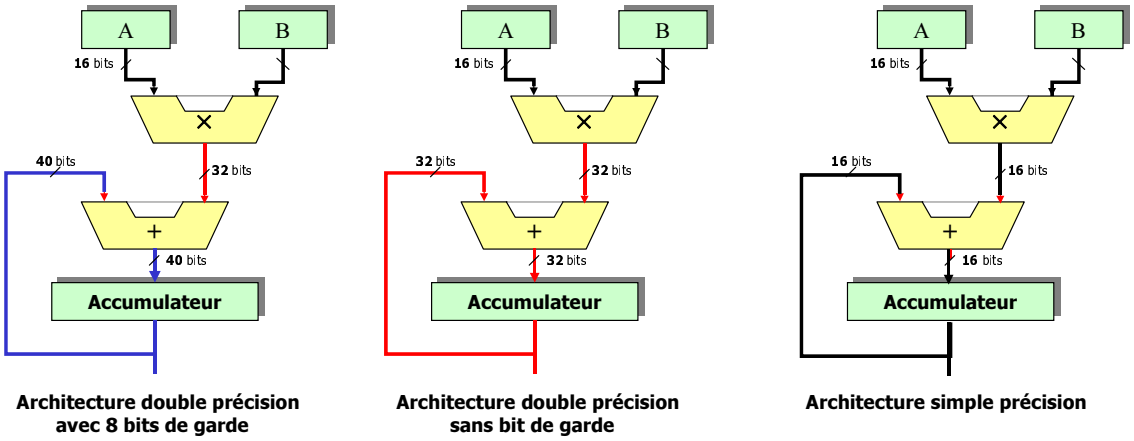


FIGURE 7.21 – Architecture de la partie opérative du processeur

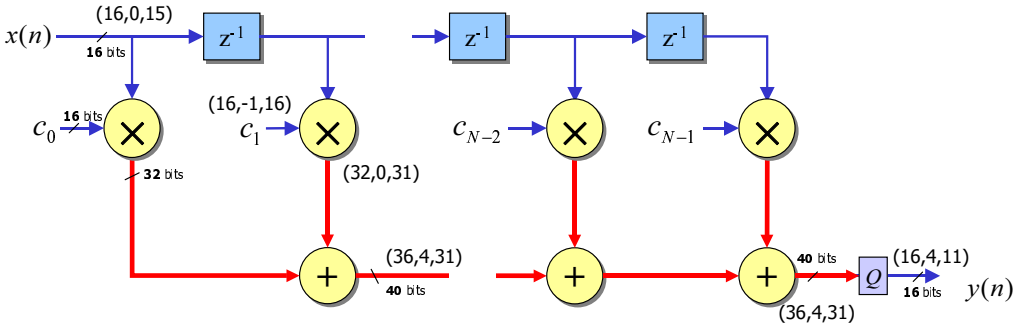


FIGURE 7.22 – Graphe flot de signal annoté avec le format complet des données pour une architecture double précision possédant 8 bits de garde.

Nous considérons dans un troisième temps une architecture simple précision dont le synoptique de la partie opérative est présenté à la figure 7.21.c. En reportant la largeur des opérateurs aux opérations du graphe flot de signal, nous obtenons le graphe présenté à la figure 7.24.

Code C de l'application pour une architecture double précision sans bit de garde

Le type virgule fixe n'existant pas en C, nous devons utiliser les types entiers. Ainsi, pour les coefficients il est nécessaire de coder la valeur entière ayant la même représentation binaire que la valeur en virgule fixe. Ceci revient à faire un décalage à droite de la virgule du nombre de bits utilisés pour la partie fractionnaire :

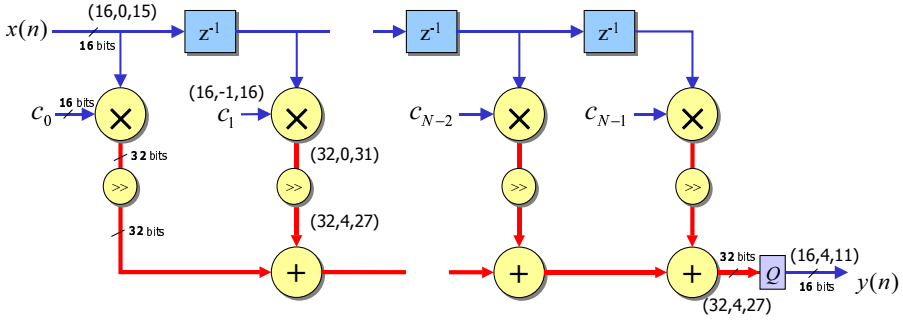


FIGURE 7.23 – Graphe flot de signal annoté avec le format complet des données pour une architecture double précision sans bit de garde.

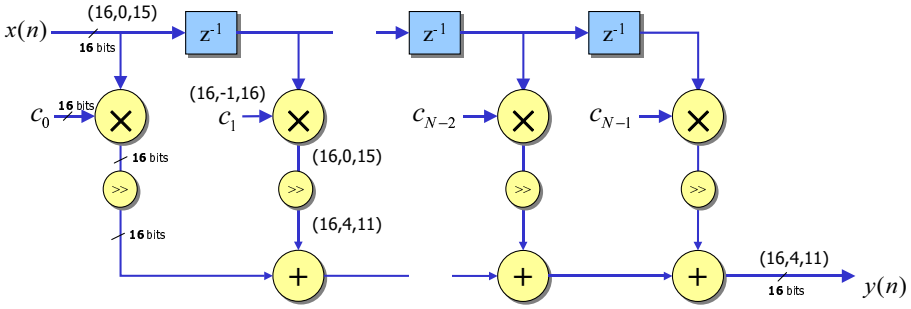


FIGURE 7.24 – Graphe flot de signal annoté avec le format complet des données pour une architecture simple précision

$$C_{int} = \lceil C_{fixed\ point} * 2^{n_c} \rceil \tag{7.82}$$

Déclaration des variables et des coefficients :

```
int input;
int x[N];
int y;
int c[N] =  {13107, 14336, 15565, , 13107, 14336} ;
long acc;
```

Code pour le calcul de la sortie du filtre

```
x[0] = Input;
 acc = (long)(x[0]*c[0]) >> 4;

for(i=N-1; i>0; i--)
{
   acc = acc + ((long)(x[i]*c[i]) >> 4); /* Calcul d'une cellule du filtre */
```

```
    x[i] = x[i-1];                  /* Vieillissement des variables internes */
}

y[j] = (int)(acc>>16);
```

L'expression `(long)( ... )` a pour objectif de forcer le compilateur à conserver les 32 bits du résultat et ne pas tronquer à 16 bits avant de faire l'addition.

L'expression `y = (int) ( ) ■ 16;` a pour objectif de renvoyer en mémoire les bits les plus significatifs du résultat (partie haute de l'accumulateur). Pour cela il est nécessaire de transférer les bits de la partie haute de l'accumulateur vers la partie basse de l'accumulateur. En effet, le compilateur travaille sur des entiers et lors d'un passage de 32 à 16 bits, il ne conserve que les bits les moins significatifs.

7.2.3.2 Implantation matérielle

Dans le cas d'une implantation matérielle de l'application au sein d'un FPGA ou d'un ASIC, l'architecture du système est définie par le concepteur. Ainsi, la largeur des opérateurs constituant l'architecture est déterminée lors du processus de conception. L'objectif de ce processus de conception est de limiter la surface de l'architecture afin de réduire les coûts et la consommation d'énergie. En conséquence, la largeur des opérateurs est minimisée tant que la précision des calculs réalisés est suffisante pour que les critères de qualité associés à l'application soient respectés.

Si le critère de qualité utilisé est le Rapport Signal à Bruit de Quantification (RSBQ), alors le processus de détermination de la largeur des données peut être modélisé par le problème d'optimisation suivant :

$$\min\left(S(\overrightarrow{b})\right) \quad \text{avec} \quad RSBQ(\overrightarrow{b}) \geq RSBQ_{min} \tag{7.83}$$

Le vecteur $\overrightarrow{b}$ représente la largeur de l'ensemble des données présentes au sein de l'application. Le terme S représente la surface de l'architecture et est fonction de la largeur des données.

Exemple du filtre RIF

Nous avons considéré une architecture composée d'un seul multiplieur et d'un seul additionneur. Ainsi, le problème d'optimisation est composé de 2 variables b_{mult} correspondant à la largeur des opérandes en entrée du multiplieur et b_{add} la largeur des opérandes de l'additionneur. Les fonctions $f_{surface}(b_{mult}, b_{add})$ et $f_{RSBQ}(b_{mult}, b_{add})$ fournissant respectivement la surface et le RSBQ en fonction des largeurs des opérateurs sont représentées à la figure 7.25. Pour une contrainte de rapport signal à bruit de quantification de 60 dB, la solution optimale utilise l'architecture suivante :

- *Multiplieur :* $12 \times 12 \rightarrow 24$ *(bits)*
- *Additionneur :* $17 + 17 \rightarrow 17$ *(bits)*

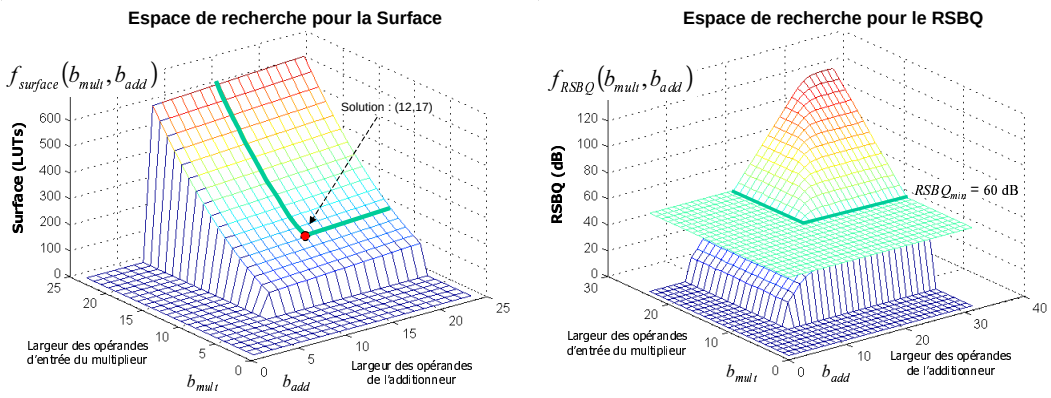


FIGURE 7.25 – Espace de recherche pour la surface et le RSBQ

7.3 Evaluation de la précision

7.3.1 Processus de quantification et de dépassement

Soit x une valeur arbitraire appartenant au domaine $\mathcal{D}$ et y une valeur du domaine de définition $\mathcal{D}_\mathcal{R}$ du codage choisi. Le domaine $\mathcal{D}_\mathcal{R}$ est borné par les valeurs X_{min} et X_{max}. Nous définissons le sous-ensemble $\mathcal{D}_D$ de $\mathcal{D}$ regroupant l'ensemble des valeurs de $\mathcal{D}$ comprises dans l'intervalle $[X_{min}; X_{max}]$. Le processus de quantification correspond à l'opération de réduction d'une valeur arbitraire x à une valeur représentable y. Ce processus est régi par deux lois présentées ci-dessous :

Loi de dépassement : cette loi permet d'associer à l'ensemble des valeurs x de $\mathcal{D}$ une valeur x appartenant au domaine $\mathcal{D}_\mathcal{D}$. Elle définit plus précisément le comportement pour les valeurs présentes en dehors du domaine $\mathcal{D}_\mathcal{D}$. Nous associons à cette loi une fonction de dépassement définie ci-dessous :

$$f_D(x) = \begin{cases} x & \forall\ x \in \mathcal{D}_\mathcal{D} \\ D(x) & \forall\ x \notin \mathcal{D}_\mathcal{D} \end{cases} \tag{7.84}$$

Loi de quantification : cette loi définit les valeurs représentables y à associer à l'ensemble des valeurs x appartenant au domaine $\mathcal{D}_\mathcal{D}$. La fonction de quantification associée est la suivante :

$$f_Q(x) = Q(x) \quad \forall\ x \in \mathcal{D}_\mathcal{D} \tag{7.85}$$

Le processus de quantification global peut s'exprimer sous la forme suivante :

$$x \rightarrow f_Q(f_D(x)) = \begin{cases} Q(x) & \forall\ x \in \mathcal{D}_\mathcal{D} \\ D(x) & \forall\ x \notin \mathcal{D}_\mathcal{D} \end{cases} \tag{7.86}$$

7.3.1.1 Lois de dépassement

Arithmétique de saturation

Cette loi appelée *loi de saturation*, consiste à choisir la valeur du domaine $\mathcal{D}_\mathcal{D}$ la plus proche de la valeur à représenter x :

$$D(x) = \begin{cases} X_{min} & \forall\ x\ < X_{min} \\ X_{max} & \forall\ x\ > X_{max} \end{cases} \tag{7.87}$$

La caractéristique de cette fonction est représentée à la figure 7.26.a. La gestion de cette loi nécessitera des opérateurs arithmétiques spécifiques. Par contre, son utilisation permet d'éviter certains problèmes et erreurs engendrés par l'utilisation de la loi modulaire.

Arithmétique modulaire

Cette loi de dépassement modulaire substitue aux valeurs de x n'appartenant pas au domaine $\mathcal{D}_\mathcal{D}$, x *modulo* $(X_{max} - X_{min})$. La caractéristique de cette loi est présentée à la figure 7.26.b.

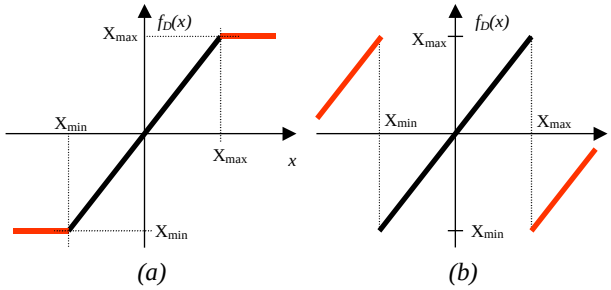


FIGURE 7.26 – Caractéristiques des lois de dépassement

7.3.1.2 Lois de quantification

Le domaine représentable $\mathcal{D}_\mathcal{R}$ est composé de N valeurs y_i avec $i = 1, 2, \ldots, N$ et le sous-domaine $\mathcal{D}_\mathcal{D}$ est subdivisé en N sous domaines juxtaposés Δ_i . La loi de quantification associe à tout x appartenant au domaine Δ_i la valeur y_i :

$$\forall\ x\ \in \Delta_i \quad Q(x) = y_i \tag{7.88}$$

Soit e l'erreur entre la valeur avant quantification, x et la valeur après quantification y_i :

$$e = x - y_i \tag{7.89}$$

Loi de quantification par arrondi

La loi de quantification par arrondi consiste à choisir la valeur représentable la plus proche de la valeur à quantifier en prenant la médiane de chaque intervalle Δ_i :

$$y_i = \frac{u_{i+1} - u_i}{2} = u_i + \frac{q}{2} \qquad \forall\ x\ \in \Delta_i = [u_i; u_{i+1}] \tag{7.90}$$

La caractéristique de cette loi est représentée à la figure 7.27.

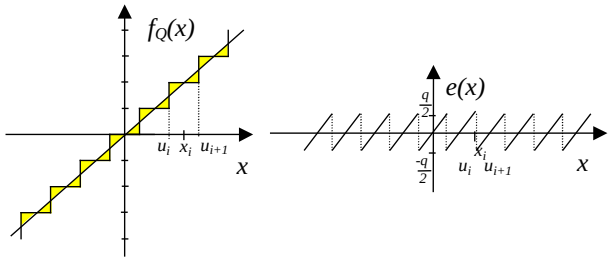


FIGURE 7.27 – Caractéristiques de la loi de quantification par arrondi

Loi de quantification par troncature

Cette loi de quantification consiste à tronquer un certain nombre de bits de poids faible. Les propriétés de cette loi de quantification sont fonctions du choix de la représentation (signe-valeur absolue ou complément à 2).

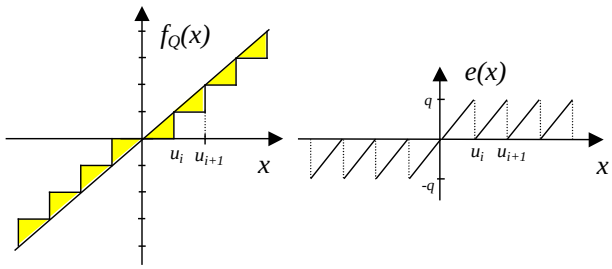


FIGURE 7.28 – Caractéristiques de la loi de quantification par troncature pour le codage SVA

Représentation signe valeur absolue

Pour cette représentation la troncature d'un certain nombre de bits de poids faible revient à choisir la valeur représentable la plus proche dont le module est inférieur à la valeur à quantifier (voir figure 7.28) :

$$y_i = \begin{cases} u_i & \forall\ x > 0 \\ u_{i+1} & \forall\ x < \end{cases} \tag{7.91}$$

Représentation en complément à 2

La quantification par troncature dans le cas d'une représentation en complément à 2 (voir figure 7.29) revient à prendre la valeur représentable immédiatement inférieure à la valeur à quantifier $y_i = u_i$.

7.3.2 Modélisation du bruit

7.3.2.1 Bruit généré par une conversion analogique-numérique

La conversion analogique numérique (CAN) consiste à passer d'un signal échantillonné d'amplitude continue $x(n)$ vers une suite de nombre $y(n)$ dont les valeurs appartiennent à un

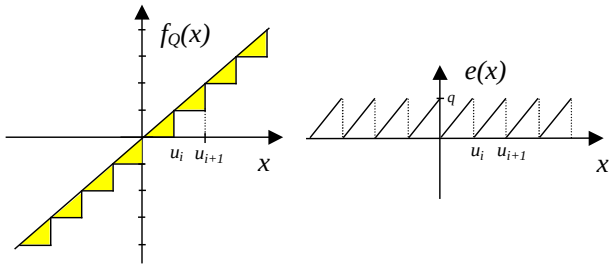


FIGURE 7.29 – Caractéristiques de la loi de quantification par troncature pour le codage CA2

intervalle de valeur (dynamique) et sont quantifiées (amplitude discrète). Ces valeurs sont un multiple entier du pas de quantification q (voir figure 7.30) :

$$y(n) = Q[x(n)] = k.q \tag{7.92}$$

Dans ce cadre, les modèles et lois de quantification présentés dans les sections précédentes peuvent être utilisés.

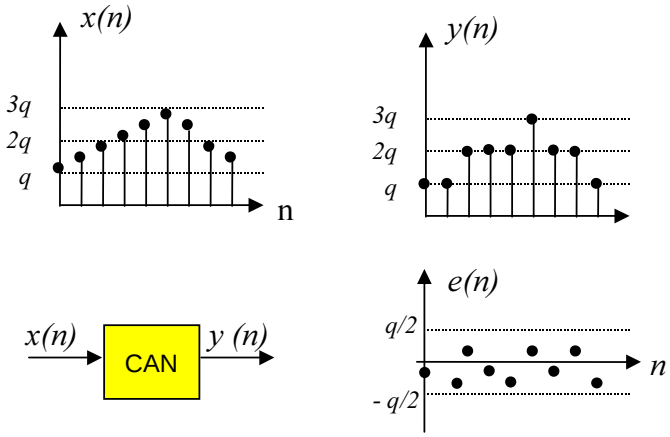


FIGURE 7.30 – Bruit de conversion analogique numérique

L'erreur de quantification $e(n) = x(n) - y(n)$ entre la valeur avant quantification x et la valeur après quantification y, peut être modélisée par une variable aléatoire stationnaire. Le processus de quantification peut être modélisé par la somme du signal $x(n)$ et d'un bruit de quantification $-e(n)$

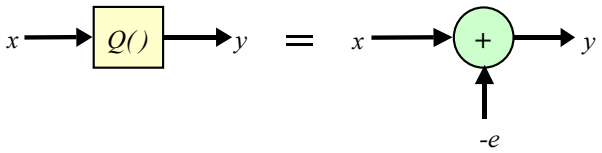


FIGURE 7.31 – Modélisation du bruit de quantification

Cette variable aléatoire est uniformément distribuée sur son intervalle de définition correspondant à $[0; q]$ pour la troncature et $[-q/2; q/2]$ pour l'arrondi. La densité de probabilité de

cette variable aléatoire e est représentée à la figure 7.32 pour le cas d'une quantification par troncature et par arrondi

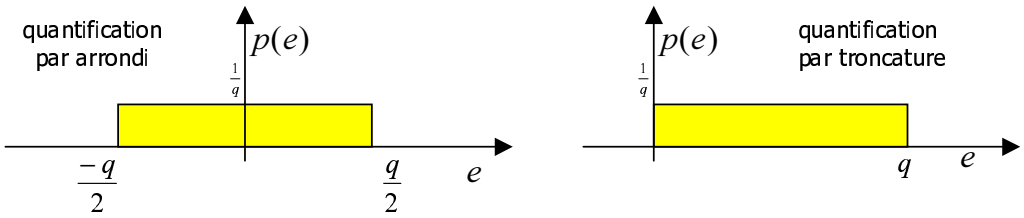


FIGURE 7.32 – Densité de probabilité du bruit généré lors de la quantification par troncature et par arrondi

La variable aléatoire $e(n)$ possède les propriétés suivantes :
- $e(n)$ est stationnaire,
- le bruit issu de la quantification d'un signal $x(n)$ n'est pas corrélé avec ce signal,
- les bruits de quantification sont statistiquement indépendants,
- $e(n)$ est un bruit blanc uniformément réparti,
- $e(n)$ est bornée par le pas de quantification,
- la densité de probabilité de $e(n)$ est uniforme sur l'intervalle de quantification,
- l'ergodicité implique que les moyennes temporelles et statistiques sont équivalentes.

Pour une loi de quantification par arrondi, la moyenne et la variance de l'erreur de quantification sont les suivantes :

$$\mu_e = 0, \qquad \sigma_e^2 = \frac{q^2}{12} \tag{7.93}$$

Pour une loi de quantification par troncature, la moyenne et la variance de l'erreur de quantification sont les suivantes :

$$\mu_e = \frac{q}{2}, \qquad \sigma_e^2 = \frac{q^2}{12} \tag{7.94}$$

7.3.2.2 Bruit généré par un changement de format

Modélisation du bruit généré par la troncature

L'objectif de cette partie est de modéliser le bruit issu d'un changement de format entraînant l'élimination d'un certain nombre de bits de poids faible. Ce bruit correspond à la modélisation de la perte d'information résultant de l'élimination des k bits les moins significatifs lors d'un changement de format. Nous considérons une donnée de départ X_e et la donnée X_s après changement de format. La représentation de ces données codées en complément à 2, est présente à la figure 7.33 et les expressions de celles-ci sont les suivantes :

$$X_e = -2^m.S + \sum_{-n}^{m-1} b_i 2^i \tag{7.95}$$

$$X_s = -2^m.S + \sum_{-j}^{m-1} b_i 2^i \tag{7.96}$$

Nous définissons q le pas de quantification de la donnée après changement de format. Celui-ci est égal à 2^{-j}.

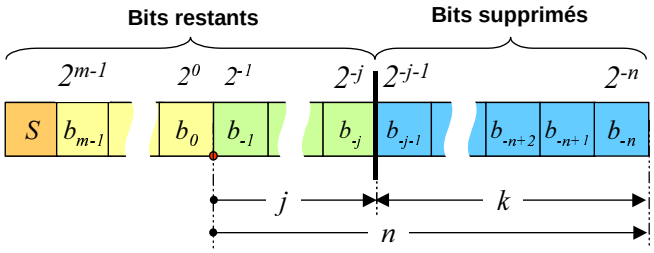


FIGURE 7.33 – Représentation des données lors d'une troncature

L'expression du bruit de quantification correspondant à la différence entre les deux variables X_e et X_s, est la suivante :

$$e = \sum_{i=j-1}^{n} b_{-i} 2^{-i} \tag{7.97}$$

L'expression 7.97 montre que le bruit généré sera toujours positif. En effet lors d'une troncature en complément à 2 la valeur tronquée est toujours inférieure à la valeur de départ.

Le bruit issu de la troncature ne pouvant prendre qu'un nombre fini de valeurs, égal à 2^k, nous pouvons modéliser le bruit généré par une variable aléatoire discrète.
Nous supposons que les valeurs binaires des bits b_{-j-1} à b_{-n} sont équiprobables, ainsi les valeurs représentant e sont équiprobables. Nous pouvons considérer que la densité de probabilité $p(e)$ représentée à la figure 7.34, est uniforme. Son expression est la suivante :

$$p(e) = \sum_{i=0}^{2^k-1} 2^{-k} \delta(e - i.2^{-n}) \tag{7.98}$$

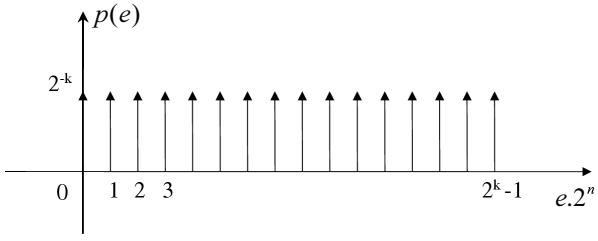


FIGURE 7.34 – Densité de probabilité du bruit généré lors d'une troncature

La moyenne de cette variable aléatoire est la suivante :

$$\mu_e = \sum_{i=-\infty}^{+\infty} y_i p(y_i) = \sum_{i=0}^{2^k-1} i.2^{-n}.2^{-k} \tag{7.99}$$

soit

$$\mu_e = 2^{-j-1}(1 - 2^{j-n}) = \frac{q}{2}(1 - 2^{-k}) \tag{7.100}$$

La variance de cette variable aléatoire est égale à :

$$\sigma_e^2 = \sum_{i=-\infty}^{+\infty} y_i^2 p(y_i) - \mu_e^2 \tag{7.101}$$

soit

$$\sigma_e^2 = \frac{2^{-2j}}{12}(1 - 2^{2(j-n)}) = \frac{q^2}{12}(1 - 2^{-2k}) \tag{7.102}$$

Lorsque le nombre de bits tronqués est important les résultats tendent vers ceux obtenus dans le cas de la quantification d'un signal à amplitude continue :

$$\mu_e = 2^{-j-1} = \frac{q}{2} \tag{7.103}$$

$$\sigma_e^2 = \frac{1}{12}(2^{-2j}) = \frac{q^2}{12} \tag{7.104}$$

Modélisation du bruit généré par l'arrondi

Dans le cas d'une quantification par arrondi conventionnel, la donnée correspondant à la valeur médiane de l'intervalle $[kq, (k+1)q]$ est codée à la valeur $(k+1)q$. La loi de quantification est définie de la façon suivante :

$$Q(x) = \begin{cases} k.q & \text{si } \; kq \leq x < (k+\frac{1}{2})q \\ (k+1).q & \text{si } \; (k+\frac{1}{2})q < x \leq (k+1)q \\ (k+1)q & \text{si } \; x = (k+\frac{1}{2})q \end{cases} \tag{7.105}$$

L'expression de la densité de probabilité de l'erreur de quantification b_{gac} représentée à la figure 7.35, est la suivante

$$p(e) = \sum_{i=-2^{k-1}}^{2^{k-1}-1} 2^{k-n}\delta(e - i.2^{-n}) \tag{7.106}$$

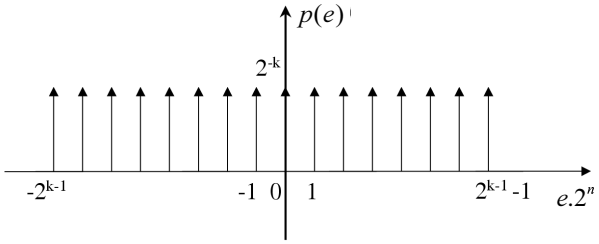


FIGURE 7.35 – Densité de probabilité du bruit généré lors d'un arrondi conventionnel

La moyenne de cette variable aléatoire est égale à :

$$\mu_{b_{gac}} = \sum_{i=-2^{k-1}}^{2^{k-1}-1} i.2^{-n}.2^{-k} = 2^{-n-1} = \frac{q}{2}(2^{-k}) \tag{7.107}$$

La valeur médiane de chaque intervalle $[k.q, (k+1).q]$ étant codée systématiquement à la valeur supérieure, la moyenne de l'erreur de quantification n'est pas nulle. Cette erreur est néanmoins nettement plus faible que celle présente lors d'une quantification par troncature. La variance de cette variable aléatoire est égale à :

$$\sigma^2_{b_{gac}} = \sum_{i=-2^{k-1}}^{2^{k-1}-1} (i.2^{-n})^2.2^{-k} - \mu_e^2 = \frac{2^{-2j}}{12}(1-2^{-2k}) = \frac{q^2}{12}(1-2^{-2k}) \tag{7.108}$$

Afin d'éliminer le biais, un arrondi convergent peut être utilisé, celui-ci va affecter la valeur médiane de façon équiprobable à la valeur supérieure et inférieure :

$$Q(x) = \begin{cases} k.q & \text{si } \; k.q \leq x < (k+\frac{1}{2})q \\ (k+1).q & \text{si } \; (k+\frac{1}{2})q < x \leq (k+1)q \\ k.q \text{ ou } (k+1).q & \text{si } \; x = (k+\frac{1}{2})q \end{cases} \tag{7.109}$$

$$\text{avec } \mathrm{P}\Big(Q(x) = kq \setminus x = (k+\frac{1}{2})q\Big) = \mathrm{P}\Big(Q(x) = (k+1)q \setminus x = (k+\frac{1}{2})q\Big)$$

Cette technique garantit l'équiprobabilité du codage de la valeur médiane en analysant la parité de la donnée. Les données impaires sont codées à la valeur supérieure et les données paires à la valeur inférieure. La densité de probabilité associée à l'erreur de quantification présentée à la figure 7.36, est égale à :

$$p(e) = \frac{2^{-k}}{2}\Big(\delta(e+2^{k-n-1}) + \delta(e-2^{k-n-1})\Big) + \sum_{i=-2^{k-1}+1}^{2^{k-1}-1} 2^{-k}\delta(e-i.2^{-n}) \tag{7.110}$$

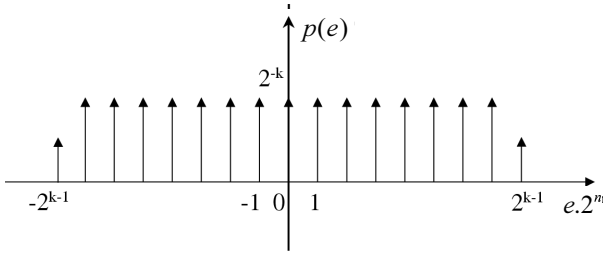


FIGURE 7.36 – Densité de probabilité du bruit généré lors d'un arrondi convergent

La moyenne de cette variable aléatoire est la suivante :

$$\mu_e = 2^{-k}\Big(2^{k-n-1} - 2^{k-n-1}\Big) + \sum_{i=-2^{k-1}+1}^{2^{k-1}-1} i.2^{-n}.2^{-k} = 0 \tag{7.111}$$

Ce type d'arrondi permet d'obtenir une moyenne de l'erreur de quantification nulle. L'expression de la variance de l'erreur de quantification est la suivante :

$$\sigma_e^2 = 2^{-k}2^{2(k-n-1)} + \sum_{i=-2^{k-1}+1}^{2^{k-1}-1} (i.2^{-n})^2.2^{-k} \tag{7.112}$$

soit

$$\sigma_e^2 = \frac{2^{-2j}}{12}(1 - 2^{-2k+1}) = \frac{q^2}{12}(1 - 2^{-2k+1}) \tag{7.113}$$

7.3.3 Filtrage du bruit

La quantification d'un signal analogique et le changement de format de données entraînent la génération d'un bruit. Ce bruit se propage au sein du système. L'objectif de cette partie est de déterminer l'influence de ce bruit en sortie du système. Considérons une source de bruit $e(n)$ traversant un système linéaire invariant dans le temps de réponse impulsionnelle $h(n)$. Soit $f(n)$ la sortie du système :

$$f(n) = e(n) * h(n) \tag{7.114}$$

La variable aléatoire $e(n)$, correspondant au bruit de quantification, est un bruit blanc, ainsi, la moyenne μ_f et la variance σ_f^2 du bruit en sortie $f(n)$ sont :

$$\mu_f = \mu_e \sum_{n=-\infty}^{+\infty} h(n) = \mu_e H(e^{j0}) \tag{7.115}$$

$$\sigma_f^2 = \sigma_e^2 \sum_{n=-\infty}^{+\infty} |h(n)|^2 = \frac{\sigma_e^2}{2\pi}\int_{-\pi}^{\pi} |H(e^{j\Omega})|^2 d\Omega \tag{7.116}$$

La puissance d'un signal aléatoire correspond au moment d'ordre 2 de celui-ci :

$$P_f = E\left(f^2\right) = \mu_f^2 + \sigma_f^2 \tag{7.117}$$

Cette puissance peut être exprimée en dB :

$$(P_f)_{dB} = 10.\log_{10}(P_f) \tag{7.118}$$

Exemple filtre RIF

Nous déterminons la puissance du bruit en sortie du filtre dans le cas des spécifications virgule fixe présentées aux figures 7.22, 7.23 et 7.24 pour une architecture en simple précision et en double précision avec et sans bits de garde. Les différentes sources de bruit potentielles au sein de ce filtre RIF sont représentées à la figure 7.37.

Architecture double précision avec bits de garde

Dans le cas d'une architecture double précision avec bits de garde nous obtenons la spécification virgule fixe présente à la figure 7.22. Les différentes sources de bruit présentes au sein de ce filtre sont les suivantes :

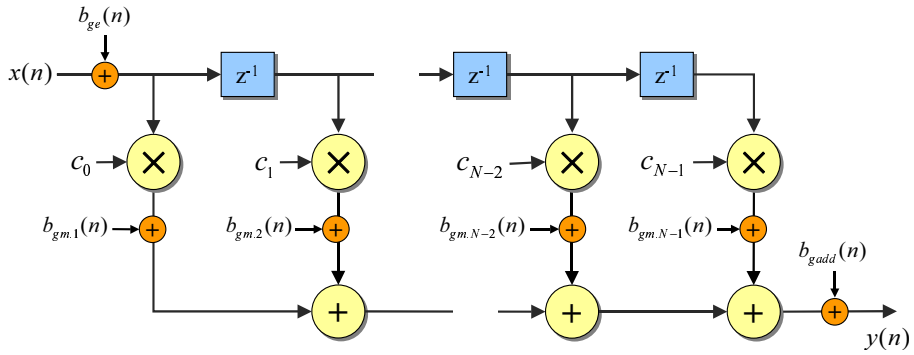


FIGURE 7.37 – Sources de bruit potentielles dans le filtre RIF

— b_{ge} *: bruit associé à l'entrée. Nous considérons que le signal* x *est issu d'un convertisseur analogique-numérique. La puissance de ce bruit est la suivante :*

$$\sigma^2_{b_{ge}} = \frac{q_x^2}{12} = \frac{2^{-2.n_x}}{12} = \frac{2^{-30}}{12} \rightarrow -101.1\ dB \tag{7.119}$$

— b_{gbadd} *: bruit présent en sortie du filtre et lié à l'élimination de 20 bits lors du renvoi de la donnée en mémoire sur 16 bits*

$$\sigma^2_{b_{gadd}} = \frac{q_y^2}{12}(1 - 2^{-2.k_y}) = \frac{2^{-2.n_y}}{12}(1 - 2^{-2.k_y}) = \frac{2^{-22}}{12}(1 - 2^{-40}) \rightarrow -77\ dB \tag{7.120}$$

Le bruit lié au renvoi de la donnée en mémoire (b_{gadd}) *est directement présent en sortie du filtre. Le bruit en entrée* b_{ge} *traverse le filtre* $h(n)$*. Ainsi, l'expression du bruit en sortie du filtre est la suivante :*

$$b_y(n) = b_{gadd}(n) + b_{ge}(n) * h(n) \tag{7.121}$$

Si nous considérons qu'une loi de quantification par arrondi est utilisée pour chaque changement de format, alors la moyenne du bruit associé à chaque source est nulle. La moyenne du bruit en sortie est nulle.

Les différentes sources de bruit ne sont pas corrélées entre elles et le bruit en entrée est un bruit blanc. En utilisant l'expression 7.143 pour le bruit b_{ge} *la variance du bruit en sortie du filtre est la suivante :*

$$\sigma^2_{b_y} = \sigma^2_{b_{gadd}} + \sigma^2_{b_{ge}} \sum_{i=0}^{N-1} |c_i|^2 \tag{7.122}$$

$$\sigma^2_{b_y} = \frac{2^{-22}}{12} + \frac{2^{-30}}{12} * 3.95 \rightarrow -77\ dB \tag{7.123}$$

Architecture double précision sans bit de garde

Dans le cas d'une architecture double précision sans bit de garde nous obtenons la spécification virgule fixe présente à la figure 7.23. Les différentes sources de bruit présentes au sein de ce filtre sont les suivantes :

— b_{ge} *: bruit associé à l'entrée. Nous considérons que le signal* x *est issu d'un convertisseur analogique-numérique. La puissance de ce bruit est la suivante :*

$$\sigma^2_{b_{ge}} = \frac{q_x^2}{12} = \frac{2^{-2.n_x}}{12} = \frac{2^{-30}}{12} \rightarrow -101.1\ dB \tag{7.124}$$

— $b_{gm.i}$ *: bruit présent en sortie du multiplieur* i *lié au décalage à droite avant l'addition. La présence d'un décalage à droite avant l'accumulation conduit à une donnée possédant 27 bits pour la partie fractionnaire. Sachant que 4 bits ont été éliminés, la puissance de ce bruit est la suivante :*

$$\sigma^2_{b_{gm.i}} = \frac{q^2_{MULT'}}{12}(1-2^{-2.k_m}) = \frac{2^{-2.n_{MULT'}}}{12}(1-2^{-2.k_m}) = \frac{2^{-54}}{12}(1-2^{-8}) \rightarrow -173.6\ dB \tag{7.125}$$

— b_{gbadd} *: bruit présent en sortie du filtre et lié à l'élimination de 16 bits lors du renvoi de la donnée en mémoire sur 16 bits*

$$\sigma^2_{b_{gadd}} = \frac{q_y^2}{12}(1-2^{-2.k_y}) = \frac{2^{-2.n_y}}{12}(1-2^{-2.k_y}) = \frac{2^{-22}}{12}(1-2^{-32}) \rightarrow -77\ dB \tag{7.126}$$

Dans cette étude nous prenons en compte les différentes sources de bruit, mais l'étude peut être simplifiée en négligeant les bruits issus du recadrage avant l'accumulation car leur contribution est insignifiante.

Les bruits issus du recadrage ($b_{gm.i}$*) et liés au renvoi de la donnée en mémoire (*b_{gadd}*) sont directement présents en sortie du filtre. Le bruit en entrée* b_{ge} *traverse le filtre* h*. Ainsi, l'expression du bruit en sortie du filtre est la suivante :*

$$b_y(n) = b_{gadd}(n) + \sum_{i=0}^{N-1} b_{gm.i}(n) + b_{ge}(n) * h(n) \tag{7.127}$$

Si nous considérons qu'une loi de quantification par arrondi est utilisée pour chaque changement de format, alors la moyenne du bruit associé à chaque source est nulle. La moyenne du bruit en sortie est nulle.

Les différentes sources de bruit ne sont pas corrélées entre elles et le bruit en entrée est un bruit blanc. En utilisant l'expression 7.143 pour le bruit b_{ge} *la variance du bruit en sortie du filtre est la suivante :*

$$\sigma^2_{b_y} = \sigma^2_{b_{gadd}} + \sum_{i=0}^{N-1} \sigma^2_{b_{gm.i}} + \sigma^2_{b_{ge}} \sum_{i=0}^{N-1} |c_i|^2 \tag{7.128}$$

$$\sigma^2_{b_y} = \frac{2^{-22}}{12}(1 - 2^{-32}) + \frac{2^{-30}}{12} * 3.95 \rightarrow -77\ dB \tag{7.129}$$

Bruit en sortie du filtre RIF pour une architecture simple précision

Dans le cas d'une architecture simple précision nous obtenons la spécification virgule fixe présente à la figure 7.24. Les différentes sources de bruit présentes au sein de ce filtre sont les suivantes :

— b_{ge} *: bruit associé à l'entrée. Nous considérons que le signal* x *est issu d'un convertisseur analogique-numérique et est ensuite codé sur 16 bits avec 15 bits pour la partie fractionnaire. La puissance de ce bruit est la suivante :*

$$\sigma^2_{b_{ge}} = \frac{q_x^2}{12} = \frac{2^{-2.n_x}}{12} = \frac{2^{-30}}{12} \tag{7.130}$$

— $b_{gm.i}$ *: bruit présent en sortie du multiplieur* i *lié à l'utilisation d'un multiplieur simple précision et au décalage à droite avant l'addition. Nous regroupons ces deux opérations au sein d'une même source de bruit. Si aucun bit n'est éliminé, la multiplication fournit une donnée possédant 31 bits pour la partie fractionnaire. L'utilisation d'un multiplieur travaillant en simple précision et la présence d'un décalage à droite avant l'accumulation conduisent à une donnée possédant 11 bits pour la partie entière. Sachant que 20 bits ont été éliminés, la puissance de ce bruit est la suivante :*

$$\sigma^2_{b_{gm}} = \frac{q^2_{MULT'}}{12}(1 - 2^{-2.k_m}) = \frac{2^{-2.n_{MULT'}}}{12}(1 - 2^{-2.k_m}) = \frac{2^{-22}}{12}(1 - 2^{-40}) \tag{7.131}$$

Les bruits issus du recadrage ($b_{gm.i}$*) et liés au renvoi de la donnée en mémoire (*b_{gadd}*) sont directement présents en sortie du filtre. Le bruit en entrée* b_{ge} *traverse le filtre* h*. Ainsi, l'expression du bruit en sortie du filtre est la suivante :*

$$b_y(n) = \sum_{i=0}^{N-1} b_{gm.i}(n) + b_{ge}(n) * h(n) \tag{7.132}$$

soit

$$b_y(n) = \sum_{i=0}^{N-1} b_{gm.i}(n) + \sum_{i=0}^{N-1} c_i.b_{ge}(n-i) \tag{7.133}$$

Si nous considérons qu'une loi de quantification par arrondi est utilisée pour chaque changement de format, alors la moyenne du bruit associé à chaque source est nulle. La moyenne du bruit en sortie est nulle.

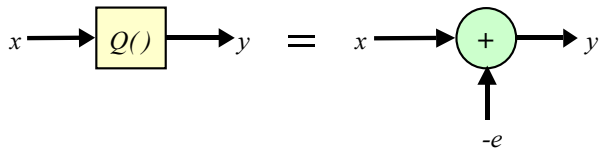
Les différentes sources de bruit ne sont pas corrélées entre elles et le bruit en entrée est un bruit blanc. En utilisant l'expression 7.143 pour le bruit b_{ge} *la variance du bruit en sortie du filtre est la suivante*

$$\sigma_{b_y}^2 = \sum_{i=0}^{N-1} \sigma_{b_{gm.i}}^2 + \sigma_{b_{ge}}^2 \sum_{i=0}^{N-1} |c_i|^2 = N.\sigma_{b_{gm.i}}^2 + \sigma_{b_{ge}}^2 \sum_{i=0}^{N-1} |c_i|^2 \tag{7.134}$$

$$\sigma_{b_y}^2 = 32 * \frac{2^{-22}}{12}(1 - 2^{-40}) + \frac{2^{-30}}{12} * 3.95 \rightarrow -61.96\ dB \tag{7.135}$$

Evaluation de la précision

- Modélisation du processus de quantification :
  - Propriétés de la variable aléatoire $e(n)$:
    - $e(n)$ est stationnaire,
    - le bruit issu de la quantification d'un signal $x(n)$ n'est pas corrélé avec ce signal,
    - les bruits de quantification sont statistiquement indépendants,
    - $e(n)$ est un bruit blanc uniformément réparti,
    - $e(n)$ est bornée par le pas de quantification,
    - la densité de probabilité de $e(n)$ est uniforme sur l'intervalle de quantification,
    - l'ergodicité implique que les moyennes temporelles et statistiques sont équivalentes.



  - Moments de la variable aléatoire $e(n)$:
    - Loi de quantification par arrondi :

$$\mu_e = 0, \qquad \sigma_e^2 = \frac{q^2}{12} \tag{7.140}$$

    - Loi de quantification par troncature :

$$\mu_e = \frac{q}{2}, \qquad \sigma_e^2 = \frac{q^2}{12} \tag{7.141}$$

- Filtrage du bruit
  - Moments du bruit en sortie du filtre $h(n)$:

$$\mu_f = \mu_e \sum_{n=-\infty}^{+\infty} h(n) = \mu_e H(e^{j0}) \tag{7.142}$$

$$\sigma_f^2 = \sigma_e^2 \sum_{n=-\infty}^{+\infty} |h(n)|^2 = \frac{\sigma_e^2}{2\pi} \int_{-\pi}^{\pi} |H(e^{j\Omega})|^2 d\Omega \tag{7.143}$$

7.4 Exemple

Pour illustrer le processus de conversion en virgule fixe, l'exemple d'un filtre IIR est traité dans cette partie.

7.4.1 Description de l'application

Un filtre à réponse impulsionnelle infinie composé de deux cellules d'ordre 1 cascadées est considéré. La fonction de transfert de ce système est la suivante

$$H(z) = H_1(z).H_2(z) \tag{7.144}$$

avec

$$H_1(z) = \frac{N_1(z)}{D_1(z)} = \frac{c_1}{1 - a_1.z^{-1}} \quad \text{et} \quad H_2(z) = \frac{N_2(z)}{D_2(z)} = \frac{c_2}{1 - a_2.z^{-1}} \tag{7.145}$$

Les valeurs des coefficients sont les suivantes :

$$c_1 = 0.35 \quad a_1 = 0.8 \tag{7.146}$$
$$c_2 = 0.48 \quad a_2 = 0.6 \tag{7.147}$$

Le graphe flot de signal de cette application est présenté à la figure 7.38 :

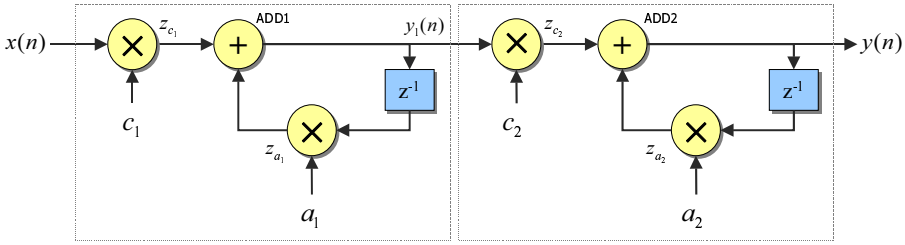


FIGURE 7.38 – Graphe flot de signal du filtre IIR

7.4.2 Détermination de la specification virgule fixe

7.4.2.1 Détermination de la dynamique

Pour l'évaluation de la dynamique, nous utilisons la norme de Chebyshev basée sur l'analyse de la réponse fréquentielle. Nous devons étudier la dynamique des données présentes en sortie de chaque addition ou série d'additions. Ainsi, nous déterminons la dynamique des données y_1 et y.

Dynamique de y_1.

La fonction de transfert entre la donnée y_1 et l'entrée globale x est la suivante :

$$\frac{Y_1(z)}{X(z)} = H_1(z) = \frac{c_1}{1 - a_1.z^{-1}} \tag{7.148}$$

Le module de la réponse fréquentielle de ce système est le suivant :

$$\left|H_1(e^{j\Omega})\right| = \frac{c_1}{\left(1 + a_1^2 - 2a_1\cos(\Omega)\right)^{1/2}} \tag{7.149}$$

Les extrema de cette réponse fréquentielle sont obtenus pour $\Omega = 0$ et $\Omega = \pi$. Dans notre cas nous obtenons les valeurs suivantes :

$$|H(e^{j0})| = \frac{c_1}{|1 - a_1|} = 1.75 \tag{7.150}$$

$$|H(^{j\pi})| = \frac{c_1}{|1 + a_1|} = 0.4375 \tag{7.151}$$

Le maximum de cette réponse fréquentielle est obtenu pour $\Omega = 0$ et cette fonction est monotone et décroissante sur $[0 - \pi]$ si a_1 est positif. Ainsi, la dynamique de la sortie du filtre H_1 est la suivante :

$$\max_n\left(|y_1(n)|\right) = \max_n\left(|x(n)|\right)\max_\Omega\left(\left|H_1(e^{j\Omega})\right|\right) = 1.75 \tag{7.152}$$

Dynamique de y.

La fonction de transfert entre la donnée y et l'entrée globale x est la suivante

$$\frac{Y(z)}{X(z)} = H(z) = H_1(z).H_2(z) \tag{7.153}$$

Pour simplifier la recherche du maximum de la réponse fréquentielle de $\left|H(e^{j\Omega})\right|$ nous utilisons les caractéristiques de chaque cellule. Chaque cellule correspond à un filtre IIR d'ordre 1 et passe-bas. Les réponses fréquentielles de H_1 et H_2 étant monotones et décroissantes, alors le maximum est obtenu pour $\Omega = 0$.

$$\left|H(e^{j0})\right| = H(z)|_{z=1} = \left|\frac{c_1 c_2}{(1 - a_1)(1 - a_2)}\right| = 2.1 \tag{7.154}$$

Ainsi la dynamique de la sortie du filtre H est la suivante :

$$\max_n\left(|y(n)|\right) = \max_n\left(|x(n)|\right)\max_\Omega\left(\left|H(e^{j\Omega})\right|\right) = 2.1 \tag{7.155}$$

7.4.2.2 Détermination de la position de la virgule

Les positions de la virgule des données x, y_1 et y sont les suivantes :

$$\begin{array}{lll} x \in]-1;1[& \rightarrow & m_x = 0 \\ y_1 \in]-1.75;1.75[& \rightarrow & m_{y_1} = 1 \\ y \in]-2.1;2.1[& \rightarrow & m_y = 2 \end{array} \tag{7.156}$$

La position de la virgule des différents coefficients du filtre est la suivante :

$$\begin{array}{lll} c_1 = 0.35 & \rightarrow & m_{c_1} = -1 \\ a_1 = 0.8 & \rightarrow & m_{a_1} = 0 \\ c_2 = 0.48 & \rightarrow & m_{c_2} = -1 \\ a_2 = 0.6 & \rightarrow & m_{a_2} = 0 \end{array} \tag{7.157}$$

Filtre H_1

La position de la virgule en sortie de la multiplication de c_1 par x est la suivante :

$$m_{z_{c1}} = m_x + m_{c_1} + 1 = 0 - 1 + 1 = 0 \tag{7.158}$$

La position de la virgule en sortie de la multiplication de a_1 par y_1 est la suivante :

$$m_{z_{a1}} = m_{y_1} + m_{a_1} + 1 = 1 + 0 + 1 = 2 \tag{7.159}$$

La position de la virgule commune pour l'addition `ADD1` est la suivante :

$$m_{ADD1} = \max\left(m_{z_{c1}}, m_{z_{a1}}, m_{y_1}\right) = \max\left(0, 2, 1\right) = 2 \tag{7.160}$$

Pour aligner la virgule en entrée de l'additionneur, il est nécessaire d'introduire un décalage à droite de $d_{z_{c1}}$ bits sur la sortie de la multiplication de c_1 par x :

$$d_{z_{c1}} = m_{ADD1} - m_{z_{c1}} = 2 - 0 = 2 \tag{7.161}$$

En sortie de l'additionneur, pour retrouver le format de la donnée, il est nécessaire d'introduire un décalage à gauche de :

$$d_{z_{y1}} = m_{ADD1} - m_{y_1} = 1 - 0 = 1 \tag{7.162}$$

Filtre H_2

La position de la virgule en sortie de la multiplication de c_2 par y_1 est la suivante :

$$m_{z_{c2}} = m_{y_1} + m_{c_2} + 1 = 1 - 1 + 1 = 1 \tag{7.163}$$

La position de la virgule en sortie de la multiplication de a_2 par y est la suivante :

$$m_{z_{a2}} = m_y + m_{a_2} + 1 = 2 + 0 + 1 = 3 \tag{7.164}$$

La position de la virgule commune pour l'addition `ADD2` est la suivante :

$$m_{ADD2} = \max\left(m_{z_{c2}}, m_{z_{a2}}, m_y\right) = \max\left(1, 3, 2\right) = 3 \tag{7.165}$$

Pour aligner la virgule en entrée de l'additionneur, il est nécessaire d'introduire un décalage à droite de $d_{z_{c2}}$ bits sur la sortie de la multiplication de c_2 par y_1 :

$$d_{z_{c2}} = m_{ADD2} - m_{z_{c2}} = 3 - 1 = 2 \tag{7.166}$$

En sortie de l'additionneur, pour retrouver le format de la donnée, il est nécessaire d'introduire un décalage à gauche de :

$$d_{z_y} = m_{ADD2} - m_y = 3 - 2 = 1 \tag{7.167}$$

Pour limiter le temps d'exécution des opérations de recadagre, les décalages à droite en sortie des multiplications peuvent être déplacés vers l'entrée du filtre ou vers les coefficients. Nous déplaçons les décalages à droite de deux bits $d_{z_{c1}}$ et $d_{z_{c2}}$ vers les coefficients c_1 et c_2. Ceci nécessite d'intégrer 2 bits supplémentaires à la partie entière de ces deux coefficients.

7.4.2.3 Détermination de la largeur des données

Pour implanter ce filtre H, nous utilisons une architecture sur 16 bits et double précision dont le synoptique est présenté à la figure 7.21. La spécification virgule fixe obtenue est présentée à la figure 7.39 avant le déplacement des opérations de décalage et à la figure 7.40 après le déplacement des opérations de décalage.

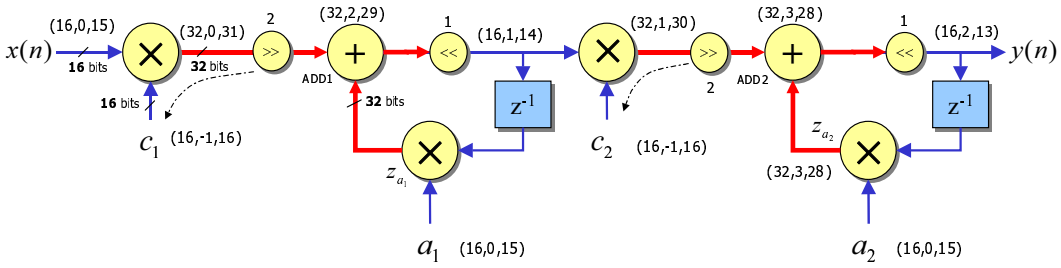


FIGURE 7.39 – Graphe flot de signal du filtre IIR avant le déplacement des opérations de décalage

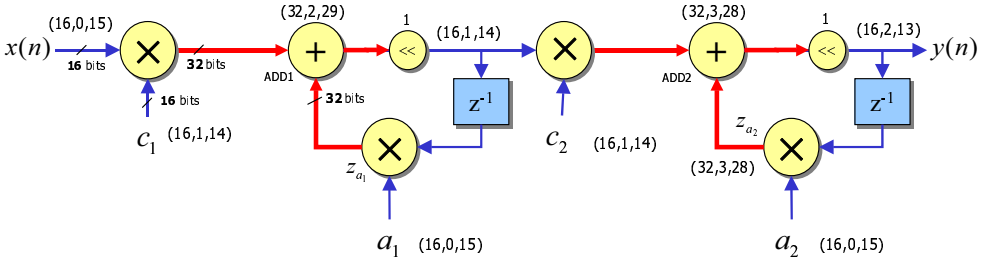


FIGURE 7.40 – Graphe flot de signal du filtre IIR après le déplacement des opérations de décalage

7.4.3 Code C de l'application

Déclaration des variables et des coefficients :

```
int x;
int y1[2];
int y[2];

int c1 = 22938;
int c2 = 31457;
int a1 = 26214;
int a2 = 19661;
```

Code pour le calcul de la sortie du filtre

```
y1[0] = (int)( ( (long)(c1 * x) + (long)(a1 * y1[1]) ) >> 16 );
y1[1] = y1[0];
```

```
y[0] = (int)( ( (long)(c2 * y1[0]) + (long)(a2 * y[1]) >> 16 );
y[1] = y[0];
```

L'expression `(long)( ... )` a pour objectif de forcer le compilateur à conserver les 32 bits du résultat et ne pas tronquer à 16 bits avant de faire l'addition.

L'expression `y = (int) ( ) ■ 16;` a pour objectif de renvoyer en mémoire les bits les plus significatifs du résultat (partie haute de l'accumulateur).

7.4.4 Evaluation de la précision

Pour la spécification virgule fixe présentée à la figure 7.40, les différentes sources de bruit, représentée à la figure 7.41, sont les suivantes :

— b_{ge} : bruit associé à l'entrée. Nous considérons que le signal x est issu d'un convertisseur analogique-numérique et est codé sur 16 bits avec 15 bits pour la partie fractionnaire. La puissance de ce bruit est la suivante :

$$\sigma^2_{b_{ge}} = \frac{q_x^2}{12} = \frac{2^{-2.n_x}}{12} = \frac{2^{-30}}{12} \rightarrow -101.1\ dB \tag{7.168}$$

— b_{g1} : bruit présent en sortie de l'additionneur lié au renvoi en mémoire de la donnée y_1. Il est nécessaire de repasser cette donnée sur 16 bits car elle est ensuite utilisée pour réaliser une multiplication. Lors de ce changement de format, 15 bits sont éliminés au niveau de la partie fractionnaire.

$$\sigma^2_{b_{g1}} = \frac{q_{y_1}^2}{12}(1 - 2^{-2.k_{y1}}) = \frac{2^{-28}}{12}(1 - 2^{-30}) \rightarrow -95.1\ dB \tag{7.169}$$

— b_{g2} : bruit présent en sortie de l'additionneur lié au renvoi en mémoire de la donnée y. Lors de ce changement de format, 15 bits sont éliminés au niveau de la partie fractionnaire.

$$\sigma^2_{b_{g2}} = \frac{q_y^2}{12}(1 - 2^{-2.k_y}) = \frac{2^{-26}}{12}(1 - 2^{-30}) \rightarrow -89.1\ dB \tag{7.170}$$

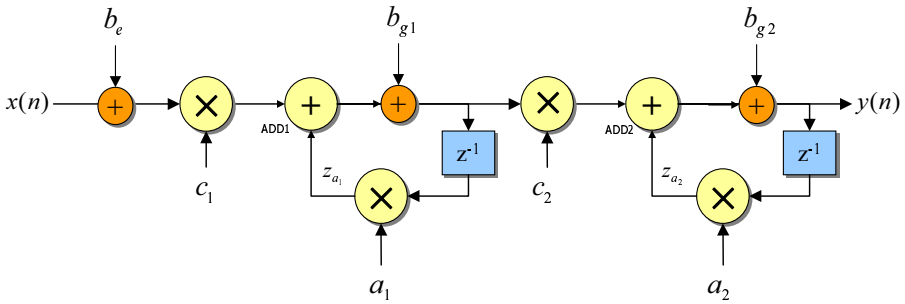


FIGURE 7.41 – Graphe flot de signal du filtre IIR intégrant les sources de bruit

L'expression du bruit en sortie du premier filtre est la suivante

$$b_{y_1}(n) = c_1.b_e(n) + b_{g1}(n) + a_1.b_{y_1}(n-1) \tag{7.171}$$

En appliquant la transformé en $\mathcal{Z}$ et en utilisant l'expression 7.145 nous obtenons l'expression suivante

$$B_{y_1}(z) = \frac{1}{D_1(z)}\left(N_1(z).B_e(z) + B_{g1}(z)\right) \tag{7.172}$$

L'expression du bruit en sortie du second filtre est la suivante

$$b_y(n) = c_2.b_{y_1}(n) + b_{g2}(n) + a_2.b_y(n-1) \tag{7.173}$$

En appliquant la transformé en $\mathcal{Z}$ et en utilisant l'expression 7.145 nous obtenons l'expression suivante

$$B_y(z) = \frac{1}{D_2(z)}\left(N_2(z).B_{y_1}(z) + B_{g2}(z)\right) \tag{7.174}$$

L'expression globale du bruit en sortie du système est la suivante :

$$B_y(z) = H_1(z).H_2(z).B_e(z) + \frac{H_2}{D_2(z)}B_{g1}(z) + \frac{1}{D_1(z)}.B_{g2}(z) \tag{7.175}$$

Si nous considérons une loi de quantification par arrondi, alors la puissance du bruit en sortie du filtre est la suivante :

$$\sigma^2_{b_y} = \frac{\sigma^2_{b_{ge}}}{2\pi}.\int_{-\pi}^{\pi}\left|H(e^{j\Omega})\right|^2 d\Omega + \frac{\sigma^2_{b_{g1}}}{2\pi}.\int_{-\pi}^{\pi}\left|\frac{H_2(e^{j\Omega})}{D_1(e^{j\Omega})}\right|^2 d\Omega + \frac{\sigma^2_{b_{g2}}}{2\pi}.\int_{-\pi}^{\pi}\left|\frac{1}{D_2(e^{j\Omega})}\right|^2 d\Omega \tag{7.176}$$

ou

$$\sigma^2_{b_y} = \sigma^2_{b_{ge}}.\sum_{n=-\infty}^{+\infty}|h(n)|^2 + \sigma^2_{b_{g1}}.\sum_{n=-\infty}^{+\infty}|h_{g1}(n)|^2 + \sigma^2_{b_{g2}}.\sum_{n=-\infty}^{+\infty}|h_{g2}(n)|^2 \tag{7.177}$$

avec $h_{g1}(n)$ la réponse impulsionnelle du système $\frac{H_2}{D_1(z)}$ et $h_{g2}(n)$ la réponse impulsionnelle du système $\frac{1}{D_2(z)}$.

L'application numérique conduit à :

$$\sigma^2_{b_y} = 3.06 \times 7.76\ 10^{-11} + 12 \times 3.1\ 10^{-10} + 2.5 \times 1.24\ 10^{-9} = 7.06\ 10^{-9} \rightarrow -81.5\ dB \tag{7.178}$$

Chapitre 8

Processeurs de traitement du signal

8.1 Introduction

8.1.1 Les solutions architecturales

Dans ce paragraphe, les différentes solutions architecturales pour l'implantation des algorithmes de traitement numérique du signal (TNS) sont présentées. Pour chaque solution, les caractéristiques en termes de coût, de consommation d'énergie, de performances, de flexibilité et de temps de développement sont détaillées. De plus, les conséquences en termes de stratégie de codage des données en virgule fixe sont exposées. Pour illustrer cette analyse, certains des critères énumérés ci-dessus sont représentés à la figure 8.1.

8.1.1.1 Processeurs à usage général

L'objectif des processeurs à usage général est de pouvoir implanter une panoplie importante d'algorithmes. En conséquence, leur architecture n'est pas spécialisée pour un domaine particulier tel que celui du traitement numérique du signal (TNS). L'architecture homogène du processeur conduit à une consommation d'énergie et un coût élevés. Ainsi, les performances en termes d'efficacité énergétique et de ratio performance coût sont relativement faibles pour les algorithmes de TNS. Dans le cas des processeurs à usage général destinés aux applications embarquées, des efforts sont réalisés afin de diminuer la consommation du processeur. De plus, face à l'accroissement du nombre d'algorithmes de TNS au sein des nouvelles applications, de nombreux processeurs à usage général intègrent des extensions permettant d'accélérer les traitements associés au domaine du TNS. Cependant, les performances en termes d'efficacité énergétique et de ratio performance coût restent inférieures à celles obtenues avec les processeurs de traitement du signal.

L'architecture homogène de ce type de processeur permet d'obtenir des compilateurs de langage de haut niveau efficaces. L'obtention d'un code assembleur de qualité à partir d'une description en langage de haut niveau permet d'éviter de longues phases de développement de code en assembleur. Ainsi, les temps de développement de ces applications peuvent être relativement réduits.

8.1.1.2 Circuits dédiés à une application

Les circuits dédiés (ASIC [1]) sont conçus et optimisés pour une application précise. Cette spécialisation du circuit à l'application cible permet d'obtenir la solution fournissant les meilleures performances en termes de temps de calcul, d'efficacité énergétique et de ratio coût performance. En contrepartie, ces circuits offrent une flexibilité très réduite, voire nulle. La complexité des différentes phases de conception, de validation, de test et de réalisation de ces circuits conduit à des coûts et des temps de développement élevés. De plus, cette tendance s'est aggravée avec l'évolution des technologies utilisées au niveau du processus de fabrication des circuits intégrés. Pour réduire les coûts et les temps de développement de ces circuits, le concept de réutilisation de blocs existants s'est généralisé. Le concepteur réalise l'assemblage de différents blocs IP (Intellectual Properties) déjà testés et validés.
La spécialisation de l'architecture à l'application est réalisée en partie par le dimensionnement des différents opérateurs. Ainsi, le codage des données en virgule fixe doit minimiser la surface du circuit tant que la contrainte de précision souhaitée en sortie de l'algorithme est respectée.

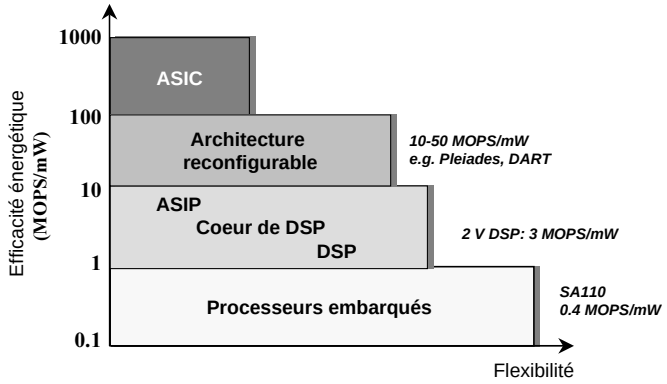


FIGURE 8.1 – Caractéristiques des solutions architecturales en termes d'efficacité énergétique et de flexibilité [Rab99]

8.1.1.3 Processeurs de traitement du signal

Nous regroupons sous l'appellation processeurs de traitement du signal, les processeurs DSP disponibles sous forme de composant, les cœurs de DSP et les ASIP [2] dédiés aux applications de TNS.
Les processeurs DSP et les cœurs de DSP sont conçus pour implanter efficacement les algorithmes spécifiques au domaine du traitement du signal. Leur architecture est programmable et regroupe les différents éléments nécessaires aux opérations de TNS. Deux catégories de processeurs DSP sont présentes sur le marché. La première catégorie correspond aux processeurs DSP conçus afin d'obtenir une efficacité énergétique élevée et un coût relativement faible. Ces caractéristiques sont obtenues en spécialisant l'architecture du processeur. En contrepartie, les compilateurs de langage de haut niveau ne permettent pas de fournir un code assembleur

1. Application Specific Integrated Circuit
2. Application Specific Instruction-set Processor

de qualité par rapport à un code développé et optimisé à la main. Ainsi, les temps de développement des applications sont relativement importants en raison du développement manuel en assembleur des parties critiques du code. La seconde catégorie des processeurs DSP correspond aux processeurs conçus afin d'obtenir des performances élevées en termes de capacité de calcul. Pour cela, des architectures homogènes et intégrant de nombreuses unités fonctionnelles pouvant travailler en parallèle sont mises en œuvre. Les compilateurs de langage de haut niveau permettent d'obtenir un code de qualité si ceux-ci exploitent efficacement le parallélisme offert par l'architecture. En contrepartie, l'efficacité énergétique et le prix sont plus élevés par rapport aux processeurs DSP spécialisés. Des solutions réalisant un compromis entre les deux approches sont également proposées.
Les ASIP sont des processeurs programmables dédiés à une classe d'applications composée de quelques algorithmes. Les ASIP représentent un intermédiaire entre les processeurs DSP et les ASIC [Mar95]. Deux familles d'ASIP correspondant à deux approches de conception différentes peuvent être définies. La première famille correspond à des processeurs proches des ASIC. Ils ont été développés afin d'obtenir une flexibilité plus importante par rapport aux ASIC à travers la possibilité de programmer le circuit. Ces processeurs vont permettre, pour les quelques applications cibles, d'obtenir des performances se rapprochant de celles des ASIC en termes d'efficacité énergétique et de ratio coût performance. La seconde famille d'ASIP correspond aux processeurs conçus à partir d'un cœur de DSP. Dans ce cas, le cœur est paramétré en fonction des applications cibles et certains accélérateurs sont intégrés afin d'obtenir de meilleures performances. La spécialisation de l'architecture à la classe d'algorithmes visés permet d'obtenir des performances en termes d'efficacité énergétique, supérieures à celles des processeurs DSP. Comme pour les processeurs et les cœurs de DSP, l'efficacité des compilateurs de langage de haut niveau dépend de la spécialisation du processeur et de l'adéquation entre les différentes phases de la génération de code et de l'architecture.
Dans le cadre des DSP, l'architecture étant figée, le processus de codage des données en virgule fixe doit permettre d'obtenir le meilleur compromis entre la précision des calculs et différents paramètres tels que le temps d'exécution, la taille du code ou la consommation d'énergie. De plus, pour les cœurs de DSP ou les ASIP dont la largeur du chemin de données est paramétrable, la méthodologie doit explorer les différentes solutions possibles.

8.1.1.4 Architectures reconfigurables

Afin d'obtenir des performances élevées et un niveau de flexibilité relativement important, différents types d'architectures reconfigurables ont été proposés.
La reconfiguration au niveau logique correspond à celle présente au niveau des composants programmables de type FPGA. Cette reconfiguration est réalisée au niveau des éléments logiques de l'architecture et de leurs interconnexions. Les configurations appliquées à ce type d'architecture nécessitent un volume d'information très important, ainsi elles sont maintenues pendant un nombre de cycles d'horloge élevé et elles ne sont pas modifiées au cours du traitement réalisé. La reconfiguration au niveau logique permet d'adapter les opérateurs et leurs interconnexions à l'application ciblée. Ainsi, les largeurs des différents opérateurs peuvent être optimisées en fonction de la contrainte de précision. Cependant, cette flexibilité au niveau des opérateurs est obtenue au détriment des performances en termes de temps d'exécution et de consommation d'énergie de ces opérateurs. En effet, l'utilisation de cellules standards ne permet pas d'optimiser les opérateurs arithmétiques par rapport à une solution *full-custom*.

Ainsi, ce type d'architecture est préconisé pour les algorithmes manipulant des données dont la largeur est relativement faible. Les méthodologies proposées [CCL01] pour le codage des données en virgule fixe sont très proches de celles utilisées pour les ASIC.

La reconfiguration au niveau fonctionnel permet de modifier le flot des données au sein de l'architecture. Les interconnexions entre les ressources et la fonctionnalité de ces ressources peuvent être reconfigurées. Ce type de reconfiguration permet d'adapter le modèle de programmation et les motifs de calcul à l'application ciblée. Si le volume d'information nécessaire à la reconfiguration de l'architecture n'est pas trop important, alors la reconfiguration peut être réalisée dynamiquement au cours de l'exécution du code.
Comme pour les DSP, la largeur des opérateurs est fixe et ne peut prendre que quelques valeurs prédéfinies. Ainsi, les méthodologies de codage des données en virgule fixe pour ce type d'architecture sont relativement proches de celles utilisées pour les DSP. La différence majeure réside dans le critère à optimiser au niveau de l'architecture pour obtenir une implantation de qualité.

8.1.1.5 Système sur puce SoC

Pour répondre aux défis de l'implantation des systèmes complexes tels que les nouveaux algorithmes destinés aux télécommunications de troisième génération, des systèmes intégrant sur une même puce différentes solutions hétérogènes sont mis en œuvre. Ces systèmes peuvent regrouper les différents éléments présentés ci-dessus. L'objectif est d'obtenir une plate-forme relativement flexible et offrant des performances élevées à travers l'intégration d'accélérateurs adaptés aux parties critiques de l'application. La flexibilité est présente à travers les parties programmables ou reconfigurables.
La méthodologie de codage des données dans le cadre d'un système sur puce nécessite de mettre en œuvre une méthodologie globale permettant d'optimiser plus ou moins en parallèle le codage des données pour les différentes solutions architecturales.

8.1.2 Domaine d'application des processeurs de traitement du signal

Le marché des DSP a connu une forte augmentation en termes de volume de production à la fin des années 90 et représentait en 2001 un marché de 4,8 milliards de $ [Str02a]. Cette forte croissance était liée à l'explosion du principal marché des DSP que représente le domaine des communications sans fil cellulaires. Ces processeurs sont aussi présents dans le domaine des modems et du contrôle moteur pour les disques durs. Après une diminution importante des ventes en 2001, la croissance moyenne entre 2001 et 2005 est de 17%. Cette évolution est liée à la reprise de la croissance au niveau mondial, du marché des communications sans fil avec les systèmes de troisième génération (UMTS), le GPRS, le WiFi et le bluetooth et à l'émergence de nouveaux marchés dans le domaine du multimédia et d'internet (voix sur IP [Str02b], décodeurs MP3 [Str00], caméras numériques). La société Forward Concepts, prévoit une croissance des ventes de DSP de 25% par an dans les années à venir [Str02a] avec le développement des systèmes de communication de troisième génération .
Les ASIP sont utilisés pour des applications nécessitant des performances supérieures aux DSP en termes de puissance de calcul, de consommation d'énergie ou de coût. Ils sont présents dans le domaine du multimédia (télévision numérique, compression audio et vidéo et graphisme 3D)

et des télécommunications (communications sans fil et par satellites, ATM) [Pa97].

8.2 Description de l'unité de traitement

8.2.1 Représentation des données

8.2.1.1 Processeurs virgule fixe et virgule flottante

Les DSP sont divisés en deux grandes catégories en fonction du codage des données en virgule fixe ou en virgule flottante. Ces deux catégories possèdent des caractéristiques et des propriétés propres qui les orientent vers des marchés différents.

Prix et consommation d'énergie Les prix et la consommation des DSP virgule fixe sont plus faibles que ceux des DSP virgule flottante, car la taille des données qu'ils manipulent est plus faible et l'unité de traitement est moins complexe.

La figure 7.3 (voir page 78) montre la dynamique et la précision obtenues pour différentes largeurs de données dans le cas de l'utilisation d'un format en virgule fixe et en virgule flottante. Lorsque la taille des données est relativement faible (16 bits) les caractéristiques en terme de précision et de dynamique des deux formats sont proches. La représentation des données en virgule flottante nécessite l'utilisation de deux parties la mantisse et l'exposant. Ainsi lorsque le nombre de bits utilisés est relativement faible, un compromis entre la dynamique et la précision doit être effectué et peu de bits peuvent être utilisés pour l'exposant. En conséquence, la dynamique obtenue n'est pas supérieure à celle des DSP virgule fixe. Pour des largeurs de données relativement faibles, les DSP sont exclusivement réalisés avec un format en virgule fixe. Le format virgule flottante ne montre de l'intérêt que pour des largeurs de données plus élevées (32 bits), permettant alors d'obtenir d'excellentes performances en terme de précision et de dynamique de codage. Le prix d'un processeur est proportionnel à sa surface et la majorité de celle-ci est occupée par les bus et la mémoire. La largeur des données manipulées par les processeurs virgule fixe étant plus faible que celle des processeurs en virgule flottante, le coût et la consommation du circuit sont moins importants.

L'unité de calcul des processeurs en virgule flottante est plus complexe car elle doit réaliser les opérations de calcul sur la mantisse et l'exposant. Ceci augmente le coût, la consommation et le temps de latence de ce type d'unité de traitement. Dans [Tou99] les expérimentations réalisées par l'auteur montrent que le temps de latence d'un opérateur de type additionneur en virgule flottante est 2,3 fois supérieur à celui d'un opérateur en virgule fixe et que la surface occupée est 4 fois supérieure.

Temps de développement des applications Lors du développement d'applications pour des processeurs virgule fixe, le concepteur doit réaliser le codage et le cadrage des données présentes au sein de l'algorithme. Il analyse la dynamique de l'ensemble des données, en déduit le codage et détermine les décalages nécessaires pour aligner ces données lors des opérations d'addition ou de soustraction. Le codage et le cadrage des données doivent être optimisés afin de garantir l'absence de débordements au sein de l'algorithme tout en maintenant la précision

maximale.

Les processeurs virgule flottante proposent une dynamique élevée qui permet de libérer l'utilisateur des tâches de codage des données et d'analyse des débordements éventuels. L'alignement des données est automatiquement réalisé par le matériel, au sein de l'unité de calcul. Ainsi le temps de développement d'applications pour ce type de processeurs est plus faible par rapport à celui des processeurs virgule fixe.

Marchés des processeurs Les DSP virgule fixe sont bien adaptés pour les applications destinés au grand public ou le coût de revient des composants constituant le système doit être le plus faible possible. Le surcoût du développement lié à la spécificité du codage en virgule fixe est rapidement amorti par les grands volumes de production. La faible consommation de ce type de composant permet de l'utiliser au sein des applications embarquées. Ces DSP sont très utilisés dans les secteurs en forte croissance tels que les télécommunications et le multimédia.

Les processeurs en virgule flottante sont destinés aux applications nécessitant une dynamique importante ou une précision élevée pour lesquelles un processeur en virgule fixe ne peut répondre à ces exigences. Ces applications sont par exemple présentent dans le domaine de l'audionumérique. Les processeurs en virgule flottante sont utilisés pour des applications réalisées en petite série ou le coût du développement représente une part très importante du coût total du produit.

Le marché des processeurs DSP est dominé par les processeurs en virgule fixe (95% des DSP en 1996) et plus particulièrement par les processeurs dont la largeur des données est de 16 bits (95% des DSP virgule fixe en 1996) [Lap96]

8.2.1.2 Format des données

Largeur naturelle La largeur naturelle des données correspond au format des données qui sont manipulées le plus efficacement par le bus et le chemin de données du processeur [LBSL96]. La majorité des opérations sur ces données est réalisée en un cycle.

La largeur naturelle des données est un paramètre fondamental au niveau du DSP car elle influence directement le prix de ce dernier en fixant la largeur minimale des opérateurs, des bus de données et de la mémoire. Afin d'adapter au mieux le DSP à l'application souhaitée, certains fabricants proposent des coeurs de DSP pouvant être synthétisés et dont la largeur naturelle est paramétrable. Ainsi le concepteur va choisir la largeur minimale répondant aux critères de précision souhaités.

Extension de la précision Les DSP permettent d'utiliser au sein de l'unité de traitement un format de données étendu. La largeur des bus d'interconnexion entre les différentes unités fonctionnelles du chemin de données est supérieure à la largeur naturelle des données afin de conserver l'ensemble des bits issus des opérateurs. Ceci permet de maintenir une précision plus élevée sur une suite d'opérations dont les opérandes restent dans l'unité de traitement et ne sont pas renvoyées en mémoire. L'efficacité de cette méthode est fortement liée au nombre

d'unités de stockage (registres) disponibles au sein du chemin de données.

L'arithmétique multi-précision permet de manipuler des données de largeur plus importante, obtenues par concaténation de plusieurs données au format naturel. Une donnée x en double précision pour un processeur 16 bits est représentée à la figure 8.2.a. Cette donnée de largeur 32 bits est composée d'une donnée 16 bits signée x_H et d'une donnée 16 bits non signée x_L.

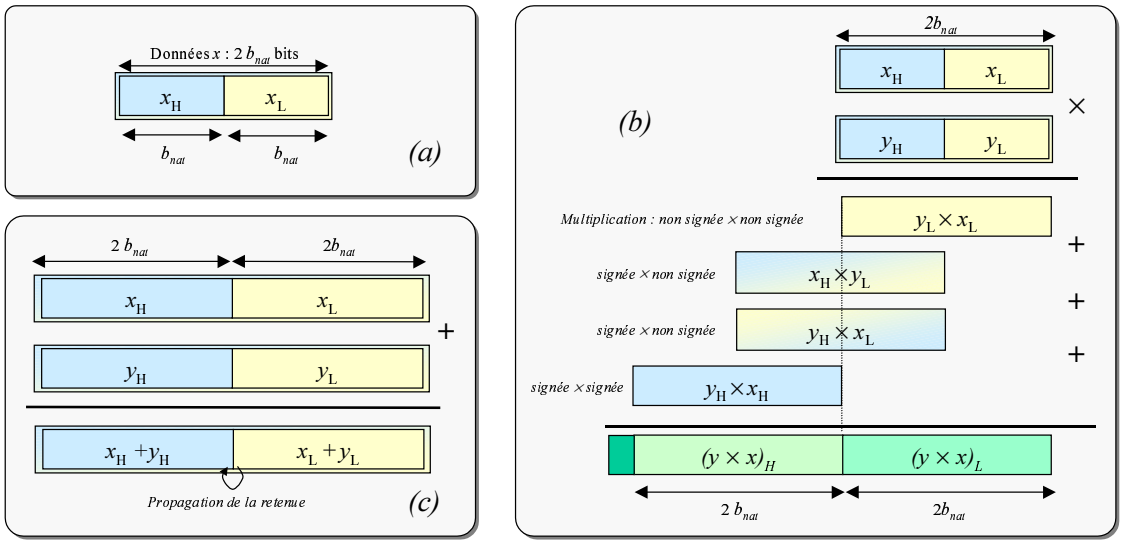


FIGURE 8.2 – Arithmétique double précision

Une opération utilisant des données en multi précision est décomposée en une suite d'opérations manipulant des opérandes au format naturel. La figure 8.2.b représente les différentes opérations à réaliser pour effectuer une multiplication. Afin de pouvoir implanter ce type de format le processeur doit posséder des opérateurs capables de travailler sur des données signées et non signées et permettre de conserver la retenue d'une opération afin de l'injecter dans la suivante.
Étant donné le nombre d'opérations à réaliser pour le calcul d'une opération de base en double précision, le temps d'exécution de celle-ci est fortement augmenté par rapport à une arithmétique simple précision. Ceci limite l'utilisation de cette arithmétique à des portions de code critiques en terme de précision.

8.2.2 Présentation des différents éléments du chemin de données

Dans une première partie nous présentons l'ensemble des éléments constituant les unités de traitement et explicitons leurs caractéristiques, nous analysons ensuite les différents types d'architecture des unités de traitement.

8.2.2.1 Multiplieur

Les DSP possèdent un multiplieur câblé permettant de réaliser une multiplication par cycle. Le temps de latence entre la lecture des données et la disponibilité des résultats de certains

multiplieurs étant supérieur à 1 cycle, l'obtention d'une multiplication par cycle est réalisée à l'aide du pipeline interne.

La majorité des DSP possède un multiplieur dont la largeur des données d'entrée est égale à la largeur naturelle et permet d'utiliser des données signées et non signées afin de supporter l'arithmétique multi-précision. La multiplication de deux nombres de taille N génère une donnée de taille $2N$, la plupart des DSP possède un multiplieur fournissant l'ensemble des $2N$ bits.

Lorsque nous sommes en arithmétique fractionnaire, la multiplication entraîne le doublement du bit de signe. Certains DSP permettent d'éliminer automatiquement le bit redondant en réalisant un décalage à gauche d'un bit. Si les données d'entrée sont codées en virgule fixe cadrée à gauche, cette technique permet d'avoir un format du résultat identique au format en entrée.

8.2.2.2 Unité Arithmétique et Logique : UAL

L'unité arithmétique et logique permet de réaliser les différentes fonctionnalités décrites ci-dessous avec un 1 cycle :

- opérations logiques : AND, OR, NOT ;
- opérations arithmétiques : addition, soustraction, incrémentation, décrementation, valeur absolue. Certaines UAL permettent de réaliser une division en plusieurs cycles.

Les UAL sont conçues pour travailler sur des données dont le format est identique à celui de l'accumulateur, mais certains DSP tels que le ADSP21xx [Ana95] possèdent une UAL ne pouvant manipuler que des données au format naturel. Dans ce cas une opération sur un registre d'accumulation nécessite plusieurs cycles.

8.2.2.3 Registres

Structure des registres Avant de présenter les structures des registres utilisées dans les unités de traitement nous définissons les deux termes suivants :

jeu de registres : ensemble des registres présents dans le chemin de données et destinés à stocker les opérandes et les résultats intermédiaires.

classe de registres : sous-ensemble de registres du jeu de registres possédant les mêmes fonctionnalités

Dans les processeurs, deux types de structure du jeu de registres sont présentes. La première consiste à utiliser un jeu de registres homogène dont tous les registres le composant sont interchangeables. Cette technique nécessite d'utiliser un banc de registres intégrant plusieurs ports de communication permettant d'accéder en lecture et en écriture aux différents registres en parallèle. Si un opérateur utilise deux opérandes issues d'une file de registres alors cette dernière doit posséder 5 ports (2 pour la lecture des opérandes par l'opérateur, 2 pour l'écriture des opérandes issues de la mémoire et 1 pour l'écriture du résultat). Cette solution est très coûteuse en terme de surface de silicium, mais garantit l'homogénéité de la structure de registres. Le DSP TMS320C62xx [Tex99b] est composé de deux files de 16 registres de 32 bits. Les deux files de registres possèdent 9 ports de lecture (32 bits) et 6 ports d'écriture

(32bits). Cette approche est utilisée dans les processeurs généraux de type RISC, les DSP virgule flottante et les nouveaux DSP virgule fixe.

La seconde approche consiste à spécialiser les registres et à les relier directement aux unités fonctionnelles, afin de réduire le nombre et la complexité des connexions. Une classe de registres est utilisée pour chaque opérande et le résultat de l'opération est stocké dans une troisième classe de registres. Cette spécialisation de la structure des registres permet de diminuer les coûts et d'augmenter la rapidité des transferts entre les différentes unités. Le jeu d'instructions associé à cette architecture est spécialisé car chaque instruction utilisant un opérateur, spécifie précisément le ou les registres qui peuvent être utilisés. Les DSP de première et de seconde génération [Lap96] utilisent ce type d'approche afin d'optimiser le coût. Ces DSP possèdent de nombreuses classes de registres (2 à 8) composées de peu de registres (1 à 8).

Il est possible de quantifier l'homogénéité du jeu de registres en calculant le rapport entre le nombre de classes de registres et le nombre total de registres. Pour un jeu de registres homogène cette métrique va tendre vers 0, et dans le cas de registres hétérogènes cette métrique tendra vers 1. L'homogénéité des registres a une influence importante sur la qualité du code généré par les compilateurs de langage de haut niveau.

Registres d'accumulation Ces registres reçoivent les valeurs intermédiaires issues des différents calculs réalisés dans l'unité de traitement. La taille de ces registres est déterminée afin de pouvoir stocker, avec le maximum de précision, les résultats intermédiaires des opérations de type *multiplication accumulation (MAC)* :

$$y_{ACC} = \sum_{i=1}^{M} x[i]y[i] \tag{8.1}$$

La multiplication de deux opérandes au format naturel fournit un résultat dont la largeur est égale au double de la largeur naturelle. Les accumulations successives d'opérandes issues du multiplieur augmentent la dynamique du résultat intermédiaire et nécessitent des bits supplémentaires pour le coder. Dans certains DSP les registres d'accumulation possèdent des bits de garde qui permettent de réaliser une suite de plusieurs accumulations sans provoquer de débordements. La taille de ce registre d'accumulation est égale à :

$$N_{ACC} = 2N_n + N_g \tag{8.2}$$

Ces N_g bits de garde vont permettre de stocker le résultat d'au moins $N_{Add} = 2^{N_g}$ additions sans nécessiter de recadrer les données. Pour les processeurs possédant un jeu de registres hétérogène, le nombre de registres d'accumulation influence la qualité du code et la souplesse de programmation du processeur. La présence d'un faible nombre de registres d'accumulation nécessite de renvoyer les résultats intermédiaires en mémoire. Ceci ralentit l'exécution du programme, augmente la taille du code et diminue la précision du calcul.

8.2.2.4 Registres à décalage

Le codage des données en virgule fixe nécessite de réaliser de nombreux décalages afin d'aligner les données. Nous trouvons au sein des unités de traitement des DSP, deux types de registres à décalage. La première catégorie correspond aux registres à décalage spécialisés qui ne peuvent réaliser que quelques décalages prédéfinis. Ces décalages sont réalisés en parallèle à l'exécution des autres opérations et ne nécessitent pas de cycles d'horloge supplémentaires. Ces registres sont affectés à la sortie ou à l'entrée d'un opérateur précis afin de fournir une plus grande flexibilité pour le cadrage des données. Ces registres sont commandés dans la plupart des cas par des bits de mode. Cette spécialisation a été réalisée afin de minimiser la complexité du registre tout en permettant d'utiliser plusieurs formats. Ce type de registres est par exemple présent en sortie des multiplieurs afin de pouvoir éliminer le second bit de signe présent avec une arithmétique fractionnaire et dans certains cas, pour modifier la position de la virgule afin d'éviter les débordements lors des accumulations réalisées par la suite.

La seconde catégorie correspond aux registres en barillet, ils possèdent la capacité de réaliser l'ensemble des décalages à droite et à gauche possibles sur une donnée avec un temps de latence qui ne dépasse pas un cycle. Ils sont intégrés au sein d'une UAL ou constituent une unité spécifique en parallèle. Ce type de registres apporte une très grande souplesse dans le cadrage des données, et permet de modifier le format de celles-ci avec un surcoût maximal d'un cycle. L'absence de ce type de registres dans un DSP est un handicap important pour l'obtention d'un code optimisé. Le DSP56000 [Mot94] ne possède que des registres permettant de réaliser le décalage de 1 bit à droite ou à gauche par cycle. Dans ce cas la minimisation du nombre de décalages est un facteur primordial dans l'obtention d'un code efficace sachant qu'un décalage de N bits nécessite N cycles.

8.2.2.5 Implantation des lois de quantification et de débordement

Implantation de la loi de dépassement Les débordements peuvent se produire lors de différentes opérations arithmétiques au sein de l'unité de traitement ou lors d'un changement de format. Les architectures des DSP permettent d'utiliser une loi de dépassement modulaire et une loi de saturation.

L'approche la plus simple en cas de débordement est de ne conserver que les N bits disponibles pour représenter le format et d'éliminer les bits les plus significatifs supplémentaires. Cette loi de dépassement modulaire possède des propriétés concernant l'erreur de codage peu intéressante, ainsi de nombreux DSP possèdent des unités permettant d'implanter une loi de saturation. Elle permet d'obtenir une erreur de codage plus faible et de conserver le signe de la donnée à coder. Les unités de saturation détectent la présence d'un débordement et substituent la donnée par celle représentant la valeur minimale ou maximale la plus proche.

Ce type d'unités est présent en sortie des accumulateurs afin de saturer la valeur de ceux-ci avant le transfert de leur contenu en mémoire. Certains DSP [Tex99b] [Bie97] [MWK98] possèdent en sortie de l'additionneur ou de l'UAL une unité de saturation permettant de gérer les débordements lors des calculs intermédiaires. Une unité de saturation est présente au sein de quelques processeurs [Bie97] [Tex99a] pour saturer le résultat de la multiplication. Dans le code complément à 2, lorsque les deux opérandes sont égales à la valeur minimale

représentable, le résultat de la multiplication ne fait pas partie du domaine de définition du code. Dans ce cas, la sortie du multiplieur est saturée à la valeur maximale représentable.

L'utilisation des unités de saturation est spécifiée au sein de l'instruction [Tex99b] ou à l'aide de bits de mode stockés au sein de registres de contrôle [Bie97] [Luc] . Dans ces deux cas la saturation des données ne nécessite pas de cycles supplémentaires. Certains DSP proposent des instructions réalisant explicitement la saturation de l'accumulateur et nécessitant un cycle d'horloge [Tex99a] [Ana95].

Cependant l'utilisation d'unités de saturation modifie les valeurs des données et introduit des non linéarités au sein du signal. Ainsi la solution la plus efficace pour gérer les problèmes de débordements est de les éviter en réalisant un cadrage correct de l'ensemble des données.

Implantation de la loi de quantification Le processus de quantification intervient dès qu'une donnée est transférée vers un registre ou un emplacement dont le format est différent de celui d'origine. De nombreux DSP permettent de choisir entre une loi de quantification par arrondi ou par troncature.

La troncature consiste à éliminer les bits de poids faible qui ne peuvent être conservés dans le nouveau format. Cette quantification peut être facilement implantée car elle consiste à sélectionner les bits les plus significatifs et à les affecter à la nouvelle donnée. En contrepartie ce procédé entraîne un biais entre la valeur réelle et celle codée.

Afin d'obtenir une moyenne de l'erreur de quantification nulle, il est possible d'utiliser une loi de quantification par arrondi, celle-ci consiste à coder la donnée avec la valeur la plus proche appartenant au nouveau format. Nous trouvons au sein des DSP deux types d'arrondi.

Dans le cas d'une quantification par arrondi conventionnel, lorsque la donnée est égale à la valeur médiane de l'intervalle $[k\Delta, (k+1)\Delta]$, la valeur représentable la plus proche est choisie de la manière suivante :

$$Q(x) = \begin{cases} k\Delta & \text{si } k\Delta \leq x < (k+\frac{1}{2})\Delta \\ (k+1)\Delta & \text{si } (k+\frac{1}{2})\Delta < x \leq (k+1)\Delta \\ (k+1)\Delta & \text{si } x = (k+\frac{1}{2})\Delta \end{cases} \tag{8.3}$$

La valeur médiane de chaque intervalle $[k\Delta, (k+1)\Delta]$ étant codée systématiquement à la valeur supérieure, l'erreur de quantification n'est pas nulle. Cette erreur est néanmoins nettement plus faible que celle présente lors d'une quantification par troncature.
Pour réaliser un arrondi sur le $k^{ème}$ bit de la donnée nous additionnons 2^{k-1} à la valeur à arrondir et nous réalisons ensuite une troncature au niveau du $k^{ème}$ bit. Certains processeurs [Ana95] [Mot94] [Tex99a] possèdent une instruction ou un mode qui supporte l'arrondi conventionnel et qui réalise automatiquement les opérations décrites ci dessus. Sinon il faut explicitement charger la valeur 2^{k-1} dans l'accumulateur avant de débuter les calculs. Ce type d'arrondi est le plus utilisé car son implantation est plus simple que celle de l'arrondi convergent présenté ci-après.

Afin d'éliminer le biais un arrondi convergent peut être utilisé [LBSL96]. Il affecte la valeur médiane de façon équiprobable à la valeur supérieure et inférieure :

$$Q(x) = \begin{cases} k\Delta & \text{si } k\Delta \leq x < (k+\frac{1}{2})\Delta \\ (k+1)\Delta & \text{si } (k+\frac{1}{2})\Delta < x \leq (k+1)\Delta \\ (k)\Delta \text{ ou } (k+1)\Delta & \text{si } x = (k+\frac{1}{2})\Delta \end{cases} \tag{8.4}$$

$$\text{avec } \mathrm{P}\Big(Q(x) = k\Delta \setminus x = (k+\frac{1}{2})\Delta\Big) = \mathrm{P}\Big(Q(x) = (k+1)\Delta \setminus x = (k+\frac{1}{2})\Delta\Big)$$

Cette technique permet de coder la valeur médiane de manière équiprobable à l'aide de l'analyse de la parité de la donnée. Les données impaires sont codées à la valeur supérieure et les données paires à la valeur inférieure. Pour réaliser ceci, le DSP DSP56000 [Mot94] teste le $k^{ème}$ bit et ne réalise l'addition de cette donnée avec 2^{k-1} que si le bit est égal à 1. Pour sa part, le DSP ADSP21xx [Ana95] réalise d'abord l'addition et ensuite met à 0 le $k^{ème}$ bit.
Ce type d'arrondi permet d'obtenir une moyenne de l'erreur de quantification nulle, mais il est présent que dans quelques DSP [Luc99] [Mot94] [Ana95], car son implantation est plus complexe.

8.2.2.6 Unités spécifiques

Afin d'accélérer l'exécution d'algorithmes fréquemment utilisés dans certains domaines d'application, les DSP fournissent des unités spécifiques permettant d'implanter plus efficacement ces algorithmes :

— Unité de gestion du codage en virgule flottante par bloc
— Unité d'accélération du décodage de Viterbi

8.2.3 Architecture des chemins de données

Les premières générations de DSP possédaient une architecture du chemin de données basée sur une structure de type MAC [Lap96]. Pour faire face à la demande croissante en puissance de calcul nécessaire pour les nouvelles applications de télécommunications et de multimédia, les concepteurs de DSP ont proposé deux types d'architectures [EB00]. La première est une extension de l'architecture MAC traditionnelle et la seconde est une architecture orthogonale s'inspirant des processeurs RISC et basée sur la mise en parallèle d'unités fonctionnelles au sein du processeur. Ces deux dernières architectures seront étudiées en 3ème année.

8.2.3.1 Architecture traditionnelle : MAC

Les premières générations de DSP ont été conçues afin d'implanter efficacement les opérations de type MAC correspondant à la structure de base de nombreux algorithmes tels que les filtres (FIR, IIR). La structure d'un filtre FIR est représentée à la figure 8.3, celui-ci est constitué de cellules élémentaires juxtaposées réalisant une opération de multiplication accumulation : le résultat en sortie de la cellule est égal à la somme du résultat précédent et celui de la multiplication du coefficient h_n et de l'opérande x_{k-n} issue de la ligne à retard. Les DSP possèdent une architecture permettant de calculer la sortie d'une cellule élémentaire par cycle. Le schéma d'une architecture traditionnelle de type MAC est présenté à la figure 8.4, les opérandes sont soit situées dans les registres Rx et Ry ou directement en mémoire.

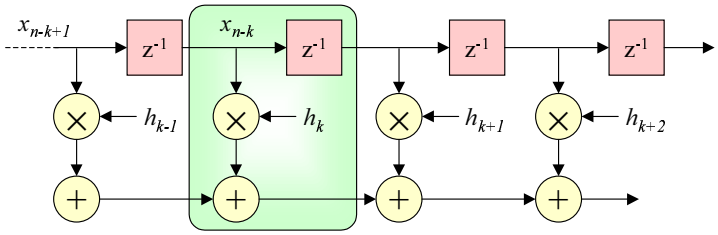


FIGURE 8.3 – Structure du filtre FIR

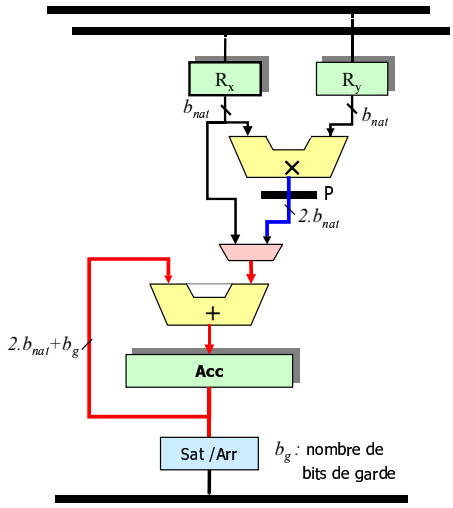


FIGURE 8.4 – Architecture traditionnelle de type MAC

Deux variantes de la structure MAC sont utilisées dans les DSP. La première réalise les opérations de multiplication accumulation de façon pipelinée, le multiplieur réalise la multiplication des opérandes de la $k^{ème}$ cellule et stocke le résultat dans un registre intermédiaire (P), en parallèle l'accumulateur réalise l'accumulation associée à la cellule précédente. Cette structure est présente dans les DSP tels que le TMS320C5x [Tex98], DSP16xx [Luc].

Dans la seconde variante, l'opération de multiplication est d'abord réalisée et le résultat est transmis à l'accumulateur afin d'effectuer l'addition. Ces deux opérations étant réalisées de manière séquentielle au cours d'un même cycle, les temps de latence du multiplieur et de l'accumulateur doivent être plus faibles par rapport à la solution précédente.

Les premières générations de DSP possédaient peu d'unités fonctionnelles, ainsi l'UAL du DSP était utilisée pour réaliser l'opération d'accumulation au sein du MAC. Les générations actuelles de DSP intègrent plus d'unités en parallèle et permettent d'utiliser un additionneur/soustracteur pour l'unité MAC, indépendant de l'UAL. De plus ces DSP incorporent des unités de manipulation de bits incluant un registre en barillet.

8.3 Description de l'unité mémoire

Les applications de TNS requièrent la lecture de nombreuses données stockées en mémoire et l'écriture des résultats dans cette dernière. Cette contrainte nécessite de posséder une structure de mémoire efficace permettant d'avoir une bande passante de communication entre les unités de calcul et la mémoire élevée afin de ne pas ralentir l'exécution des instructions. Les facteurs importants pour obtenir une architecture efficace sont l'organisation de la mémoire et l'interconnexion entre la mémoire et le chemin de données.

L'exemple le plus souvent utilisé pour montrer l'importance de l'architecture de la mémoire dans un processeur de traitement du signal est le filtre FIR. Celui-ci est constitué de cellules de type MAC et d'une ligne à retards. Cette opération MAC étant réalisée en 1 cycle dans la majorité des DSP, l'architecture mémoire doit permettre d'effectuer l'ensemble des communications nécessaires entre l'unité de calcul et la mémoire en 1 cycle, afin de ne pas ralentir le traitement des données. Les différents accès à la mémoire sont les suivants :

— recherche de l'instruction
— lecture de la donnée x_{n-k}
— lecture du coefficient h_k
— vieillissement des données $x_{n-k-1} = x_{n-k}$

Afin d'éliminer le dernier accès les processeurs utilisent un mode d'adressage particulier qui permet de vieillir automatiquement les données, ceci permet de réduire le nombre d'accès par cycle à 3.

L'architecture traditionnelle des processeurs généraux est basée sur le modèle de Von Neuman pour lequel une mémoire unique est utilisée pour stocker les données et le programme. Pour obtenir des performances acceptables ces processeurs utilisent des mémoires caches.

L'architecture des PTS est basée sur le modèle Harvard dont le principe est la séparation de la mémoire pour les données et le programme. Les deux espaces mémoire sont totalement indépendants, ils possèdent leurs propres bus de données et d'adresses et leur propre générateur d'adresses.
Différentes variantes du modèle Harvard ont été proposées afin d'augmenter le nombre d'échanges simultanés entre la mémoire et les unités de calcul. Ces variantes sont présentées à la figure 8.5 [Lee88]. Le choix des diverses variantes est fonction de la structure de localisation des opérandes [Ara97] utilisée dans le PTS.

8.3.1 Modèles de localisation des opérandes

8.3.1.1 Modèle *registre mémoire*

Les architectures basées sur le modèle *registre-mémoire*, utilisent une opérande présente en mémoire et une opérande stockée dans un registre [Ara97]. Au cours d'un cycle, une seule lecture d'opérande en mémoire est réalisée, ainsi il est possible d'utiliser la structure Harvard de base et sa première déclinaison qui possède une seule mémoire pour les données. Le

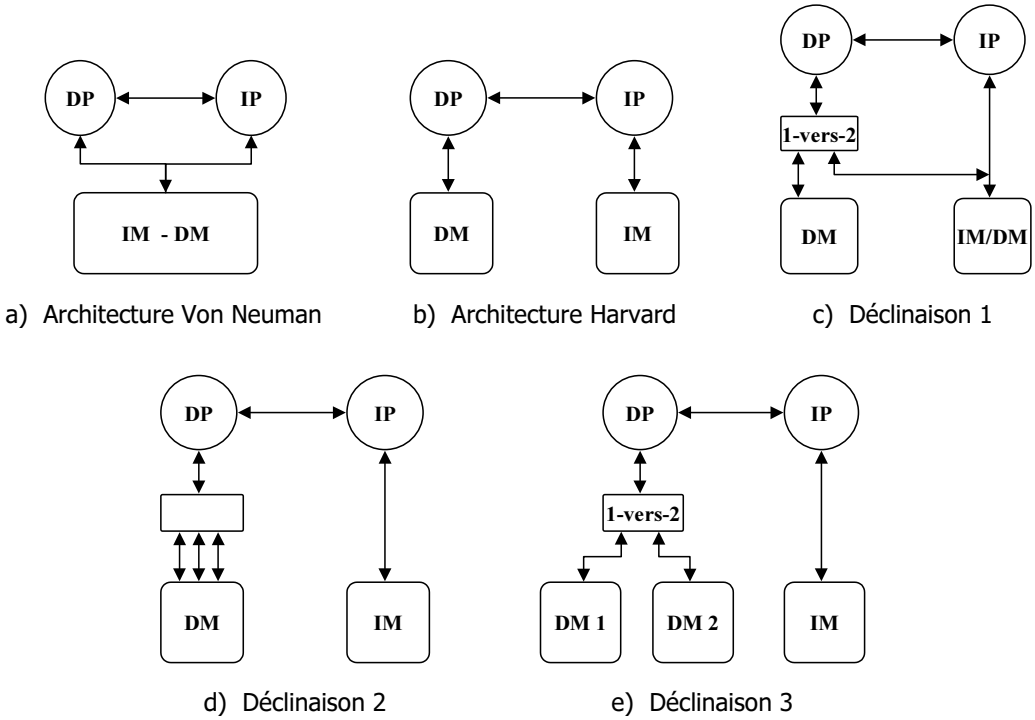


FIGURE 8.5 – Architecture Harvard et ses déclinaisons

temps d'exécution d'une instruction est fonction du temps d'accès à la mémoire et du temps d'exécution de l'opération :

$$t_{inst} = t_{acc} + t_{exec} \tag{8.5}$$

Le challenge pour ce type d'architecture est de minimiser le temps d'accès à la mémoire interne. Pour cela les DSP utilisent des mémoires statiques car elles sont plus rapides que les mémoires dynamiques. Le temps d'accès est aussi diminué en minimisant la taille des blocs adressés à l'aide d'une mémoire paginée. Le type d'adressage associé est présenté dans le paragraphe 8.3.3.2. De nombreux DSP conçus par Texas Instruments possèdent cette architecture mémoire : TMS320C25/50/54x [Tex98] [Tex99a].

8.3.1.2 Modèle *registre-registre* ou *'load/store'*

Afin de s'affranchir du temps d'accès à la mémoire une structure évitant la lecture directe d'une opérande en mémoire est utilisée. Les deux opérandes sont stockées dans des registres et le chargement de ceux-ci est effectué en parallèle avec l'exécution d'une opération dans l'unité de calcul. Afin de respecter les contraintes de dépendance des données, l'opération courante s'effectue en même temps que le chargement des opérandes de l'instruction suivante. L'instruction spécifie les deux registres des opérandes sources et le registre de stockage du résultat, ainsi que l'adresse des 2 opérandes à charger pour l'instruction suivante. Ce type de structure permet de diminuer le temps d'exécution de l'instruction par rapport à la structure précédente, en le rendant indépendant du temps d'accès à la mémoire :

$$t_{inst} = t_{exec} \qquad \text{si} \quad t_{acc} < t_{inst} \tag{8.6}$$

Ce modèle de localisation des opérandes nécessite pour un opérateur de lire deux données par cycle. Deux approches sont utilisées afin de réaliser ces deux lectures.

Utilisation d'une mémoire multi-ports Cette structure est composée d'une mémoire multi-ports permettant de lire plusieurs données en parallèle au cours d'un même cycle. Dans ce cas la majorité des architectures mémoire correspond à la seconde déclinaison. Elle est composée d'une mémoire programme mono port et d'une mémoire données double ports permettant de lire deux données en parallèle. L'aspect multi-ports de la mémoire va augmenter la surface et le coût du circuit. Cette structure est très utilisée dans les processeurs de type VLIW comme le TMS320C62x [Tex99b] [TH97]. L'association de ce modèle mémoire et d'une file de registres permet de garantir l'homogénéité de l'architecture.

Utilisation de deux bancs de mémoires La seconde approche correspond à la troisième déclinaison, elle comprend deux bancs de mémoires X et Y possédant chacun ses propres bus de données (XDB et YDB) et d'adresses et sa propre unité de génération d'adresses. Cette structure permet de lire deux données en parallèle au cours d'un cycle sans utiliser de mémoires multi-ports.

Cette solution est moins coûteuse en terme de surface de silicium par rapport à des mémoires multi-ports. Mais ce type d'architecture ne permet de lire qu'une seule donnée par banc de mémoire et par cycle, ainsi il est nécessaire de répartir efficacement les données dans les deux bancs de mémoires afin de pouvoir charger le maximum de données en parallèle.

8.3.2 Spécificité des DSP

Comme nous l'avons vu précédemment le temps d'accès à la mémoire est un facteur essentiel pour l'obtention de performances élevées, ainsi pour les ASIP ou système sur une puce la taille de la mémoire ROM et RAM est ajustée afin de pouvoir contenir l'ensemble du programme et des données nécessaires à l'application. Ceci permet de ne pas utiliser de mémoires externes et ainsi de ne pas augmenter le temps d'accès à la mémoire, le nombre de broches, la surface du système et la consommation.
Dans le cadre des DSP l'application n'étant pas connue à priori, il est possible d'utiliser de la mémoire externe pour augmenter la taille de l'espace mémoire, mais ceci est très pénalisant en terme de coût (nécessité d'une mémoire externe rapide), et de consommation.

8.3.3 Mode d'adressage

Afin de respecter les performances souhaitées au niveau de la bande passante de communication avec la mémoire, il est nécessaire d'avoir des unités spécialisées (unité de génération d'adresses) permettant de calculer les adresses des données rapidement et en parallèle avec l'exécution des autres opérations. En effet la charge de calcul concernant les adresses est proche de celle des données [Peg99].

Les algorithmes de TNS sont caractérisés par des accès réguliers et séquentiels à la mémoire. Les unités de génération d'adresses vont exploiter ces propriétés afin d'optimiser le calcul des adresses en proposant des solutions câblées permettant d'implanter les modes d'accès les plus couramment utilisés. Ceux-ci sont associés à la structure *somme de produits*. Les DSP permettent d'utiliser de nombreux modes d'adressage afin d'optimiser le calcul des adresses pour les diverses applications de traitement du signal. Nous allons présenter différents modes d'adressage [LBSL96] et plus particulièrement le mode d'adressage indirect par registres qui est le plus adapté aux applications de TNS.

8.3.3.1 Mode d'adressage immédiat

Dans le mode d'adressage immédiat, la valeur de la donnée est directement codée dans l'instruction, ceci est utilisé pour initialiser les registres d'adresses ou pour stocker une constante utilisée dans une opération arithmétique. Lorsque la taille de la donnée est faible il est possible de coder l'instruction en un seul mot composé de l'opération à réaliser et de la valeur de la donnée. Mais pour une donnée de valeur élevée, il est nécessaire d'utiliser 2 mots pour coder l'instruction, ceci est pénalisant en terme de densité de code et de temps d'exécution.

8.3.3.2 Mode d'adressage direct

Dans le mode d'adressage direct, l'adresse de la donnée est codée directement dans l'instruction, si l'ensemble de l'adresse est spécifié il est nécessaire d'ajouter un second mot pour coder l'instruction.
Afin d'encoder partiellement l'adresse, un système de mémoire paginée peut être utilisé. L'adresse de la donnée est composée d'un numéro de page stocké dans un registre (pointeur de page) et d'un déplacement dans la page qui sera stocké dans l'instruction. La taille du déplacement est fonction de la longueur de la page et doit être relativement faible. Le choix de la taille de la page résulte d'un compromis entre le nombre de pages qui ne doit pas être trop élevé afin de minimiser le nombre de modifications du pointeur de page et la longueur de la page qui doit rester faible afin d'encoder le déplacement avec un minimum de bits. Dans le cas du TMS320C54x la taille d'une page est de 128 mots, ainsi l'adresse de la donnée est codée dans l'instruction sur 7 bits. Ce type d'adressage est intéressant si le programme accède de manière séquentielle à des données regroupées dans des pages identiques. Des techniques d'optimisations existent pour minimiser le surcoût associé aux diverses initialisations du pointeur de page [Sud98]

8.3.3.3 Mode d'adressage indirect

Dans le mode d'adressage indirect, la donnée est pointée en mémoire via un registre d'adresse. Ceci permet de spécifier dans l'instruction le registre à utiliser et non l'adresse de la donnée et ainsi de réduire fortement le nombre de bits nécessaires pour encoder la localisation de la donnée. Le nombre de registres d'adresses est limité afin de ne pas utiliser trop de bits dans l'instruction pour le spécifier. Ce mode d'adressage est le plus utilisé dans les algorithmes de TNS pour les raisons suivantes :

- encodage efficace de l'adresse dans l'instruction ;
- mode d'adressage naturel des tableaux (adressage par pointeur) ;

— possibilité de calculer l'adresse de l'instruction suivante en parallèle avec l'exécution de l'instruction courante.

Les unités de génération d'adresses permettent d'utiliser 4 types d'adressage spécifiques afin de répondre aux contraintes d'adressage des données de certains algorithmes, sans ajouter d'instructions supplémentaires.

Adressage linéaire Afin de pouvoir se déplacer rapidement dans un tableau des possibilités de post-modifications sont disponibles. Ceci permet d'incrémenter ou décrémenter la valeur du registre d'adresse en parallèle à l'exécution d'une opération. Il est possible de se déplacer de façon non contiguë en ajoutant un offset à l'adresse. Cet offset est soit stocké dans un registre quelconque spécifié dans l'instruction ou dans un registre dédié appelé *registre d'offset*.

Indexage La valeur de l'adresse est obtenue à partir de l'addition de deux registres d'adresses ou d'un registre d'adresse et d'une constante codée dans l'instruction. La première partie de l'adresse représente l'adresse de base du bloc de données et n'est pas modifiée lors de l'accès à ce bloc et la seconde partie représente le déplacement par rapport à l'adresse de base.
Ce mode d'adressage est utilisé lorsque le code doit accéder à des blocs de données de structure identique et dont l'adresse de base est différente. Il est utilisé pour le passage de paramètres lors de l'appel de routines et pour la gestion de la pile.

Modulo Ce mode d'adressage permet de gérer des buffers circulaires utilisés dans les algorithmes de filtrage. La gestion de ce type de buffers nécessite de vérifier après chaque incrémentation (décrémentation) du registre d'adresse, si celui-ci pointe sur la dernière (première) valeur du tableau. Dans ce cas, il faut rediriger le pointeur sur le début (fin) du tableau.
Deux méthodes sont utilisées au niveau des générateurs d'adresses pour réaliser ce type d'adressage. La première méthode utilise 2 registres pour stocker l'adresse de début et de fin du buffer circulaire afin de pouvoir détecter l'accès à la dernière ou à la première valeur du tableau. Dans le second cas, le DSP stocke dans un registre uniquement la taille du buffer circulaire et impose une contrainte sur l'adresse de départ du buffer : celle-ci doit être un multiple de 2^k relié à la taille N du buffer par l'expression suivante :

$$2^k > N \tag{8.7}$$

Bit inversé Ce mode d'adressage est utilisé pour l'adressage des données en entrée ou en sortie de l'algorithme de la transformée de Fourier rapide (FFT). L'adressage *bit inversé* consiste à inverser l'ordre des N derniers bits d'un registre d'adresse pour une FFT de taille 2^N. L'unité de génération d'adresses réalise cette opération en inversant directement les bits du registre d'adresse ou en propageant la retenue en ordre inverse lors des calculs arithmétiques sur ce registre d'adresse.

8.4 Description de l'unité de contrôle

8.4.1 Type de code et Pipeline

8.4.1.1 Présentation du pipeline

L'efficacité d'un PTS réside dans sa capacité à réaliser une instruction par cycle. Chaque instruction étant composée d'une séquence d'opérations, il est nécessaire d'exécuter ces opérations en parallèle. La technique du pipeline permet de décomposer les différentes phases d'une instruction et de les réaliser en parallèle [Lee89]. Une instruction classique est composée des 5 phases suivantes :

- Pipeline sur les instructions
  - 1. Recherche de l'instruction
  - 2. Décodage de l'instruction
- Pipeline sur les données
  - 3. Lecture des opérandes
  - 4. Exécution de l'instruction
  - 5. Écriture du résultat

A chaque cycle le pipeline réalise une opération sur des instructions différentes. Au cours du même cycle le pipeline réalise le décodage de l'instruction $Inst_n$, la recherche en mémoire de l'instruction suivante $Inst_{n+1}$ et la lecture des opérandes de l'instruction précédente $Inst_{n-1}$

La profondeur du pipeline (nombre d'étages) est un compromis entre la facilité de programmer et de contrôler celui-ci et la rapidité du processeur. Les processeurs traditionnels utilisent un pipeline dont la longueur varie entre 3 et 5 étages. Dans des processeurs de type VLIW, la longueur du pipeline est plus importante, ceci permet de simplifier chaque opération à réaliser dans celui-ci et ainsi d'utiliser des horloges de fréquence plus élevée, le TMS3320C62x possède un pipeline de 11 étages et permet d'utiliser une fréquence d'horloge de 200 MHz [Tex99b].

La mise en parallèle des différentes phases d'une instruction peut entraîner les aléas de fonctionnement suivants [HP96] [Yar99] :

- *aléas de ressource :* cette situation se produit lorsque le processeur ne peut accepter une combinaison d'instructions utilisant les mêmes ressources en même temps ;
- *aléas de données :* ce cas se présente lorsque les données d'une instruction dépendent du résultat d'une instruction précédente qui n'est pas encore terminée ;
- *aléas de contrôle :* ceci correspond à la rupture du flot de contrôle du programme lors d'un branchement, d'un appel ou d'un retour de sous routines ou d'interruption. Lorsqu'une instruction de branchement est détectée une ou plusieurs instructions consécutives sont déjà rentrées dans le pipeline, il est alors nécessaire de les éliminer du pipeline afin de respecter le flot de contrôle du programme.

8.4.1.2 Type de codage des instructions

Dans cette partie nous présentons les deux types de codage d'instructions [Ga97] [Ga96] utilisés pour programmer les processeurs utilisant un pipeline, nous présentons les deux catégories

de processeurs liés aux types de codage et les mécanismes associés pour gérer les aléas de données et de ressources.

Codage des instructions stationnaire dans le temps Chaque instruction du jeu d'instructions définit l'ensemble des opérations à réaliser pour chaque unité fonctionnelle, au cours du cycle [Lee89]. Dans le cas de l'architecture de la figure 8.4, trois actions sont à réaliser, la première correspond à l'opération à exécuter et les deux dernières au chargement des deux registres avec les opérandes présentes en mémoire. Le schéma du pipeline pour ce type de code est présenté à la figure 8.6. Un exemple d'instruction utilisant ce principe est présenté ci dessous :

```
MPY Rx,Ry,P  ADD ACC,ACC,P  LD Rx,*(ARx++)   LD Ry,*(ARy++)
```

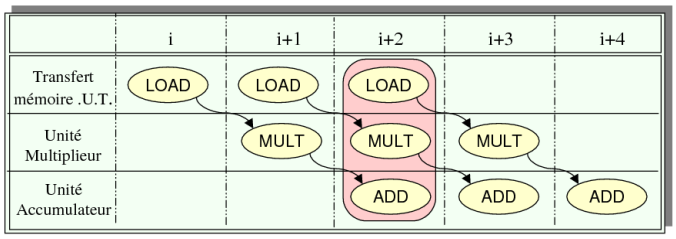


FIGURE 8.6 – Synoptique du pipeline sur les données pour un codage stationnaire dans le temps

L'instruction spécifie l'ensemble des opérations à réaliser sur les données présentes dans les différentes parties du chemin de données. Le programmeur doit assurer l'alimentation du pipeline et est responsable de la gestion des aléas liés à la dépendance des données, ceux-ci sont analysés à partir d'une table de réservation qui permet de visualiser l'utilisation des ressources et des données.

Codage des instructions stationnaire au niveau des données Dans ce cas l'instruction spécifie une séquence complète d'opérations à réaliser sur un ensemble de données. Les différentes opérations nécessitent plusieurs cycles, ainsi le résultat n'est pas immédiatement disponible. Le schéma du pipeline pour ce type de code est présenté à la figure 8.7. Un exemple d'instruction utilisant ce type de codage est présenté ci dessous :

```
MAC *(ARx++),*(ARy++),ACC
```

Ce type de codage permet d'obtenir un code plus lisible et facilite la programmation. L'utilisateur ne doit pas spécifier l'ensemble des actions à réaliser au cours d'un cycle mais les opérations à effectuer sur les données. Cette approche est plus proche de l'algorithme à coder.

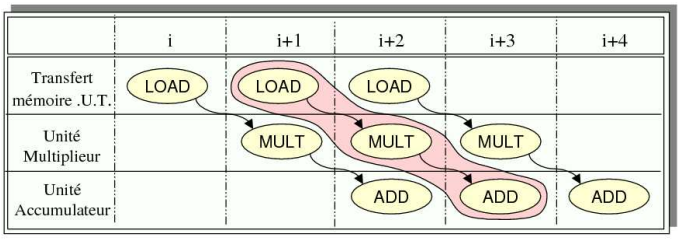


FIGURE 8.7 – Synoptique du pipeline sur les données pour un codage stationnaire au niveau des données

8.4.2 Format du jeu d'instructions

Deux catégories de format d'instructions sont possibles [Ga97] [Ga96], le choix de l'une ou l'autre sera fonction du domaine d'applications du processeur.

8.4.2.1 Format encodé

Le challenge pour les processeurs embarqués, est de minimiser la taille de la puce afin de réduire les coûts, tout en maintenant des performances en terme de puissance de calcul élevées. Ainsi le nombre d'instructions pour réaliser une opération et la largeur des instructions doivent être minimisés afin de diminuer la taille de la mémoire programme [Peg99]. Pour obtenir des performances élevées, l'instruction doit encoder le maximum de parallélisme.

Le second facteur important dans les applications embarquées est la consommation du circuit. Elle est proportionnelle à la taille des bus et de la mémoire. La minimisation de la taille des instructions va permettre de diminuer la consommation liée à la mémoire programme.

Le choix du format des instructions résulte d'un compromis entre le parallélisme offert par l'instruction et sa taille. Elle doit être compacte afin de minimiser la taille de la mémoire programme et elle doit offrir le maximum de parallélisme pour utiliser efficacement les ressources du processeur. Ce jeu d'instructions est construit à partir des statistiques obtenues sur des applications représentatives (benchmarks) qui permettent de déterminer les instructions et les unités les plus utilisées et de révéler les opportunités de parallélisme [Za96]. La diversité des algorithmes retenus pour réaliser les statistiques, est un facteur primordial dans le degré de spécialisation d'un DSP à un domaine d'applications donné.

Cette minimisation du nombre de bits est réalisée à l'aide des concepts suivants [LBSL96] :
- réduction du nombre d'opérations disponibles au sein du jeu d'instructions ;
- réduction du nombre de modes d'adressage et des possibilités de mise à jour des registres d'adresses ;
- restrictions sur les opérandes sources et destinations des instructions (spécialisation des registres) ;
- utilisation de bits de mode.

Ce type de restriction va sacrifier une partie du parallélisme disponible et rendre la structure du jeu d'instructions hétérogène en spécialisant les registres sources et de destination des

opérateurs et en limitant les modes d'adressage disponibles.

Ce format étant encodé la signification des différents bits de l'instruction est fonction de l'instruction. Ainsi l'unité de décodage destinée à transformer les différents bits de l'instruction en signaux de contrôle pour les unités fonctionnelles, est relativement complexe.

Ce type de codage est utilisé pour les processeurs destinés aux applications pour le grand public et en particulier dans le domaine des télécommunications (téléphone portable, pager) ou les facteurs prix et consommation sont primordiaux.

8.4.3 Structure de contrôle

8.4.3.1 Boucles matérielles

Les structures de boucle représentent l'une des structures de contrôle les plus couramment utilisées dans les algorithmes de TNS. Dans la plupart des cas, ces boucles permettent d'implanter un calcul numérique répétitif sur les éléments d'un vecteur, ainsi le nombre d'itérations est connu à priori. Le nombre d'instructions présentes dans le corps de la boucle pouvant être relativement faible, il est indispensable d'implanter efficacement ce type de structure et de minimiser le surcoût lié au contrôle de celle-ci.

La plupart des PTS possède une unité de contrôle appelée boucle matérielle qui permet de coder la structure de boucle en une seule instruction et de ne pas utiliser de cycles supplémentaires pour réaliser le test de la valeur du compteur d'itérations, le branchement conditionnel et la décrémentation de ce compteur. L'unité de contrôle possède un premier registre pour stocker le nombre d'itérations et un second pour spécifier l'adresse de fin de la boucle. Ces deux paramètres sont initialisés dans l'instruction de description de la boucle. Certains DSP proposent plusieurs supports de boucle matérielle afin de pouvoir imbriquer plusieurs boucles.

8.4.3.2 Instructions de branchement

Comme nous l'avons vu dans le paragraphe 8.4.1.1, la présence d'instructions de branchement au sein du code introduit des aléas de fonctionnement du pipeline. La première solution pour gérer ces aléas, est de vider le pipeline des instructions ne correspondant pas au flot de contrôle et de ne pas exécuter d'opérations entre l'exécution du branchement et la première instruction pointée par ce branchement. Cette technique appelée *branchement multi cycles*, augmente le temps d'exécution lié à la gestion du branchement. Ce temps est fonction de la profondeur du pipeline.

Pour diminuer les pénalités liées aux traitements des branchements les techniques de branchements retardés sont utilisées dans les DSP. Plusieurs instructions devant être exécutées avant le branchement sont placées, après celui-ci. Ces instructions seront exécutées entre l'instruction de branchement et la première instruction pointée par celui ci.

8.4.3.3 Instructions de mode

Les DSP possèdent des registres d'état ou de mode afin d'implanter un contrôle à l'aide de ce registre et non au sein de l'instruction. Ces registres de mode sont utilisés pour contrôler les parties du chemin de données qui varient peu au cours d'une séquence d'instructions. Ces registres sont initialisés ou modifiés à l'aide d'instructions spécifiques, avant l'utilisation de l'unité fonctionnelle.

8.4.3.4 Instructions à prédicat

Les instructions à prédicat sont des instructions conditionnelles du type : `si` *condition* `alors` *exécuter opération*. Cette structure permet de minimiser le nombre d'instructions pour coder une structure conditionnelle du type `If Then Else` en éliminant les différentes instructions de branchement. Ceci est illustré à la figure 8.8. Le TMS320C62x [Tex99b] permet de rendre l'ensemble de ces instructions conditionnelles à l'aide des prédicats.

```
If(a<10)                    SUB  R2,10,R1
     b=c+1;          [R1]   ADD  R3,R6,1
else                 [!R1]  SUB  R3,R6,2
     b=c-2;
```

FIGURE 8.8 – Instructions à prédicat

Annexe A

Tables des transformées

A.1 Transformées en Z

Type	$X_i(z)$	$x_i(n)$
polynomial en z	$\sum_k c_k^{-k}$	$\sum_k c_k \delta(n-k)$
pôle réel simple	$\frac{1}{1-pz^{-1}}$	$p^n u(n)$
pôle réel double	$\frac{pz^{-1}}{(1-pz^{-1})^2}$	$np^n u(n)$
pôle réel double	$\frac{1}{(1-pz^{-1})^2}$	$(n+1)p^n u(n)$
pôle réel triple	$\frac{p^2 z^{-1}}{(1-pz^{-1})^3}$	$\frac{n(n-1)}{2} p^n u(n)$
pôle complexe conjugué	$\frac{r\sin(\omega_0)z}{(z-re^{i\omega_0})(z-re^{-i\omega_0})}$	$r^n \sin(\omega_0 n)u(n)$

TABLE A.1 – Tables des transformées en z

$x(t)$	$x(n)$	$X(z)$
	$\delta(n)$	1
	$\delta(n-k)$	z^{-k}
$u(t)$	$u(n)$	$\frac{z}{z-1}$
t	n	$\frac{Tz}{(z-1)^2}$
$\frac{1}{2}t^2$	$\frac{1}{2}(nT)^2$	$\frac{T^2z(z+1)}{2(z-1)^3}$
$a^{\frac{t}{T}}$	a^n	$\frac{z}{z-a}$

A.2 Transformées de Fourier

$x(t)$	$x(n)$	$X(z)$
e^{-at}	e^{-anT}	$\dfrac{z}{z - e^{-aT}}$
$\sin \omega_0 t$	$\sin \omega_0 nT$	$\dfrac{z \sin \omega_0 T}{z^2 - 2z \cos \omega_0 T + 1}$
$\cos \omega_0 t$	$\cos \omega_0 nT$	$\dfrac{z(z - \cos \omega_0 T)}{z^2 - 2z \cos \omega_0 T + 1}$
$e^{-at} \sin \omega_0 t$	$e^{-at} \sin \omega_0 nT$	$\dfrac{ze^{-aT} \sin \omega_0 T}{z^2 - 2ze^{-aT} \cos \omega_0 T + e^{-2aT}}$
$e^{-at} \cos \omega_0 t$	$e^{-at} \cos \omega_0 nT$	$\dfrac{z^2 - ze^{-aT} \cos \omega_0 T}{z^2 - 2ze^{-aT} \cos \omega_0 T + e^{-2aT}}$
$ax_1(t) + bx_2(t)$	$ax_1(n) + bx_2(n)$	$aX_1(z) + bX_2(z)$
$x(t - kT)$	$x(n - k)$	$z^{-k} X(z)$

TABLE A.2 – Tables des transformées en Z de fonctions usuelles. T est la période d'échantillonnage du signal transformé dans lequel on a posé $t = nT$.

$x(n)$	$X(e^{j\Omega})$
$\delta(n)$	1
$\delta(n-n_0)$	$e^{-j\Omega n_0}$
1	$\sum_{k=-\infty}^{\infty} 2\pi\delta(\Omega+2\pi k)$
$u(n)$	$\frac{1}{1-e^{-j\Omega}}+\sum_{k=-\infty}^{\infty}\pi\delta(\Omega+2\pi k)$
$a^n u(n) \quad (\lvert a\rvert<1)$	$\frac{1}{1-ae^{-j\Omega}}$
$(n+1)a^n u(n) \quad (\lvert a\rvert<1)$	$\frac{1}{(1-ae^{-j\Omega})^2}$
$x(n)=\begin{cases}1 & \text{si } 0\le n\le N\\ 0 & \text{ailleurs}\end{cases}$	$\frac{\sin\left(\Omega(N+1)/2\right)}{\sin(\Omega/2)}e^{-j\Omega N/2}$
$\frac{\sin\Omega_c n}{\pi n}$	$X(e^{j\Omega})=\begin{cases}1 & \text{si } \lvert\Omega\rvert<\Omega_c\\ 0 & \text{si } \Omega_c<\lvert\Omega\rvert<\pi\end{cases}$
$e^{j\Omega_0 n}$	$\sum_{k=-\infty}^{\infty} 2\pi\delta(\Omega-\Omega_0+2\pi k)$
$\cos(\Omega_0 n+\phi)$	$\sum_{k=-\infty}^{\infty}[\pi e^{-j\phi}\delta(\Omega+\Omega_0+2\pi k)+\pi e^{j\phi}\delta(\Omega-\Omega_0+2\pi k)]$
$\sin(\Omega_0 n+\phi)$	$j\sum_{k=-\infty}^{\infty}[\pi e^{-j\phi}\delta(\Omega+\Omega_0+2\pi k)-\pi e^{j\phi}\delta(\Omega-\Omega_0+2\pi k)]$

TABLE A.3 – Tables des principales transformées de Fourier

Annexe B

Abaques de filtrage analogique

B.1 Filtres de Butterworth

Les coefficients des filtres prototypes de Butterworth sont donnés dans la deuxième colonne du tableau B.1 pour des filtres d'ordre 2 à 8. Les coefficients donnés sont ceux du dénominateur de la fonction de transfert $H(p)$ du filtre analogique.
Les réponses fréquentielles de ces filtres sont données figure B.1. Leur fréquence de coupure est normalisée à $\omega = 1$. Ces courbes permettent de déterminer l'ordre du filtre.

Le programme Matlab ci dessous permet de déterminer l'ordre d'un filtre de Butterworth, puis de créer un polynôme le représentant. Dans le cas du filtre passe-bas prototype, on posera `Wn`$= 1$.

```
[N, Wn] = BUTTORD(Wp, Ws, Rp, Rs);
% Wp and Ws are the passband and stopband edge frequencies, normalized from 0 to 1
% where 1 corresponds to pi radians/sample
% Rp (dB) and Rs (dB) are the passband ripple and stopband attenuation
% Wn is the Butterworth natural frequency or 3 dB frequency
% N is the filter order

[b,a] = BUTTER(N,Wn,'s');
% Designs an Nth order lowpass digital
% Butterworth filter and returns the filter coefficients in length
% N+1 vectors B (numerator) and A (denominator)
```

B.2 Filtres de Bessel

Les coefficients des filtres prototypes de Bessel sont donnés dans la troisième colonne du tableau B.1 pour des filtres d'ordre 2 à 8. Les coefficients donnés sont ceux du dénominateur de la fonction de transfert $H(p)$ du filtre analogique.
Les réponses fréquentielles de ces filtres sont données figure B.2. Leur fréquence de coupure est normalisée à $\omega = 1$. Ces courbes permettent de déterminer l'ordre du filtre.

Le programme Matlab ci dessous permet de créer un polynôme de Bessel.

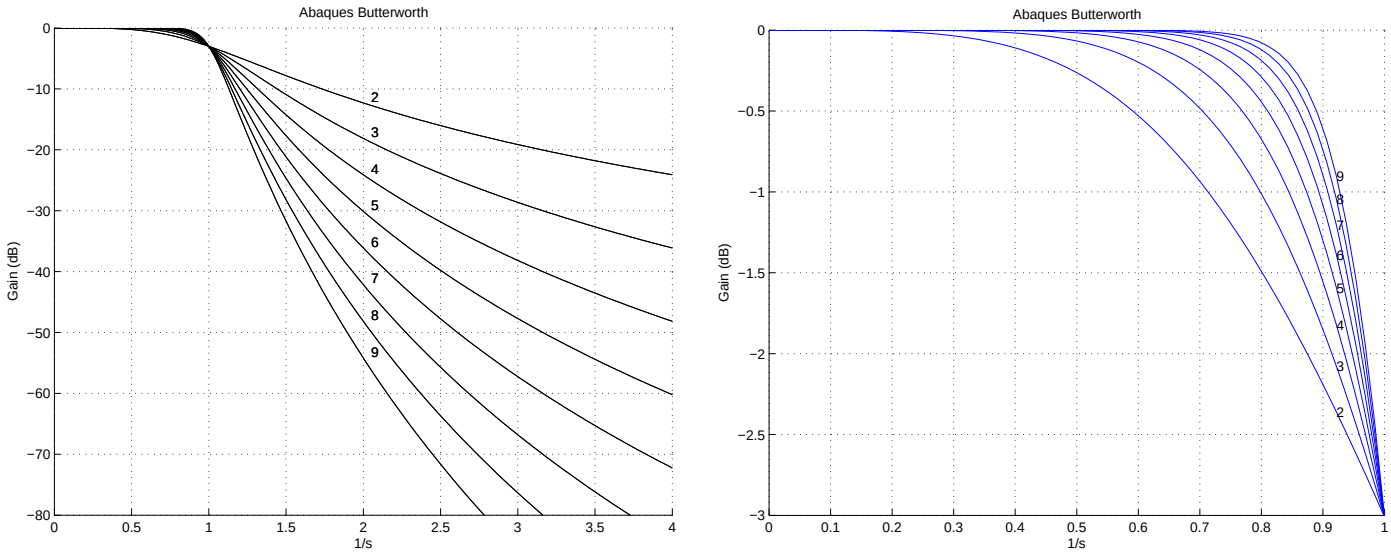


FIGURE B.1 – Réponse en fréquence des filtres de Butterworth

```
[b,a] = BESSELF(N,Wn);
% Designs an Nth order lowpass digital
% Bessel filter and returns the filter coefficients in length
% N+1 vectors B (numerator) and A (denominator)
```

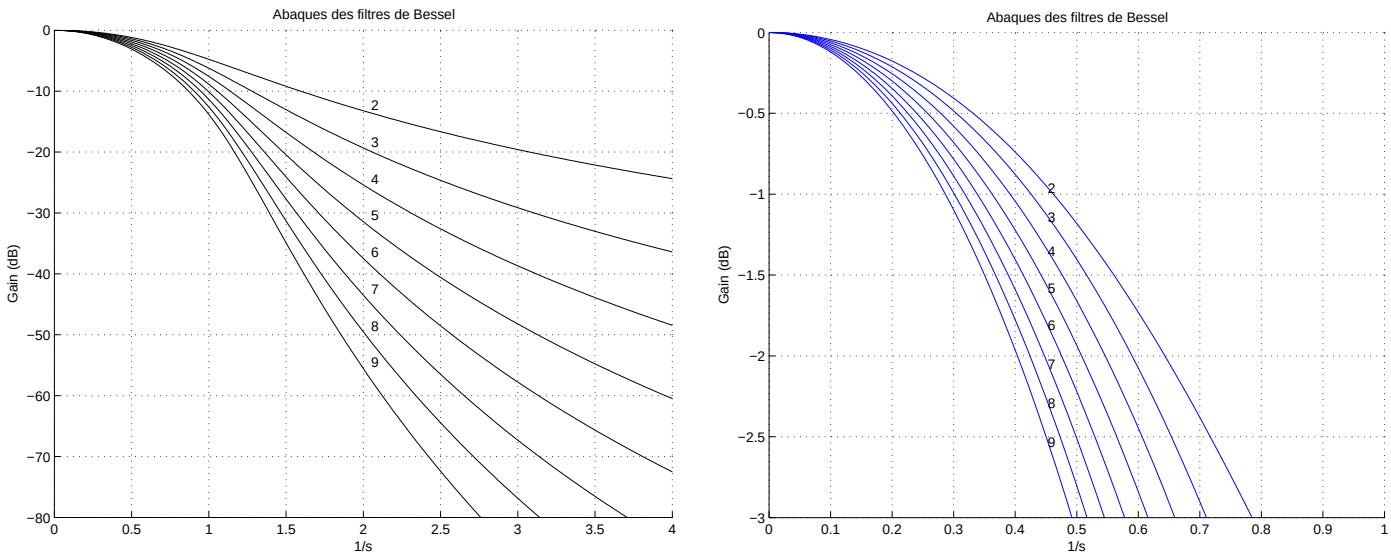


FIGURE B.2 – Réponse en fréquence des filtres de Bessel

Cellule	Butterworth	Bessel	Chebyshev $0.1dB$ à $0.1dB$ en BP
1	$p^2 + 1.4142p + 1$	$0.618p^2 + 1.3616p + 1$	$0.3017p^2 + 0.7158p + 1$
1	$p^2 + 1.0000p + 1$	$0.4771p^2 + 0.9996p + 1$	$0.5918p^2 + 0.5736p + 1$
2	$p + 1$	$0.756p + 1$	$1.031p + 1$
1	$p^2 + 1.8477p + 1$	$0.3889p^2 + 0.7742p + 1$	$0.7518p^2 + 0.3972p + 1$
2	$p^2 + 0.7853p + 1$	$0.4889p^2 + 1.3396p + 1$	$1.6053p^2 + 2.0475p + 1$
1	$p^2 + 1.6180p + 1$	$0.4128p^2 + 1.1401p + 1$	$0.8368p^2 + 0.2787p + 1$
2	$p^2 + 0.6180p + 1$	$0.3245p^2 + 0.6215p + 1$	$1.5725p^2 + 1.3712p + 1$
3	$p + 1$	$0.665p + 1$	$1.855p + 1$
1	$p^2 + 1.9318p + 1$	$0.3887p^2 + 1.2217p + 1$	$3.797p^2 + 3.2506p + 1$
2	$p^2 + 1.4142p + 1$	$0.3504p^2 + 0.9686p + 1$	$0.8854p^2 + 0.2031p + 1$
3	$p^2 + 0.5176p + 1$	$0.2756p^2 + 0.513p + 1$	$1.4360p^2 + 0.8999p + 1$
1	$p^2 + 1.8019p + 1$	$0.3394p^2 + 1.0944p + 1$	$0.9153p^2 + 0.1534p + 1$
2	$p^2 + 1.2469p + 1$	$0.301p^2 + 0.8303p + 1$	$3.0283p^2 + 2.056p + 1$
3	$p^2 + 0.4450p + 1$	$0.238p^2 + 0.4332p + 1$	$1.3276p^2 + 0.6237p + 1$
4	$p + 1$	$0.593p + 1$	$2.654p + 1$
1	$p^2 + 1.9615p + 1$	$0.3161p^2 + 0.1112p + 1$	$6.8675p^2 + 4.4178p + 1$
2	$p^2 + 1.6629p + 1$	$0.2979p^2 + 0.9753p + 1$	$0.935p^2 + 0.1196p + 1$
3	$p^2 + 1.1111p + 1$	$0.2621p^2 + 0.7202p + 1$	$1.2517p^2 + 0.4561p + 1$
4	$p^2 + 0.3901p + 1$	$0.208p^2 + 0.3727p + 1$	$2.4026p^2 + 1.3103p + 1$

TABLE B.1 – Coefficients des filtres prototypes. Les filtres sont données en cascade de cellules du second ordre et correspondent à $1/H(p)$

B.3 Filtres de Chebyshev

Les coefficients des filtres prototypes de Chebyshev d'ondulation $0.1dB$ dans la bande passante sont donnés dans la quatrième colonne du tableau B.1 pour des filtres d'ordre 2 à 8. Les coefficients donnés sont ceux du dénominateur de la fonction de transfert $H(p)$ du filtre analogique.
Les réponses fréquentielles des filtres prototypes de Chebyshev d'ondulation $0.1dB$ et $1dB$ dans la bande passante sont données figures B.3 et B.4. Leur fréquence de coupure est normalisée à $\omega = 1$. Ces courbes permettent de déterminer l'ordre du filtre.

Le programme Matlab ci dessous permet de déterminer l'ordre d'un filtre de Chebyshev d'ondulation $0.1dB$, puis de créer un polynôme le représentant. Dans le cas du filtre passe-bas prototype, on posera `Wn`$= 1$.

```
[N, Wn] = CHEB1ORD(Wp, Ws, Rp, Rs);
% Wp and Ws are the passband and stopband edge frequencies, normalized from 0 to 1
% where 1 corresponds to pi radians/sample
% Rp (dB) and Rs (dB) are the passband ripple and stopband attenuation
% Wn is the Chebyshev natural frequency or 3 dB frequency
% N is the filter order

[b,a] = CHEBY1(N,0.1,Wn,'s')
% Designs an Nth order lowpass digital Chebyshev filter
% with 0.1dB of passband ripple and returns the filter coefficients in length
% N+1 vectors B (numerator) and A (denominator)
```

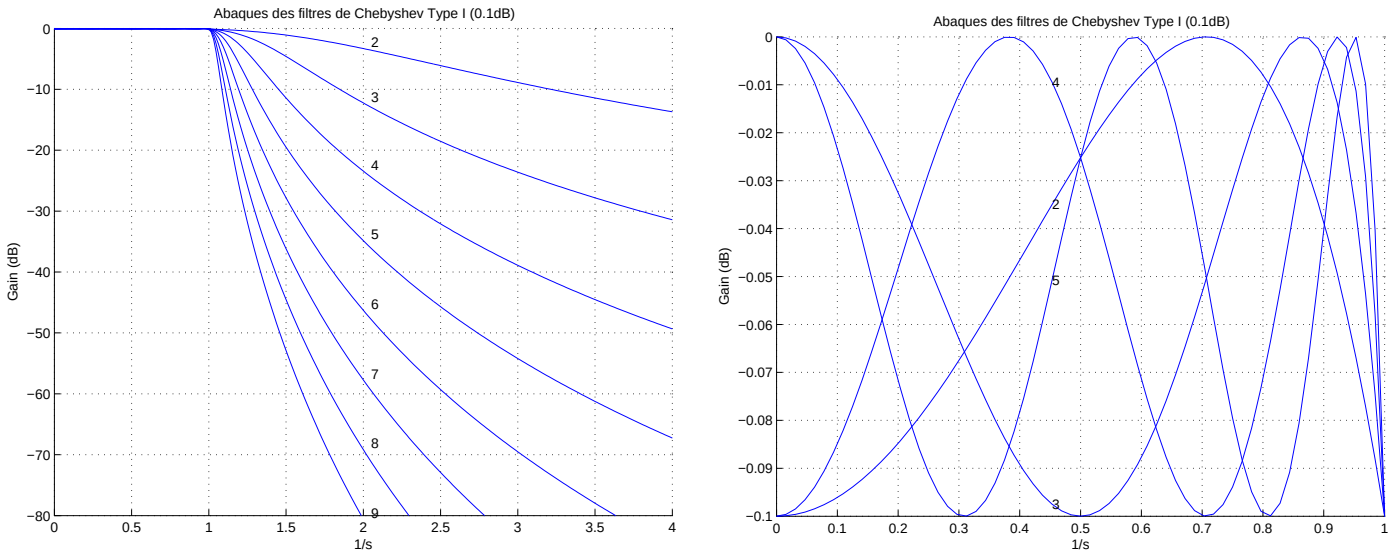


FIGURE B.3 – Réponse en fréquence des filtres de Chebyshev à $0.1dB$ d'ondulation en bande passante

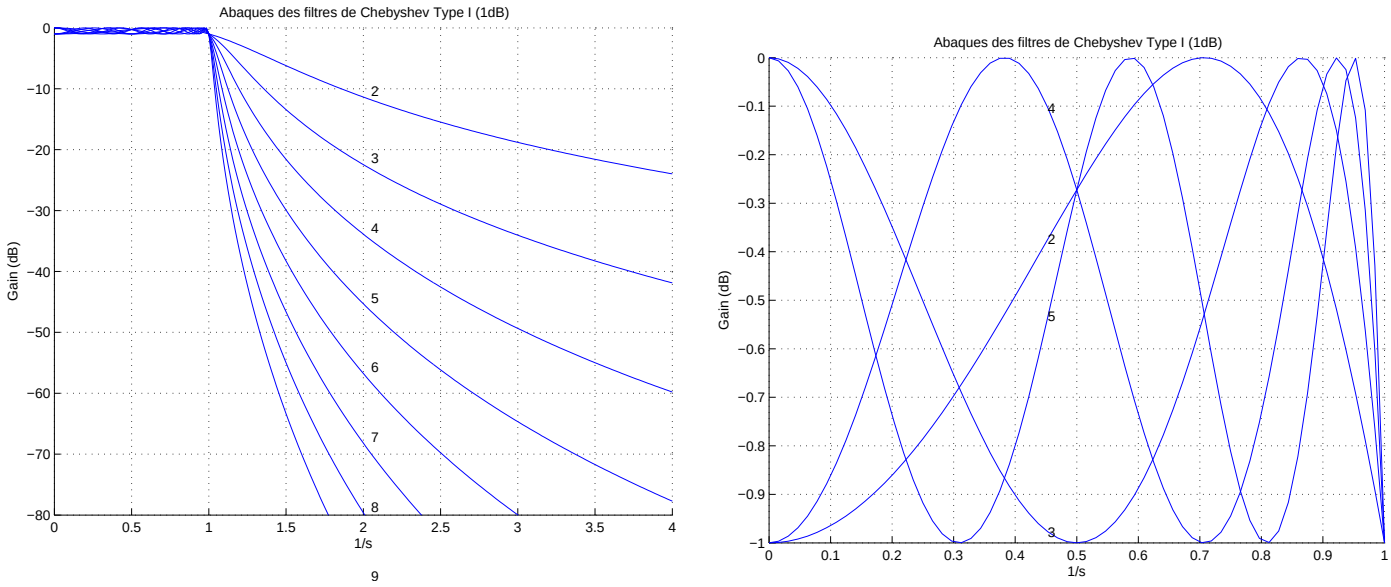


FIGURE B.4 – Réponse en fréquence des filtres de Chebyshev à $1dB$ d'ondulation en bande passante

B.4 Filtres elliptiques de Cauer

Les réponses fréquentielles des filtres prototypes de Cauer (elliptiques) d'ondulation $0.1dB$ et $1dB$ dans la bande passante, et d'atténuation en bande atténuée de $30dB$ et $50dB$, sont données figures B.5 et B.6. Leur fréquence de coupure est normalisée à $\omega = 1$. Ces courbes permettent de déterminer l'ordre du filtre.

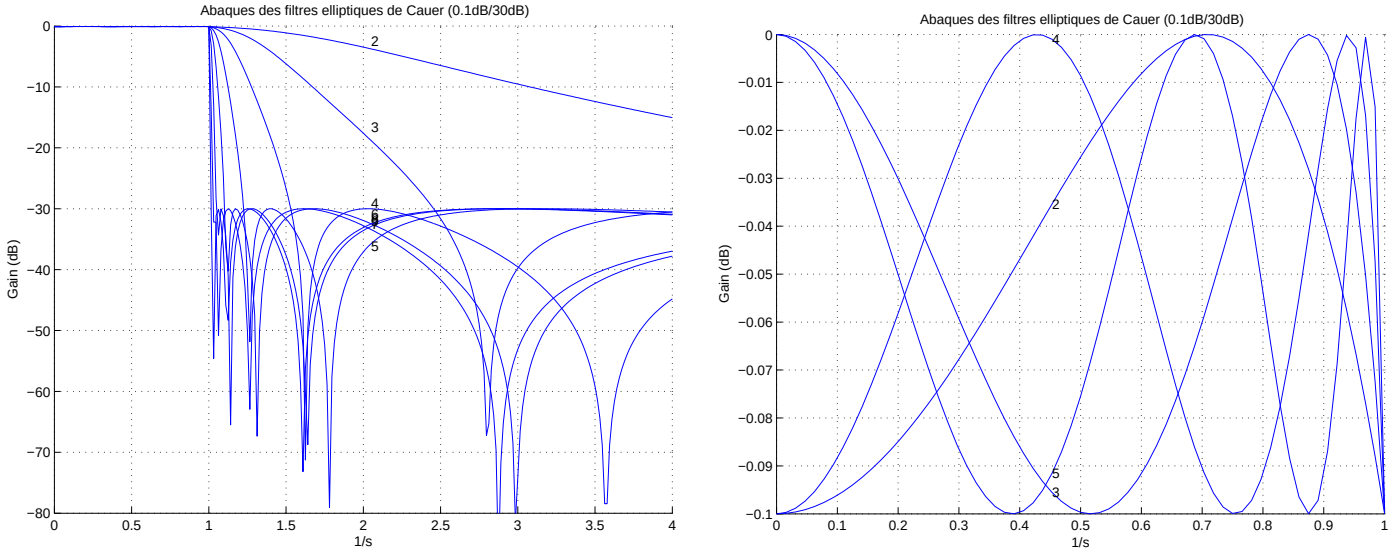


FIGURE B.5 – Réponse en fréquence des filtres elliptiques de Cauer à $0.1dB$ d'ondulation en bande passante et $30dB$ d'atténuation en bande atténuée

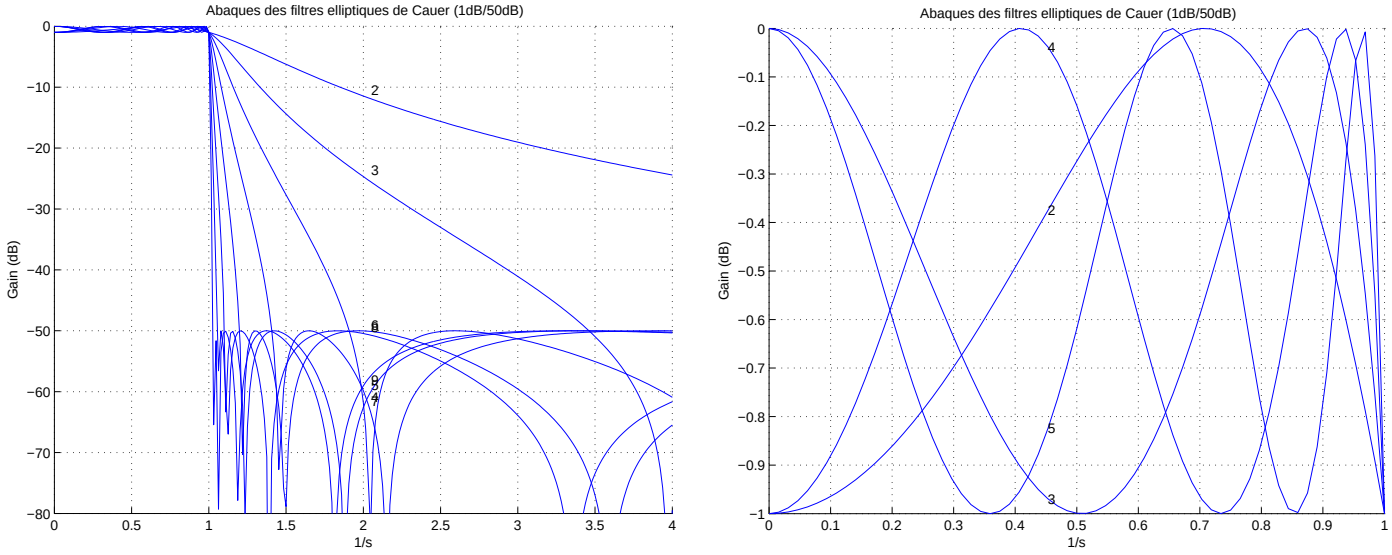


FIGURE B.6 – Réponse en fréquence des filtres elliptiques de Cauer à $1dB$ d'ondulation en bande passante et $50dB$ d'atténuation en bande atténuée

Bibliographie

[Ana95] Analog Device. *ADSP-2100 Family User's Manual*, 3 edition, September 1995.

[Ara97] G. Araujo. *Code Generation Algorithms for Digital Signal Processors*. PhD thesis, Princeton University Department of EE, Princeton, May 1997.

[Bie97] J. Bier. DSP16xxx Targets Communications Apps. *Microprocessor Report*, 11(12), September 1997.

[CCL01] G. Constantinides, P. Cheung, and W. Luk. Heuristic Datapath Allocation for Multiple Wordlength Systems. In *Design Automation and Test in Europe (DATE 01)*, pages 791–796, March 2001.

[EB00] J. Eyre and J. Bier. The Evolution of DSP Processors. *IEEE Signal Processing Magazine*, 17(2) :44–51, March 2000.

[Ga96] G. Goossens and al. Programmable Chips in Consumer Electronics and Telecommunications. In G. De Micheli and M SAmi, editors, *Hardware Software Co-Design*, pages 135–164. Kluwer Academic Publishers, 1996.

[Ga97] G. Goossens and al. Embedded Software in Real-Time Signal Processing Systems : Design Technologies. *Proceedings of the IEEE*, 85(3) :436–453, March 1997.

[HP96] J. L. Hennessy and D. A. Patterson. *Achitecture des Ordinnateurs - Une approche quantitative*. Morgan Kaufmann Publishers, San Francisco, 2 edition, 1996.

[Ka91] M. Kunt and al. *Techniques modernes de Traitement Numérique du Signal*. Collection CNET-ENST, Presses Romandes, Masson, 1991.

[Lap96] P. Lapsley. Fixed-Point DSP Processors : History and Trends. In *U.C. Berkeley (VLSI Signal Processing)*, April 1996.

[LBSL96] P. Lapsley, J. Bier, A. Shoham, and E. A. Lee. *DSP Processor Fundamentals : Architectures and Features*. Berkeley Design Technology, Inc, Fremont, CA, 1996.

[Lee88] E. Lee. Programmable DSP Architectures : Part I. *IEEE ASSP Magazine*, pages 4–14, Oct. 1988.

[Lee89] E. Lee. Programmable DSP Architectures : Part II. *IEEE ASSP Magazine*, pages 4–14, Janv. 1989.

[Luc] Lucent Technologies. *DSP16xx*. Lucent Technologies.

[Luc99] Lucent Technologies, Motorola. *SC140 DSP Core Reference Manual*. Lucent Technologies, December 1999.

[Mar95] P. Marwedel. Code Generation for Embedded Processors : an Introduction. In P. Marwedel and G. Goossens, editors, *Code Generation for Embedded Processors*. KluwerAcademic Publishers, 1995.

[Mot94] Motorola. *DSP56000 24-Bit Digital Signal Processor User'S Manual*. Motorola, 1994.

[MWK98] C. Moerman, R. Woudsma, and P. Kievits. Embedded DSP Technologies in Consumer applications. In *9th International Conference on Signal Processing Applications and Technology (ICSPAT 98)*, Toronto, September 1998. Miller Freeman.

[OS75] A. V. Oppenheim and R. W. Schafer. *Digital Signal Processing*. Prentice-Hall, 1975.

[OS99] A. V. Oppenheim and R. W. Schafer. *Discrete-Time Signal Processing, second edition*. Prentice-Hall, 1999.

[Pa97] P. Paulin and al. Embedded Software in Real-Time Signal Processing Systems : Application and Architecture Trends. *Proceedings of the IEEE*, 85(3) :419–435, March 1997.

[Peg99] A. Pegatoquet. *Méthode d'estimation de performance logicielle : application au développement rapide de code optimisé pour une classe de DSP*. PhD thesis, Université de Nice Sophia Antipolis, 1999.

[PM73] T.W. Parks and J.H. McClellan. A unified approach to the design of optimum linear phase digital filters. *IEEE Transactions on Circuit Theory*, 20 :697–701, Nov. 1973.

[Rab99] J.M. Rabaey. Managing Power Dissipation in the Generation after Next Wireless Systems. In *2ème journées francophones d'études Faible Tension Faible Consommation FTFC'99*, Paris, France, 26-28 mai 1999.

[Str00] W. Strauss. Internet Audio-Beyond Mp3. Technical Report 1030, Foward Concepts, Tempe, Arizona, USA, May 2000.

[Str02a] W. Strauss. Forward Concepts' Dsp Market Bulletin. Technical report, Foward Concepts, Tempe, Arizona, USA, March 2002.

[Str02b] W. Strauss. VoIP and Packet Voice DSP Markets. Technical Report 2201, Foward Concepts, Tempe, Arizona, USA, April 2002.

[Sud98] A. Sudarsanam. *Code Optimization Libraries for Retargetable Compilation for Embedded Digital Signal Processors*. PhD thesis, Princeton University Department of EE, Princeton, May 1998.

[Tex98] Texas Instruments. *TMS320C5X User's Guide*. Texas Instruments, June 1998.

[Tex99a] Texas Instruments. *TMS320C54X Dsp Cpu And Peripherals Reference Set Volume I*. Texas Instruments, Dallas, January 1999.

[Tex99b] Texas Instruments. *TMS320C6000 Cpu And Instruction Set Reference Guide*. Texas Instruments, Dallas, Sept 1999.

[TH97] J. Turley and H. Hakkarainen. TI's New 'C6x DSP Screams at 1,600 MIPS. *Microprocessor Report*, 11(2), Frebruary 1997.

[Tou99] J.M. Toureilles. *Conception d'architectures pour le traitement du signal en précision finie*. PhD thesis, Université de Rennes I, Janvier 1999.

[Yar99] K. Yarlagadda. Programmable DSP Architectures. In *DSP World*, Orlando, Florida, Nov. 1999. Miller Freeman.

[Za96] V. Zivojnovic and al. DSP Processor/Compiler Co-Design : A Quantitative Approach. In *Proceedings of the 9th International Symposium on System Synthesis (ISSS 96)*, La Jolla, November 1996.